opentext™

OpenText™ Documentum™ Content Management

Enterprise Content Services Reference Guide

Understand content services such as core, business process management, collaboration, search, and transformation.

EDCPKSVC250400-ARC-EN-01

OpenText™ Documentum™ Content Management
Enterprise Content Services Reference Guide
EDCPKSVC250400-ARC-EN-01
Rev.: 2025-July-15

This documentation has been created for OpenText™ Documentum™ Content Management CE 25.4.
It is also valid for subsequent software releases unless OpenText has made newer documentation available with the product, on an OpenText website, or by any other means.

Open Text Corporation

275 Frank Tompa Drive, Waterloo, Ontario, Canada, N2L 0A1

Tel: +1-519-888-7111
Toll Free Canada/USA: 1-800-499-6544 International: +800-4996-5440
Fax: +1-519-888-0677
Support: https://support.opentext.com
For more information, visit https://www.opentext.com

© 2025 Open Text

Patents may cover this product, see https://www.opentext.com/patents.

Disclaimer

No Warranties and Limitation of Liability

Every effort has been made to ensure the accuracy of the features and techniques presented in this publication. However, Open Text Corporation and its affiliates accept no responsibility and offer no warranty whether expressed or implied, for the accuracy of this publication.

Table of Contents

1 Enterprise Content Services services 9
1.1 Developing ECS consumers 9
1.2 Services and products 9

2 Core services 13
2.1 Object service 13
2.1.1 create operation 13
2.1.2 createPath operation 18
2.1.3 get operation 19
2.1.4 update operation 26
2.1.5 delete operation 32
2.1.6 copy operation 35
2.1.7 move operation 39
2.1.8 validate operation 42
2.1.9 getObjectContentUrls operation 44
2.1.10 registerEvent operation 45
2.1.11 unRegisterEvent operation 47
2.1.12 getPermission operation 49
2.1.13 getLinkedRepeatingString operation 50
2.1.14 hasAttributes operation 52
2.1.15 removeRendition operation 55
2.1.16 markVersion operation 57
2.1.17 updateNonCurrentVersion operation 60
2.1.18 saveAsNew operation 63
2.1.19 Working with lightweight objects 65
2.2 VersionControl service 74
2.2.1 getCheckoutInfo operation 74
2.2.2 checkout operation 76
2.2.3 checkin operation 78
2.2.4 AccessControl service 91
2.2.5 Lifecycle service 104
2.2.6 Schema service 119
2.2.7 Query service 140
2.2.8 QueryStore service 147
2.2.9 Virtual Document service 157
2.2.10 RepositoryInquiry service 178
2.3 DocumentumUtil service 180
2.3.1 dateTimeToIDfTime operation 180
2.4 LoginTicket service 182
2.4.1 getDCTMLoginTicket operation 182

3 Business process management services 185
3.1 Workflow service 185
3.1.1 Workflow SBO dependency 185
3.1.2 getProcessTemplates operation 186
3.1.3 getProcessInfo operation 188
3.1.4 startProcess operation 190
3.1.5 terminateWorkflow operation 194
3.1.6 completeWorkitem operation 195
3.2 Task Management service 197
3.2.1 Human task management 198
3.2.2 Generic human roles 198
3.2.3 TaskListFactory SBO dependency 199
3.2.4 claim operation 199
3.2.5 start operation 202
3.2.6 stop operation 202
3.2.7 release operation 203
3.2.8 suspend operation 206
3.2.9 suspendUntil operation 209
3.2.10 resume operation 212
3.2.11 complete operation 215
3.2.12 remove operation 218
3.2.13 fail operation 221
3.2.14 setPriority operation 224
3.2.15 addAttachment operation 227
3.2.16 getAttachmentInfos operation 230
3.2.17 getAttachments operation 234
3.2.18 deleteAttachments operation 237
3.2.19 addComment operation 240
3.2.20 getComments operation 243
3.2.21 skip operation 246
3.2.22 forward operation 246
3.2.23 delegate operation 249
3.2.24 getRendering operation 252
3.2.25 getRenderingTypes operation 252
3.2.26 getTaskInfo operation 252
3.2.27 getTaskDescription operation 255
3.2.28 setFault operation 258
3.2.29 deleteFault operation 258
3.2.30 getInput operation 258
3.2.31 getOutput operation 261
3.2.32 setOutput operation 264

3.2.33 deleteOutput operation 268
3.2.34 getFault operation 268
3.2.35 activate operation 271
3.2.36 nominate operation 271
3.2.37 setGenericHumanRole operation 274
3.2.38 getMyTaskAbstracts operation 275
3.2.39 getMyTasks operation 279
3.2.40 query operation 283

4 Collaboration services 289
4.1 Comment service 289
4.1.1 Dependencies 289
4.1.2 createComment operation 289
4.1.3 enumComments operation 290
4.1.4 getComment operation 291
4.1.5 markRead operation 291
4.1.6 markUnread operation 292
4.1.7 updateComment operation 292

5 Analytics services 293
5.1 Analytics service 293
5.1.1 Classifying documents 293
5.1.2 Objects related to this service 293
5.1.3 analyze operation 295

6 Search services. 301
6.1 Search service 301
6.1.1 Full-text and database searches 301
6.1.2 External source repositories 302
6.1.3 Non-blocking (asynchronous) searches 302
6.1.4 Caching mechanism 303
6.1.5 Computing clusters 303
6.1.6 Clustering SBO dependency 304
6.1.7 Objects related to this service 304
6.1.8 getRepositoryList operation 309
6.1.9 execute operation 311
6.1.10 stopSearch operation 315
6.1.11 getClusters operation 317
6.1.12 getSubclusters operation 319
6.1.13 getResultsProperties operation 322

7 Transformation services 325
7.1 Profile service 325

7.1.1 Objects related to this service 325
7.1.2 addProfile operation 326
7.1.3 addProfiles operation 326
7.1.4 getProfileById operation 327
7.1.5 getProfileByName operation 328
7.1.6 getProfiles operation 328
7.1.7 removeProfile operation 337
7.1.8 updateProfile operation 337
7.2 Transformation service 338
7.2.1 Objects related to this service 338
7.2.2 addJob operation 339
7.2.3 cleanUpJobs operation 341
7.2.4 deleteJob operation 342
7.2.5 getJobInfo operation 343
7.2.6 importAndAddJob operation 344
7.2.7 transformJob operation 347

8 Enterprise Integration services 359
8.1 ERP Integration Service 359
8.1.1 Prerequisites 359
8.1.2 Installation 359
8.1.3 Description of services 362
8.1.4 Application-level service examples 365
8.1.5 Sample applications 370
8.1.6 Known problems and limitations 372

9 Compliance management services 373
9.1 Policy service 373
9.1.1 Prerequisites and dependencies 373
9.1.2 Objects related to this service 374
9.1.3 getPolicies operation 376
9.1.4 getAppliedPolicies operation 378
9.1.5 apply operation 380
9.1.6 remove operation 382
9.2 Formal Record Service 384
9.2.1 Prerequisites and dependencies 385
9.2.2 Objects related to this service 386
9.2.3 getValidFormalRecordTypes operation 386
9.2.4 getFormalRecordTemplates operation 386
9.2.5 createFormalRecord operation 389
9.2.6 declareFormalRecord operation 391
9.3 Retention Markup Service 394
9.3.1 Prerequisites and dependencies 396

9.3.2 Objects related to this service 396
9.3.3 getRetentionMarkups operation 397
9.3.4 getAppliedRetentionMarkups operation 399
9.3.5 apply operation 402
9.3.6 remove operation 404
9.3.7 getRetentionMarkupProperties operation 406
9.3.8 getRetentionMarkupsQuery operation 409
9.4 Physical Records Library Service 412
9.4.1 Prerequisites and dependencies 414
9.4.2 Objects related to this service 414
9.4.3 createRequest operation 415
9.4.4 getLibraryRequests operation 417
9.4.5 cancelRequest operation 419
9.4.6 getUserLibraryRequests 420
9.4.7 getLibraryRequestProperties 421
9.4.8 getLibraryRequestsQuery 424
9.4.9 getUserLibraryRequestQuery 428
9.5 Federated Proxy Service 431
9.5.1 Prerequisites and dependencies 432
9.5.2 Objects related to this service 432
9.5.3 declareProxy operation 432
9.6 Electronic Signature Service 434
9.6.1 Prerequisites and dependencies 435
9.6.2 Objects related to this service 435
9.6.3 add operation 436
9.6.4 verify operation 438

10 Content delivery services 441
10.1 Content Delivery service 441
10.1.1 Dependencies and prerequisites 441
10.1.2 Objects related to this service 441
10.1.3 publishSite operation 444
10.1.4 publish operation 446
10.1.5 ingest operation 449

Chapter 1

Enterprise Content Services services

This guide is intended to provide a reference to Enterprise Content Services. Enterprise Content Services (ECS) are based on the OpenText™ Documentum™ Content Management Foundation SOAP API framework. Enterprise Content Services are a set of services based on OpenText Documentum Content Management (CM) Foundation SOAP API technology that provide a service-oriented interface to content management and archiving software products. This guide provides an API reference as well as sample code that shows how to use each service operation. For information about developing Foundation SOAP API consumers, as well as for information about developing your own Foundation SOAP API, see *(EDCPKSVC2502400-PGD)*. It is assumed that the reader has a basic understanding of SOAP-based web services, as well as the ability to understand code written in Java or C#.

1.1 Developing ECS consumers

This reference guide provides specific information and samples that will help you develop consumers of ECS services. All of the services are based on a common framework (Foundation SOAP API), and can be consumed using the Foundation SOAP API Java and .NET client libraries (also called the productivity layer), or with standard web services tools.

1.2 Services and products

Many of these services are delivered with the Foundation SOAP API product delivered with OpenText™ Documentum™ Content Management Server.

Foundation SOAP API services and server runtime are delivered in the `dfs.ear` archive. The service addresses use the context root <protocol>://<host>:<port>/services. For example, the address of the ObjectService WSDL could be:

```
http://DfsHost:9080/services/core/ObjectService?WSDL
```

The following tables list the service provided as part of Foundation SOAP API, categorized by module, and the services delivered with other products:

Table 1-1: Services delivered with Foundation SOAP API

Module name	Service
core	"Object service" on page 13
	"AccessControl service" on page 91
	"Lifecycle service" on page 104

Module name	Service
	"Query service" on page 140
	"QueryStore service" on page 147
	"Virtual Document service" on page 157
	"RepositoryInquiry service" on page 178
bpm	"Workflow service" on page 185
	"Task Management service" on page 197
ci	"Analytics service" on page 293
collaboration	"Comment service" on page 289
search	"Search service" on page 301

Services not delivered as part of Foundation SOAP API require the deployment of separate archives, but share the same context root, that is <protocol>://<host>:<port>/services.

Table 1-2: Services delivered with other products

Product	Archive	Module	Service
OpenText™ Documentum™ Content Management Transformation Services	transformation.ear	transformation	"Profile service" on page 325
	transformation.ear	transformation	"Transformation service" on page 338
Process Services for SAP	erp.ear	erp	"ERP Integration Service" on page 359
OpenText™ Documentum™ Content Management Records Manager	rm.ear, rps.ear	policy	"Policy service" on page 373
	rm.ear	formalrecord	"Formal Record Service" on page 384
OpenText™ Documentum™ Content Management Retention Policy Services	rm.ear, rps.ear	retentionmarkup	"Retention Markup Service" on page 394
Physical Records Manager	rm.ear, rps.ear	physicallibrary	"Physical Records Library Service" on page 412
Federated Records Service	rm.ear, rps.ear	federatedproxy	"Federated Proxy Service" on page 431
Documentum Compliance Manager	compliance.ear	compliance	"Electronic Signature Service" on page 434

Product	Archive	Module	Service
Interactive Delivery Services	scs.ear	scs	"Content Delivery service" on page 441

Chapter 2

Core services

2.1 Object service

The Object service provides a set of operations on repository objects, such as create, get, update, delete, and move. The object service operations are intentionally version agnostic: each operation uses appropriate default behaviors as relates to object versions. All of the Object service operations can operate on multiple objects (contained in either a DataPackage or an ObjectIdentitySet), enabling clients to optimize service usage by minimizing the number of service interactions.

2.1.1 create operation

The create operation creates a set of new repository objects based on the DataObject instances contained in a DataPackage passed to the operation. Because each DataObject represents a new repository object, its ObjectIdentity is populated with only a repository name. OpenText Documentum Content Management (CM) Server assigns a unique object identifier when the object is created in the repository.

To create an object in a specific location, or to create objects that have relationships to one another defined in the repository, the client can define Relationship instances in a DataObject passed to the operation. The most common example of this would be to create a Relationship between a newly created document and the folder in which it is to be created.

2.1.1.1 Java syntax

```
DataPackage create(DataPackage dataPackage,
                   OperationOptions operationOptions)
        throws CoreServiceException, ServiceException
```

2.1.1.2 C# syntax

```
DataPackage Create(DataPackage dataPackage,
                   OperationOptions operationOptions)
```

2.1.1.3 Parameters

Parameter	Data type	Description
dataPackage	DataPackage	A collection of DataObject instances that identify the repository objects to be created.
operationOptions	OperationOptions	An object containing profiles and properties that specify operation behaviors.

2.1.1.4 Profiles

Generally the expected behavior of the create operation can be logically determined by the objects contained within the DataPackage passed to the operation. For example, if DataObject instances contained in the DataPackage include content, the operation assumes that it should transfer the content and create contentful repository objects. Similarly, if DataObject instances contain Relationship objects, the relationships are created along with the primary object. The profile that does have direct bearing on the create operation is the ContentTransferProfile, which determines the mode used to transfer content from the remote client to the repository. The ContentTransferProfile will generally be set in the service context.

Other profiles, such as ContentProfile, PropertyProfile, and RelationshipProfile, will control the composition of the DataPackage returned by the create operation, which by default will contain an ObjectIdentity only. These profiles can allow the client to obtain detailed information about created objects if required without performing an additional query.

2.1.1.5 Response

Returns a DataPackage containing one DataObject for each repository object created by the create operation. By default, each DataObject contains only the ObjectIdentity of the created object and no other data. The client can modify this behavior by using Profile objects if it requires more data about the created objects.

2.1.1.6 Examples

2.1.1.6.1 Simple object creation

The following sample creates a folder in the repository in the default location:

Example 2-1: Java: Simple object creation

```
public DataPackage createNewFolder()
    throws ServiceException
{
    ObjectIdentity folderIdentity = new ObjectIdentity();
    folderIdentity.setRepositoryName(defaultRepositoryName);
    DataObject dataObject = new DataObject(folderIdentity, "dm_folder");
    PropertySet properties = new PropertySet();
    String folderName = "aTestFolder-" + System.currentTimeMillis();
    properties.set("object_name", folderName);
    dataObject.setProperties(properties);

    DataPackage dataPackage = new DataPackage(dataObject);

    OperationOptions operationOptions = null;
    return objectService.create(dataPackage, operationOptions);
}
```

Example 2-2: C#: Simple object creation

```
public DataPackage CreateNewFolder()
{
    ObjectIdentity folderIdentity = new ObjectIdentity();
    folderIdentity.RepositoryName = DefaultRepository;
    DataObject dataObject = new DataObject(folderIdentity, "dm_folder");
    PropertySet properties = new PropertySet();
    String folderName = "aTestFolder-" + System.DateTime.Now.Ticks;
    properties.Set("object_name", folderName);
    dataObject.Properties = properties;

    DataPackage dataPackage = new DataPackage(dataObject);

    OperationOptions operationOptions = null;
    return objectService.Create(dataPackage, operationOptions);
}
```

2.1.1.6.2 Creating and linking

The following sample creates and object and uses a ReferenceRelationship to link it into an existing folder:

Example 2-3: Java: Creating and linking

```
public DataObject createAndLinkToFolder(String folderPath)
{
    // create a contentless document to link into folder
    String objectName = "linkedDocument" + System.currentTimeMillis();
    String repositoryName = defaultRepositoryName;
    ObjectIdentity sampleObjId = new ObjectIdentity(repositoryName);
    DataObject sampleDataObject = new DataObject(sampleObjId, "dm_document");
    sampleDataObject.getProperties().set("object_name", objectName);

    // add the folder to link to as a ReferenceRelationship
    ObjectPath objectPath = new ObjectPath(folderPath);
    ObjectIdentity<ObjectPath> sampleFolderIdentity
```

```
        = new ObjectIdentity<ObjectPath>(objectPath,
                                         defaultRepositoryName);
    ReferenceRelationship sampleFolderRelationship = new ReferenceRelationship();
    sampleFolderRelationship.setName(Relationship.RELATIONSHIP_FOLDER);
    sampleFolderRelationship.setTarget(sampleFolderIdentity);
    sampleFolderRelationship.setTargetRole(Relationship.ROLE_PARENT);
    sampleDataObject.getRelationships().add(sampleFolderRelationship);

    // create a new document linked into parent folder
    try
    {
        OperationOptions operationOptions = null;
        DataPackage dataPackage = new DataPackage(sampleDataObject);
        objectService.create(dataPackage, operationOptions);
    }
    catch (ServiceException e)
    {
        throw new RuntimeException(e);
    }

    return sampleDataObject;
}
```

Example 2-4: C#: Creating and linking

```
public DataObject CreateAndLinkToFolder(String folderPath)
{
    // create a contentless document to link into folder
    String objectName = "linkedDocument" + System.DateTime.Now.Ticks;
    String repositoryName = DefaultRepository;
    ObjectIdentity sampleObjId = new ObjectIdentity(repositoryName);
    DataObject sampleDataObject = new DataObject(sampleObjId, "dm_document");
    sampleDataObject.Properties.Set("object_name", objectName);

    // add the folder to link to as a ReferenceRelationship
    ObjectPath objectPath = new ObjectPath(folderPath);
    ObjectIdentity sampleFolderIdentity = new ObjectIdentity
        (objectPath, DefaultRepository);
    ReferenceRelationship sampleFolderRelationship = new ReferenceRelationship();
    sampleFolderRelationship.Name = Relationship.RELATIONSHIP_FOLDER;
    sampleFolderRelationship.Target = sampleFolderIdentity;
    sampleFolderRelationship.TargetRole = Relationship.ROLE_PARENT;
    sampleDataObject.Relationships.Add(sampleFolderRelationship);

    // create a new document linked into parent folder
    OperationOptions operationOptions = null;
    DataPackage dataPackage = new DataPackage(sampleDataObject);
    objectService.Create(dataPackage, operationOptions);

    return sampleDataObject;
}
```

2.1.1.6.3 Creating multiple related objects

The following sample creates a folder with a Relationship to a new document. The create service will create both the document and the folder, and link the document into the folder.

Example 2-5: Java: Creating multiple related objects

```
public DataPackage createFolderAndLinkedDoc()
        throws ServiceException
{
    // create a folder data object
    String folderName = "Otest-folder-" + System.currentTimeMillis();
    DataObject folderDataObj
            = new DataObject(new ObjectIdentity(defaultRepositoryName),
                             "dm_folder");
    PropertySet folderDataObjProperties = new PropertySet();
    folderDataObjProperties.set("object_name", folderName);
    folderDataObj.setProperties(folderDataObjProperties);

    // create a contentless document DataObject
    String doc1Name = "aTestDoc-" + System.currentTimeMillis();
    DataObject docDataObj
        = new DataObject(new ObjectIdentity(defaultRepositoryName),
                         "dm_document");
    PropertySet properties = new PropertySet();
    properties.set("object_name", doc1Name);
    docDataObj.setProperties(properties);

    // add the folder as a parent of the folder
    ObjectRelationship objRelationship = new ObjectRelationship();
    objRelationship.setTarget(folderDataObj);
    objRelationship.setName(Relationship.RELATIONSHIP_FOLDER);
    objRelationship.setTargetRole(Relationship.ROLE_PARENT);
    docDataObj.getRelationships().add(new ObjectRelationship(objRelationship));

    // create the folder and linked document
    DataPackage dataPackage = new DataPackage();
    dataPackage.addDataObject(docDataObj);
    OperationOptions operationOptions = null;
    return objectService.create(dataPackage, operationOptions);
}
```

Example 2-6: C#: Creating multiple related objects

```
public DataPackage CreateFolderAndLinkedDoc()
{
    // create a folder data object
    String folderName = "Otest-folder-" + System.DateTime.Now.Ticks;
    DataObject folderDataObj = new DataObject(new ObjectIdentity(DefaultRepository),
                                              "dm_folder");
    PropertySet folderDataObjProperties = new PropertySet();
    folderDataObjProperties.Set("object_name", folderName);
    folderDataObj.Properties = folderDataObjProperties;

    // create a contentless document DataObject
    String doc1Name = "aTestDoc-" + System.DateTime.Now.Ticks;
    DataObject docDataObj = new DataObject(new ObjectIdentity(DefaultRepository),
                                           "dm_document");
    PropertySet properties = new PropertySet();
    properties.Set("object_name", doc1Name);
    docDataObj.Properties = properties;

    // add the folder as a parent of the folder
```

```
    ObjectRelationship objRelationship = new ObjectRelationship();
    objRelationship.Target = folderDataObj;
    objRelationship.Name = Relationship.RELATIONSHIP_FOLDER;
    objRelationship.TargetRole = Relationship.ROLE_PARENT;
    docDataObj.Relationships.Add(new ObjectRelationship(objRelationship));

    // create the folder and linked document
    DataPackage dataPackage = new DataPackage();
    dataPackage.AddDataObject(docDataObj);
    OperationOptions operationOptions = null;
    return objectService.Create(dataPackage, operationOptions);
}
```

2.1.2 createPath operation

The createPath operation creates a folder structure (from the cabinet down) in a repository. The path is passed to the service as an ObjectPath, which contains a path String in the format "/cabinetName/folderName...", which can be extended to any depth. If any of the folders specified in the path exist, no exception is thrown. This allows you to use the operation to create the complete path, or to add new folders to an existing path.

2.1.2.1 Java syntax

```
DataPackage ObjectIdentity createPath(ObjectPath objectPath, String repositoryName)
        throws CoreServiceException, ServiceException
```

2.1.2.2 C# syntax

```
ObjectIdentity CreatePath(ObjectPath objectPath, String repositoryName)
```

2.1.2.3 Parameters

Parameter	Data type	Description
objectPath	ObjectPath	Contains a String in the form "/cabinetName/ folderName..." that describes the complete path to create.

2.1.2.4 Response

Returns the ObjectIdentity of the final object in the path. For example, if the path is "/cabinetName/childFolder1/childFolder2", the operation will return the ObjectIdentity of childFolder2.

2.1.2.5 Example

The following sample creates a path consisting of a cabinet and a folder. If the cabinet exists, only the folder is created. If the cabinet and folder both exist, the operation does nothing.

Example 2-7: Java: Creating a folder in a cabinet

```
public ObjectIdentity createFolderInCabinet()
    throws ServiceException
{
    ObjectPath objPath = new ObjectPath("/DFSTestCabinet/createPathTestFolder");
    return objectService.createPath(objPath, defaultRepositoryName);
}
```

Example 2-8: C#: Creating a folder in a cabinet

```
public ObjectIdentity CreateFolderInCabinet()
{
    ObjectPath objPath = new ObjectPath("/DFSTestCabinet/createPathTestFolder");
    return objectService.CreatePath(objPath, DefaultRepository);
}
```

2.1.3 get operation

The get operation retrieves a set of objects from the repository based on the contents of an ObjectIdentitySet. The get operation always returns the version of the object specified by ObjectIdentity; if the ObjectIdentity identifies a non-CURRENT version, the get operation returns the non-CURRENT version. The operation will also return related objects if instructed to do so by RelationshipProfile settings.

The get operation supports retrieval of content from external sources available to the Search service (see "Getting content from external sources" on page 25).

When getting a virtual document, you can supply a VdmRetrieveProfile to provide settings controlling virtual document assembly. See "VdmRetrieveProfile" on page 171.

2.1.3.1 Java syntax

```
DataPackage DataPackage get(ObjectIdentitySet forObjects,
                OperationOptions operationOptions)
        throws CoreServiceException, ServiceException
```

2.1.3.2 C# syntax

```
DataPackage Get(ObjectIdentitySet forObjects,
                OperationOptions operationOptions)
```

2.1.3.3 Parameters

Parameter	Data type	Description
forObjects	ObjectIdentitySet	Contains a list of ObjectIdentity instances specifying the repository objects to be retrieved.
operationOptions	OperationOptions	An object containing profiles and properties that specify operation behaviors. If this object is null, default operation behaviors will take effect.

2.1.3.4 Profiles

See "Controlling data returned by get operation" on page 21.

2.1.3.5 Response

Returns a DataPackage containing DataObject instances representing objects retrieved from the repository. The client can control the complexity of the data in each DataObject using Profile settings passed in operationOptions or stored in the service context. The following table summarizes the data returned by the operation in the default case; that is, if no profiles are set:

Data	Default behavior
Properties	Returns all non-system.
Content	Returns none.
Relations	Returns none.
Permission	Returns none.

For more information see "Controlling data returned by get operation" on page 21.

2.1.3.6 Example

The following example uses an ObjectId to reference a repository object and retrieves it from the repository:

Example 2-9: Java: Basic object retrieval

```
public DataObject getObjectWithDefaults(ObjectIdentity objIdentity)
    throws ServiceException
{
    objIdentity.setRepositoryName(defaultRepositoryName);
    ObjectIdentitySet objectIdSet = new ObjectIdentitySet();
    List<ObjectIdentity> objIdList = objectIdSet.getIdentities();
    objIdList.add(objIdentity);

    OperationOptions operationOptions = null;
    DataPackage dataPackage = objectService.get(objectIdSet, operationOptions);

    return dataPackage.getDataObjects().get(0);
}
```

Example 2-10: C#: Basic object retrieval

```
public DataObject GetObjectWithDefaults(ObjectIdentity objIdentity)
{
    objIdentity.RepositoryName = DefaultRepository;
    ObjectIdentitySet objectIdSet = new ObjectIdentitySet();
    List<ObjectIdentity> objIdList = objectIdSet.Identities;
    objIdList.Add(objIdentity);

    OperationOptions operationOptions = null;
    DataPackage dataPackage = objectService.Get(objectIdSet, operationOptions);

    return dataPackage.DataObjects[0];
}
```

2.1.3.7 Controlling data returned by get operation

You can control the type and quantity of data returned as part of a DataObject by the get operation using profiles, which are made accessible to the get operation via the operationOptions parameter or the service context. The following profiles can be used:

- PropertyProfile
- PermissionProfile
- RelationshipProfile
- ContentProfile

2.1.3.7.1 Filtering properties using PropertyProfile

By default, the get operation returns all non-system properties of an object (PropertyFilterMode.ALL_NON_SYSTEM). The following example shows how to configure the PropertyProfile so that the get operation returns no properties:

Example 2-11: Java: PropertyProfile

```
PropertyProfile propertyProfile = new PropertyProfile();
propertyProfile.setFilterMode(PropertyFilterMode.NONE);
OperationOptions operationOptions = new OperationOptions();
operationOptions.setPropertyProfile(propertyProfile);
ObjectIdentitySet objectIdSet = new ObjectIdentitySet(objIdentity);
```

Example 2-12: C#: PropertyProfile

```
PropertyProfile propertyProfile = new PropertyProfile();
propertyProfile.FilterMode = PropertyFilterMode.NONE;
OperationOptions operationOptions = new OperationOptions();
operationOptions.PropertyProfile = propertyProfile;
ObjectIdentitySet objectIdSet = new ObjectIdentitySet(objIdentity);
```

Another useful option is to set the filter mode to SPECIFIED_BY_INCLUDE, and provide a list of the specific properties that the client program requires.

Example 2-13: Java: Specifying included properties

```
PropertyProfile propertyProfile = new PropertyProfile();
propertyProfile.setFilterMode(PropertyFilterMode.SPECIFIED_BY_INCLUDE);
ArrayList<String> includeProperties = new ArrayList<String>();
includeProperties.add("title");
includeProperties.add("object_name");
includeProperties.add("r_object_type");
propertyProfile.setIncludeProperties(includeProperties);
OperationOptions operationOptions = new OperationOptions();
operationOptions.setPropertyProfile(propertyProfile);
```

Example 2-14: C#: Specifying included properties

```
PropertyProfile propertyProfile = new PropertyProfile();
propertyProfile.FilterMode = PropertyFilterMode.SPECIFIED_BY_INCLUDE;
List<string> includeProperties = new List<string>();
includeProperties.Add("title");
includeProperties.Add("object_name");
includeProperties.Add("r_object_type");
propertyProfile.IncludeProperties = includeProperties;
OperationOptions operationOptions = new OperationOptions();
operationOptions.PropertyProfile = propertyProfile;
```

Conversely, you can set the filter mode to SPECIFIED_BY_EXCLUDE and provide a list of excluded properties.

Example 2-15: Java: Specifying excluded properties

```
PropertyProfile propertyProfile = new PropertyProfile();
propertyProfile.setFilterMode(PropertyFilterMode.SPECIFIED_BY_EXCLUDE);
ArrayList<String> excludeProperties = new ArrayList<String>();
excludeProperties.add("title");
excludeProperties.add("object_name");
excludeProperties.add("r_object_type");
propertyProfile.setExcludeProperties(excludeProperties);
OperationOptions operationOptions = new OperationOptions();
operationOptions.setPropertyProfile(propertyProfile);
```

Example 2-16: C#: Specifying excluded properties

```
PropertyProfile propertyProfile = new PropertyProfile();
propertyProfile.FilterMode = PropertyFilterMode.SPECIFIED_BY_EXCLUDE;
List<string> excludeProperties = new List<string>();
excludeProperties.Add("title");
excludeProperties.Add("object_name");
excludeProperties.Add("r_object_type");
propertyProfile.ExcludeProperties = excludeProperties;
OperationOptions operationOptions = new OperationOptions();
operationOptions.PropertyProfile = propertyProfile;
```

2.1.3.7.2 Controlling Relationship instances

By default, the get operation returns no Relationship instances. You can use a RelationshipProfile to specify exactly what relation types and relation target roles the get operation will return, and to what depth to return Relationship instances. You can use relationProfile.setResultDataMode(ResultDataMode.OBJECT) or (ResultDataMode.REFERENCE) to control whether the get operation returns a reference relationship or an object relationship.

The following example adds a RelationshipProfile to operationOptions to specify that all relations are returned as part of the data object, to any depth:

Example 2-17: Java: RelationshipProfile

```
RelationshipProfile relationProfile = new RelationshipProfile();
relationProfile.setResultDataMode(ResultDataMode.REFERENCE);
relationProfile.setTargetRoleFilter(TargetRoleFilter.ANY);
relationProfile.setNameFilter(RelationshipNameFilter.ANY);
relationProfile.setDepthFilter(DepthFilter.UNLIMITED);
OperationOptions operationOptions = new OperationOptions();
operationOptions.setRelationshipProfile(relationProfile);
```

Example 2-18: C#: RelationshipProfile

```
RelationshipProfile relationProfile = new RelationshipProfile();
relationProfile.ResultDataMode = ResultDataMode.REFERENCE;
relationProfile.TargetRoleFilter = TargetRoleFilter.ANY;
relationProfile.NameFilter = RelationshipNameFilter.ANY;
relationProfile.DepthFilter = DepthFilter.UNLIMITED;
```

```
OperationOptions operationOptions = new OperationOptions();
operationOptions.RelationshipProfile = relationProfile;
```

The next example adds a RelationshipProfile to operationOptions to specify that only the proximate parent relations of the data object are returned.

Example 2-19: Java: Filtering relationships, parent only

```
RelationshipProfile relationProfile = new RelationshipProfile();
relationProfile.setResultDataMode(ResultDataMode.REFERENCE);
relationProfile.setTargetRoleFilter(TargetRoleFilter.SPECIFIED);
relationProfile.setTargetRole(Relationship.ROLE_PARENT);
relationProfile.setNameFilter(RelationshipNameFilter.ANY);
relationProfile.setDepthFilter(DepthFilter.SINGLE);
OperationOptions operationOptions = new OperationOptions();
operationOptions.setRelationshipProfile(relationProfile);
```

Example 2-20: C#: Filtering relationships, parent only

```
RelationshipProfile relationProfile = new RelationshipProfile();
relationProfile.ResultDataMode = ResultDataMode.REFERENCE;
relationProfile.TargetRoleFilter = TargetRoleFilter.SPECIFIED;
relationProfile.TargetRole = Relationship.ROLE_PARENT;
relationProfile.NameFilter = RelationshipNameFilter.ANY;
relationProfile.DepthFilter = DepthFilter.SINGLE;
OperationOptions operationOptions = new OperationOptions();
operationOptions.RelationshipProfile = relationProfile;
```

2.1.3.7.3 Filtering content

By default, the get operation returns no content. The client can use a ContentProfile to specify that content be returned by the get operation, and to filter specific content types.

To specify that any content be returned, set the format filter to ANY, as shown in the following sample:

Example 2-21: Java: Returning any content format

```
ContentProfile contentProfile = new ContentProfile();
contentProfile.setFormatFilter(FormatFilter.ANY);
OperationOptions operationOptions = new OperationOptions();
operationOptions.setContentProfile(contentProfile);
operationOptions.setProfile(contentProfile);
```

Example 2-22: C#: Returning any content format

```
ContentProfile contentProfile = new ContentProfile();
contentProfile.FormatFilter = FormatFilter.ANY;
OperationOptions operationOptions = new OperationOptions();
```

```
operationOptions.ContentProfile = contentProfile;
operationOptions.SetProfile(contentProfile);
```

To specify that only content of a specified format be returned, set the format filter to SPECIFIED and set the format using the setFormat method, as shown in the following sample:

Example 2-23: Java: Returning a specified content format

```
ContentProfile contentProfile = new ContentProfile();
contentProfile.setFormatFilter(FormatFilter.SPECIFIED);
contentProfile.setFormat("gif");
OperationOptions operationOptions = new OperationOptions();
operationOptions.setContentProfile(contentProfile);
```

The format string can be either the content mime type or the name of a dm_format object in the repository.

Example 2-24: C#: Returning a specified content format

```
ContentProfile contentProfile = new ContentProfile();
contentProfile.FormatFilter = FormatFilter.SPECIFIED;
contentProfile.Format = "gif";
OperationOptions operationOptions = new OperationOptions();
operationOptions.ContentProfile = contentProfile;
```

You can also filter content by page number or page modifier.

2.1.3.8 Getting content from external sources

The Object service get operation can retrieve content from external, ECI-enabled repositories available to the Search service. The Search service getRepositoryList operation returns a list of available sources (both OpenText Documentum Content Management (CM) repositories and external services). The Search service execute operation returns the results of a query against a list of available sources. The get operation can return content based on the query result, as shown here:

Example 2-25: Java: Retrieving content based on query results

```
List<DataObject> dataObjects = result.getDataObjects();
ObjectIdentitySet identities = new ObjectIdentitySet();
for(DataObject data : dataObjects)
{
    identities.addIdentity(data.getIdentity());
}
DataPackage package = objectService.get(identities, operationOptions);
```

For content retrieved from external sources, profiles, such as RelationshipProfile and PropertyProfile, are largely inapplicable. A ContentProfile is required to specify that content be retrieved; however filter settings within the ContentProfile are ignored.

2.1.4 update operation

The update operation updates a set of repository objects using data supplied in a set of DataObject instances passed in a DataPackage. The update operation will only update the CURRENT version of an object. If passed an ObjectIdentity that identifies a non-CURRENT object, the operation will throw an exception. The updated repository object will be saved as the CURRENT version.

The ObjectIdentity of each DataObject passed to the update operation must uniquely identify an existing repository object. The DataObject instances can contain updates to properties, content, and relationships, and only needs to include data that requires update.

If a DataObject contains ReferenceRelationship instances, the corresponding relationships are created or updated in the repository. The update operation can also remove existing relationships. It can therefore be used, for example, to unlink an object from a folder and link it into another folder. If the DataObject contains ObjectRelationship instances, then the related objects are either updated or created, depending on whether they already exist in the repository. If the object does not exist, it is created; if it does exist, it is updated.

2.1.4.1 Java syntax

```
DataPackage update(DataPackage dataPackage,
                   OperationOptions options)
        throws CoreServiceException, ServiceException
```

2.1.4.2 C# syntax

```
DataPackage Update(DataPackage dataPackage,
                   OperationOptions options)
```

2.1.4.3 Parameters

Parameter	Data type	Description
dataPackage	DataPackage	A collection of DataObject instances that contain modifications to repository objects. The ObjectIdentity of each DataObject instance must uniquely identity the repository object to update. The DataObject instance need only contain data that is to be modified on the repository object; data that is to remain unchanged need not be supplied.
options	OperationOptions	An object containing profiles and properties that specify operation behaviors.

2.1.4.4 Profiles

Generally the expected behavior of the update operation can be logically determined by the objects contained within the DataPackage passed to the operation. For example, if DataObject instances contained in the DataPackage include content, the operation assumes that it should transfer and update content. Similarly, if DataObject instances contain Relationship objects, the relationships are created or updated. The profile that does have direct bearing on the update operation is the ContentTransferProfile, which determines the mode used to transfer content from the remote client to the repository. The ContentTransferProfile will generally be set in the service context.

Note: Profiles can be used to filter out data passed to the update operation, so that the data will not be used in the update. Especially note that if a DataObject passed to the update operation contains system properties, you should leave the PropertyFilter set to PropertyFilterMode.ALL_NON_SYSTEM (this is the default) to avoid errors caused by attempts to update system properties. Use PropertyFilterMode.ALL only if you explicitly want to change a system property.

Other profiles, such as ContentProfile, PropertyProfile, and RelationshipProfile, will control the contents of the DataPackage returned by the update operation, which by default will contain an ObjectIdentity only. These profiles can allow the client to obtain detailed information about updated objects if required without performing an additional query.

2.1.4.5 Response

The update operation returns a DataPackage, which by default is populated with DataObject instances that contain only an ObjectIdentity. This default behavior can be changed through the use of Profile objects set in the OperationOptions or service context.

2.1.4.6 Examples

2.1.4.6.1 Updating properties

To update the properties of an existing repository object, the client can pass a DataObject that has an ObjectIdentity that identities it as the existing object, and just those properties that require update. This keeps the data object passed to the update service as small as possible. If the client wants to test whether the updates have been applied by examining the DataPackage object returned by the update operation, it will need to use a PropertyProfile to instruct the service to return all properties. Otherwise the update operation will by default return DataObject instances with only an ObjectIdentity.

The following example updates the properties of an existing document. It passes a PropertyProfile object in operationOptions, causing the update operation to return all properties. It creates a new DataObject with an ObjectIdentity mapping it to an existing document in the repository, and passes this new DataObject to the update operation.

Example 2-26: Java: Updating properties

```
public DataPackage updateObjectProperties(ObjectIdentity objectIdentity,
           String newTitle, String newSubject, String[] newKeywords)
           throws ServiceException
{
    PropertyProfile propertyProfile = new PropertyProfile();

    // Setting the filter to ALL can cause errors if the DataObject
    // passed to the operation contains system properties, so to be safe
    // set the filter to ALL_NON_SYSTEM unless you explicitly want to update
    // a system property
    propertyProfile.setFilterMode(PropertyFilterMode.ALL_NON_SYSTEM);
    OperationOptions operationOptions = new OperationOptions();
    operationOptions.setProfile(propertyProfile);

    DataObject dataObject = new DataObject(objectIdentity);

    PropertySet properties = new PropertySet();
    properties.set("title", newTitle);
    properties.set("subject", newSubject);
    properties.set("keywords", newKeywords);
    dataObject.setProperties(properties);

    try
    {
        return objectService.update(new DataPackage(dataObject),
                    operationOptions);
    } catch (ServiceException sE)
    {
        sE.printStackTrace();
    }
```

```
    return null;
}
```

Example 2-27: C#: Updating object properties

```
public DataPackage UpdateObjectProperties(ObjectIdentity objectIdentity,
                                          String newTitle,
                                          String newSubject,
                                          String[] newKeywords)
{
    PropertyProfile propertyProfile = new PropertyProfile();

    // Setting the filter to ALL can cause errors if the DataObject
    // passed to the operation contains system properties, so to be safe
    // set the filter to ALL_NON_SYSTEM unless you explicitly want to update
    // a system property
    propertyProfile.FilterMode = PropertyFilterMode.ALL_NON_SYSTEM;
    OperationOptions operationOptions = new OperationOptions();
    operationOptions.SetProfile(propertyProfile);

    DataObject dataObject = new DataObject(objectIdentity);

    PropertySet properties = new PropertySet();
    properties.Set("title", newTitle);
    properties.Set("subject", newSubject);
    properties.Set("keywords", newKeywords);
    dataObject.Properties = properties;

    return objectService.Update(new DataPackage(dataObject), operationOptions);
}
```

2.1.4.6.2 Modifying a repeating properties (attributes) list

In some cases your client may need to make a specific change to a list of repeating properties (also called repeating attributes), such as appending values to the end of the list, inserting an item into the list, or removing an item from the list. To accomplish this you can add one or more ValueAction instances to the ArrayProperty.

A ValueAction list is synchronized with the ArrayProperty that contains it, such that an item in position p in the ValueAction list corresponds to a value stored at position p of the ArrayProperty. In this example the first item in the ValueAction list (INSERT, 0) corresonds to the first item in the ArrayProperty (snakes). The index value (0) specifies a position in the repeating property of the repository object.

Note that if you insert or delete items in a repeated properties list, the positions of items to the right of the alteration will be offset by 1 or -1. This will affect subsequent processing of the ValueAction list, which is processed from beginning to end. You must account for this effect when coding a ValueAction list, such as by ensuring that the repeating properties list is processed from right to left.

Example 2-28: Java: Modifying repeating properties

```
public DataPackage updateRepeatProperty(ObjectIdentity objectIdentity)
           throws ServiceException
{
    PropertyProfile propertyProfile = new PropertyProfile();

    // Setting the filter to ALL can cause errors if the DataObject
    // passed to the operation contains system properties, so to be safe
    // set the filter to ALL_NON_SYSTEM unless you explicitly want to update
    // a system property
    propertyProfile.setFilterMode(PropertyFilterMode.ALL_NON_SYSTEM);
    serviceContext.setProfile(propertyProfile);

    DataObject dataObject = new DataObject(objectIdentity);

    String[] moreDangers =
    { "snakes", "sharks" };
    ArrayProperty keywordProperty = new StringArrayProperty("keywords",
               moreDangers);

    ValueAction insertAction = new ValueAction(ValueActionType.INSERT, 0);
    ValueAction appendAction = new ValueAction(ValueActionType.APPEND, 1);
    keywordProperty.setValueActions(insertAction, appendAction);
    PropertySet properties = new PropertySet();
    properties.set(keywordProperty);
    dataObject.setProperties(properties);

    OperationOptions operationOptions = null;
    return objectService.update(new DataPackage(dataObject),
               operationOptions);
}
```

Example 2-29: C#: Modifying repeating properties

```
public DataPackage UpdateRepeatProperty(ObjectIdentity objectIdentity)
{
    PropertyProfile propertyProfile = new PropertyProfile();

    // Setting the filter to ALL can cause errors if the DataObject
    // passed to the operation contains system properties, so to be safe
    // set the filter to ALL_NON_SYSTEM unless you explicitly want to update
    // a system property
    propertyProfile.FilterMode = PropertyFilterMode.ALL_NON_SYSTEM;
    DemoServiceContext.SetProfile(propertyProfile);

    DataObject dataObject = new DataObject(objectIdentity);

    String[] moreDangers = { "snakes", "sharks" };
    ArrayProperty<string> keywordProperty = new StringArrayProperty
        ("keywords", moreDangers);

    ValueAction insertAction = new ValueAction(ValueActionType.INSERT, 0);
    ValueAction appendAction = new ValueAction(ValueActionType.APPEND, 1);
    keywordProperty.SetValueActions(insertAction, appendAction);
    PropertySet properties = new PropertySet();
    properties.Set(keywordProperty);
    dataObject.Properties = properties;

    OperationOptions operationOptions = null;
    return objectService.Update(new DataPackage(dataObject), operationOptions);
}
```

2.1.4.6.3 Updating object relationships

If the client adds a Relationship object to a DataObject passed to the update operation, the processing of the Relationship object depends on two factors:

- Whether the Relationship is an ObjectRelationship (which contains a DataObject) or a ReferenceRelationship (which contains only an ObjectIdentity).
- Whether the Relationship object represents an existing object in the repository.

If the Relationship object is an ObjectRelationship, the update operation will update an existing repository object represented by the ObjectRelationship, or create a new repository object if no such repository object exists. If the Relationship object is a ReferenceRelationship, the update operation will create a relationship (by modifying repository metadata) between the repository object represented by the DataObject and an existing repository object referenced by the ReferenceRelationship.

To remove a relationship, rather than add it, you can set the RelationshipIntentModifier to REMOVE (otherwise it is implicitly set to ADD).

To illustrate, the following example unlinks a document from one folder and links it into another folder:

Example 2-30: Java: Updating and re-linking a folder

```
public DataPackage updateRelinkFolder(ObjectIdentity docId,
                                      ObjectIdentity sourceFolderId,
                                      ObjectIdentity targetFolderId)
    throws ServiceException
{
    DataObject docDataObj = new DataObject(docId, "dm_document");

    // add the source folder as a parent relationship of the document
    ReferenceRelationship removeRelationship = new ReferenceRelationship();
    removeRelationship.setTargetRole(Relationship.ROLE_PARENT);
    removeRelationship.setName(Relationship.RELATIONSHIP_FOLDER);
    removeRelationship.setTarget(sourceFolderId);
    docDataObj.getRelationships().add(removeRelationship);

    // specify that the folder is to be unlinked
    removeRelationship.setIntentModifier(RelationshipIntentModifier.REMOVE);

    // add the folder into which to link document
    ReferenceRelationship addRelationship = new ReferenceRelationship();
    addRelationship.setTargetRole(Relationship.ROLE_PARENT);
    addRelationship.setName(Relationship.RELATIONSHIP_FOLDER);
    addRelationship.setTarget(targetFolderId);
    docDataObj.getRelationships().add(addRelationship);

    OperationOptions operationOptions = null;
    return objectService.update(new DataPackage(docDataObj), operationOptions);
}
```

Example 2-31: C#: Updating and re-linking a folder

```
public DataPackage UpdateRelinkFolder(ObjectIdentity docId,
                                      ObjectIdentity sourceFolderId,
                                      ObjectIdentity targetFolderId)
```

```
{
    DataObject docDataObj = new DataObject(docId, "dm_document");

    // add the source folder as a parent relationship of the document
    ReferenceRelationship removeRelationship = new ReferenceRelationship();
    removeRelationship.TargetRole = Relationship.ROLE_PARENT;
    removeRelationship.Name = Relationship.RELATIONSHIP_FOLDER;
    removeRelationship.Target = sourceFolderId;
    docDataObj.Relationships.Add(removeRelationship);

    // specify that the folder is to be unlinked
    removeRelationship.IntentModifier = RelationshipIntentModifier.REMOVE;

    // add the folder into which to link document
    ReferenceRelationship addRelationship = new ReferenceRelationship();
    addRelationship.TargetRole = Relationship.ROLE_PARENT;
    addRelationship.Name = Relationship.RELATIONSHIP_FOLDER;
    addRelationship.Target = targetFolderId;
    docDataObj.Relationships.Add(addRelationship);

    OperationOptions operationOptions = null;
    return objectService.Update(new DataPackage(docDataObj), operationOptions);
}
```

2.1.5 delete operation

2.1.5.1 Description

The Object service delete operation deletes a set of objects from the repository. By default, for each object that it deletes, it deletes all versions. The specific behaviors of the delete operation are controlled by a DeleteProfile, which should be passed to the operation as part of OperationOptions.

2.1.5.2 Java syntax

```
void delete(ObjectIdentitySet objectsToDelete,
            operationOptions OperationOptions)
        throws CoreServiceException, ServiceException
```

2.1.5.3 C# syntax

```
void Delete(ObjectIdentitySet objectsToDelete,
            operationOptions OperationOptions)
```

2.1.5.4 Parameters

Parameter	Data type	Description
objectsToDelete	ObjectIdentitySet	A collection of ObjectIdentity instances that uniquely identify repository objects to be deleted.
operationOptions	OperationOptions	An object containing profiles and properties that specify operation behaviors. If this object is null, default operation behaviors will take effect.

2.1.5.5 DeleteProfile

The DeleteProfile, normally passed within OperationOptions, controls specific behaviors of the delete operation. The following table describes the profile settings:

Field	Description
isDeepDeleteFolders	If true, deletes all folders under a folder specified in objectsToDelete. This setting does not specify whether non-folder objects that are linked into other folders are deleted from the repository. Default value is false.
isDeepDeleteChildrenInFolders	If true, for each folder specified in objectsToDelete, removes all objects descended from the folder from the repository. However, this setting does not specify whether child objects of virtual documents that reside in other folders are removed from the repository. Default value is false.
isDeepDeleteVdmInFolders	If true, for each folder specified in objectsToDelete, removes all virtual document children descended from virtual documents residing in the folder tree, even if the child objects of the virtual document reside in folders outside the folder tree descended from the specified folder. Default value is false.
versionStrategy	Determines the behavior or the delete operation as pertains to versions, using a value of the DeleteVersionStrategy enum. Possible values are SELECTED_VERSIONS, UNUSED_VERSIONS, ALL_VERSIONS. Default value is ALL_VERSIONS.

Field	Description
isPopulateWithReferences	Specifies whether reference objects should be dereferenced during population, that is, when files/objects are added to the operation. True will indicate that the reference objects themselves will be added to the operation. False will indicate that reference objects will be dereferenced and the remote object will be added to the operation. The default is false.

2.1.5.6 Example

The following example deletes all versions of a document from the repository, as well as all descended folders and child objects residing within those folders. However, it does not delete children of virtual documents that reside in folders outside the tree descended from the specified folder.

Example 2-32: Java: Deep delete

```
public void objServiceDelete(String path) throws ServiceException
{
    ObjectPath docPath = new ObjectPath(path);
    ObjectIdentity<ObjectPath> objIdentity = new ObjectIdentity<ObjectPath>();
    objIdentity.setValue(docPath);
    objIdentity.setRepositoryName(defaultRepositoryName);
    ObjectIdentitySet objectIdSet = new ObjectIdentitySet(objIdentity);

    DeleteProfile deleteProfile = new DeleteProfile();
    deleteProfile.setDeepDeleteFolders(true);
    deleteProfile.setDeepDeleteChildrenInFolders(true);
    OperationOptions operationOptions = new OperationOptions();
    operationOptions.setDeleteProfile(deleteProfile);

    objectService.delete(objectIdSet, operationOptions);
}
```

Example 2-33: C#: Deep delete

```
public void ObjServiceDelete(String path)
{
    ObjectPath docPath = new ObjectPath(path);
    ObjectIdentity objIdentity = new ObjectIdentity();
    objIdentity.Value = docPath;
    objIdentity.RepositoryName = DefaultRepository;
    ObjectIdentitySet objectIdSet = new ObjectIdentitySet(objIdentity);

    DeleteProfile deleteProfile = new DeleteProfile();
    deleteProfile.IsDeepDeleteFolders = true;
    deleteProfile.IsDeepDeleteChildrenInFolders = true;
    OperationOptions operationOptions = new OperationOptions();
    operationOptions.DeleteProfile = deleteProfile;

    objectService.Delete(objectIdSet, operationOptions);
}
```

2.1.6 copy operation

2.1.6.1 Description

The copy operation copies a set of repository objects from one location to another, either within a single repository, or from one repository to another. During the copy operation, the service can optionally make modifications to the objects being copied.

Note: For the service to copy an object from one repository to another, the ServiceContext must be set up to provide the service with access to both repositories. This can be done by setting up a separate RepositoryIdentity for each repository, or by use of a BasicIdentity, which provides default user credentials for multiple repositories.

2.1.6.2 Java syntax

```
DataPackage copy(ObjectIdentitySet fromObjects,
                 ObjectLocation targetLocation,
                 DataPackage modifyObjects,
                 OperationOptions operationOptions)
        throws CoreServiceException, ServiceException
```

2.1.6.3 C# syntax

```
DataPackage Copy(ObjectIdentitySet fromObjects,
                 ObjectLocation targetLocation,
                 DataPackage modifyObjects,
                 OperationOptions operationOptions)
```

2.1.6.4 Parameters

Parameter	Data type	Description
fromObjects	ObjectIdentitySet	A collection of ObjectIdentity instances that identify the repository objects to be copied.
targetLocation	ObjectLocation	Contains an ObjectIdentity that identifies the location (a cabinet or folder) into which the repository objects are to be copied.

Parameter	Data type	Description
modifyObjects	DataPackage	Optionally contains a set of DataObject instances that contain modifications (such as changes to property values, content, or relationships) to all or some of the repository objects being copied. The ObjectIdentity of each DataObject must uniquely identify one of the copied objects. The modifications supplied in the DataObject are applied during the copy operation.
operationOptions	OperationOptions	An object containing profiles and properties that specify operation behaviors.

2.1.6.5 CopyProfile

The CopyProfile, normally passed within OperationOptions, controls specific behaviors of the copy operation. The following table describes the profile settings:

Field	Description
isDeepCopyFolders	If true, copies all folders and their contents descended from any folder specified in fromObjects. Default value is false.
isNonCurrentObjectsAllowed	If true, allows copy of non-CURRENT objects; otherwise throws an exception on attempt to copy non-CURRENT object. Default value is false.

2.1.6.6 Response

Returns a DataPackage containing one DataObject for each repository object created by the copy operation. By default, each DataObject contains only the ObjectIdentity of the created object and no other data. The client can modify this behavior by using Profile objects if it requires more data about the copied objects.

2.1.6.7 Examples

2.1.6.7.1 Copy across repositories

The following example copies a single object to a secondary repository. Note that the service context must contain Identity instances that provide the service with access credentials to both repositories.

Example 2-34: Java: Copy across repositories

```
public void objServiceCopyAcrossRepositories(String sourceObjectPathString,
                                             String targetLocPathString)
    throws ServiceException
{
    // identify the object to copy
    ObjectPath objPath = new ObjectPath(sourceObjectPathString);
    ObjectIdentity<ObjectPath> docToCopy = new ObjectIdentity<ObjectPath>();
    docToCopy.setValue(objPath);
    docToCopy.setRepositoryName(defaultRepositoryName);

    // identify the folder to copy to
    ObjectPath folderPath = new ObjectPath();
    folderPath.setPath(targetLocPathString);
    ObjectIdentity<ObjectPath> toFolderIdentity = new ObjectIdentity<ObjectPath>();
    toFolderIdentity.setValue(folderPath);
    toFolderIdentity.setRepositoryName(secondaryRepositoryName);
    ObjectLocation toLocation = new ObjectLocation();
    toLocation.setObjectIdentity(toFolderIdentity);

    OperationOptions operationOptions = null;
    objectService.copy(new ObjectIdentitySet(docToCopy),
        toLocation, null, operationOptions);
}
```

Example 2-35: C#: Copy across repositories

```
public void ObjServiceCopyAcrossRepositories(String sourceObjectPathString,
                                             String targetLocPathString)
{
    // identify the object to copy
    ObjectPath objPath = new ObjectPath(sourceObjectPathString);
    ObjectIdentity docToCopy = new ObjectIdentity();
    docToCopy.Value = objPath;
    docToCopy.RepositoryName = DefaultRepository;

    // identify the folder to copy to
    ObjectPath folderPath = new ObjectPath();
    folderPath.Path = targetLocPathString;
    ObjectIdentity toFolderIdentity = new ObjectIdentity();
    toFolderIdentity.Value = folderPath;
    toFolderIdentity.RepositoryName = SecondaryRepository;
    ObjectLocation toLocation = new ObjectLocation();
    toLocation.Identity = toFolderIdentity;

    OperationOptions operationOptions = null;
    objectService.Copy(new ObjectIdentitySet(docToCopy),
        toLocation, null, operationOptions);
}
```

2.1.6.7.2 Copy with modifications

The following sample copies a document to a new location, and at the same time changes its object_name property:

Example 2-36: Java: Copy with modifications

```
public void objServiceCopyWithMods(String sourceObjectPathString,
                                   String targetLocPathString)
    throws ServiceException
{
    // identify the object to copy
    ObjectPath objPath = new ObjectPath(sourceObjectPathString);
    ObjectIdentity<ObjectPath> docToCopy = new ObjectIdentity<ObjectPath>();
    docToCopy.setValue(objPath);
    docToCopy.setRepositoryName(defaultRepositoryName);

    // identify the folder to copy to
    ObjectPath folderPath = new ObjectPath();
    folderPath.setPath(targetLocPathString);
    ObjectIdentity<ObjectPath> toFolderIdentity = new ObjectIdentity<ObjectPath>();
    toFolderIdentity.setValue(folderPath);
    toFolderIdentity.setRepositoryName(defaultRepositoryName);
    ObjectLocation toLocation = new ObjectLocation();
    toLocation.setObjectIdentity(toFolderIdentity);

    // specify changes to make when copying
    DataObject modDataObject = new DataObject(docToCopy);
    modDataObject.setType("dm_document");
    PropertySet modProperties = modDataObject.getProperties();
    modProperties.set("object_name", "copiedDocument-" +
        System.currentTimeMillis());
    DataPackage dataPackage = new DataPackage(modDataObject);

    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.getIdentities().add(docToCopy);
    OperationOptions operationOptions = null;
    objectService.copy(objIdSet, toLocation, dataPackage, operationOptions);
 }
```

Example 2-37: C#: Copy with modifications

```
public void ObjServiceCopyWithMods(String sourceObjectPathString,
                                   String targetLocPathString)
{
    // identify the object to copy
    ObjectPath objPath = new ObjectPath(sourceObjectPathString);
    ObjectIdentity docToCopy = new ObjectIdentity();
    docToCopy.Value = objPath;
    docToCopy.RepositoryName = DefaultRepository;

    // identify the folder to copy to
    ObjectPath folderPath = new ObjectPath();
    folderPath.Path = targetLocPathString;
    ObjectIdentity toFolderIdentity = new ObjectIdentity();
    toFolderIdentity.Value = folderPath;
    toFolderIdentity.RepositoryName = DefaultRepository;
    ObjectLocation toLocation = new ObjectLocation();
    toLocation.Identity = toFolderIdentity;

    // specify changes to make when copying
    DataObject modDataObject = new DataObject(docToCopy);
    modDataObject.Type = "dm_document";
    PropertySet modProperties = modDataObject.Properties;
    modProperties.Set("object_name", "copiedDocument-" + System.DateTime.Now.Ticks);
```

```
    DataPackage dataPackage = new DataPackage(modDataObject);

    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.Identities.Add(docToCopy);
    OperationOptions operationOptions = null;
    objectService.Copy(objIdSet, toLocation, dataPackage, operationOptions);
}
```

2.1.7 move operation

2.1.7.1 Description

The move operation moves a set of repository objects from one location to another within a repository, and provides the optional capability of updating the repository objects as they are moved. The move operation will only move the CURRENT version of an object, unless non-CURRENT objects are specifically permitted by a MoveProfile. By default, if passed an ObjectIdentity that identifies a non-CURRENT object, the operation will throw an exception.

Note: Moving an object across repositories is not supported.

2.1.7.2 Java syntax

```
DataPackage move(ObjectIdentitySet fromObjects,
                 ObjectLocation sourceLocation,
        throws CoreServiceException, ServiceException
```

2.1.7.3 C# syntax

```
DataPackage Move(ObjectIdentitySet fromObjects,
                 ObjectLocation sourceLocation,
                 ObjectLocation targetLocation,
                 DataPackage modifyObjects
                 OperationOptions operationOptions)
```

2.1.7.4 Parameters

Parameter	Data type	Description
fromObjects	ObjectIdentitySet	A collection of ObjectIdentity instances that identify the repository objects to be moved.
sourceLocation	ObjectLocation	Contains an ObjectIdentity that identifies the location (a cabinet or folder) from which the repository objects are to be moved.

Parameter	Data type	Description
targetLocation	ObjectLocation	Contains an ObjectIdentity that identifies the location (a cabinet or folder) into which the repository objects are to be moved.
modifyObjects	DataPackage	Optionally contains a set of DataObject instances that contain modifications (such as changes to property values, content, or relationships) to all or some of the repository objects being moved. The ObjectIdentity of each DataObject must uniquely identify one of the moved objects. The modifications supplied in the DataObject are applied during the move operation.
operationOptions	OperationOptions	Contains profiles and properties that specify operation behaviors. operationOptions can contain a MoveProfile, which provides options specific to this operation.

2.1.7.5 MoveProfile

The MoveProfile, normally passed within OperationOptions, controls specific behaviors of the move operation. The following table describes the profile settings:

Field	Description
isNonCurrentObjectsAllowed	If true, allows move of non-CURRENT objects; otherwise throws an exception on attempt to move non-CURRENT object. Default value is false.

2.1.7.6 Response

Returns a DataPackage containing one DataObject for each repository object created by the move operation. By default, each DataObject contains only the ObjectIdentity of the created object and no other data. The client can modify this behavior by using Profile objects if it requires more data about the moved objects.

2.1.7.7 Example

Example 2-38: Java: Moving an object

```
public void objServiceMove(String sourceObjectPathString,
                           String targetLocPathString,
                           String sourceLocPathString)
    throws ServiceException
{
    // identify the object to move
    ObjectPath objPath = new ObjectPath(sourceObjectPathString);
    ObjectIdentity<ObjectPath> docToCopy =
        new ObjectIdentity<ObjectPath>();
    docToCopy.setValue(objPath);
    docToCopy.setRepositoryName(defaultRepositoryName);

    // identify the folder to move from
    ObjectPath fromFolderPath = new ObjectPath();
    fromFolderPath.setPath(sourceLocPathString);
    ObjectIdentity<ObjectPath> fromFolderIdentity =
        new ObjectIdentity<ObjectPath>();
    fromFolderIdentity.setValue(fromFolderPath);
    fromFolderIdentity.setRepositoryName(defaultRepositoryName);
    ObjectLocation fromLocation = new ObjectLocation();
    fromLocation.setObjectIdentity(fromFolderIdentity);

    // identify the folder to move to
    ObjectPath folderPath = new ObjectPath();
    folderPath.setPath(targetLocPathString);
    ObjectIdentity<ObjectPath> toFolderIdentity =
        new ObjectIdentity<ObjectPath>();
    toFolderIdentity.setValue(folderPath);
    toFolderIdentity.setRepositoryName(defaultRepositoryName);
    ObjectLocation toLocation = new ObjectLocation();
    toLocation.setObjectIdentity(toFolderIdentity);

    OperationOptions operationOptions = null;
    objectService.move(new ObjectIdentitySet(docToCopy),
                       fromLocation,
                       toLocation,
                       new DataPackage(),
                       operationOptions);
}
```

Example 2-39: C#: Moving an object

```
public void ObjServiceMove(String sourceObjectPathString,
                           String targetLocPathString,
                           String sourceLocPathString)
{
    // identify the object to move
    ObjectPath objPath = new ObjectPath(sourceObjectPathString);
    ObjectIdentity docToCopy = new ObjectIdentity();
    docToCopy.Value = objPath;
    docToCopy.RepositoryName = DefaultRepository;
```

```
    // identify the folder to move from
    ObjectPath fromFolderPath = new ObjectPath();
    fromFolderPath.Path = sourceLocPathString;
    ObjectIdentity fromFolderIdentity = new ObjectIdentity();
    fromFolderIdentity.Value = fromFolderPath;
    fromFolderIdentity.RepositoryName = DefaultRepository;
    ObjectLocation fromLocation = new ObjectLocation();
    fromLocation.Identity = fromFolderIdentity;

    // identify the folder to move to
    ObjectPath folderPath = new ObjectPath(targetLocPathString);
    ObjectIdentity toFolderIdentity = new ObjectIdentity();
    toFolderIdentity.Value = folderPath;
    toFolderIdentity.RepositoryName = DefaultRepository;
    ObjectLocation toLocation = new ObjectLocation();
    toLocation.Identity = toFolderIdentity;

    OperationOptions operationOptions = null;
    objectService.Move(new ObjectIdentitySet(docToCopy),
                       fromLocation,
                       toLocation,
                       new DataPackage(),
                       operationOptions);
}
```

2.1.8 validate operation

The validate operation validates a set of DataObject instances against repository data dictionary (schema) rules, testing whether the DataObject instances represent valid repository objects, and whether the DataObject properties represent valid repository properties.

2.1.8.1 Java syntax

```
ValidationInfoSet validate(DataPackage dataPackage)
        throws CoreServiceException, ServiceException
```

2.1.8.2 C# syntax

```
ValidationInfoSet Validate(DataPackage dataPackage)
```

2.1.8.3 Parameters

Parameter	Data type	Description
dataPackage	DataPackage	A collection of DataObject instances to be validated by the operation.

2.1.8.4 Response

Returns a ValidationInfoSet, which contains a list of ValidationInfo objects. Each ValidationInfo contains a DataObject and a list of any ValidationIssue instances that were raised by the operation. A ValidationIssue can be of enum type ERROR, UNDEFINED, or WARNING. The following image shows the ValidationInfoSet model:

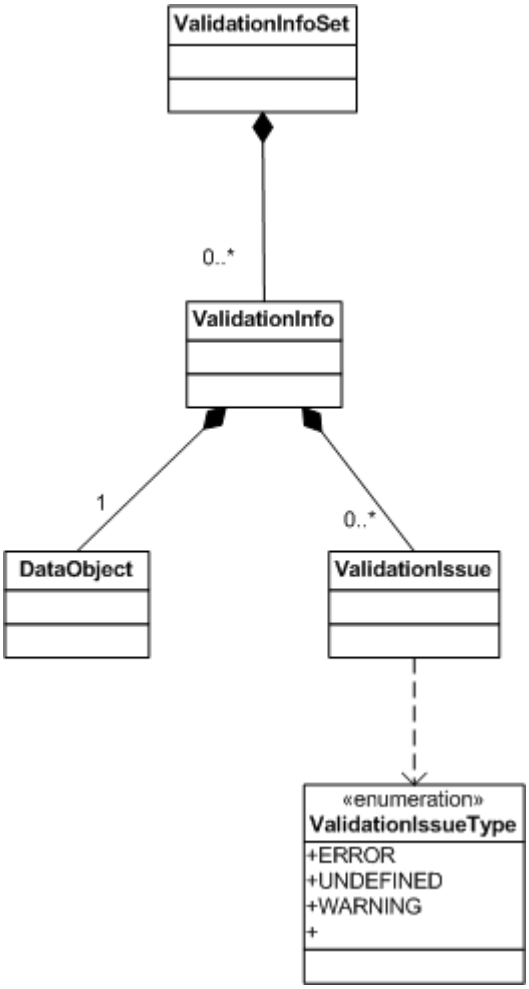

Figure 2-1: ValidationInfoSet

2.1.9 getObjectContentUrls operation

2.1.9.1 Description

The getObjectContentUrls operation gets a set of UrlContent objects based on a set of ObjectIdentity instances. The UrlContent contains an Accelerated Content Services URL that points to content available via Accelerated Content Services. This URL is temporary.

2.1.9.2 Java syntax

```
List<ObjectContentSet> getObjectContentUrls(ObjectIdentitySet forObjects)
        throws CoreServiceException, ServiceException
```

2.1.9.3 C# syntax

```
List<ObjectContentSet> GetObjectContentUrls(ObjectIdentitySet forObjects)
```

2.1.9.4 Parameters

Parameter	Data type	Description
forObjects	ObjectIdentitySet	A collection of ObjectIdentity instances for which to obtain UrlContent objects.

2.1.9.5 Response

Returns a list of ObjectContentSet objects, each of which contains a list of UrlContent objects. Note that more than one UrlContent can be returned for each ObjectIdentity. Additional Content instances represent renditions of the repository object.

2.1.9.6 Example

Example 2-40: Java: Moving an object

```
public void objServiceMove(String sourceObjectPathString,
                           String targetLocPathString,
                           String sourceLocPathString)
    throws ServiceException
{
    // identify the object to move
    ObjectPath objPath = new ObjectPath(sourceObjectPathString);
    ObjectIdentity<ObjectPath> docToCopy = new ObjectIdentity<ObjectPath>();
    docToCopy.setValue(objPath);
    docToCopy.setRepositoryName(defaultRepositoryName);

    // identify the folder to move from
    ObjectPath fromFolderPath = new ObjectPath();
    fromFolderPath.setPath(sourceLocPathString);
    ObjectIdentity<ObjectPath> fromFolderIdentity = new ObjectIdentity<ObjectPath>();
    fromFolderIdentity.setValue(fromFolderPath);
    fromFolderIdentity.setRepositoryName(defaultRepositoryName);
    ObjectLocation fromLocation = new ObjectLocation();
    fromLocation.setObjectIdentity(fromFolderIdentity);
```

```
    // identify the folder to move to
    ObjectPath folderPath = new ObjectPath();
    folderPath.setPath(targetLocPathString);
    ObjectIdentity<ObjectPath> toFolderIdentity = new ObjectIdentity<ObjectPath>();
    toFolderIdentity.setValue(folderPath);
    toFolderIdentity.setRepositoryName(defaultRepositoryName);
    ObjectLocation toLocation = new ObjectLocation();
    toLocation.setObjectIdentity(toFolderIdentity);

    OperationOptions operationOptions = null;
    objectService.move(new ObjectIdentitySet(docToCopy),
                       fromLocation,
                       toLocation,
                       new DataPackage(),
                       operationOptions);
}
```

2.1.10 registerEvent operation

The registerEvent operation registers the current user to receive notifications when the specified event occurs on specified objects. After you register an event, an entry is created in the dmi_registry table. For example, if you have registered a save event to the object, then when you save the object, an audit record is registered in the dm_audittrail table for that event. A corresponding mail notification is also registered in the dmi_queue_item table. Therefore, whenever you save the object, the user is notified.

2.1.10.1 Java syntax

```
public Boolean registerEvent(ObjectIdentitySet forObjects, String event,
String message, int priority, boolean sendMail) throws CoreServiceException
```

2.1.10.2 C# syntax

```
bool RegisterEvent(ObjectIdentitySet forObjects, string eventName,
string message, int priority, bool sendMail);
```

2.1.10.3 Parameters

Parameter	Data type	Description
forObjects	ObjectIdentitySet	A collection of ObjectIdentity instances for which events to be registered.
event	String	Defines the event you want to receive notification.
message	String	Defines a message sent to the user receiving the event notification.

Parameter	Data type	Description
priority	int	Defines a priority level for the event. Use an integer number for this argument.
sendMail	boolean	Directs the server to send electronic mail to the user whenever an event is registered.

2.1.10.4 Response

Returns a boolean value containing true, if the event is registered successfully. If the event is not registered, it returns as false.

2.1.10.5 Examples

Example 2-41: Java: registerEvent

```
public void objServiceRegisterEvent
(String path,String event,String message,int priority,Boolean sendMail)
throws ServiceException
{
        ObjectPath docPath = new ObjectPath(path);
        ObjectIdentity<ObjectPath> objIdentity = new
        ObjectIdentity<ObjectPath>();
        objIdentity.setValue(docPath);
        objIdentity.setRepositoryName(defaultRepositoryName);
        ObjectIdentitySet objectIdSet = new ObjectIdentitySet(objIdentity);
        objectService.registerEvent(objectIdSet,null,message,priority,sendMail);
}
```

Example 2-42: C#: registerEvent

```
public void TestRegisterEvent()
{
  string event_name = "dm_save";
  string message = "object is saved";
  int priority = 1;
  bool sendMail = false;

  Console.WriteLine("\nTesting register event for test document.");
  try
  {
    DataPackage resultDataPackage
    =
objectServiceDemo.CreateWithContentDefaultContext(SampleContentManager.gifImage1FilePath)
;
    DataObject resultObject = resultDataPackage.DataObjects[0];
    Console.WriteLine("Created object with content " +
resultObject.Identity.GetValueAsString());

    ObjectIdentitySet forObjects = new ObjectIdentitySet();
    forObjects.AddIdentity(resultObject.Identity);

    objectServiceDemo.RegisterEvent(forObjects, event_name, message, priority, sendMail);
    Console.WriteLine("DM_SAVE event is registered");
   }
      catch (FaultException<SerializableException> ex)
   {
```

```
        Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
   }
    catch (Exception exx)
    {
     Console.WriteLine(exx.StackTrace);
    }
     finally
    {
     sampleContentManager.DeleteCreatedObjects();
    }
}
```

2.1.11 unRegisterEvent operation

The unRegisterEvent operation removes the current user from the registered event when the specified event occurs on specified object. After the user is removed from an registered event, the user does not receive any notification.

2.1.11.1 Java syntax

```
public Boolean unRegisterEvent(ObjectIdentitySet forObjects, String event) throws
CoreServiceException
```

2.1.11.2 C# syntax

```
bool UnRegisterEvent(ObjectIdentitySet forObjects, string eventName);
```

2.1.11.3 Parameters

Parameter	Data type	Description
forObjects	ObjectIdentitySet	A collection of ObjectIdentity instances for which events are to be removed from registration.
event	String	Defines the event you do not want to receive notification.

2.1.11.4 Response

Returns a boolean value containing true, if the event is removed from the registration successfully. If the event is not removed from the registration, it returns as false.

2.1.11.5 Examples

Example 2-43: Java: unRegisterEvent

```
public void objServiceUnRegisterEvent
(String path,String event)
throws ServiceException
{
        ObjectPath docPath = new ObjectPath(path);
        ObjectIdentity<ObjectPath> objIdentity = new
        ObjectIdentity<ObjectPath>();
        objIdentity.setValue(docPath);
        objIdentity.setRepositoryName(defaultRepositoryName);
        ObjectIdentitySet objectIdSet = new ObjectIdentitySet(objIdentity);
        objectService.unRegisterEvent(objectIdSet,event);
}
```

Example 2-44: C#: unRegisterEvent

```
public void TestUnRegisterEvent()
{
 string event_name = "dm_save";
 string message = "object is saved";
 int priority = 1;
 bool sendMail = false;

 Console.WriteLine("\nTesting unregister event for test document.");
 try
 {
  DataPackage resultDataPackage
  =
objectServiceDemo.CreateWithContentDefaultContext(SampleContentManager.gifImage1FilePath)
;
  DataObject resultObject = resultDataPackage.DataObjects[0];
  Console.WriteLine("Created object with content " +
resultObject.Identity.GetValueAsString());

  ObjectIdentitySet forObjects = new ObjectIdentitySet();
  forObjects.AddIdentity(resultObject.Identity);

  objectServiceDemo.RegisterEvent(forObjects, event_name, message, priority, sendMail);
  Console.WriteLine("DM_SAVE event is registered");

  objectServiceDemo.UnRegisterEvent(forObjects, event_name);

  Console.WriteLine("DM_SAVE event is unregistered");
 }
  catch (FaultException<SerializableException> ex)
  {
   Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
  }
  catch (Exception exx)
  {
   Console.WriteLine(exx.StackTrace);
  }
  finally
```

```
  {
   sampleContentManager.DeleteCreatedObjects();
  }
}
```

2.1.12 getPermission operation

The getPermission operation returns the permissions of object that current user has on it. Every object has different permissions according to users. With this operation, users get to know about their permissions on any particular object with passed object identity. In Documentum CM Server, it is stored in the integer format. However, with this operation, the permission is returned in the string format (for example, NONE, BROWSE, READ, RELATE, VERSION, WRITE, DELETE).

2.1.12.1 Java syntax

```
public String getPermission(ObjectIdentity objectIdentity) throws CoreServiceException
```

2.1.12.2 C# syntax

```
string GetPermission(ObjectIdentity objectIdentity);
```

2.1.12.3 Parameters

Parameter	Data type	Description
objectIdentity	ObjectIdentity	Contains the object ID and repository name.

2.1.12.4 Response

Returns these permissions NONE, BROWSE, READ, RELATE, VERSION, WRITE, and DELETE. If any user has READ permission on the object, then it returns NONE | BROWSE | READ.

2.1.12.5 Examples

Example 2-45: Java: getPermission

```
public String testGetPermission()
{
    System.out.println("\nTesting get permission...");
    String permission="";
    try
    {
        sampleContentManager.createDemoObjects();
        DataPackage
dataPackage=sampleContentManager.getDataPackageOfNewlyCreatedObjects();

        ObjectIdentity objectIdentity =
dataPackage.getDataObjects().get(0).getIdentity();
        System.out.println("Getting permission for
```

```
objectId :"+objectIdentity.getValueAsString());
        permission = objServiceGetPermission.getPermission(objectIdentity);
    }
    catch(Exception e)
    {
        e.printStackTrace();
    }
    return permission;
}// complete code is available in dfs-sdk DfsSamples
```

Example 2-46: C#: getPermission

```
public void TestGetPermission()
{
    Console.WriteLine("\nTesting get permission of contentless document.");
    try
    {
      DataPackage resultDataPackage
      =
objectServiceDemo.CreateWithContentDefaultContext(SampleContentManager.gifImage1FilePath)
;
      DataObject resultObject = resultDataPackage.DataObjects[0];
      Console.WriteLine("Created object with content " +
resultObject.Identity.GetValueAsString());
      string permission = objectServiceDemo.GetPermission(resultObject.Identity);
      Console.WriteLine("Permission for this object id"+
resultObject.Identity.GetValueAsString()+" is :" + permission);
     }
     catch (FaultException<SerializableException> ex)
     {
      Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
     }
      catch (Exception exx)
     {
      Console.WriteLine(exx.StackTrace);
     }
        finally
     {
      sampleContentManager.DeleteCreatedObjects();
     }
}// complete code is available in dfs-sdk DotNetClientDocSamples
```

2.1.13 getLinkedRepeatingString operation

The getLinkedRepeatingString operation gets a linked repeating string value based on the value of a source repeating string attribute.

If you have a source attribute as keywords and target attribute as authors as follows:

- keywords: “Guest”,“Admin”,“Administrator”,“User”
- authors: “User”,“Admin”,“Administrator”,“Guest”

If you invoke getLinkedRepeatingString and pass the source attribute value as “Guest”, then it returns “User”. It is because “User” is found at the third index.

2.1.13.1 Java syntax

```
String getLinkedRepeatingString(String sourceAttributeName, String sourceAttributeValue,
String targetAttributeName, ObjectIdentity objectIdentity) throws CoreServiceException
```

2.1.13.2 C# syntax

```
string GetLinkedRepeatingString(string sourceAttributeName, string sourceAttributeValue,
string targetAttributeName, ObjectIdentity objectIdentity)
```

2.1.13.3 Parameters

Parameter	Data type	Description
sourceAttributeName	String	Name of the source attribute.
sourceAttributeValue	String	Value of the source attribute.
targetAttributeName	String	Name of the target attribute name for which you need to search for the linked repeating string.
objectIdentity	ObjectIdentity	Contains the object ID and repository name.

2.1.13.4 Response

Returns a repeating string value based on the source attribute.

2.1.13.5 Examples

Example 2-47: Java: getLinkedRepeatingString

```
public String testGetLinkedRepeatingString()
{
    System.out.println("\nTesting get linked repeating string(for dm_document) ...");

    String targetAttributeRepeatingString="";
    String sourceAttributeValue="User";
    String sourceAttributeName="keywords";
    String targteAttributeName="authors";

    ObjectIdentity objectIdentity=null;

    System.out.println("Name of Source Attribute :"+sourceAttributeName);
    System.out.println("Name of Target Attribute :"+targteAttributeName);
    System.out.println("Source Attribute Value :"+sourceAttributeValue);

    try
    {
        DataPackage dataPackage=objServiceGetLinkedRepeatingString.getCreatedDocument();
        objectIdentity=dataPackage.getDataObjects().get(0).getIdentity();

        System.out.println("Getting linked repeating string for
objectId :"+objectIdentity.getValueAsString());
        targetAttributeRepeatingString =
objServiceGetLinkedRepeatingString.getLinkedRepeatingString(sourceAttributeName,
sourceAttributeValue, targteAttributeName,objectIdentity);
    }
```

```
    catch(Exception e)
    {
        e.printStackTrace();
    }
    return targetAttributeRepeatingString;
}// complete code is available in dfs-sdk DfsSamples
```

Example 2-48: C#: getLinkedRepeatingString

```
public void TestGetLinkedRepeatingString()
{
   Console.WriteLine("\nTesting get linked repeating string.");

   string targetAttributeRepeatingString = "";
   string sourceAttributeValue = "User";
   string sourceAttributeName = "keywords";
   string targetAttributeName = "authors";

   try
   {
    DataPackage resultDataPackage
    = objectServiceDemo.GetObjectWithRepeatingString();
    DataObject resultObject = resultDataPackage.DataObjects[0];
    Console.WriteLine("Created object with content " +
resultObject.Identity.GetValueAsString());
    string targetAttributeValue =
objectServiceDemo.GetLinkedRepeatingString(sourceAttributeName, sourceAttributeValue,
targetAttributeName, resultObject.Identity);
    Console.WriteLine("Linked Repeating String for this object id" +
resultObject.Identity.GetValueAsString() + " is :" + targetAttributeValue);
   }
    catch (FaultException<SerializableException> ex)
   {
    Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
   }
    catch (Exception exx)
    {
     Console.WriteLine(exx.StackTrace);
    }
     finally
    {
     sampleContentManager.DeleteCreatedObjects();
    }
}// complete code is available in dfs-sdk DotNetClientDocSamples
```

2.1.14 hasAttributes operation

The hasAttributes operation verifies if the specified list of attributes exist in the given object. Pass the object identity that contains the object ID and repository name. Along with object identity, pass PropertySet that contains the list of attributes that needs to verify if they exist. This operation returns the property set that contains the list of attributes that are available in the object.

2.1.14.1 Java syntax

```
public PropertySet hasAttributes(ObjectIdentity objectIdentity, PropertySet propSet)
throws CoreServiceException
```

2.1.14.2 C# syntax

```
public PropertySet HasAttributes(ObjectIdentity objectIdentity, PropertySet propSet)
```

2.1.14.3 Parameters

Parameter	Data type	Description
objectIdentity	ObjectIdentity	Contains the object ID and repository name.
propSet	PropertySet	Contains the list of attributes that need to be verified.

2.1.14.4 Response

Returns either the PropertySet object that contains the list of attributes existing in the object or throws an exception.

2.1.14.5 Examples

Example 2-49: Java: hasAttributes

```
public void testHasAttributes()
{
   try
   {
      System.out.println("\nTesting checkout information lookup...");
      System.out.println("Creating demo object " );
      sampleContentManager.createDemoObject();

      ObjectIdentity sampleObject = new ObjectIdentity(new Qualification(
            sampleObjectQual), getRepository());

      ObjectIdentity docObjId = new ObjectIdentity();
      docObjId.setRepositoryName(getRepository());
      docObjId.setValue(new ObjectPath(gifImageObjPath));
      System.out.println("invoking Object Service hasAttributes for sample object ");
      PropertySet ps = new PropertySet();
      ps.set("r_version_label", "");
      ps.set("r_immutable_flag", false);
      ps.set("title1", "");
      ps.set("r_object_id", "");

      PropertySet propSet = objectServiceDemo.hasAttributes(docObjId, ps);

      if(propSet == null)
      {
         System.out.println("ObjectService hasAttributes() returnes null");
      }
      Iterator<Property> propertyIterator = propSet.iterator();

      while (propertyIterator.hasNext()) {
         Property property = propertyIterator.next();
         String attributeName = property.getName();
```

```
         System.out.println("prop name " + property.getName() + "  exists : " +
property.getValueAsString());
      }
      if(propSet.get("title1").getValueAsString().equals("false") )
      System.out.println("object does not have title1 attribute");
      else
         System.out.println("object has title1 attribute");

   }
   catch(ServiceException e)
   {
      e.printStackTrace();
   }
   catch(Throwable t)
   {
      System.out.println("Exception occurred " + t.getMessage());
      t.printStackTrace();
   }
   finally
   {
      sampleContentManager.deleteTestCabinet();
   }

}// complete code is available in dfs-sdk DfsSamples
```

Example 2-50: C#: getLinkedRepeatingString

```
public void TestHasAttributes()
{
   Console.WriteLine("\nTesting has attributes");
   try
   {
    DataPackage resultDataPackage
    =
objectServiceDemo.CreateWithContentDefaultContext(SampleContentManager.gifImage1FilePath)
;
    DataObject resultObject = resultDataPackage.DataObjects[0];
    Console.WriteLine("Created object with content " +
resultObject.Identity.GetValueAsString());

    PropertySet ps = new PropertySet();
    ps.Set("r_version_label", "");
    ps.Set("r_immutable_flag", false);
    ps.Set("title1", "");
    ps.Set("r_object_id", "");

    PropertySet resultSet = objectServiceDemo.HasAttributes(resultObject.Identity, ps);

    if(resultSet==null)
    {
     Console.WriteLine("Result property set is null");
     return;
    }

    if (resultSet.Get("title1").GetValueAsString().Equals("false"))
    Console.WriteLine("Object does not have title1 attribute");

    else
    Console.WriteLine("Object has title attribute");
    }
    catch (FaultException<SerializableException> ex)
    {
     Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
    }
     catch (Exception exx)
```

```
    {
     Console.WriteLine(exx.StackTrace);
    }
     finally
    {
     sampleContentManager.DeleteCreatedObjects();
    }
}// complete code is available in dfs-sdk DotNetClientDocSamples
```

2.1.15 removeRendition operation

The removeRendition operation allows users to delete the rendition if users have the DELETE permission on the selected document. This operation can be performed on multiple objects. Pass the object identity set and rendition format.

2.1.15.1 Java syntax

```
public Boolean removeRendition(ObjectIdentitySet forObjects, String renditionFormat)
throws CoreServiceException
```

2.1.15.2 C# syntax

```
bool RemoveRendition(ObjectIdentitySet forObjects, string renditionFormat)
```

2.1.15.3 Parameters

Parameter	Data type	Description
forObjects	ObjectIdentitySet	Contains the object ID list.
renditionFormat	String	Contains the format to be removed.

2.1.15.4 Response

Returns either true if the removal of rendition format is successful or throws an exception.

2.1.15.5 Examples

Example 2-51: Java: removeRendition

```
public void testRemoveRendition()
{
   System.out.println("\nTesting Remove Rendition of an object...");
   ObjectIdentity objIdentity = null;
   try
   {
      objServiceRemoveRenditionDemo.createDocWithRenditions();
      sampleContentManager.createDemoObjects();
      //String path = SampleContentManager.gifImageObjPath;
      String docQualWithName = docQual + "'" +
objServiceRemoveRenditionDemo.getDefaultDocumentName() + "'";
```

```
      objIdentity = new ObjectIdentity(new Qualification(
            docQualWithName), getRepository());
      String deleteFormat = "pdf";
      objServiceRemoveRenditionDemo.removeRendition(objIdentity, deleteFormat);

      //ASSERT
      DataObject dataObj = objectServiceGet.getWithContent(objIdentity, "",
ContentTransferMode.MTOM);
      List<Content> contents = dataObj.getContents();
      for(Content content : contents)
      {
         if(deleteFormat.equalsIgnoreCase(content.getFormat()))
         {
            System.out.println("ERROR: Rendition with format  " + deleteFormat + " is
not deleted. Please check the logs" );
            break;
         }
      }
      System.out.println("Remove rendition done " );
   }
   catch(Exception e)
   {
      e.printStackTrace();
   }
   finally
   {
      try
      {
         sampleContentManager.deleteObject(objIdentity);
      }
      catch(Exception e)
      {
         e.printStackTrace();
      }
   }
}
}// complete code is available in dfs-sdk DfsSamples
```

Example 2-52: C#: removeRendition

```
public void TestRemoveRendition()
{
   Console.WriteLine("\nTesting Remove Rendition");
   try
   {
    objectServiceDemo.CreateDocWithRenditions();
    sampleContentManager.CreateDemoObjects();
    string docQual = "dm_document "
    + "where object_name = ";
    String docQualWithName = docQual + "'" + objectServiceDemo.GetDefaultDocumentName()
+ "'";
    ObjectIdentity objIdentity = new ObjectIdentity(new Qualification(
    docQualWithName), DefaultRepository);
    String deleteFormat = "pdf";
    objectServiceDemo.RemoveRendition(new ObjectIdentitySet(objIdentity), deleteFormat);

    //ASSERT
    DataObject dataObj = objectServiceDemo.GetWithContent(objIdentity, "",
ContentTransferMode.MTOM);
    List<Content> contents = dataObj.Contents;
    foreach (Content content in contents)
    {
     if (deleteFormat.Equals(content.Format))
    {
     Console.WriteLine("ERROR: Rendition with format  " + deleteFormat + " is not
deleted. Please check the logs");
```

```
     break;
    }
    }
     Console.WriteLine("Remove rendition done ");
    }
     catch (FaultException<SerializableException> ex)
    {
     Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
    }
     catch (Exception exx)
    {
     Console.WriteLine(exx.StackTrace);
    }
     finally
    {
     sampleContentManager.DeleteCreatedObjects();
    }
}// complete code is available in dfs-sdk DotNetClientDocSamples
```

2.1.16 markVersion operation

The markVersion operation marks the version with value, sets value in version label for the given r_object_id, and removes the same value from any other existing revision of the selected document. This operation can be performed on multiple objects.

2.1.16.1 Java syntax

```
public Boolean markVersion(ObjectIdentitySet forObjects, String version) throws
CoreServiceException
```

2.1.16.2 C# syntax

```
bool MarkVersion(ObjectIdentitySet forObjects, string version)
```

2.1.16.3 Parameters

Parameter	Data type	Description
forObjects	ObjectIdentitySet	Contains the object ID list.
version	String	Version value that the user wants to mark.

2.1.16.4 Response

Returns either true if the marking of version is successful or throws an exception.

2.1.16.5 Examples

Example 2-53: Java: markVersion

```
public void testMarkVersion()
{
   try
   {
      System.out.println("\nTesting checkout information lookup...");
      System.out.println("Creating demo object " );
      sampleContentManager.createDemoObject();
      System.out.println("Creating test_version version for sample object " );
      sampleContentManager.createVersionForSampleObject("test_version");
      System.out.println("Creating test_version1 version for sample object " );
      sampleContentManager.createVersionForSampleObject("test_version1");
      System.out.println("Creating test_version2 version for sample object " );
      sampleContentManager.createVersionForSampleObject("test_version2");
      System.out.println("Creating APPROVED version for sample object " );
      sampleContentManager.createVersionForSampleObject("APPROVED");
      System.out.println("Getting non current version of the sample object " );
      ObjectIdentity nonCurrObjIdentity = new ObjectIdentity(new Qualification(
            nonCurrentQual), getRepository());

      ObjectIdentity currObjIdentity = new ObjectIdentity(new Qualification(
            nonCurrentQual1), getRepository());

      System.out.println("invoking markVersion APPROVED for sample object ");
      objectServiceMarkVersionDemo.markVersion(nonCurrObjIdentity, "APPROVED");
      System.out.println("invoking markVersion CURRENT for sample object ");
      boolean res = objectServiceMarkVersionDemo.markVersion(nonCurrObjIdentity,
"CURRENT");
      System.out
            .println("Sucessfully marked version CURRENT  for object " + nonCurrentQual);


      ObjServiceGet objectServiceGet = new ObjServiceGet(getRepository(),
            getUserName(), getPassword(), isRemoteMode());
      System.out.println("Getting the object with markVersion set ");
      DataObject nonCurrDataObject =
objectServiceGet.getObjectFilterProperties(nonCurrObjIdentity);
      DataObject currDataObject =
objectServiceGet.getObjectFilterProperties(currObjIdentity);

      String nonCurrVer =
nonCurrDataObject.getProperties().get("r_version_label").getValueAsString();
      String currVer =
currDataObject.getProperties().get("r_version_label").getValueAsString();

      System.out.println("object version with markVersion set on non current object " +
nonCurrVer);
      System.out.println("object with current version " + currVer);
      //After the markVersion call the versions 'CURRENT' and 'APPROVED' should be
removed from the current object and should be
      //set on the noncurrent version of the object
      if(nonCurrVer.contains("CURRENT") && nonCurrVer.contains("APPROVED") &&
            ! currVer.contains("CURRENT") && ! currVer.contains("APPROVED"))
      {
         System.out.println("************** Object service markVersion test completed
successfully ***********");
      }

      if (nonCurrDataObject == null)
      {
         throw new Exception("dataObject is null");
```

```
      }
      ObjectExaminer objExaminer = new ObjectExaminer(nonCurrDataObject);
      if (objExaminer.printProperties() == 0)
      {
         throw new Exception("Object Examiner properties = 0");
      }
      System.out
            .println("Successfully got the object and its properties");

   }
   catch(ServiceException e)
   {
      e.printStackTrace();
   }
   catch(Throwable t)
   {
      System.out.println("Exception occurred " + t.getMessage());
      t.printStackTrace();
   }
   finally
   {
      sampleContentManager.deleteTestCabinet();
   }
}// complete code is available in dfs-sdk DfsSamples
```

Example 2-54: C#: markVersion

```
public void TestMarkVersion()
{
  Console.WriteLine("\nTesting mark version.");
  try
  {
   String nonCurrentQual = "dm_document (ALL) "
   + "where object_name = 'DFS_sample_image' "
   + "and ANY r_version_label = 'test_version'";

   String nonCurrentQual1 = "dm_document (ALL) "
   + "where object_name = 'DFS_sample_image' "
   + "and ANY r_version_label = 'test_version'";

   Console.WriteLine("Creating demo objects ");
   sampleContentManager.CreateDemoObject();

   versionSampleObect();

   ObjectIdentity nonCurrObjIdentity = new ObjectIdentity(new Qualification(
   nonCurrentQual),DefaultRepository);

   ObjectIdentity currObjIdentity = new ObjectIdentity(new Qualification(
   nonCurrentQual1), DefaultRepository);

   Console.WriteLine("Invoking MarkVersion APPROVED for sample object ");

   objectServiceDemo.MarkVersion(new ObjectIdentitySet(nonCurrObjIdentity), "APPROVED");
   Console.WriteLine("Invoking MarkVersion CURRENT for sample object ");

   bool res = objectServiceDemo.MarkVersion(new ObjectIdentitySet(nonCurrObjIdentity),
"CURRENT");

   if (res)
   Console.WriteLine("Sucessfully Marked Version CURRENT ");
   else
   Console.WriteLine("Mark version failed ");

   }
   catch (FaultException<SerializableException> ex)
```

```
   {
    Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
   }
    catch (Exception exx)
   {
    Console.WriteLine(exx.StackTrace);
   }
    finally
   {
    sampleContentManager.DeleteCreatedObjects();
   }
}// complete code is available in dfs-sdk DotNetClientDocSamples
```

2.1.17 updateNonCurrentVersion operation

The updateNonCurrentVersion operation updates the attribute value for CURRENT and NON CURRENT Version document. This operation can be performed on multiple objects. Pass the object identity set and property set.

2.1.17.1 Java syntax

```
public Boolean updateNonCurrentVersion(ObjectIdentitySet forObjects, PropertySet
propSet) throws CoreServiceException
```

2.1.17.2 C# syntax

```
bool UpdateNonCurrentVersion(ObjectIdentitySet forObjects, PropertySet propSet)
```

2.1.17.3 Parameters

Parameter	Data type	Description
forObjects	ObjectIdentitySet	Contains the object ID list.
propSet	PropertySet	Contains the list of attributes.

2.1.17.4 Response

Returns either true if updating of version is successful or throws an exception.

2.1.17.5 Examples

Example 2-55: Java: updateNonCurrentVersion

```
public void testUpdateNonCurrentVersion()
{
   try
   {
      System.out.println("\nTesting checkout information lookup...");
      System.out.println("Creating demo object " );
      sampleContentManager.createDemoObject();

      System.out.println("Creating test_version version for sample object " );
      sampleContentManager.createVersionForSampleObject("test_version");

      System.out.println("Creating test_version1 version for sample object " );
      sampleContentManager.createVersionForSampleObject("test_version1");

      System.out.println("Creating test_version2 version for sample object " );
      sampleContentManager.createVersionForSampleObject("test_version2");

      System.out.println("Creating APPROVED version for sample object " );
      sampleContentManager.createVersionForSampleObject("APPROVED");

      System.out.println("Getting non current version of the sample object " );
      ObjectIdentity nonCurrObjIdentity = new ObjectIdentity(new Qualification(
            nonCurrentQual), getRepository());

      System.out.println("invoking updateNonCurrentVersion for sample object ");
      PropertySet ps = new PropertySet();
      ps.set("r_version_label", "test_version0.0");
      ps.set("r_immutable_flag", false);
      ps.set("title", "test_version0.0_title");

      objectServiceDemo.updateNonCurrentVersion(nonCurrObjIdentity, ps);
           ObjectIdentity nonCurrObjIdentityWithNewVer = new ObjectIdentity(new
Qualification(
                   nonCurrentQual1), getRepository());

      ObjServiceGet objectServiceGet = new ObjServiceGet(getRepository(),
            getUserName(), getPassword(), isRemoteMode());
      System.out.println("Getting the object after updateNonCurrentVersion ");
      DataObject nonCurrDataObject =
objectServiceGet.getObjectFilterProperties(nonCurrObjIdentityWithNewVer);
      //DataObject currDataObject =
objectServiceGet.getObjectFilterProperties(currObjIdentity);

      String nonCurrVer =
nonCurrDataObject.getProperties().get("r_version_label").getValueAsString();
      if(nonCurrVer.equalsIgnoreCase("test_version0.0"))
      {
         System.out.println("updateNonCurrentVersion update version attribute successful
");
      }
      else{
         System.out.println("updateNonCurrentVersion update version attribute failed ");
      }
      String nonCurrVerTitle =
nonCurrDataObject.getProperties().get("title").getValueAsString();
      if(nonCurrVerTitle.equalsIgnoreCase("test_version0.0_title"))
      {
         System.out.println("updateNonCurrentVersion update title attribute successful
");
      }
      else{
         System.out.println("updateNonCurrentVersion update title attribute failed ");
      }

      String nonCurrVerImmutFlag =
```

```
nonCurrDataObject.getProperties().get("r_immutable_flag").getValueAsString();
      if(nonCurrVerImmutFlag.equalsIgnoreCase("false"))
      {
         System.out.println("updateNonCurrentVersion update r_immutable_flag attribute
successful ");
      }
      else{
         System.out.println("updateNonCurrentVersion update r_immutable_flag attribute
failed ");
      }

      if(nonCurrVerTitle.equalsIgnoreCase("test_version0.0_title") &&
            nonCurrVerImmutFlag.equalsIgnoreCase("false") &&
      nonCurrVer.equalsIgnoreCase("test_version0.0"))
      {
         System.out.println("************** Object service updateNonCurrentVersion test
completed successfully ***********");
      }

      if (nonCurrDataObject == null)
      {
         throw new Exception("dataObject is null");
      }
      ObjectExaminer objExaminer = new ObjectExaminer(nonCurrDataObject);
      if (objExaminer.printProperties() == 0)
      {
         throw new Exception("Object Examiner properties = 0");
      }
      System.out
            .println("Successfully got the object and its properties");

   }
   catch(ServiceException e)
   {
      e.printStackTrace();
   }
   catch(Throwable t)
   {
      System.out.println("Exception occurred " + t.getMessage());
      t.printStackTrace();
   }
   finally
   {
      sampleContentManager.deleteTestCabinet();
   }
}
}// complete code is available in dfs-sdk DfsSamples
```

Example 2-56: C#: updateNonCurrentVersion

```
public void TestUpdateNonCurrentVersion()
{
  Console.WriteLine("\nTesting Update Non Current Version ");

  String nonCurrentQual = "dm_document (ALL) "
  + "where object_name = 'DFS_sample_image' "
  + "and ANY r_version_label = 'test_version'";

  String nonCurrentQual1 = "dm_document (ALL) "
  + "where object_name = 'DFS_sample_image' "
  + "and ANY r_version_label = 'test_version0.0'";

  try
  {
   Console.WriteLine("Creating demo object ");
   sampleContentManager.CreateDemoObject();
```

```
   Console.WriteLine("Getting non current version of the sample object ");
   ObjectIdentity nonCurrObjIdentity = new ObjectIdentity(new Qualification(
   nonCurrentQual), DefaultRepository);

   versionSampleObect();
   PropertySet ps = new PropertySet();
   ps.Set("r_version_label", "test_version0.0");
   ps.Set("r_immutable_flag", false);
   ps.Set("title", "test_version0.0_title");

   bool res = objectServiceDemo.UpdateNonCurrentVersion(new
ObjectIdentitySet(nonCurrObjIdentity), ps);
   ObjectIdentity nonCurrObjIdentityWithNewVer = new ObjectIdentity(new Qualification(
   nonCurrentQual1), DefaultRepository);

   Console.WriteLine("Getting the object after updateNonCurrentVersion ");
   DataObject nonCurrDataObject =
objectServiceDemo.GetObjectFilterProperties(nonCurrObjIdentityWithNewVer);

   String nonCurrVerTitle = nonCurrDataObject.Properties.Get("title").GetValueAsString();

   if (res && nonCurrVerTitle.Equals("test_version0.0_title"))
   {
    Console.WriteLine("UpdateNonCurrentVersion update title attribute successful ");
   }
   else
   {
    Console.WriteLine("UpdateNonCurrentVersion update title attribute failed ");
   }
    Console.WriteLine("************** Object service updateNonCurrentVersion test
completed successfully ***********");
   }
    catch (FaultException<SerializableException> ex)
   {
    Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
   }
    catch (Exception exx)
   {
    Console.WriteLine(exx.StackTrace);
   }
    finally
   {
    sampleContentManager.DeleteCreatedObjects();
   }
}// complete code is available in dfs-sdk DotNetClientDocSamples
```

2.1.18 saveAsNew operation

The saveAsNew operation saves the object as new object. Pass the object identity that contains object ID and repository name and one boolean parameter (true or false). The boolean parameter true copies the content. This operation is similar to the copy method but does not provide options to link with folder. This operation creates a copy of the document with or without content. However, creating the relationship must be done separately.

2.1.18.1 Java syntax

```
public String saveAsNew(ObjectIdentity objectIdentity,Boolean withContent) throws
CoreServiceException
```

2.1.18.2 C# syntax

```
public string SaveAsNew(ObjectIdentity objectIdentity, bool withContent)
```

2.1.18.3 Parameters

Parameter	Data type	Description
forObjects	ObjectIdentitySet	Contains the object ID list.
WithContent	Boolean	Contains true or false to copy content.

2.1.18.4 Response

Returns either the object ID of new object or throws an exception.

2.1.18.5 Examples

Example 2-57: Java: saveAsNew

```
public String testSaveAsNew()
{
    System.out.println("\nTesting save as new...");
    String newObjectId="";
    Boolean withContent = false; // this means not to copy content
    try
    {
        sampleContentManager.createDemoObjects();
        DataPackage
dataPackage=sampleContentManager.getDataPackageOfNewlyCreatedObjects();

        ObjectIdentity objectIdentity =
dataPackage.getDataObjects().get(0).getIdentity();
        System.out.println("Saving as new :"+objectIdentity.getValueAsString());
        newObjectId = objServiceSaveAsNew.saveAsNew(objectIdentity,withContent);
    }
    catch(Exception e)
    {
        e.printStackTrace();
    }
    return newObjectId;
}
}// complete code is available in dfs-sdk DfsSamples
```

Example 2-58: C#: saveAsNew

```
public void TestSaveAsNew()
{
  Console.WriteLine("\nTesting save as new for contentless document.");
  try
  {
```

```
   DataPackage resultDataPackage
   =
objectServiceDemo.CreateWithContentDefaultContext(SampleContentManager.gifImage1FilePath)
;
   Boolean withContent = false;
   DataObject resultObject = resultDataPackage.DataObjects[0];
   Console.WriteLine("Created object with content " +
resultObject.Identity.GetValueAsString());
   string permission = objectServiceDemo.SaveAsNew(resultObject.Identity,withContent);
   Console.WriteLine("Saving object " + resultObject.Identity.GetValueAsString() + " as
new object :" + permission);
  }
  catch (FaultException<SerializableException> ex)
  {
   Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
  }
   catch (Exception exx)
  {
   Console.WriteLine(exx.StackTrace);
  }
   finally
  {
   sampleContentManager.DeleteCreatedObjects();
  }
}// complete code is available in dfs-sdk DotNetClientDocSamples
```

2.1.19 Working with lightweight objects

The following sections provide some general information about lightweight objects (also referred to as lightweight SysObjects, because they must be subtypes of dm_sysobject), and describe how to work with lightweight objects using Foundation SOAP API. For more information about lightweight objects, including information about creating shareable and lightweight types using Documentum Query Language (DQL), see *(EDCCS250400-DGD)*.

2.1.19.1 Overview of lightweight SysObjects

Lightweight SysObjects are part of an object model enhancement introduced to share system managed metadata among objects which only hold application specific data. For example, policies for security, retention, and storage are stored in a shareable system object that is shared among all the lightweight objects. Because the system managed metadata is stored only once, it significantly reduces the disk storage requirements and improves the ingestion performance. However, an application needs to be aware of lightweight SysObjects in order to display and use them.

2.1.19.2 Lightweight type

A lightweight type is a type whose implementation is optimized to reduce the storage space needed in the database for instances of the type. All lightweight SysObjects types are subtypes of a shareable type. When a lightweight SysObject is created, it references a shareable supertype object. As additional lightweight SysObjects are created, they can reference the same shareable object. That shareable object is called the lightweight SysObject's parent, and the lightweight SysObject is the child. Each lightweight SysObject shares the information in its shareable parent object. In that way, instead of having multiple nearly identical rows in the SysObject tables to support all the instances of the lightweight type, a single parent object exists for multiple lightweight SysObjects.

You may see a lightweight SysObject referred to as a lightweight object, or sometimes abbreviated as LWSO. All of these terms are equivalent.

Lightweight objects are useful if you have a large number of attribute values that are identical for a group of objects. This redundant information can be shared among the LWSOs from a single copy of the shared parent object. For example, Enterprise A-Plus Financial Services receives many payment checks each day. They record the images of the checks and store the payment information in SysObjects. They will retain this information for several years and then get rid of it. For their purposes, all objects created on the same day can use a single ACL, retention information, creation date, version, and other attributes. That information is held by the shared parent object. The LWSO has information about the specific transaction.

Using lightweight SysObjects can provide the following benefits:

- Lightweight types take up less space in the underlying database tables than a standard subtype.
- Importing lightweight objects into a repository is faster than importing standard SysObjects.

2.1.19.3 Shareable type

A shareable type is a type whose instances can share its property values with instances of lightweight types. It is possible for multiple lightweight objects to share the property values of one shareable object. The shareable object that is sharing its properties with the lightweight object is called the parent object, and the lightweight object is called its child.

2.1.19.4 Materialization and lightweight SysObjects

When a lightweight object shares a parent object with other lightweight objects, we say that the lightweight object is unmaterialized. All the unmaterialized lightweight objects share the attributes of the shared parent, so, in effect, the lightweight objects all have identical values for the attributes in the shared parent. This situation can change if some operation needs to change a parent attribute for one of (or a subset of) the lightweight objects. Since the parent is shared, the change in an attribute would affect all the children. If the change should only affect one child, that child object needs to have its own copy of the parent. When a lightweight object has its own private copy of a parent, we say that the object is materialized. Documentum CM Server creates rows in the tables of the shared type for the object, copying the values of the shared properties into those rows. The lightweight object no longer shares the property values with the instance of the shared type, but with its own private copy of that shared object.

For example, if you checkout a lightweight object, it is materialized. A copy of the original parent is created with the same r_object_id value as the child and the lightweight object is updated to point to the new parent. Since the private parent has the same r_object_id as the lightweight child, a materialized lightweight object behaves similar to a standard object. As another example, if you delete a non-materialized lightweight object, the shared parent is not deleted (whether or not there are any remaining lightweight children). If you delete a materialized lightweight object, the lightweight child and the private parent are deleted.

When, or if, a lightweight object instance is materialized is dependent on the object type definition. You can define a lightweight type such that instances are materialized automatically when certain operations occur, only on request, or never.

2.1.19.5 How Foundation SOAP API represents lightweight SysObjects

Foundation SOAP API represents lightweight SysObjects (also called lightweight objects or light objects) using a specific type of Relationship named Relationship.LIGHT_OBJECT_RELATIONSHIP, which represents the relationship between a lightweight object and a shared parent.

In a relationship, one object is the container of the Relationship (it encapsulates a Relationship instance), and the other object is the target in the relationship (and is encapsulated by the Relationship instance). The direction of the Relationship is determined by the role of the target object in the relationship. For example, if you want the container object to be the lightweight object and the target to be the shared parent, you would set the targetRole property of the Relationship to Relationship.ROLE_PARENT. The following example demonstrates this. The snippet assumes that a sharedParent DataObject and a lightObject DataObject have both been instantiated.

```
ObjectRelationship lightObjectRelationship = new ObjectRelationship();
lightObjectRelationship.setTarget(sharedParent);
lightObjectRelationship.setName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
lightObjectRelationship.setTargetRole(Relationship.ROLE_PARENT);
lightObject.getRelationships().add(lightObjectRelationship);
```

You could also construct the DataObject the other way around, so that the shared parent is the container and the lightweight object is the target. In this case you would set the Relationship.targetRole property to Relationship.ROLE_CHILD.

It is important to note that the LIGHT_OBJECT_RELATIONSHIP is a relationship between a lightweight object and a shared parent. A lightweight object with a private parent (that is, a materialized lightweight object) is represented as a DataObject of a lightweight type, but with no LIGHT_OBJECT_RELATIONSHIP.

2.1.19.5.1 Lightweight and shareable characteristics of a DataObject

To get lightweight objects from the repository, you can specify the following settings in a RelationshipProfile:

```
RelationshipProfile relationshipProfile = new RelationshipProfile();
relationshipProfile.setNameFilter(RelationshipNameFilter.SPECIFIED);
relationshipProfile.setRelationName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
relationshipProfile.setTargetRoleFilter(TargetRoleFilter.ANY);
relationshipProfile.setResultDataMode(ResultDataMode.OBJECT);
```

You can test whether a retrieved DataObject represents a shareable or lightweight object by examining its properties. (You do not have to set these properties to create or update a lightweight or shareable object, but they can be useful when querying for objects or examining objects retrieved from the repository.)

- A DataObject represents a lightweight object if it has the properties i_sharing_parent and i_sharing_type.
- A DataObject is a shareable parent if it has an i_sharing_type property but no i_sharing_parent property.

You can test whether an object is materialized by testing whether it contains any LIGHT_OBJECT_RELATIONSHIP instances. If a lightweight object is not materialized, it will contain a LIGHT_OBJECT_RELATIONSHIP to its shared parent. If it is materialized, it will have no LIGHT_OBJECT_RELATIONSHIP.

Similarly, a shareable object (an object of a shareable type) will have a LIGHT_OBJECT_RELATIONSHIP to each lightweight object that shares it.

2.1.19.6 Shareable and lightweight types

To create an object in the repository, you need to specify its object type in its DataObject. For example, the following instantiates two DataObject instances, one representing the shared parent of a lightweight object, the other representing the lightweight object:

```
DataObject sharedParent = new DataObject(new ObjectIdentity(defaultRepositoryName),
                                         "my_shared_type");
DataObject lightObject = new DataObject(new ObjectIdentity(defaultRepositoryName),
                                        "my_light_type");
```

To successfully make sharedParent the shared parent of lightObject, the shared parent object must be of a shareable type, and the lightweight object must be of a lightweight type. Moreover, the lightweight type must share the shareable type.

When a lightweight type is created, it can explicitly share a shareable type, as shown in the this DQL:

```
CREATE LIGHTWEIGHT TYPE type_name SHARES a_shareable_type
```

Or, instead of sharing a shareable type explicitly, a lightweight type can subtype another lightweight type (in DQL using the WITH SUPERTYPE clause), in which case the newly created type shares its parent's shareable type.

You can get information about lightweight and shareable types by examining the dm_type instance that contains data about the type, using the Query or the Schema service. The following attributes provide information related to lightweight objects:

- The type_category attribute will equal 0x00000002 if the type is shareable.
- The type_category attribute will equal 0x00000004 if the type is lightweight.
- The shared_parent_name attribute, on the lightweight type, stores the name of the shareable type shared by the lightweight type.

2.1.19.7 Creating a lightweight object with a shared parent

To create a lightweight object with a shared parent, create a LIGHT_OBJECT_RELATIONSHIP between a DataObject representing the lightweight object and a DataObject representing the parent.

Example 2-59: Java: Creating a lightweight object with a shared parent

```
public void createLightObject(String shareTypeName, String lightTypeName)
        throws ServiceException
{
    DataObject sharedParent = new DataObject
        (new ObjectIdentity(defaultRepositoryName),
                                            shareTypeName);
    sharedParent.getProperties().set("object_name", "mySharedParent");

    DataObject lightObject = new DataObject
        (new ObjectIdentity(defaultRepositoryName),
                                           lightTypeName);
    lightObject.getProperties().set("object_name", "myLightObject");

    ObjectRelationship lightObjectRelationship = new ObjectRelationship();
    lightObjectRelationship.setTarget(sharedParent);
    lightObjectRelationship.setName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
    lightObjectRelationship.setTargetRole(Relationship.ROLE_PARENT);
    lightObject.getRelationships().add(lightObjectRelationship);

    objectService.create(new DataPackage(lightObject), null);
}
```

2.1.19.8 Re-parenting a lightweight object

Re-parenting a lightweight object means to link it to a new shared parent object. You can do this by updating the object with a LIGHT_OBJECT_RELATIONSHIP to the new shareable parent.

Example 2-60: Java: Re-parenting a lightweight object

```
public void reparentLightObject(String shareTypeName, String lightTypeName)
    throws ServiceException
{
    DataObject sharedParentA = new DataObject
        (new ObjectIdentity(defaultRepositoryName),
                                                                  shareTypeName);
    sharedParentA.getProperties().set("object_name", "sharedParentA");

    DataObject sharedParentB = new DataObject
        (new ObjectIdentity(defaultRepositoryName),
                                                                  shareTypeName);
    sharedParentB.getProperties().set("object_name", "sharedParentB");

    DataObject lightObject = new DataObject
        (new ObjectIdentity(defaultRepositoryName),
                                                                 lightTypeName);
    lightObject.getProperties().set("object_name", "myLightObject");

    // create the lightweight object with shared parent sharedParentA
    ObjectRelationship lightObjectRelationship = new ObjectRelationship();
    lightObjectRelationship.setTarget(sharedParentA);
    lightObjectRelationship.setName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
    lightObjectRelationship.setTargetRole(Relationship.ROLE_PARENT);
    lightObject.getRelationships().add(lightObjectRelationship);

    OperationOptions options = null;
    objectService.create(new DataPackage(lightObject), options);

    // now update the lightweight object, re-parenting to sharedParentB
    lightObject.getRelationships().clear();
    ObjectRelationship newParentRelationship = new ObjectRelationship();
    newParentRelationship.setTarget(sharedParentB);
    newParentRelationship.setName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
    newParentRelationship.setTargetRole(Relationship.ROLE_PARENT);
    lightObject.getRelationships().add(newParentRelationship);

    objectService.update(new DataPackage(lightObject), options);
}
```

2.1.19.9 Deleting a lightweight object

Deleting a lightweight object requires that the object first be materialized (that is, its shareable parent is cloned and becomes a private parent). The following example creates a lightweight object, materializes it, then deletes it:

Example 2-61: Java: Deleting a lightweight object

```
public void deleteLightObject(String shareTypeName, String lightTypeName)
        throws ServiceException
{
    // create a lightweight object
    DataObject sharedParent = new DataObject
        (new ObjectIdentity(defaultRepositoryName),
                                                 shareTypeName);
```

```
    sharedParent.getProperties().set("object_name", "mySharedParent");

    DataObject lightObject = new DataObject
        (new ObjectIdentity(defaultRepositoryName),
                                            lightTypeName);
    lightObject.getProperties().set("object_name", "myLightObject");

    ObjectRelationship lightObjectRelationship = new ObjectRelationship();
    lightObjectRelationship.setTarget(sharedParent);
    lightObjectRelationship.setName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
    lightObjectRelationship.setTargetRole(Relationship.ROLE_PARENT);
    lightObject.getRelationships().add(lightObjectRelationship);

    objectService.create(new DataPackage(lightObject), null);

    / materialize the lightweight object (required before deletion)
    ObjectRelationship relationshipToRemove = new ObjectRelationship();
    relationshipToRemove.setTarget(sharedParent);
    relationshipToRemove.setName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
    relationshipToRemove.setTargetRole(Relationship.ROLE_PARENT);
    relationshipToRemove.setIntentModifier(RelationshipIntentModifier.REMOVE);
    lightObject.getRelationships().add(relationshipToRemove);

    OperationOptions options = null;
    objectService.update(new DataPackage(lightObject), options);

    // delete the lightweight object (along with private parent)
    ObjectIdentitySet objectsToRemove = new ObjectIdentitySet();
    objectsToRemove.addIdentity(lightObject.getIdentity());
    objectsToRemove.addIdentity(sharedParent.getIdentity());

    objectService.delete(objectsToRemove, options);
}
```

2.1.19.10 Getting lightweight objects

To get a DataPackage containing lightweight objects, you need to provide a RelationshipProfile (either in the service context or in OperationOptions) that specifies LIGHT_OBJECT_RELATIONSHIP as the relationship name. The following example will return lightweight as well as associated parent objects:

Example 2-62: Java: Getting lightweight objects

```
public DataPackage getLightweightObjects(ObjectIdentity objId)
        throws ServiceException
    {
        RelationshipProfile relationshipProfile = new RelationshipProfile();
        relationshipProfile.setNameFilter(RelationshipNameFilter.SPECIFIED);
        relationshipProfile.setRelationName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
        relationshipProfile.setTargetRoleFilter(TargetRoleFilter.ANY);
        relationshipProfile.setResultDataMode(ResultDataMode.OBJECT);
        relationshipProfile.setDepthFilter(DepthFilter.SPECIFIED);
        relationshipProfile.setDepth(1);

        objectService.getServiceContext().setProfile(relationshipProfile);

        return objectService.get(new ObjectIdentitySet(objId), null);
    }
```

2.1.19.11 Materializing a lightweight object

Materializing a lightweight object means to disassociate it from its shared parent and associate it with a private parent (which is a clone of the shareable parent). To do this, update the lightweight object, removing its LIGHT_OBJECT_RELATIONSHIP to the shared parent. (If the identity of the parent object is not known, you can materialize the object by removing a LIGHT_OBJECT_RELATIONSHIP with a null target.) The following example creates a lightweight object, then materializes it:

Example 2-63: Java: Materializing a lightweight object

```
public void materializeLightObject(String shareTypeName, String lightTypeName)
        throws ServiceException
{
    DataObject sharedParent = new DataObject
        (new ObjectIdentity(defaultRepositoryName),
                                            shareTypeName);
    sharedParent.getProperties().set("object_name", "sharedParent");

    DataObject lightObject = new DataObject
        (new ObjectIdentity(defaultRepositoryName),
                                           lightTypeName);
    lightObject.getProperties().set("object_name", "myLightObject");

    // create the lightweight object
    ObjectRelationship lightObjectRelationship = new ObjectRelationship();
    lightObjectRelationship.setTarget(sharedParent);
    lightObjectRelationship.setName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
    lightObjectRelationship.setTargetRole(Relationship.ROLE_PARENT);
    lightObject.getRelationships().add(lightObjectRelationship);

    OperationOptions options = null;
    objectService.create(new DataPackage(lightObject), options);

    // materialize by removing LIGHT_OBJECT_RELATIONSHIP
    lightObject.getRelationships().clear();
    ObjectRelationship relationshipToRemove = new ObjectRelationship();

    // you can also set target to null if parent is unknown
    relationshipToRemove.setTarget(sharedParent);
    relationshipToRemove.setName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
    relationshipToRemove.setTargetRole(Relationship.ROLE_PARENT);
    relationshipToRemove.setIntentModifier(RelationshipIntentModifier.REMOVE);
    lightObject.getRelationships().add(relationshipToRemove);

    objectService.update(new DataPackage(lightObject), options);
}
```

When the identity of the shared parent is unknown, you can remove the LIGHT_OBJECT_RELATIONSHIP, setting a null identity for the shareable parent.

```
    // you can also set target to null if parent is unknown
    relationshipToRemove.setTarget(null);
    relationshipToRemove.setName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
    relationshipToRemove.setTargetRole(Relationship.ROLE_PARENT);
    relationshipToRemove.setIntentModifier(RelationshipIntentModifier.REMOVE);
    lightObject.getRelationships().add(relationshipToRemove);
```

2.1.19.12 Dematerializing a lightweight object

You dematerialize a materialized lightweight object by making it the child of a shared object. If the identity of the shared object is unknown, you can make the identity of the parent in the relationship null. In this case the lightweight object will be parented to its original shared parent (that is, the shareable object from which its private parent was cloned). If the identity of the shared parent is provided, then the lightweight object can be simultaneously dematerialized and re-parented.

The following example creates a lightweight object with no shared parent, then updates it, making it the child of a new shared parent:

Example 2-64: Java: Dematerializing a lightweight object

```
public void dematerializeLightObject(String shareTypeName, String lightTypeName)
        throws ServiceException
{
    DataObject lightObject = new DataObject
        (new ObjectIdentity(defaultRepositoryName),
                                          lightTypeName);
    lightObject.getProperties().set("object_name", "myLightObject");

    OperationOptions options = null;
    // This creates a materialized lightweight object
    // because there is no LIGHT_OBJECT_RELATIONSHIP
    objectService.create(new DataPackage(lightObject), options);

          DataObject sharedParent = new DataObject
            (new ObjectIdentity(defaultRepositoryName),
                                              shareTypeName);
    sharedParent.getProperties().set("object_name", "sharedParent");

    // parent the lightweight object to a shared parent (dematerialize)
    ObjectRelationship newParentRelationship = new ObjectRelationship();
    newParentRelationship.setTarget(sharedParent);
    newParentRelationship.setName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
    newParentRelationship.setTargetRole(Relationship.ROLE_PARENT);
    lightObject.getRelationships().add(newParentRelationship);

    objectService.update(new DataPackage(lightObject), options);
}
```

When the identity of the shared parent is unknown, you can add a the LIGHT_OBJECT_RELATIONSHIP, setting a null identity for the shareable parent.

```
// you can also set target to null if parent is unknown
relationshipToRemove.setTarget(null);
relationshipToRemove.setName(Relationship.LIGHT_OBJECT_RELATIONSHIP);
relationshipToRemove.setTargetRole(Relationship.ROLE_PARENT);
relationshipToRemove.setIntentModifier(RelationshipIntentModifier.ADD);
lightObject.getRelationships().add(relationshipToRemove);
```

2.2 VersionControl service

The VersionControl service provides operations that apply to specific object versions, such as checking in, checking out, or deleting an object version.

2.2.1 getCheckoutInfo operation

2.2.1.1 Description

Provides checkout information about the specified objects, specifically whether the objects are checked out, and the user name of the user who has them checked out.

2.2.1.2 Java syntax

```
List<CheckoutInfo> getCheckoutInfo(ObjectIdentitySet objectIdentitySet)
        throws CoreServiceException, ServiceException
```

2.2.1.3 C# syntax

```
List<CheckoutInfo> GetCheckoutInfo(ObjectIdentitySet objectIdentitySet)
```

2.2.1.4 Parameters

Parameter	Data type	Description
objectIdentitySet	ObjectIdentitySet	A collection of ObjectIdentity instances that uniquely identify the repository objects about which to obtain checkout information.

2.2.1.5 Response

Returns a List of CheckoutInfo instances. Checkout info encapsulates data about a specific checked out repository object. The following table shows the CheckoutInfo fields:

Field	Field type	Description
identity	ObjectIdentity	Uniquely identifies the checked out object.
userName	String	The name of the user who has the object checked out.
isCheckedOut	boolean	Indicates whether the repository object is checked out.

2.2.1.6 Example

The following example gets checkout information about an object and prints it to the console:

Example 2-65: Java: Getting checkout info

```
public CheckoutInfo checkoutInfo(ObjectIdentity objIdentity)
    throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    IVersionControlService versionSvc
        = serviceFactory.getRemoteService(IVersionControlService.class,
                    serviceContext);

    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.getIdentities().add(objIdentity);
    List<CheckoutInfo> objList;
    OperationOptions operationOptions = null;
    versionSvc.checkout(objIdSet, operationOptions);
    objList = versionSvc.getCheckoutInfo(objIdSet);
    CheckoutInfo checkoutInfo = objList.get(0);

    if (checkoutInfo.isCheckedOut())
    {
        System.out.println("Object "
                            + checkoutInfo.getIdentity()
                            + " is checked out.");
        System.out.println("Lock owner is " + checkoutInfo.getUserName());
    }
    else
    {
        System.out.println("Object "
                            + checkoutInfo.getIdentity()
                            + " is not checked out.");
        }
    versionSvc.cancelCheckout(objIdSet,null);
    return checkoutInfo;
}
```

Example 2-66: C#: Getting checkout info

```
public CheckoutInfo CheckoutInfo(ObjectIdentity objIdentity)
{
    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.Identities.Add(objIdentity);
    List<CheckoutInfo> objList;
    OperationOptions operationOptions = null;
    versionControlService.Checkout(objIdSet, operationOptions);
    objList = versionControlService.GetCheckoutInfo(objIdSet);
    CheckoutInfo checkoutInfo = objList[0];

    if (checkoutInfo.IsCheckedOut)
    {
        Console.WriteLine("Object "
                            + checkoutInfo.Identity
                            + " is checked out.");
        Console.WriteLine("Lock owner is " + checkoutInfo.UserName);
    }
    else
    {
        Console.WriteLine("Object "
                            + checkoutInfo.Identity
                            + " is not checked out.");
    }
```

```
    versionSvc.cancelCheckout(objIdSet,null);
    return checkoutInfo;
}
```

2.2.2 checkout operation

2.2.2.1 Description

The checkout operation checks out a set of repository objects. Any version of the object can be checked out.

The checkout operation by default returns no content and no properties. These defaults can be changed using ContentProfile and PropertyProfile instances passed in OperationOptions or set in the service context.

When checking out a virtual document, you can supply a VdmRetrieveProfile. See "VdmRetrieveProfile" on page 171.

2.2.2.2 Java syntax

```
DataPackage checkout(ObjectIdentitySet objectIdentitySet,
                     OperationOptions operationOptions)
        throws CoreServiceException, ServiceException
```

2.2.2.3 C# syntax

```
DataPackage Checkout(ObjectIdentitySet objectIdentitySet,
                     OperationOptions operationOptions)
```

2.2.2.4 Parameters

Parameter	Data type	Description
objectIdentitySet	ObjectIdentitySet	A collection of ObjectIdentity instances that uniquely identify the repository objects to check out.
operationOptions	OperationOptions	Contains profiles and properties that specify operation behaviors. In the case of the checkout operation, the profiles primarily provide filters that modify the contents of the returned DataPackage.

2.2.2.5 Response

Returns a DataPackage containing DataObject instances representing the checked out repository objects. The DataObject instances contain complete properties, and any object content is transferred. The client can change these defaults by setting Profile instances in OperationOptions.

2.2.2.6 Example

Example 2-67: Java: Checking an object out

```
public DataPackage checkout(ObjectIdentity objIdentity)
    throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    IVersionControlService versionSvc
        = serviceFactory.getRemoteService(IVersionControlService.class,
                    serviceContext);

    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.getIdentities().add(objIdentity);

    OperationOptions operationOptions = null;
    DataPackage resultDp;
    try
    {
        resultDp = versionSvc.checkout(objIdSet, operationOptions);
    }
    catch (Exception e)
    {
        e.printStackTrace();
        throw new RuntimeException(e);
    }
    System.out.println("Checkout successful");

    List<VersionInfo> vInfo = versionSvc.getVersionInfo(objIdSet);
    VersionInfo versionInfo = vInfo.get(0);

    System.out.println("Printing version info for " +
        versionInfo.getIdentity());
    System.out.println("isCurrent is " + versionInfo.isCurrent());
    System.out.println("Version is " + versionInfo.getVersion());

    System.out.println("Symbolic labels are: ");
    for (String label : versionInfo.getSymbolicLabels())
    {
        System.out.println(label);
    }

    versionSvc.cancelCheckout(objIdSet,null);
    System.out.println("Checkout cancelled");
    return resultDp;
}
```

Example 2-68: C#: Checking an object out

```
public DataPackage Checkout(ObjectIdentity objIdentity)
{

    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.Identities.Add(objIdentity);

    OperationOptions operationOptions = null;
```

```
    DataPackage resultDp;

    resultDp = versionControlService.Checkout(objIdSet, operationOptions);
    Console.WriteLine("Checkout successful");

    List<VersionInfo> vInfo = versionControlService.
        GetVersionInfo(objIdSet);
    VersionInfo versionInfo = vInfo[0];

    Console.WriteLine("Printing version info for " + versionInfo.Identity);
    Console.WriteLine("IsCurrent is " + versionInfo.IsCurrent);
    Console.WriteLine("Version is " + versionInfo.Version);
    Console.WriteLine("Symbolic labels are: ");
    foreach (String label in versionInfo.SymbolicLabels)
    {
        Console.WriteLine(label);
    }

    versionSvc.cancelCheckout(objIdSet,null);
    Console.WriteLine("Checkout cancelled");
    return resultDp;
}
```

2.2.3 checkin operation

2.2.3.1 Description

The checkin operation checks in a set of repository objects using data contained in a DataPackage. It provides control over how the checked in object is versioned and whether the object remains checked out and locked by the user after the changes are versioned, and provides a mechanism for applying symbolic version labels to the checked-in versions. The ObjectIdentity of each DataObject passed to the operation is expected to match the identity of a checked out repository object.

Note: If you are checking in an object that contains system properties, you should set the FilterMode in PropertyProfile to ALL_NON_SYSTEM, not to ALL, unless you explicitly want to update a system property. Otherwise the update of the system property may lead to errors.

2.2.3.2 Java syntax

```
DataPackage checkin(DataPackage dataPackage,
                    VersionStrategy versionStrategy,
                    boolean isRetainLock,
                    List<String> symbolicLabels
                    OperationOptions operationOptions)
throws CoreServiceException, ServiceException
```

2.2.3.3 C# syntax

```
DataPackage Checkin(DataPackage dataPackage,
                    VersionStrategy versionStrategy,
                    boolean isRetainLock,
                    List<String> symbolicLabels
                    OperationOptions operationOptions)
```

2.2.3.4 Parameters

Parameter	Data type	Description
dataPackage	DataPackage	Contains a set of DataObject instances that are to be checked in as new versions of checked out repository objects.
versionStrategy	VersionStrategy	Specifies option for incrementing the version number of the new version.
isRetainLock	boolean	Specifies whether the object is to remain checked out and locked by the user after the new version is saved.
symbolicLabels	List<String>	A list of symbolic version labels, which are applied to all repository objects represented in the DataPackage.
operationOptions	OperationOptions	Contains profiles and properties that specify operation behaviors. In the case of the checkout operation, the profiles primarily provide filters that modify the contents of the returned DataPackage.

2.2.3.4.1 VersionStrategy values

The VersionStrategy values represent the numbering strategy that is applied to a new repository object version when it is checked in.

TargetRole value	Description
IMPLIED	Use the default behavior configured on Documentum CM Server for versioning. This is typically to check the object in as a new version and to increment the version number as the next minor version.

TargetRole value	Description
NEXT_MINOR	Check the object in as a new version and to increment the version number as the next minor version. For example, if the version number is currently 1.1, give the new version the number 1.2.
NEXT_MAJOR	Check the object in as a new version and to increment the version number as the next major version. For example, if the version number is currently 1.1, give the new version the number 2.0.
SAME_VERSION	Save the new object as the same version as the current version, overwriting the current version. This requires that the user have WRITE permissions on the object.
BRANCH_VERSION	Check the object in as a new branch version. (Since 6.6.)

2.2.3.5 CheckinProfile

The CheckinProfile, normally passed within OperationOptions, controls specific behaviors of the checkin operation. The following table describes the profile settings:

Field	Description
checkinOnlyVDMRoot	Indicates whether or not the virtual document root is checked in by itself or with all of its child components. Default value is false.
deleteLocalFileHint	If using UCF content transfer, delete local file content after checkin to repository. Default value is false. This hint will not be honored if content transfer mode is not UCF.
makeCurrent	Indicates whether or not the checked in version of the file is to be made the current version. Default value is true.

2.2.3.6 Response

Returns a DataPackage containing one DataObject for each repository object version created by the checkin operation. By default, each DataObject contains only the ObjectIdentity of the new version and no other data. The client can modify this behavior by using Profile objects if it requires more data about the new versions.

2.2.3.7 Example

The following example checks in a single DataObject as a new version. Note that it explicitly sets a ContentProfile for the that is applied on checkout and subsequent checkin. Note as well that new content is explicitly added to the object prior to checkin.

Example 2-69: Java: Checking an object in

```
public DataPackage checkin(ObjectIdentity objIdentity, String newContentPath)
    throws Exception
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    IVersionControlService versionSvc
        = serviceFactory.getService(IVersionControlService.class, serviceContext);
    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.getIdentities().add(objIdentity);

    OperationOptions operationOptions = new OperationOptions();
    ContentProfile contentProfile = new ContentProfile(FormatFilter.ANY, null,
                                    PageFilter.ANY,
                                    -1,
                                    PageModifierFilter.ANY,
                                    null);
    operationOptions.setContentProfile(contentProfile);

    DataPackage checkinPackage = versionSvc.checkout(objIdSet,
        operationOptions);

    DataObject checkinObj = checkinPackage.getDataObjects().get(0);

    checkinObj.setContents(null);
    FileContent newContent = new FileContent();
    newContent.setLocalPath(newContentPath);
    newContent.setRenditionType(RenditionType.PRIMARY);
    newContent.setFormat("gif");
    checkinObj.getContents().add(newContent);

    boolean retainLock = false;
    List<String> labels = new ArrayList<String>();
    labels.add("test_version");
    DataPackage resultDp;
    try
    {
        resultDp = versionSvc.checkin(checkinPackage,
                                      VersionStrategy.NEXT_MINOR,
                                      retainLock,
                                      labels,
                                      operationOptions);
    }
    catch (ServiceException sE)
    {
        sE.printStackTrace();
        throw new RuntimeException(sE);
    }
    return resultDp;
}
```

Example 2-70: C#: Checking an object in

```
public DataPackage Checkin(ObjectIdentity objIdentity, String newContentPath)
{
    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.Identities.Add(objIdentity);
```

```
    OperationOptions operationOptions = new OperationOptions();
    ContentProfile contentProfile = new ContentProfile(FormatFilter.ANY, null,
                                    PageFilter.ANY,
                                    -1,
                                    PageModifierFilter.ANY, null);
    operationOptions.ContentProfile = contentProfile;

    DataPackage checkinPackage = versionControlService.Checkout(objIdSet,
        operationOptions);

    DataObject checkinObj = checkinPackage.DataObjects[0];

    checkinObj.Contents = null;
    FileContent newContent = new FileContent();
    newContent.LocalPath = newContentPath;
    newContent.RenditionType = RenditionType.PRIMARY;
    newContent.Format = "gif";
    checkinObj.Contents.Add(newContent);

    bool retainLock = false;
    List<String> labels = new List<String>();
    labels.Add("test_version");
    DataPackage resultDp;
    try
    {
        resultDp = versionControlService.Checkin(checkinPackage,
                                                 VersionStrategy.NEXT_MINOR,
                                                 retainLock,
                                                 labels,
                                                 operationOptions);
    }
        catch (FaultException<SerializableException> ex)
    {
        Console.WriteLine(String.Format("Got FaultException[{0}]
                with message: {1}\n", ex.Detail, ex.Message));
    }
    catch (Exception exx)
    {
        Console.WriteLine(exx.StackTrace);
    }
    return resultDp;
}
```

2.2.3.8 Notes on checking in virtual documents

If you are using UCF content transfer, UCF provides special support for checking in virtual document children. If you are not using UCF transfer mode, then your client code has to has to package the virtual document and its children correctly for the checkin to succeed.

When you are using UCF to transfer the content of a virtual document, you need only include the root of the document in the DataPackage; however, the operation will succeed only if:

- The checkout operation was executed using UCF transfer mode and the contents of the virtual document that are now to be checked in were transferred during the operation.
- UCF transfer mode is selected for the checkin operation also.

If your client is using Base64 or MTOM transfer mode, then in order to checkin the virtual document with its children content, the client will have to explicitly provide the content as separate documents in separate DataObject instances. In other words the client will have to take responsibility for checking in the virtual document children In this case the VirtualDocumentService checkin process is sequential, so the root of the virtual document must be the last DataObject in the DataPackage. Note that in this case no special processing of XML content or Microsoft documents with links is provided.

2.2.3.9 cancelCheckout operation

2.2.3.9.1 Description

The cancelCheckout operation cancels checkout of a set of repository objects.

2.2.3.9.2 Java syntax

```
public void cancelCheckout(ObjectIdentitySet objectIdentitySet, OperationOptions options)
        throws CoreServiceException
```

2.2.3.9.3 C# syntax

```
public void CancelCheckout(ObjectIdentitySet objectIdentitySet, OperationOptions options)
```

2.2.3.9.4 Parameters

Parameter	Data type	Description
objectIdentitySet	ObjectIdentitySet	A collection of ObjectIdentity instances that uniquely identify the repository objects on which to cancel checkout.
operationOptions	OperationOptions	Contains profiles and properties that specify operation behaviors. In the case of the cancel checkout operation, the profiles primarily provide filters that what to cancel.

2.2.3.9.5 Example

Example 2-71: Java: Cancelling checkout

```
public void cancelCheckout(ObjectIdentity objIdentity)
    throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    IVersionControlService versionSvc =
        serviceFactory.getRemoteService(IVersionControlService.class,
            serviceContext);

    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.getIdentities().add(objIdentity);

    versionSvc.cancelCheckout(objIdSet,null);
}
```

Example 2-72: C#: Cancelling checkout

```
public void CancelCheckout(ObjectIdentity objIdentity)
{
    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.Identities.Add(objIdentity);

    versionControlService.CancelCheckout(objIdSet,null);

}
```

2.2.3.10 deleteVersion operation

2.2.3.10.1 Description

The deleteVersion operation deletes a specific version of a repository object. If the deleted object is the CURRENT version, the previous version in the version tree is promoted to CURRENT.

2.2.3.10.2 Java syntax

```
void deleteVersion(ObjectIdentitySet objectsToDelete)
        throws CoreServiceException, ServiceException
```

2.2.3.10.3 C# syntax

```
void DeleteVersion(ObjectIdentitySet objectsToDelete)
```

2.2.3.10.4 Parameters

Parameter	Data type	Description
objectsToDelete	ObjectIdentitySet	A collection of ObjectIdentity instances that uniquely identify the repository object versions to delete.

2.2.3.10.5 Example

The following sample deletes a specific version of a repository object. The ObjectIdentity representing the repository object can be an ObjectId or a Qualification that identifies a non-CURRENT version.

Example 2-73: Java: Deleting a specific version

```
public void deleteVersionDemo(ObjectIdentity objIdentity)
    throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    IVersionControlService versionSvc
        = serviceFactory.getRemoteService(IVersionControlService.class,
                                          serviceContext);

    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.getIdentities().add(objIdentity);

    versionSvc.deleteVersion(objIdSet);
}
```

Example 2-74: C#: Deleting a specific version

```
public void DeleteVersionDemo(ObjectIdentity objIdentity)
{
    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.Identities.Add(objIdentity);

    versionControlService.DeleteVersion(objIdSet);
}
```

2.2.3.11 deleteAllVersions operation

2.2.3.11.1 Description

The deleteAllVersions operation deletes all versions of a repository object. An ObjectIdentity indicating the object to delete can reference any version in the version tree.

2.2.3.11.2 Java syntax

```
void deleteAllVersions(ObjectIdentitySet objectIdentitySet)
        throws CoreServiceException, ServiceException
```

2.2.3.11.3 C# syntax

```
void DeleteAllVersions(ObjectIdentitySet objectIdentitySet)
```

2.2.3.11.4 Parameters

Parameter	Data type	Description
objectIdentitySet	ObjectIdentitySet	A collection of ObjectIdentity instances that uniquely identify the repository objects of which to delete all versions.

2.2.3.11.5 Example

The following sample deletes all versions of an object. The qualification it uses can represent a CURRENT or a non-CURRENT version.

Example 2-75: Java: Deleting all versions of an object

```
public void deleteAllVersionsDemoQual()
    throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    IVersionControlService versionSvc
        = serviceFactory.getRemoteService(IVersionControlService.class,

                                          serviceContext);

    String nonCurrentQual = "dm_document (ALL) " +
                            "where object_name = 'DFS_sample_image' " +
                            "and ANY r_version_label = 'test_version'";
    Qualification<String> qual = new Qualification<String>(nonCurrentQual);
    ObjectIdentity<Qualification> objIdentity = new ObjectIdentity<Qualification>();
    objIdentity.setValue(qual);
    objIdentity.setRepositoryName(defaultRepositoryName);
    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.getIdentities().add(objIdentity);

    versionSvc.deleteAllVersions(objIdSet);
}
```

Example 2-76: C#: Deleting all versions of an object

```
public void DeleteAllVersionsDemoQual()
{
    string nonCurrentQual = "dm_document (ALL) " +
                            "where object_name = 'DFS_sample_image' " +
                            "and ANY r_version_label = 'test_version'";
    Qualification qual = new Qualification(nonCurrentQual);
    ObjectIdentity objIdentity = new ObjectIdentity();
    objIdentity.Value = qual;
    objIdentity.RepositoryName = DefaultRepository;
    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.Identities.Add(objIdentity);

    versionControlService.DeleteAllVersions(objIdSet);
}
```

2.2.3.12 getCurrent operation

2.2.3.12.1 Description

The getCurrent operation exports the CURRENT version of a repository object, transferring any object content to the client. The getCurrent operation returns the CURRENT version of a repository object even when passed an ObjectIdentity identifying a non-CURRENT version.

By default, the getCurrent operation returns no content, and only non-system properties. These defaults can be changed using ContentProfile and PropertyProfile instances passed in operationOptions or set in the service context.

2.2.3.12.2 Java syntax

```
DataPackage getCurrent(ObjectIdentitySet forObjects,
                       OperationOptions operationOptions)
    throws CoreServiceException, ServiceException
```

2.2.3.12.3 C# syntax

```
DataPackage GetCurrent(ObjectIdentitySet forObjects,
                       OperationOptions operationOptions)
```

2.2.3.12.4 Parameters

Parameter	Data type	Description
forObjects	ObjectIdentitySet	A collection of ObjectIdentity instances that uniquely identify the repository objects of which the CURRENT version will be exported.

Parameter	Data type	Description
operationOptions	OperationOptions	Contains profiles and properties that specify operation behaviors. In the case of the getCurrent operation, the profiles primarily provide filters that modify the contents of the returned DataPackage.

2.2.3.12.5 Response

Returns a DataPackage populated using the same defaults as the Object service get operation (see "Response" on page 20). These defaults can be modified by setting Profile instances in operationOptions or the service context (see "Controlling data returned by get operation" on page 21).

2.2.3.12.6 Example

Example 2-77: Java: Getting the current object

```
public DataObject getCurrentDemo(ObjectIdentity objIdentity)
    throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    IVersionControlService versionSvc
        = serviceFactory.getRemoteService(IVersionControlService.class,
                                          serviceContext);

    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.getIdentities().add(objIdentity);

    OperationOptions operationOptions = null;
    DataPackage resultDataPackage = versionSvc.getCurrent(objIdSet,
        operationOptions);
    return resultDataPackage.getDataObjects().get(0);
}
```

Example 2-78: C#: Getting the current object

```
public DataObject GetCurrentDemo(ObjectIdentity objIdentity)
{
  ObjectIdentitySet objIdSet = new ObjectIdentitySet();
  objIdSet.Identities.Add(objIdentity);

  OperationOptions operationOptions = null;
  DataPackage resultDataPackage = versionControlService.GetCurrent(objIdSet,
    operationOptions);
  return resultDataPackage.DataObjects[0];
}
```

2.2.3.13 getVersionInfo operation

2.2.3.13.1 Description

The getVersionInfo operation provides information about a version of a repository object.

2.2.3.13.2 Java syntax

```
List<VersionInfo> getVersionInfo(ObjectIdentitySet objectIdentitySet)
    throws CoreServiceException, ServiceException
```

2.2.3.13.3 C# syntax

```
List<VersionInfo> GetVersionInfo(ObjectIdentitySet objectIdentitySet)
```

2.2.3.13.4 Parameters

Parameter	Data type	Description
ObjectIdentitySet	ObjectIdentitySet	A collection of ObjectIdentity instances that uniquely identify the repository objects about which to provide version information.

2.2.3.13.5 Response

Returns a List of VersionInfo instances corresponding to the DataObject instances in the ObjectIdentitySet. Each VersionInfo contains data about a specific version of a repository object. The following table shows the VersionInfo fields:

Field	Field type	Description
identity	ObjectIdentity	Uniquely identifies the object version.
isCurrent	boolean	Specifies whether this is the CURRENT version of the repository object.
version	String	The system version label, for example 1.1.
symbolicLabel	List	A List of String values representing all symbolic version labels applied to this version, including (if applicable) CURRENT.

Field	Field type	Description
nextMajorVersion	String	The next major implicit version label for an object. A version's implicit label is the numerical version label typically assigned to the object by the server.
nextMinorVersion	String	The next minor implicit version label for an object. A version's implicit label is the numerical version label typically assigned to the object by the server.
nextBranchVersion	String	The next possible branch version label of an object. When a user creates a branch version, the server adds two more places to the version's current implicit version label. For example, if a user branches a version with the implicit label 1.2, the new branch version is assigned the label 1.2.1.0.

2.2.3.13.6 Example

Example 2-79: Java: Getting version info

```
public void versionInfoDemoQual(String nonCurrentQual)
    throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    IVersionControlService versionSvc
        = serviceFactory.getRemoteService(IVersionControlService.class,
                                          serviceContext);

    Qualification<String> qual = new Qualification<String>(nonCurrentQual);
    ObjectIdentity<Qualification> objIdentity = new ObjectIdentity<Qualification>();
    objIdentity.setValue(qual);
    objIdentity.setRepositoryName(defaultRepositoryName);
    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.getIdentities().add(objIdentity);

    List<VersionInfo> vInfo = versionSvc.getVersionInfo(objIdSet);
    VersionInfo versionInfo = vInfo.get(0);

    System.out.println("Printing version info for " + versionInfo.getIdentity());
    System.out.println("isCurrent is " + versionInfo.isCurrent());
    System.out.println("Version is " + versionInfo.getVersion());

    System.out.println("Symbolic labels are: ");
    for (String label : versionInfo.getSymbolicLabels())
    {
        System.out.println(label);
    }
}
```

Example 2-80: C#: Getting version info

```
public void VersionInfoDemoQual(String nonCurrentQual)
{
    Qualification qual = new Qualification(nonCurrentQual);
    ObjectIdentity objIdentity = new ObjectIdentity();
    objIdentity.Value = qual;
    objIdentity.RepositoryName = DefaultRepository;
    ObjectIdentitySet objIdSet = new ObjectIdentitySet();
    objIdSet.Identities.Add(objIdentity);

    List<VersionInfo> vInfo = versionControlService.GetVersionInfo(objIdSet);
    VersionInfo versionInfo = vInfo[0];

    Console.WriteLine("Printing version info for " + versionInfo.Identity);
    Console.WriteLine("isCurrent is " + versionInfo.IsCurrent);
    Console.WriteLine("Version is " + versionInfo.Version);

    Console.WriteLine("Symbolic labels are: ");
    foreach (string label in versionInfo.SymbolicLabels)
    {
        Console.WriteLine(label);
    }
}
```

2.2.4 AccessControl service

The AccessControl service provides operations for creating, getting, updating and deleting Access Control Lists (ACLs). ACLs are used by Documentum CM Server to implement object-level permissions and folder security.

This section describes the Foundation SOAP API data model used by the AccessControl service to represent ACLs. It then discusses the AccessControl service operations and provides a sample for each operation. Finally, it discusses two important topics that involve using the AccessControl service in combination with other services: how to apply an ACL to an object using the Object service; and how to fetch a list of ACLs from the repository using the Query service and convert the query results into the data model understood by the AccessControl service.

2.2.4.1 For more information

To make best use of the AccessControl service, it's important to have a thorough understanding of how object security and folder security work on Documentum CM Server, and how ACLs fit into the picture. Complete coverage of this topic is beyond the scope of this chapter, but it is discussed thoroughly in *(EDCCS250400-AGD)*, with some useful additional material (particularly regarding alias sets) in *(EDCCS250400-GGD)*.

The AccessControl service data model is an object-oriented abstraction that represents an ACL repository object (dm_acl). For more information to understand the structure and semantics of the dm_acl repository object, see *(EDCCS250400-ORD)*. It is particularly useful to understand the dm_acl repository object, because for some purposes you may need to work directly ACL-related properties in objects and in query results.

2.2.4.2 AccessControl service data model

The AccessControl data model is organized at the top level into a collection of Acls called an AclPackage, which is analogous in form and function to the Object service DataPackage. The Acl object itself contains an AclIdentity, with settings that uniquely identify the Acl, and a collection of AclEntry instances, which associate users and groups with lists of permissions. The following class diagram shows the structure of the data model, and following sections describe the data model classes:

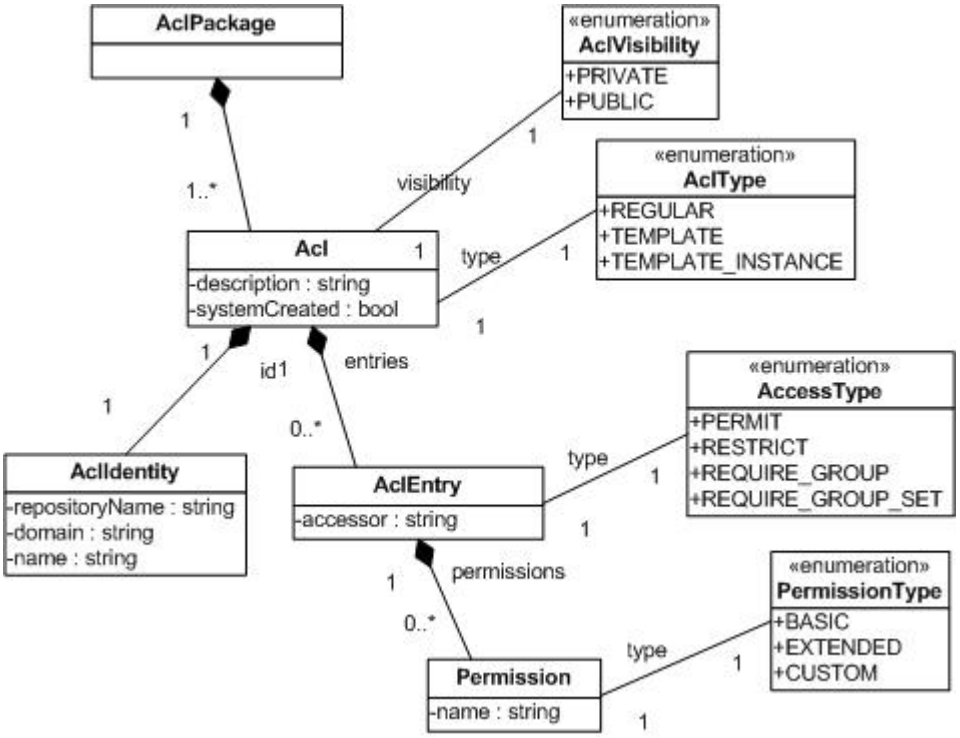

Figure 2-2: AccessControl service data model

2.2.4.2.1 AclPackage

An AclPackage instance contains a collection of Acl instances (it encapsulates a List<Acl>).

2.2.4.2.2 Acl

An Acl object represents an ACL repository object. An ACL is applied to a repository object (specifically a SysObject) to define object-level security, or to a folder for use in folder security. The entries in the ACL control indicate the users and groups who can access the objects to which the ACL is attached, and the level of permissions for the users and groups. If the security mode for a repository is set to "acl", then every object in the repository will have an ACL.

The aclVisibility field specifies the accessibility of the ACL to users. It can either be PUBLIC, meaning that all users of the repository can see the ACL, or PRIVATE, meaning that only the owner of the ACL or a Superuser can see the ACL.

The systemCreated field indicates whether the ACL is a system ACL. This is a read-only field: a client cannot create a system ACL using the AccessControl service. System ACLs on Documentum CM Server are a subset of public ACLs: they are public ACLs that are owned by the repository owner.

The AclType field indicates whether Documentum CM Server will treat the ACL as a regular ACL, a template, or a template instance. For more information, see *(EDCCS250400-AGD)* and *(EDCCS250400-GGD)*.

- A regular ACL is the default type, and Documentum CM Server does not apply any special handling.
- A template ACL typically has one or more accessor values set to alias specifications. When used in ACLs, aliases are placeholders for user and group names. Use template ACLs with aliases in applications where the concrete user names may change depending on context.
- When a template ACL to is applied to an object, the server copies the template, creating a template-instance, resolves the aliases based on an alias set, and replaces them in the copy with the real names, and assigns the copy to the object. The copy is always a system ACL. If a template ACL or the alias set used to resolve the template's aliases is modified, the server automatically updates the template-instances derived from the template.

An Acl contains an AclIdentity instance that uniquely identifies the ACL repository object (see "AclIdentity" on page 94), and a List of AclEntry instances that assign permissions to users and groups (see "AclEntry" on page 94).

2.2.4.2.3 AclIdentity

An AclIdentity is used to identify a unique ACL in a repository and domain, using three properties:

- *`repositoryName`*: a String specifying the name of the repository.
- *`domain`*: a String representing the owner of the ACL, which can be a user name, group name, or alias.
- *`name`*: a String specifying the name of the ACL, which must be unique on the repository within a domain.

2.2.4.2.4 AclEntry

An Acl instance encapsulates a List of AclEntry instances, which assign a list of permissions to a user or group (the accessor). An AclEntry is typically contained in an Acl with other AclEntry instances, which together define access for multiple accessors.

The *`accessor`* property is a String specifying a user name, group name, or alias. Some commonly used aliases are dm_world (all users) and dm_owner (the owner of the object to which the ACL is applied).

A list of `Permission` instances (see "Permission" on page 95) are associated with the accessor.

The `AccessType` of an AclEntry indicates to Documentum CM Server how it should interpret the permission assignments for the accessor; that is, should it grant the permission, restrict the permission, or apply some other logic.

AccessType	Description
PERMIT	Grant the user or group this set of permissions. Note that if an accessor appears more than once in a list of AclEntry instances, the accessor is granted the most restrictive level of basic permissions specified. For example, if a user is a member of a group that is granted BROWSE permissions, but in another AclEntry is granted VERSION permissions, Documentum CM Server will give this user BROWSE permissions on the object.

AccessType	Description
RESTRICT	An AclEntry of type RESTRICT removes the right to the base object-level permission level specified in the entry, and to any extended privileges listed in the AclEntry. For BASIC permissions, the user or group members have access at the level up to the specified restriction. Access restriction entries are useful when you want give a group a particular base object-level permission, but restrict access for individual members or a subgroup of members.
REQUIRE_GROUP	An AclEntry of type REQUIRE_GROUP requires a user requesting access to an object governed by the ACL to be a member of the group identified in the entry. This is true even if the user is explicitly granted the permission in another entry.
REQUIRE_GROUP_SET	A REQUIRE_GROUP_SET entry requires a user requesting access to an object governed by the ACL to be a member of at least one group in a set of groups. An ACL that enforces a required group set typically has multiple required group set entries. Each entry identifies one group in the set. The user must belong to at least one of the groups identified by the REQUIRE_GROUP_SET entries in the ACL.

2.2.4.2.5 Permission

Each Permission has a permissionType field that can be set to BASIC, EXTENDED, or CUSTOM. BASIC permissions are compound (sometimes called hierarchical), meaning that there are levels of permission, with each level including all lower-level permissions. For example, if a user has RELATE permissions on an object, the user is also granted READ and BROWSE permissions. This principle does not apply to extended permissions, which have to be granted individually.

The following table shows the PermissionType enum constants and Permission constants:

Permission type	Permission	Description
BASIC	NONE	No access is permitted. The object is not visible to the user.
	BROWSE	The user can view attribute values of content, including its location.

Permission type	Permission	Description
	READ	The user can read content but not update.
	RELATE	The user can relate objects to this object (can create a relationship to this object).
	VERSION	The user can version the object.
	WRITE	The user can write and update the object.
	DELETE	The user can delete the object.
EXTENDED	X_CHANGE_LOCATION	The user can change move an object from one folder to another. All users having at least Browse permission on an object are granted Change Location permission by default for that object.
	X_CHANGE_OWNER	The user can change the owner of the object.
	X_CHANGE_PERMIT	The user can change the basic permissions on the object.
	X_CHANGE_STATE	The user can change the document lifecycle state of the object.
	X_DELETE_OBJECT	The user can delete the object. The delete object extended permission is not equivalent to the base Delete permission. Delete Object extended permission does not grant Browse, Read, Relate, Version, or Write permission.
	X_EXECUTE_PROC	The user can run the external procedure associated with the object. All users having at least Browse permission on an object are granted Execute Procedure permission by default for that object.

Note that Documentum CM Server will automatically grant X_CHANGE_LOCATION and X_EXECUTE_PROC extended permissions to a user that has a BASIC permission level of BROWSE or higher.

2.2.4.3 create operation

The create operation creates new ACL objects based on ACL instances contained in an AclPackage passed to the operation.

Note that the create operation cannot be used to create a system ACL.

2.2.4.3.1 Java syntax

```
AclPackage create(AclPackage aclPackage)
        throws CoreServiceException, ServiceException
```

2.2.4.3.2 C# syntax

```
AclPackage Create(AclPackage aclPackage)
```

2.2.4.3.3 Examples

The following example creates a REGULAR ACL in the repository, with PRIVATE visibility.

Note that EXECUTE_PROC and CHANGE_LOCATION privileges are assigned automatically by Documentum CM Server, if the accessor is assigned basic privileges of BROWSE or higher, as is done in the sample.

Example 2-81: Java: Creating a private ACL

```
public AclPackage createAcl(String repoName, String userName, String aclName)
  throws ServiceException
{
    AclIdentity aclIdentity = new AclIdentity();
    aclIdentity.setRepositoryName(repoName);
    aclIdentity.setDomain(userName);
    aclIdentity.setName(aclName);

    Permission basicReadPermission = new Permission();
    basicReadPermission.setName(Permission.READ);
    basicReadPermission.setType(PermissionType.BASIC);

    Permission basicDeletePermission = new Permission();
    basicDeletePermission.setName(Permission.DELETE);
    basicDeletePermission.setType(PermissionType.BASIC);

    List<AclEntry> entryList = new ArrayList<AclEntry>();

    AclEntry aclEntry = new AclEntry();
    aclEntry.setAccessType(AccessType.PERMIT);
    aclEntry.setAccessor("dm_world");
    List<Permission> permissionList = new ArrayList<Permission>();
    permissionList.add(basicReadPermission);
    aclEntry.setPermissions(permissionList);

    AclEntry aclEntry1 = new AclEntry();
    aclEntry1.setAccessType(AccessType.PERMIT);
    aclEntry1.setAccessor("dm_owner");
    List<Permission> permissionList1 = new ArrayList<Permission>();
    permissionList1.add(basicDeletePermission);
    aclEntry1.setPermissions(permissionList1);

    entryList.add(aclEntry);
```

```
    entryList.add(aclEntry1);

    Acl acl = new Acl();
    acl.setIdentity(aclIdentity);
    acl.setDescription("a test acl" + System.currentTimeMillis());
    acl.setSystemCreated(false); // defaults to false
    //acl.setType(AclType.REGULAR); //defaults to REGULAR
    //acl.setVisibility(AclVisibility.PRIVATE); //defaults to PRIVATE
    acl.setEntries(entryList);

    AclPackage aclPackage = new AclPackage();
    List<Acl> aclList = new ArrayList<Acl>();
    aclList.add(acl);
    aclPackage.setAcls(aclList);

    return accessControlService.create(aclPackage);
}
```

Example 2-82: C#: Creating a private Acl

```
public AclPackage CreateAcl(String repoName, String userName, String aclName)
{
    AclIdentity aclIdentity = new AclIdentity();
    aclIdentity.RepositoryName = repoName;
    aclIdentity.Domain = userName;
    aclIdentity.Name = aclName;

    Permission basicReadPermission = new Permission();
    basicReadPermission.Name = Permission.READ;
    basicReadPermission.Type = PermissionType.BASIC;

    Permission basicDeletePermission = new Permission();
    basicDeletePermission.Name = Permission.DELETE;
    basicDeletePermission.Type = PermissionType.BASIC;

    List<AclEntry> entryList = new List<AclEntry>();

    AclEntry aclEntry = new AclEntry();
    aclEntry.AccessType = AccessType.PERMIT;
    aclEntry.Accessor = "dm_world";
    List<Permission> permissionList = new List<Permission>();
    permissionList.Add(basicReadPermission);
    aclEntry.Permissions = permissionList;

    AclEntry aclEntry1 = new AclEntry();
    aclEntry1.AccessType = AccessType.PERMIT;
    aclEntry1.Accessor = "dm_owner";
    List<Permission> permissionList1 = new List<Permission>();
    permissionList1.Add(basicDeletePermission);
    aclEntry1.Permissions = permissionList1;

    entryList.Add(aclEntry);
    entryList.Add(aclEntry1);

    Acl acl = new Acl();
    acl.Identity = aclIdentity;
    acl.Description = "a test acl" + System.DateTime.Now.Ticks;
    // acl.setType(AclType.REGULAR); // defaults to REGULAR
    // acl.setVisibility(AclVisibility.PRIVATE); // defaults to PRIVATE
    acl.Entries = entryList;

    AclPackage aclPackage = new AclPackage();
    List<Acl> aclList = new List<Acl>();
    aclList.Add(acl);
    aclPackage.Acls = aclList;
```

```
    return accessControlService.Create(aclPackage);
}
```

2.2.4.4 get operation

The get operation retrieves an AclPackage containing ACL objects based on the ACL identities passed to the operation.

2.2.4.4.1 Java syntax

```
AclPackage get(List<AclIdentity> aclIdentities)

        throws CoreServiceException, ServiceException
```

2.2.4.4.2 C# syntax

```
AclPackage Get(List<AclIdentity> aclIdentities)
```

2.2.4.4.3 Example

Example 2-83: Java: Getting an ACL

```
public AclPackage getAcl(String repository, String domain,
String name)
    throws ServiceException
{
    AclIdentity aclIdentity = new AclIdentity();
    aclIdentity.setRepositoryName(repository);
    aclIdentity.setDomain(domain);
    aclIdentity.setName(name);
    List<AclIdentity> aclIdentityList = new ArrayList<AclIdentity>();
    aclIdentityList.add(aclIdentity);
    return accessControlService.get(aclIdentityList);
}
```

Example 2-84: C#: Getting an ACL

```
public AclPackage GetAcl(String repository, String domain,
String name)
{
    AclIdentity aclIdentity = new AclIdentity();
    aclIdentity.RepositoryName = repository;
    aclIdentity.Domain = domain;
    aclIdentity.Name = name;
    List<AclIdentity> aclIdentityList = new List<AclIdentity>();
    aclIdentityList.Add(aclIdentity);
    return accessControlService.Get(aclIdentityList);
}
```

2.2.4.5 update operation

The update operation updates a list of existing ACL objects based on the Acl instances contained in a AclPackage passed to the operation.

The update operation does not merge the data in an Acl instance into an existing ACL repository object. It will instead replace all of the attribute values in an ACL object based on the data in the Acl instance. Therefore, the best practice for using the update operation is to first retrieve the Acl instance using the get operation, make the necessary modifications, then pass the modified instance to the update operation.

2.2.4.5.1 Java syntax

```
AclPackage update(AclPackage aclPackage)
        throws CoreServiceException, ServiceException
```

2.2.4.5.2 C# syntax

```
AclPackage Update(AclPackage aclPackage)
```

2.2.4.5.3 Example

Example 2-85: Java: Updating an ACL

```
public AclPackage updateAcl(AclIdentity aclIdentity,
String userName)
    throws ServiceException
{
        List<AclIdentity> idList = new ArrayList<AclIdentity>();
        idList.add(aclIdentity);
     AclPackage aclPackage = accessControlService.get(idList);

     AclEntry aclEntry = new AclEntry();
     aclEntry.setAccessor(userName);
        Permission basicDeletePermission = new Permission();
        basicDeletePermission.setName(Permission.DELETE);
        basicDeletePermission.setType(PermissionType.BASIC);
        aclEntry.setAccessType(AccessType.PERMIT);
        List<Permission> permissionList = new ArrayList<Permission>();
        permissionList.add(basicDeletePermission);
        aclEntry.setPermissions(permissionList);

     Acl acl = aclPackage.getAcls().get(0);
     acl.getEntries().add(aclEntry);

     return accessControlService.update(aclPackage);
}
```

Example 2-86: C#: Updating an ACL

```
 public AclPackage UpdateAcl(AclIdentity aclIdentity,
    String userName)

    List<AclIdentity> idList = new List<AclIdentity>();
    idList.Add(aclIdentity);
    AclPackage aclPackage = accessControlService.Get(idList);
```

```
    AclEntry aclEntry = new AclEntry();
    aclEntry.Accessor = userName;
    Permission basicDeletePermission = new Permission();
    basicDeletePermission.Name = Permission.DELETE;
    basicDeletePermission.Type = PermissionType.BASIC;
    aclEntry.AccessType = AccessType.PERMIT;
    List<Permission> permissionList = new List<Permission>();
    permissionList.Add(basicDeletePermission);
    aclEntry.Permissions = permissionList;

    Acl acl = aclPackage.Acls[0];
    acl.Entries.Add(aclEntry);

    return accessControlService.Update(aclPackage);
}
```

2.2.4.6 delete operation

The delete operation deletes ACL objects based on a list of ACL identities passed to the operation.

2.2.4.6.1 Java syntax

```
void delete(List<AclIdentity> aclIdentities)
         throws CoreServiceException, ServiceException
```

2.2.4.6.2 C# syntax

```
void Delete(List<AclIdentity> aclIdentities)
```

2.2.4.6.3 Example

Example 2-87: Java: Deleting an ACL

```
public void deleteAcl(String repository, String domain,
    String name)
        throws ServiceException
{
    AclIdentity aclIdentity = new AclIdentity();
    aclIdentity.setRepositoryName(repository);
    aclIdentity.setDomain(domain);
    aclIdentity.setName(name);
    List<AclIdentity> aclIdentityList = new ArrayList<AclIdentity>();
    aclIdentityList.add(aclIdentity);
    accessControlService.delete(aclIdentityList);
 }
```

Example 2-88: C#: Deleting an ACL

```
public void DeleteAcl(String repository, String domain,
    String name)
{
     AclIdentity aclIdentity = new AclIdentity();
     clIdentity.RepositoryName = repository;
     clIdentity.Domain = domain;
```

```
     clIdentity.Name = name;
     ist<AclIdentity> aclIdentityList = new List<AclIdentity>();
     clIdentityList.Add(aclIdentity);
            accessControlService.Delete(aclIdentityList);
}
```

2.2.4.7 Applying an ACL to an object

To apply an ACL to a repository object, you can set the acl_domain and acl_name properties of the object to the domain and name of an existing ACL. Note, you can use a similar technique to perform other ACL related functions, such as assigning a default ACL to a folder or user.

Example 2-89: Java: Applying an ACL to a repository object

```
public void updateAcl(ObjectIdentity objectIdentity,
AclIdentity aclIdentity)
            throws ServiceException
{
    DataObject dataObject = new DataObject(objectIdentity, "dm_sysobject");
      dataObject.getProperties().set("acl_domain", aclIdentity.getDomain());
      dataObject.getProperties().set("acl_name", aclIdentity.getName());
      OperationOptions operationOptions = null;
      objectService.update(new DataPackage(dataObject), operationOptions);
}
```

Example 2-90: C#: Applying and ACL to a repository object

```
public void UpdateAcl(ObjectIdentity objectIdentity,
AclIdentity aclIdentity)
{
    DataObject dataObject = new DataObject(objectIdentity, "dm_sysobject");
    dataObject.Properties.Set("acl_domain", aclIdentity.Domain);
    dataObject.Properties.Set("acl_name", aclIdentity.Name);
    OperationOptions operationOptions = null;
    objectService.Update(new DataPackage(dataObject), operationOptions);
}
```

2.2.4.8 Querying for sets of ACLs

For a client to interact with a service in a loosely-coupled manner, it will likely need to cache data about available ACLs in a client-side object, rather than making repeated service calls to get data about individual ACLs or small sets of ACLs. This can be done with the Query service, but the client will need to take care of converting the data returned by the query into objects that are expected by the AccessControl service. This will require some familiarity with the attributes of the dm_acl repository object and how they map to data members in the AccessControl service data model.

The following example runs a PassthroughQuery to get all of the regular, non-system-created ACLs owned by a specific user, and returns the result as an AclPackage:

Example 2-91: Java: Getting an AclPackage based on a query

```
public AclPackage getOwnerPrivateAcls(String ownerName,
String repository)
        throws ServiceException
{
    // instantiate a Query service proxy
      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IQueryService queryService = null;
      if (remoteMode)
        {
            queryService = serviceFactory.getRemoteService(IQueryService.class,
                    accessControlService.getServiceContext());
        } else
        {
            queryService = serviceFactory.getLocalService(
                    IQueryService.class, accessControlService.getServiceContext());
        }

        // run the query
        PassthroughQuery query = new PassthroughQuery();
        query.setQueryString("select owner_name, object_name from dm_acl " +
            "where r_is_internal='0' and acl_class='0' and
        owner_name='" + ownerName + "'");
        query.addRepository(defaultRepositoryName);
        QueryExecution queryEx = new QueryExecution();
        queryEx.setCacheStrategyType(CacheStrategyType.DEFAULT_CACHE_STRATEGY);
        queryEx.setMaxResultCount(-1); // use server-defined limit
        OperationOptions operationOptions = null;
        QueryResult queryResult = queryService.execute(query, queryEx,
                operationOptions);
        System.out.println("QueryId == " + query.getQueryString());
        System.out.println("CacheStrategyType == "
                + queryEx.getCacheStrategyType());
        DataPackage resultDp = queryResult.getDataPackage();
        List<DataObject> dataObjects = resultDp.getDataObjects();
        System.out.println("Total objects returned is: " +
        dataObjects.size());

     //convert the results into a List<AclIdentity>
        List<AclIdentity> identityList = new ArrayList<AclIdentity>();
        for (DataObject dObj : dataObjects)
        {
            PropertySet docProperties = dObj.getProperties();
            AclIdentity aclIdentity = new AclIdentity();
            aclIdentity.setDomain(docProperties.get("owner_name").
            getValueAsString());
            aclIdentity.setName(docProperties.get("object_name").
            getValueAsString());
            aclIdentity.setRepositoryName(repository);
            identityList.add(aclIdentity);
        }

        // get and return the AclPackage
        return accessControlService.get(identityList);
}
```

Example 2-92: C#: Getting an AclPackage based on a query

```
public AclPackage GetOwnerPrivateAcls(String ownerName, String repository)
{
    // instantiate a Query service proxy
    ServiceFactory serviceFactory = ServiceFactory.Instance;
    IQueryService queryService = null;
```

```
    queryService = serviceFactory.GetRemoteService<IQueryService>(
            accessControlService.GetServiceContext());

    // build and run the query
    PassthroughQuery query = new PassthroughQuery();
    query.QueryString = "select owner_name, object_name
        from dm_acl " +
        "where r_is_internal='0' and acl_class='0' and
                owner_name='" + ownerName + "'";
    query.AddRepository(DefaultRepository);
    QueryExecution queryEx = new QueryExecution();
    queryEx.CacheStrategyType = CacheStrategyType.DEFAULT_CACHE_STRATEGY;
    queryEx.MaxResultCount = -1; // user server-defined limit
    OperationOptions operationOptions = null;
    QueryResult queryResult = queryService.Execute(query, queryEx,
            operationOptions);
    DataPackage resultDp = queryResult.DataPackage;
    List<DataObject> dataObjects = resultDp.DataObjects;
    Console.WriteLine("Total objects returned is: " + dataObjects.Count);

    //convert the results into a List<AclIdentity>
    List<AclIdentity> identityList = new List<AclIdentity>();
    foreach (DataObject dObj in dataObjects)
    {
        PropertySet docProperties = dObj.Properties;
        AclIdentity aclIdentity = new AclIdentity();
        aclIdentity.Domain = docProperties.Get("owner_name").
                GetValueAsString();
        aclIdentity.Name = docProperties.Get("object_name").
                GetValueAsString();
        aclIdentity.RepositoryName = repository;
        identityList.Add(aclIdentity);
    }

    // get and return the AclPackage
    return accessControlService.Get(identityList)
}
```

2.2.5 Lifecycle service

The Lifecycle service provides operations for runtime use of lifecycles, such as attaching objects to lifecycles, detaching objects from lifecycles, moving objects from one lifecycle state to another, and examining the lifecycles associated with objects.

This section provides some essential information about lifecycles, examines the object model and operations of the Lifecycle service, and discusses how to query the repository for data related to lifecycles using the Query service.

For more information about lifecycles and Documentum CM Server, see *(EDCCS250400-GGD)*. For more information about designing lifecycles, see *(EDCPC250400-UGD)*.

2.2.5.1 Understanding lifecycles

A lifecycle is defined by a business policy (a dm_policy object on the repository). The lifecycle determines:

- the states that an object can be in during its life
- the order in which the object can progress (or regress) through the states
- business rules that determine criteria for entering each state
- processes that occur when the object enters each state

Lifecycles are typically used in combination with workflows to implement business policies and procedures related to document management. Typically, lifecycles are created and updated at design-time using Documentum Composer (although it would be possible to build your own lifecycle editor using the Object service).

2.2.5.1.1 Lifecycle states and operations

Lifecycle states are of two types: normal states and exception states.

Normal states are ordered linearly, so that a document moves through the lifecycle in steps forward or backward. The first state in the sequence is called the *base state*. The *promote* operation moves an object forward from one state to the next. The *demote* operation moves a document backward from one state to the previous; or optionally a document can be demoted from any state back to the base state.

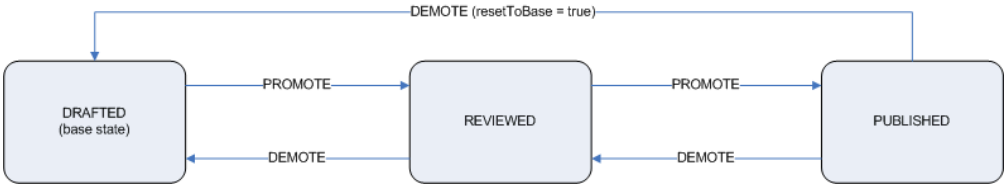

Figure 2-3: Normal states and transitions

Exception states add another direction of movement, which has only a single step. Each normal state can have an associated exception state. The *suspend* operation moves a document from a normal state to the associated exception state. The *resume* operation moves the object from the exception state to the associated normal state, or optionally back to the base state.

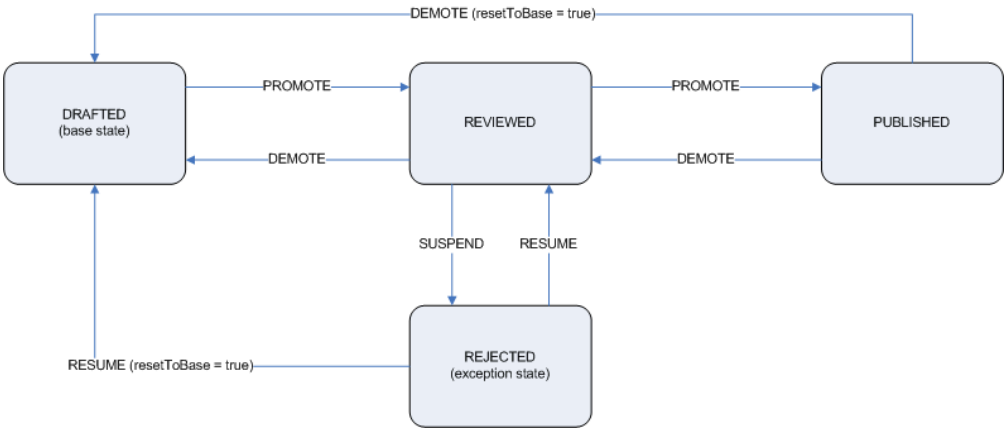

Figure 2-4: Lifecycle with normal states and an exception state

Lifecycle operations are constrained by properties associated with specific states. These properties are generally set at the time the lifecycle is designed. The allow_demote property must be true to permit the lifecycle service to demote an object from the state. The return_to_base property must be true to permit the lifecycle service to demote or resume an object to the base state. The lifecycle service specifies this option in the isResetToBase property of the LifecycleExecutionProfile.

Within the constrains defined by the lifecycle, the movements of the object through its lifecycle are controlled by the client application logic, and the application user.

2.2.5.1.2 Lifecycle primary type and subtypes

A lifecycle defines the repository types to which it can be applied. The lifecycle can have only one primary type (which must be a dm_sysobject or subtype). It can be applied either to all subtypes of the primary type, or to a subset of the subtypes of the primary type (which can be an empty subset). The attributes that specify the types are defined in the dm_policy object. The client application needs to be aware of these settings if it wants to validate whether an object can be attached to a lifecycle.

2.2.5.1.3 Default lifecycle of a type

A dm_sysobject or subtype has a default lifecycle (which can be null), defined in the Documentum CM Server data dictionary. This default lifecycle will be applied to an object by the Lifecycle service attach operation if no lifecycle is specified in the data passed to the operation.

Note: The default lifecycle is set in the dmi_dd_type_info object for the type and can be obtained using a DQL query as follows:

```
select default_policy_id from dmi_dd_type_info
where type_name = '<typename>'
```

2.2.5.1.4 Lifecycle attachment

In the repository, objects are associated with a lifecycle by two properties of dm_sysobject:

- r_policy_id – the object Id of the dm_policy object
- r_current_state, which is the state number of the current state of the object in the lifecycle

When we talk about attaching an object to a lifecycle, or applying a lifecycle to an object, we are really talking about setting these properties. r_policy_id is a single attribute, so an object can be attached to only one lifecycle.

The Lifecycle service attach operation calculates r_policy_id and r_current_state based on information provided by the client. To use the Lifecycle service attach operation, the client program must uniquely identify both the lifecycle and the object to attach to the lifecycle, using instances of ObjectIdentity. The client must also specify the state name, if the attached object is to be placed in a state other than the base state. (See "transformJob operation" on page 347.) Note that the lifecycle service can place an object to a state only if the allow_attach property is set on that state.

2.2.5.1.5 Setting an object's lifecycle scope alias set

Note: Aliases, alias sets, alias scope, and alias scope resolution are fairly complex topics that are outside the domain of this document. For more information on aliases, see *(EDCCS250400-GGD)*.

A lifecycle definition can reference one or more alias sets, specifically in the alias_set_ids repeating attribute of the dm_policy object. When an object is attached to a lifecycle, Documentum CM Server chooses one of the alias sets (if any are listed) in the lifecycle definition as the alias set to use in the lifecycle scope to resolve any aliases found in the attached object's properties; a reference to the chosen alias set is placed in the r_alias_set_id property of the attached object. In a non-workflow context, lifecycle scope is the first scope to be searched during alias resolution.

The Lifecycle service attach operation allows the client to explicitly determine which alias set is chosen as the lifecycle alias set. If the attach operation does not specify an alias set name, then Documentum CM Server will use its own logic to choose an alias set from the alias_set_ids list. The methods used by Documentum CM Server to select the alias set from alias_set_ids, and to resolve aliases using the lifecycle scope and other scopes, are described in *(EDCCS250400-GGD)*.

2.2.5.2 Classes used by the Lifecycle service

The following classes are used as data objects by the Lifecycle service:

Class name	Description
LifecycleInfo	Provides information required to attach an object to a lifecycle and state.
AttachLifecycleInfo	The AttachLifecycleInfo class contains all the information required to attach a lifecycle to a document.
LifecycleOperation	Contains data prescribing a lifecycle promote, demote, suspend, or resume operation.
LifecycleExecutionProfile	Contains data setting behavior options for the execute operation.

2.2.5.2.1 LifecycleInfo

The LifecycleInfo class contains information about the current lifecycle and state to which an object is attached.

Field name	Data type	Description
objectId	ObjectIdentity	Uniquely identifies the object whose lifecycle information is represented by this instance.
policyId	ObjectIdentity	Uniquely identifies the dm_policy object (that is, the lifecycle) to which the object is attached.
policyName	String	The name of the lifecycle to which this object is attached.
stateName	String	The name of the current lifecycle state of the object.
stateLabel	String	A string representing the state name for display and localization.
enabledOperations	List<LifecycleOperation>	Specifies the lifecycle operations that are available based on the object's current lifecycle state.

2.2.5.2.2 AttachLifecycleInfo

An instance of AttachLifecycleInfo contains data prescribing attachment of an object to a lifecycle.

Field name	Data type	Description
objectId	ObjectIdentity	Uniquely identifies the object that is to be attached to the lifecycle. This value cannot be null.
policyId	ObjectIdentity	Uniquely identifies the lifecycle (dm_policy instance) to which the object is to be attached. If this value is null, the object will be attached to the default lifecycle of the object type.
policyScope	String	The name of an alias set listed in the alias_set_ids repeating attribute of the dm_policy object. This determines the alias_set used for lifecycle scope resolution of aliases in the attached object. If policyScope is null, Documentum CM Server will determine the object's lifecycle scope.
stateName	String	A String representing the state in which to place the object on attachment to the lifecycle. If stateName is null, the object will be placed in the base state of the lifecycle. If an attach operation attempts is not able to set the object to this lifecycle state, an exception is thrown.

2.2.5.2.3 LifecycleOperation

The LifecycleOperation class models a lifecycle promote, demote, suspend, or resume operation and provides information required to execute that operation on a repository object. The lifecycle state name is not provided: this value is calculated based on the current state of the lifecycle and the specific operation to be executed.

Field name	Data type	Description
name	String	The operation name, which should be set to one the LifecycleOperation static fields: PROMOTE, DEMOTE, SUSPEND, or RESUME.
objectId	ObjectIdentity	The identity object on which to execute the lifecycle operation.
label	String	A string representing the operation for display and localization.

2.2.5.2.4 LifecycleExecutionProfile

An instance of the LifecycleExecutionProfile class is passed in an operationOptions parameter to specify behavior options to the execute operation.

Field name	Data type	Description
isBypassEntryCriteria	Boolean	If this value is true, the operation will bypass any entry criteria defined by a state and force promotion into that state. Has no effect on a DEMOTE operation.
isResetToBase	Boolean	If true, will cause a DEMOTE or RESUME operation to set the object lifecycle state to the base lifecycle state. For this to work, the return_to_base property of the current state must be set to true.
testOnly	Boolean	If true, the execute operation will not change the object state, but will only check the possibility of performing the operation. Not valid for DEMOTE operations. An exception is thrown if the operation would have been unsuccessful.

2.2.5.3 attach operation

The attach operation processes a collection of AttachLifecycleInfo objects, each of which specifies a lifecycle and an object to attach to the lifecycle. The operation can specify the lifecycle state that an object will be placed in when the object is attached to the lifecycle. If no state is specified, the object is placed in the lifecycle's base state. If no lifecycle is specified, the object is attached to the default lifecycle of the object type. For an object to be attached to a state, the allow_attach property of the state must be set to true. (This property would normally be set at design time by the creator of the lifecycle.)

The attach operation can also set the lifecycle alias scope (also called policy scope) of the object, by specifying the alias set name of an alias listed in the lifecycle's alias_set_ids attribute. If no alias set name is specified by the attach operation, thenDocumentum CM Server logic determines the lifecycle alias scope for the object.

2.2.5.3.1 Java syntax

```
public void attach(List<AttachLifecycleInfo> lifecycleInfos,
OperationOptions options)
        throws CoreServiceException, ServiceException
```

2.2.5.3.2 C# syntax

```
public void Attach(List<AttachLifecycleInfo> lifecycleInfos,
OperationOptions options)
```

2.2.5.3.3 Parameters

Parameter	Data type	Description
lifecycleInfos	List<AttachLifecycleInfo>	Each LifeCycleInfo instance provides information required to attach an object to a lifecycle and state.
options	OperationOptions	Reserved for future use.

2.2.5.3.4 Example

The following example attaches a single object to a lifecycle:

Example 2-93: Java: Attaching an object to a lifecycle

```
public void attachLifecycle(ObjectIdentity objId, ObjectIdentity
policyId, String aliasSetName)
            throws ServiceException
{
    AttachLifecycleInfo attachLcInfo = new AttachLifecycleInfo();

    // this must be the name of an alias set listed in
    // the alias_set_ids attribute of the dm_policy (lifecycle) object
    // if null or empty the Documentum Server will set the policy scope
    attachLcInfo.setPolicyScope(aliasSetName);
```

```
    attachLcInfo.setPolicyId(policyId);
    attachLcInfo.setObjectId(objId);
    OperationOptions operationOptions = null;
    List<AttachLifecycleInfo> attachLcInfoList =
    new ArrayList<AttachLifecycleInfo>();
    attachLcInfoList.add(attachLcInfo);

    lifecycleService.attach(attachLcInfoList, operationOptions);
}
```

Example 2-94: C#: Attaching an object to a lifecycle

```
public void AttachLifecycle(ObjectIdentity objId,
                            ObjectIdentity policyId,
                            String aliasSetName)
{
   AttachLifecycleInfo attachLcInfo = new AttachLifecycleInfo();

   // this must be the name of an alias set listed in
   // the alias_set_ids attribute of the dm_policy (lifecycle) object
   // if null or empty the Documentum Server will set the policy scope
   attachLcInfo.PolicyScope = aliasSetName;

   attachLcInfo.PolicyId = policyId;
   attachLcInfo.ObjectId = objId;
   OperationOptions operationOptions = null;
   List<AttachLifecycleInfo> attachLcInfoList =
        new List<AttachLifecycleInfo>();
   attachLcInfoList.Add(attachLcInfo);

   lifecycleService.Attach(attachLcInfoList, operationOptions);
}
```

2.2.5.4 detach operation

The detach operation processes a collection of objects, detaching each object from its lifecycle.

2.2.5.4.1 Java syntax

```
public void detach(ObjectIdentitySet objectIds,
OperationOptions options)
           throws CoreServiceException, ServiceException
```

2.2.5.4.2 C# syntax

```
public void Detach(ObjectIdentitySet objectIds,
OperationOptions options)
```

2.2.5.4.3 Parameters

Parameter	Data type	Description
objectIds	ObjectIdentitySet	A collection of objects to detach from any lifecycle to which they are currently attached.
operationOptions	OperationOptions	Reserved for future use.

2.2.5.4.4 Example

The following example detaches an object from its lifecycle:

Example 2-95: Java: Detaching an object from a lifecycle

```
public void detachLifecycle(ObjectIdentity objectIdentity)
throws ServiceException
{
    ObjectIdentitySet objIdSet = new ObjectIdentitySet(objectIdentity);
    OperationOptions operationOptions = null;
    lifecycleService.detach(objIdSet, operationOptions);
}
```

Example 2-96: C#: Detaching an object from a lifecycle

```
public void DetachLifecycle(ObjectIdentity objectIdentity)
{
    ObjectIdentitySet objIdSet = new ObjectIdentitySet(objectIdentity);
    OperationOptions operationOptions = null;
    lifecycleService.Detach(objIdSet, operationOptions);
}
```

2.2.5.5 execute operation

The execute operation processes a collection of LifecycleOperation objects, each of which specifies an object and a lifecycle operation to execute on that object.

2.2.5.5.1 Java syntax

```
public void execute(List<LifecycleOperation> lifecycleOperations,
OperationOptions options)
            throws CoreServiceException, ServiceException
```

2.2.5.5.2 C# syntax

```
public void Execute(List<LifecycleOperation> lifecycleOperations,
OperationOptions options)
```

2.2.5.5.3 Parameters

Parameter	Data type	Description
lifecycleOperations	List<LifecycleOperation>	Each LifecycleOperation instance specifies an object and a lifecycle operation to execute on that object. The LifecycleOperation does not specify the lifecycle state name. This value is calculated based on the current life cycle state and the specific operation to be executed.
options	OperationOptions	May contain a LifecycleExecution profile, which specifies specific operation behaviors. See "LifecycleExecutionProfile" on page 110.

2.2.5.5.4 Examples

The following demonstration promotion and demotion of a lifecycle to the base state:

Example 2-97: Java: Promoting a lifecycle

```
public void promoteLifecycle(ObjectIdentity objectIdentity)
throws ServiceException
{
    LifecycleOperation lifecycleOperation = new LifecycleOperation();
    lifecycleOperation.setName(LifecycleOperation.PROMOTE);
    lifecycleOperation.setLabel("Promote");
    lifecycleOperation.setObjectId(objectIdentity);

    List<LifecycleOperation> lcOperationsList = new ArrayList<LifecycleOperation>();
    lcOperationsList.add(lifecycleOperation);

    OperationOptions operationOptions = null;
    lifecycleService.execute(lcOperationsList, operationOptions);
}
```

Example 2-98: C#: Promoting a lifecycle

```
public void PromoteLifecycle(ObjectIdentity objectIdentity)
{
    LifecycleOperation lifecycleOperation = new LifecycleOperation();
    lifecycleOperation.Name = LifecycleOperation.PROMOTE;
    lifecycleOperation.Label = "Promote";
    lifecycleOperation.ObjectId = objectIdentity;

    List<LifecycleOperation> lcOperationsList = new List<LifecycleOperation>();
    lcOperationsList.Add(lifecycleOperation);

    OperationOptions operationOptions = null;
    lifecycleService.Execute(lcOperationsList, operationOptions);
}
```

Example 2-99: Java: Demoting a lifecycle and resetting to base state

```
public void demoteLifecycleToBase(ObjectIdentity objectIdentity)
throws ServiceException
{
    LifecycleOperation lifecycleOperation = new LifecycleOperation();
    lifecycleOperation.setName(LifecycleOperation.DEMOTE);
    lifecycleOperation.setLabel("Demote");
    lifecycleOperation.setObjectId(objectIdentity);

    LifecycleExecutionProfile lcExecProfile = new LifecycleExecutionProfile();
    lcExecProfile.setResetToBase(true);
    OperationOptions operationOptions = new OperationOptions();
    operationOptions.getProfiles().add(lcExecProfile);

    List<LifecycleOperation> lcOperationsList = new ArrayList<LifecycleOperation>();
    lcOperationsList.add(lifecycleOperation);

    lifecycleService.execute(lcOperationsList, operationOptions);
}
```

Example 2-100: C#: Demoting a lifecycle and resetting to base state

```
public void DemoteLifecycleToBase(ObjectIdentity objectIdentity)
{
    LifecycleOperation lifecycleOperation = new LifecycleOperation();
    lifecycleOperation.Name = LifecycleOperation.DEMOTE;
    lifecycleOperation.Label = "Demote";
    lifecycleOperation.ObjectId = objectIdentity;

    LifecycleExecutionProfile lcExecProfile = new LifecycleExecutionProfile();
    lcExecProfile.ResetToBase = true;
    OperationOptions operationOptions = new OperationOptions();
    operationOptions.Profiles.Add(lcExecProfile);

    List<LifecycleOperation> lcOperationsList = new List<LifecycleOperation>();
    lcOperationsList.Add(lifecycleOperation);

    lifecycleService.Execute(lcOperationsList, operationOptions);
}
```

2.2.5.6 getLifecycle operation

The getLifecycle operation processes a collection of ObjectIdentity instances and returns a collection of LifecycleInfo objects, each containing information about the lifecycle to which a specific object is attached.

2.2.5.6.1 Java syntax

```
public List<LifecycleInfo> getLifecycle(ObjectIdentitySet objectIds,
OperationOptions options)
        throws CoreServiceException, ServiceException
```

2.2.5.6.2 C# syntax

```
public List<LifecycleInfo> GetLifecycle(ObjectIdentitySet objectIds,
OperationOptions options)
```

2.2.5.6.3 Parameters

Parameter	Data type	Description
objectIds	ObjectIdentitySet	A collection of objects about which to obtain lifecycle information.
operationOptions	OperationOptions	Reserved for future use.

2.2.5.6.4 Returns

Returns a collection of LifecycleInfo objects, each of which provides information about the lifecycle to which the object in the objectIds collection is attached. See "LifecycleInfo" on page 108.

2.2.5.6.5 Example

The following sample gets lifecycle information about a single object. If a lifecycle has no attached, the LifecycleInfo will return the null ID (0000000000000000) as the ID of the dm_policy object.

Example 2-101: Java: Getting lifecycle information about an object

```
public void showLifecycleInfo(ObjectIdentity objectIdentity)
           throws ServiceException
{
    ObjectIdentitySet objIdSet = new ObjectIdentitySet(objectIdentity);
    List<LifecycleInfo> lcInfoList = lifecycleService.
    getLifecycle(objIdSet, null);
    LifecycleInfo lcInfo = lcInfoList.get(0);
    // if no lifecycle is attached, the policyId will have the null id
    if (lcInfo.getPolicyId().getValueAsString().equals("0000000000000000"))
    {
        System.out.println("No lifecycle attached to object.");
    } else
    {
        System.out.println("Lifecycle name is " + lcInfo.getPolicyName());
        System.out.println("Current state is " + lcInfo.getStateName());
        System.out.println("Available operations are:");
```

```
        for (LifecycleOperation lcOperation : lcInfo.
        getEnabledOperations())
        {
            System.out.println("    " + lcOperation.getName());
        }
    }
}
```

Example 2-102: C#: Getting lifecycle information about an object

```
public void ShowLifecycleInfo(ObjectIdentity objectIdentity)
{
    ObjectIdentitySet objIdSet = new ObjectIdentitySet
        (objectIdentity);
    List<LifecycleInfo> lcInfoList = lifecycleService.
        GetLifecycle(objIdSet, null);
    LifecycleInfo lcInfo = lcInfoList[0];
    // if no lifecycle is attached, the policyId will have the null id
    if (lcInfo.PolicyId.GetValueAsString().Equals("0000000000000000"))
    {
        Console.WriteLine("No lifecycle attached to object.");
    }
    else
    {
        Console.WriteLine("Lifecycle name is " + lcInfo.PolicyName);
        Console.WriteLine("Current state is " + lcInfo.StateName);
        Console.WriteLine("Available operations are:");
        foreach (LifecycleOperation lcOperation in lcInfo.
                EnabledOperations)
        {
            Console.WriteLine("    " + lcOperation.Name);
        }
    }
}
```

2.2.5.7 Querying for lifecycle properties

Most client applications will need to query the repository for information about lifecycles. For example, it may need to display a list of available lifecycles to a user, or it may need to validate whether a lifecycle can be attached to a specific state, or whether a lifecycle operation is permitted by a lifecycle state. You can use the Query service to obtain information about lifecycles. To formulate the query and to make use of the returned data, you will need to understand the properties of the dm_policy object. For more information about the dm_policy object, see *(EDCCS250400-ORD)*.

Example 2-103: Java: Querying for lifecycle information

```
public DataPackage getLifecycles(String ownerName, String repository)
        throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    IQueryService queryService = null;
    if (remoteMode)
    {
        queryService = serviceFactory.getRemoteService(IQueryService.class,
                    lifecycleService.getServiceContext());
    } else
```

```
    {
        queryService = serviceFactory.getLocalService(
                IQueryService.class, lifecycleService.getServiceContext());
    }

    PassthroughQuery query = new PassthroughQuery();

    // this query does not necessarily contain everything you will need
    // but shows most of the lifecycle-related properties on dm_policy
    query.setQueryString("select r_object_id, " +
                             "object_name, " +
                             "acl_name, " +
                             "included_type, " +
                             "include_subtypes, " +
                             "state_name, " +
                             "state_description, " +
                             "state_class, " +
                             "r_resume_state, " +
                             "r_current_state, " +
                             "entry_criteria_id, " +
                             "user_criteria_id, " +
                             "action_object_id, " +
                             "user_action_id, " +
                             "exception_state, " +
                             "allow_attach, " +
                             "allow_schedule, " +
                             "return_to_base, " +
                             "allow_demote, " +
                             "alias_set_ids, " +
                             "return_condition " +
                             "from dm_policy");
    query.addRepository(defaultRepositoryName);
    QueryExecution queryEx = new QueryExecution();
    queryEx.setCacheStrategyType(CacheStrategyType.DEFAULT_CACHE_STRATEGY);
    queryEx.setMaxResultCount(-1); // use server-defined limit
    OperationOptions operationOptions = null;
    QueryResult queryResult = queryService.execute(query, queryEx,
                operationOptions);
    return queryResult.getDataPackage();
}
```

Example 2-104: C#: Querying for lifecycle information

```
public DataPackage GetLifecycles(String ownerName, String repository)
{
    ServiceFactory serviceFactory = ServiceFactory.Instance;
    IQueryService queryService = null;

    queryService = serviceFactory
        .GetRemoteService<IQueryService>(lifecycleService.
                GetServiceContext());

    PassthroughQuery query = new PassthroughQuery();

    // this query does not necessarily contain everything you will need
    // but shows most of the lifecycle-related properties on dm_policy
    query.QueryString = "select r_object_id, " +
                        "object_name, " +
                        "acl_name, " +
                        "included_type, " +
                        "include_subtypes, " +
                        "state_name, " +
                        "state_description, " +
                        "state_class, " +
                        "r_resume_state, " +
                        "r_current_state, " +
```

```
                        "entry_criteria_id, " +
                        "user_criteria_id, " +
                        "action_object_id, " +
                        "user_action_id, " +
                        "exception_state, " +
                        "allow_attach, " +
                        "allow_schedule, " +
                        "return_to_base, " +
                        "allow_demote, " +
                        "alias_set_ids, " +
                        "return_condition " +
                        "from dm_policy";
    query.AddRepository(repository);
    QueryExecution queryEx = new QueryExecution();
    queryEx.CacheStrategyType = CacheStrategyType.DEFAULT_CACHE_STRATEGY;
    queryEx.MaxResultCount = -1; // user server-defined limit
    OperationOptions operationOptions = null;
    QueryResult queryResult = queryService.Execute(query, queryEx,
                operationOptions);
    return queryResult.DataPackage;
}
```

2.2.6 Schema service

The Schema service provides operations for retrieving information about repository schemas. A schema is a formal definition of repository metadata, including types, properties, and relationships. For the current release only the DEFAULT repository schema is supported, which provides metadata information concerning the data dictionary. In future releases a repository will potentially have an arbitrary number of named schemas. The Schema service can be used for creating a data structure against which a client can perform offline validation of objects against repository metadata.

2.2.6.1 Common schema classes

The following sections describe common descriptor classes used by the Schema service:

2.2.6.1.1 TypeInfo

The TypeInfo class is a descriptor for repository object types. For more information on the types, see *(EDCCS250400-ORD)*.

Property	Data type	Description
getName setName	String	The name of the repository object type.
getDescription setDescription	String	A description of the repository object type.
getLabel setLabel	String	The localized displayed string for the type name.

Property	Data type	Description
getDisplayInfos setDisplayInfos	List<DisplayInfo>	Information to display a summary of the repository object type, which consists of its name and a List of its attribute names.
getParentName setParentName	String	The name of the parent type of this repository object type.
getPropertyInfos setPropertyInfos	List<PropertyInfo>	A List of PropertyInfo objects describing the properties of this repository object type. See "PropertyInfo" on page 120.
getRelationshipInfos setRelationshipInfos	List<RelationshipInfo>	A list of RelationshipInfo objects indicating all the relationships in which this object type can participate. See "RelationshipInfo" on page 122.

2.2.6.1.2 PropertyInfo

The PropertyInfo class is a descriptor for a repository property (also called attribute).

Field	Field type	Description
name	String	The property name.
description	String	Description of the property.
helpText	String	Help text to display in UI for this property.
searchOperations	List<SearchOperation>	A List of search operations allowed against this Property. Refer to the Javadoc for documentation of the PropertyInfo.SearchOperation enum constants.
dataType	DataType	The repository data type of this Property. Possible values are BOOLEAN, CUSTOM, DATE, DOUBLE, INTEGER, LONG, SHORT, OBJECT_ID, STRING.

Field	Field type	Description
defaultSearchOperation	SearchOperation	The default search operation to use against this Property. Refer to the Javadoc for documentation of the SearchOperation enum constants.
length	int	Maximum length for string properties. Undefined for all other data types.
label	String	Localized display string for the property name.
defaultValues	ArrayProperty	Default values for this property. (These are the actual, raw values.)
dependencies	List<String>	List of property names that this property depends on (in terms of their values), if isDynamic is true.
valueAssist	ValueAssist	Provides default value assistance, that is, values to display to enable user to select a value for this property in a dialog box control. These values are provide for both static value assist (a fixed list of values), or dynamic value assist (values derived from the values of other properties). If isDynamic = true,, then the value assist is dynamic, and getValueAssist provides default values. For information on getting dynamic values, see "getDynamicAssistValues operation" on page 133.
valueMap	List<ValueInfo>	A map of possible values for this property onto localizable display strings. This data can be cached and used to look up display strings for values obtained from the getDynamicAssistValues operation.
isArray	boolean	True if multiple values are allowed. In repository terms this is a repeating attribute.

Field	Field type	Description
isDynamic	boolean	If true, value assistance for this property is obtained dynamically based on a query or on the value of other attributes. For information on getting dynamic values, see "getDynamicAssistValues operation" on page 133.
isHidden	boolean	If true, property is to be hidden in the user interface.
isNotNull	boolean	If true, property cannot have null values.
isReadOnly	boolean	True if property is read-only.
isRequired	boolean	If true, user must provide this value for this property in the user interface (dialog box).
isSearchable	boolean	True if searches allowed on this property.

2.2.6.1.3 ValueInfo

A PropertyInfo instance stores a List<ValueInfo>. This List can be used to lookup the localizable display label representing the value if value assistance is available for the property.

Field	Field type	Description
value	Property	A Property instance that stores the raw value that functions as the key in the value map.
label	String	Localizable display label for the value.

2.2.6.1.4 RelationshipInfo

The RelationshipInfo is a descriptor that provides access to information about a Relationship defined by the underlying metadata of the schema. Relationship instances can be based on metadata stored using one of the following strategies:

- The implicit relationships folder and virtual document. These are hard-coded values passed as strings.
- Metadata stored in dm_relation_type.
- Metadata stored in dmc_relationship_def.

The following table shows RelationshipInfo fields:

Field	Field type	Description
name	String	The name of the relationship.
description	String	A description of the relationship.
label	String	Localizable display string for relationship name.
currentType	String	The name of the type that the relationship is resolved against.
currentTypeRole	String	Role of the current type.
targetType	String	The repository object type of the source object in the relationship. Any object that participates in the relationship must be of this type or a subtype of this type.
targetTypeRole	String	Role that target type can play in this relationship.
degree	RelationshipDegree	An enum constant indicating the kind of mapping between the two terms of the relationship, with possible values of ONE_TO_ONE, ONE_TO_MANY, and MANY_TO_ONE. This data is available for relationships based on dmc_relationship_def only; otherwise it is null (in which case the Relationship is not completely defined).
propertyInfos	List<PropertyInfo>	Possible properties that can be set on the actual relationship object.

2.2.6.2 SchemaProfile

A SchemaProfile specifies categories of data returned by the Schema service. The following table describes the SchemaProfile fields:

Field	Description
isIncludeProperties	If true, return information regarding properties.
isIncludeValues	If true, return information regarding value assistance for properties.
isIncludeRelationships	If true, return information regarding relationships for a specified type.
isIncludeTypes	If true, return information regarding repository object types.
scope	A String value that specifies a scope setting that confines attributes returned to a subset delimited to a specific scope. Typically scope is a value designating an application, such as webtop.

2.2.6.3 getSchemaInfo operation

2.2.6.3.1 Description

Retrieves schema information for the default schema of the specified repository. (Named schemas will be supported in a future release.)

2.2.6.3.2 Java syntax

```
SchemaInfo getSchemaInfo(String repositoryName,
                         String schemaName,
                         OperationOptions operationOptions)
    throws CoreServiceException ServiceException
```

2.2.6.3.3 C# syntax

```
SchemaInfo GetSchemaInfo(String repositoryName,
                         String schemaName,
                         OperationOptions operationOptions)
```

2.2.6.3.4 Parameters

Parameter	Data type	Description
repositoryName	String	The name of the repository about which to obtain schema information.
schemaName	String	The name of the repository schema. If null or an empty string, examine the default repository schema.
operationOptions	OperationOptions	Contains profiles and properties that specify operation behaviors. In the case of this operation, a SchemaProfile can be passed to control the information returned.

2.2.6.3.5 Response

Returns a SchemaInfo instance containing the following information about a repository schema:

Field	Field type	Description
name	String	The name of the schema. Null if this is the default schema.
description	String	The description of the schema. Null if this is the default schema.
label	String	Default label for this schema. Null if this is the default schema.
typeInfos	List<TypeInfo>	A list of TypeInfo instances showing the types defined in the schema/repository.

2.2.6.3.6 Example

Example 2-105: Java: Getting schema info

```
public SchemaInfo getSchemaInfo() throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    ISchemaService schemaSvc
        = serviceFactory.getRemoteService(ISchemaService.class, serviceContext);

    SchemaProfile schemaProfile = new SchemaProfile();
    schemaProfile.setIncludeTypes(true);
    serviceContext.setProfile(schemaProfile);

    SchemaInfo schemaInfo = schemaSvc.getSchemaInfo(defaultRepositoryName,
                                                    "DEFAULT",
                                                    null);
    System.out.println("Schema name is: " + schemaInfo.getName());
    System.out.println("Schema description is: " + schemaInfo.getDescription());
    System.out.println("Schema label is: " + schemaInfo.getLabel());
    List<TypeInfo> typeInfoList = schemaInfo.getTypeInfos();
    System.out.println("Printing schema type info:");
    for (TypeInfo typeInfo : typeInfoList)
    {
        System.out.println(typeInfo.getName());
    }
    return schemaInfo;
}
```

Example 2-106: C#: Getting schema info

```
public void SchemaInfoDemo()
{
    SchemaProfile schemaProfile = new SchemaProfile();
    schemaProfile.IncludeTypes = true;
    DemoServiceContext.SetProfile(schemaProfile);

    SchemaInfo schemaInfo = schemaService.GetSchemaInfo(DefaultRepository,
        "DEFAULT", null);
    Console.WriteLine("Schema name is: " + schemaInfo.Name);
    Console.WriteLine("Schema description is: " + schemaInfo.Description);
    Console.WriteLine("Schema label is: " + schemaInfo.Label);
    List<TypeInfo> typeInfoList = schemaInfo.TypeInfos;
    Console.WriteLine("Printing schema type info:");
    foreach (TypeInfo typeInfo in typeInfoList)
    {
        Console.WriteLine(typeInfo.Name);
    }
}
```

2.2.6.4 getRepositoryInfo operation

2.2.6.4.1 Description

Retrieves schema information about a repository specified by name, including a list of repository schemas. For the current release, only the DEFAULT repository schema is supported.

2.2.6.4.2 Java syntax

```
RepositoryInfo getRepositoryInfo(String repositoryName,
                                 OperationOptions operationOptions)
    throws CoreServiceException, ServiceException
```

2.2.6.4.3 C# syntax

```
RepositoryInfo GetRepositoryInfo(String repositoryName,
                                 OperationOptions operationOptions)
```

2.2.6.4.4 Parameters

Parameter	Data type	Description
repositoryName	String	Name of the repository to examine.
operationOptions	OperationOptions	Contains profiles and properties that specify operation behaviors. In the case of this operation, a SchemaProfile can be passed to control the information returned.

2.2.6.4.5 Response

Returns a RepositoryInfo descriptor object containing the following data:

Field	Field type	Description
name	String	The name of the repository.
description	String	Description of the repository.
label	String	Localizable display string for the repository name.
schemaNameList	List<String>	A list of the repository schemas.
defaultSchemaName	List<String>	The name of the default schema. Typically the value is “DEFAULT”.

2.2.6.4.6 Example

Example 2-107: Java: Getting repository info

```
public RepositoryInfo getRepositoryInfo() throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    ISchemaService schemaSvc
        = serviceFactory.getRemoteService(ISchemaService.class, serviceContext);

    SchemaProfile schemaProfile = new SchemaProfile();
    schemaProfile.setIncludeTypes(true);
    serviceContext.setProfile(schemaProfile);

    OperationOptions operationOptions = null;
    RepositoryInfo repositoryInfo = schemaSvc.
        getRepositoryInfo(defaultRepositoryName,
                          operationOptions);
    System.out.println("Name: " + repositoryInfo.getName());
    System.out.println("Default schema name: " + repositoryInfo.
        getDefaultSchemaName());
    System.out.println("Label: " + repositoryInfo.getLabel());
    System.out.println("Description: " + repositoryInfo.getDescription());
    System.out.println("Schema names:");
    List<String> schemaList = repositoryInfo.getSchemaNames();
    for (String schemaName : schemaList)
    {
        System.out.println(schemaName);
    }

    return repositoryInfo;
}
```

Example 2-108: C#: Getting repository info

```
public RepositoryInfo RepositoryInfoDemo()
{
    OperationOptions operationOptions = new OperationOptions();
    RepositoryInfo repositoryInfo = schemaService.
        GetRepositoryInfo(DefaultRepository,
                                          operationOptions);

    Console.WriteLine(repositoryInfo.Name);
    Console.WriteLine("Default schema name: " + repositoryInfo.
        DefaultSchemaName);
    Console.WriteLine("Label: " + repositoryInfo.Label);
    Console.WriteLine("Description: " + repositoryInfo.Description);
    Console.WriteLine("Schema names:");
    List<String> schemaList = repositoryInfo.SchemaNames;
    foreach (String schemaName in schemaList)
    {
        Console.WriteLine(schemaName);
    }
    return repositoryInfo;
}
```

2.2.6.5 getTypeInfo operation

2.2.6.5.1 Description

The getTypeInfo operation returns information about a repository type specified by name.

2.2.6.5.2 Java syntax

```
TypeInfo getTypeInfo(String repositoryName,
                     String schemaName,
                     String typeName,
                     OperationOptions operationOptions)
        throws CoreServiceException, ServiceException
```

2.2.6.5.3 C# syntax

```
TypeInfo GetTypeInfo(String repositoryName,
                     String schemaName,
                     String typeName,
                     OperationOptions operationOptions)
```

2.2.6.5.4 Parameters

Parameter	Data type	Description
repositoryName	String	The name of the repository to examine.
schemaName	String	The name of the repository schema. For the current release set this value to DEFAULT or null.
typeName	String	The name of the type about which information is to be retrieved.
operationOptions	OperationOptions	Contains profiles and properties that specify operation behaviors. In the case of this operation, a SchemaProfile can be passed to control the information returned.

2.2.6.5.5 Response

Returns a TypeInfo instance with populated with information about the specified type. For details, see "TypeInfo" on page 119. For more information on the repository types, see *(EDCCS250400-ORD)*.

2.2.6.5.6 Example

Example 2-109: Java: Getting type info

```
public TypeInfo getTypeInfo() throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    ISchemaService schemaSvc
        = serviceFactory.getRemoteService(ISchemaService.class,
                    serviceContext);

    SchemaProfile schemaProfile = new SchemaProfile();
    schemaProfile.setIncludeProperties(true);
    schemaProfile.setIncludeValues(true);
    serviceContext.setProfile(schemaProfile);

    OperationOptions operationOptions = null;
    TypeInfo typeInfo = schemaSvc.getTypeInfo(defaultRepositoryName,
                                              null,
                                              "dm_document",
                                              operationOptions);
    System.out.println("Name: " + typeInfo.getName());
    System.out.println("Label: " + typeInfo.getLabel());
    System.out.println("Description: " + typeInfo.getDescription());
    System.out.println("Parent name : " + typeInfo.getParentName());
    List<PropertyInfo> propertyInfoList;
    propertyInfoList = typeInfo.getPropertyInfos();
    System.out.println("Properties: ");
    for (PropertyInfo propertyInfo : propertyInfoList)
    {
        System.out.print("  " + propertyInfo.getName());
        System.out.println("  " + propertyInfo.
                getDataType().toString());
    }
    return typeInfo;
}
```

Example 2-110: C#: Getting type info

```
public TypeInfo TypeInfoDemo()
{
    SchemaProfile schemaProfile = new SchemaProfile();
    schemaProfile.IncludeProperties = true;
    schemaProfile.IncludeValues = true;
    DemoServiceContext.SetProfile(schemaProfile);

    OperationOptions operationOptions = null;
    TypeInfo typeInfo = schemaService.GetTypeInfo(DefaultRepository,
                                                  null,
                                                  "dm_document",
                                                  operationOptions);

    Console.WriteLine("Name: " + typeInfo.Name);
    Console.WriteLine("Label: " + typeInfo.Label);
    Console.WriteLine("Description: " + typeInfo.Description);
    Console.WriteLine("Parent name : " + typeInfo.ParentName);
    List<PropertyInfo> propertyInfoList;
    propertyInfoList = typeInfo.PropertyInfos;
```

```
    Console.WriteLine("Properties: ");
    foreach (PropertyInfo propertyInfo in propertyInfoList)
    {
        Console.WriteLine("  " + propertyInfo.Name);
        Console.WriteLine("  " + propertyInfo.DataType.ToString());
    }
    return typeInfo;
}
```

2.2.6.6 getPropertyInfo operation

2.2.6.6.1 Description

The getPropertyInfo operation returns data about a repository property specified by repository, schema, type, and name.

2.2.6.6.2 Java syntax

```
PropertyInfo getPropertyInfo(String repositoryName,

                             String schemaName,                             String
typeName,

                             String propertyName

                             OperationOptions operationOptions)

        throws CoreServiceException, ServiceException
```

2.2.6.6.3 C# syntax

```
PropertyInfo GetPropertyInfo(String repositoryName,

                             String schemaName,                             String
typeName,

                             String propertyName

                             OperationOptions operationOptions)
```

2.2.6.6.4 Parameters

Parameter	Data type	Description
repositoryName	String	The name of the repository to examine.
schemaName	String	The name of the repository schema. For the current release set this value to DEFAULT or null.
typeName	String	The name of the repository type in which information about this property is to be retrieved.

Parameter	Data type	Description
propertyName	String	The name of the repository property about which to retrieve information.
operationOptions	OperationOptions	Contains profiles and properties that specify operation behaviors. In the case of this operation, a SchemaProfile can be passed to control the information returned.

2.2.6.6.5 Response

Returns a PropertyInfo instance with populated with information about the specified property. For details, see "PropertyInfo" on page 120.

2.2.6.6.6 Example

Example 2-111: Java: Getting property info

```
public PropertyInfo demoGetPropertyInfo() throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    ISchemaService schemaSvc
        = serviceFactory.getRemoteService(ISchemaService.class,
                    serviceContext);

    OperationOptions operationOptions = null;
    PropertyInfo propertyInfo = schemaSvc.getPropertyInfo
        (defaultRepositoryName,
                                        null,
                                        "dm_document",
                                        "subject",
                                        operationOptions);
    System.out.println("Name: " + propertyInfo.getName());
    System.out.println("Label: " + propertyInfo.getLabel());
    System.out.println("Description: " + propertyInfo.getDescription());

    return propertyInfo;
}
```

Example 2-112: C#: Getting property info

```
public PropertyInfo DemoGetPropertyInfo()
{
    OperationOptions operationOptions = null;
    PropertyInfo propertyInfo = schemaService.GetPropertyInfo(DefaultRepository,
                                                              null,
                                                              "dm_document",
                                                              "subject",
                                                              operationOptions);
    Console.WriteLine("Name: " + propertyInfo.Name);
    Console.WriteLine("Label: " + propertyInfo.Label);
    Console.WriteLine("Description: " + propertyInfo.Description);

    return propertyInfo;
}
```

2.2.6.7 getDynamicAssistValues operation

2.2.6.7.1 Description

The getDynamicAssistValues operation retrieves information about dynamic value assistance for a specified repository property. Value assistance provides a list of valid values for a property, which are used to populate a pick list associated with a field on a dialog box. Dynamic value assistance uses a query or a routine to list possible values for an attribute, generally based on the values of other attributes, rather than a literal list. A value assist list (whether literal or dynamic) can be complete—meaning that no values for the property are valid other than those in the list, or incomplete—meaning that the user is allowed to provide values in addition to those in the list.

2.2.6.7.2 Java syntax

```
ValueAssist getDynamicAssistValues(String repositoryName,

String schemaName, String typeName, String propertyName, PropertySet propertySet,
OperationOptions operationOptions)
throws CoreServiceException, ServiceException
```

2.2.6.7.3 C# syntax

```
ValueAssist GetDynamicAssistValues(String repositoryName,

String schemaName, String typeName, String propertyName, PropertySet propertySet,
OperationOptions operationOptions)
```

2.2.6.7.4 Parameters

Parameter	Data type	Description
repositoryName	String	The name of the repository to examine.
schemaName	String	The name of the repository schema. For the current release set this value to DEFAULT or null.
typeName	String	The name of the repository type in which information about the property is to be retrieved.
propertyName	String	The name of the repository property about which to retrieve information.

Parameter	Data type	Description
operationOptions	OperationOptions	Contains profiles and properties that specify operation behaviors. In the case of this operation, a SchemaProfile can be passed to control the information returned.

2.2.6.7.5 Response

Returns a ValueAssist object containing data about any value assistance configured in the repository for the property in question.

Field	Field type	Description
values	List<String>	A List of the raw values to be used as value assistance.
isAllowUserValues	boolean	If true, this property allows users to add their own values, in addition to those provided by value assistance. If false, the user can choose only values that are provided by value assistance.

Notice that only the raw values for value assistance are returned. This is an optimization to minimize payload size for this operation. To look up the labels for the values, you can use the getPropertyInfo operation to retrieve and cache values locally for properties, then use the getValueMap method of the PropertyInfo object to look up the label on the relevant property, using the raw value returned in ValueAssist as a key.

2.2.6.7.6 Example

The following example shows basic usage of the getDynamicAssistValues operation:

Example 2-113: Java: Getting dynamic assist values

```
public void demoGetValueInfo() throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    ISchemaService schemaSvc
        = serviceFactory.getRemoteService(ISchemaService.class,
                    serviceContext);

    SchemaProfile schemaProfile = new SchemaProfile();
    schemaProfile.setIncludeValues(true);
    OperationOptions operationOptions = new OperationOptions();
    operationOptions.setSchemaProfile(schemaProfile);

    System.out.println("Printing value info:");
    ValueAssist valueAssist = schemaSvc.
        getDynamicAssistValues(defaultRepositoryName,
                                            null,
```

```
                                    "dm_document",
                                        "subject",
                                             null,
                                operationOptions);
    if (valueAssist == null)
    {
        System.out.println("valueAssist is null.");
        return;
    }
    for (String value : valueAssist.getValues())
    {
        System.out.println("  " + value);
    }
}
```

Example 2-114: C#: Getting dynamic assist values

```
public void DemoGetValueInfo()
{
    SchemaProfile schemaProfile = new SchemaProfile();
    schemaProfile.IncludeValues = true;
    OperationOptions operationOptions = new OperationOptions();
    operationOptions.SchemaProfile = schemaProfile;

    Console.WriteLine("Printing value info:");
    ValueAssist valueAssist = schemaService.
        GetDynamicAssistValues(DefaultRepository,
                                                null,
                                    "dm_document",
                                        "subject",
                                                null,
                                  operationOptions);
    if (valueAssist == null)
    {
        Console.WriteLine("valueAssist is null.");
        return;
    }
    foreach (String value in valueAssist.Values)
    {
        Console.WriteLine("  " + value);
    }
}
```

2.2.6.8 getValueAssistSnapshot operation

2.2.6.8.1 Description

The getValueAssistSnapshot operation retrieves a snapshot of value constraints for all object properties.

2.2.6.8.2 Java syntax

```
ValueAssistSnapshot
getValueAssistSnapshot(ValueAssistRequest request,PropertySet changedProperties,

                                        OperationOptions options)
 throws CoreServiceException
```

2.2.6.8.3 C# syntax

```
ValueAssistSnapshot
GetValueAssistSnapshot(ValueAssistRequest request,PropertySet changedProperties,

                                        OperationOptions options)
```

2.2.6.8.4 Parameters

Parameter	Data type	Description
request	ValueAssistRequest	Identifies the object for which value assistance is requested.
changedProperties	PropertySet	Property values that must be taken into consideration when evaluating value constraints. For ObjectIdentity, the default value null is used for type requests or repository object values.
OperationOptions	options	Contains profiles and properties that specify operation behaviors. In the case of this operation, a SchemaProfile can be passed to filter the properties returned.

2.2.6.8.5 Response

Returns a snapshot of the value constraints containing the following information:

Field	Field type	Description
repositoryName	String	Repository name of the object for the snapshot.
typeName	String	Type name of the object for the snapshot.
propertyInfos	List<PropertyInfo>	Property information about value assistance.

2.2.6.8.6 Example

The following example shows basic usage of the getValueAssistSnapshot operation:

Example 2-115: Java: Getting a snapshot of value constraints for objects with changed property settings

```
public void testGetValueAssistSnapshotOfObject()
            throws Exception {
          String fileName = "test";
          ObjectIdentity objIdentity = null;
          try {
            objIdentity = new ObjectIdentity(m_repName);
            DataObject dataObject = new DataObject
            (objIdentity, assistTypeName);
            PropertySet properties = dataObject.getProperties();
            properties.set("object_name", fileName);
            properties.set("str_attr", "TEST_ASSIST");
            properties.set("zz_depends", "Depends");
            properties.set(new StringArrayProperty("zz_repeat_atozmap",
                    new String[] { "c" }));
            this.logDebug("set the object attribute str_attr=TEST_ASSIT");
            this.logDebug("set the object attribute zz_depends=Depends");
            this.logDebug("set the object attribute zz_repeat_atozmap=c");
            DataObject result = m_objectservice.create(
                    new DataPackage(dataObject), null).getDataObjects().get(0);
            objIdentity = result.getIdentity();
            assertNotNull(objIdentity);
            getSchemaProfile().setIncludeValues(true);
            OperationOptions option = new OperationOptions();

            ValueAssistRequest<ObjectIdentity> var =
            new ValueAssistRequest<ObjectIdentity>(
                    objIdentity);
            var.setType(ValueAssistRequestType.OBJECT_IDENTITY);

            PropertySet propertySet = new PropertySet();
            option.getSchemaProfile().setPropertyFilter(
                    PropertyInfoFilter.NON_INHERITED);
            ValueAssistSnapshot vas = m_service.getValueAssistSnapshot(var,
                    propertySet, option);
            assertEquals(assistTypeName, vas.getTypeName());
            this.logDebug("Here are " + vas.getPropertyInfos().size()
                    + " properties");
            assertEquals(5, vas.getPropertyInfos().size());
            for (PropertyInfo pi : vas.getPropertyInfos()) {
                ValueAssist va = pi.getValueAssist();
                this.logDebug("property name:" + pi.getName());
                if (va != null)
                    this.logDebug(va.getValues().toString());
                if (pi.getName().equals("int_attr")) {

                    assertEquals(5, va.getValues().size());
                    assertEquals("0", pi.getDefaultValues().get(0));
                    assertEquals("2", va.getValues().get(0));
                }
                if (pi.getName().equals("zz_dynamap")) {

                    assertEquals(4, va.getValues().size());
                    assertEquals("A", va.getValues().get(0));

                }
                if (pi.getName().equals("zz_depends")) {

                    assertEquals(1, va.getValues().size());
                    assertEquals("Depends2", va.getValues().get(0));

                }
```

```
            }

            propertySet.set(new StringArrayProperty("zz_repeat_atozmap",
                    new String[] { "a" }));
            this.logDebug("setthe zz_repeat_atozamap={a}
            as changedPropetySet");
            propertySet.set(new StringArrayProperty("str_attr",
                    new String[] { "a" }));
            this.logDebug("setthe str_attr={a} as changedPropetySet");
            vas = m_service.getValueAssistSnapshot(var,
            propertySet, option);
            assertEquals(assistTypeName, vas.getTypeName());
            this.logDebug("Here are " + vas.getPropertyInfos().size()
                    + " properties");
            assertEquals(5, vas.getPropertyInfos().size());
            for (PropertyInfo pi : vas.getPropertyInfos()) {
                ValueAssist va = pi.getValueAssist();
                this.logDebug("property name:" + pi.getName());
                if (va != null)
                    this.logDebug(va.getValues().toString());
                if (pi.getName().equals("int_attr")) {

                    assertEquals(10, va.getValues().size());
                    assertEquals("0", pi.getDefaultValues().get(0));
                    assertEquals("1", va.getValues().get(0));
                }
                if (pi.getName().equals("zz_dynamap")) {

                    assertEquals(3, va.getValues().size());
                    assertEquals("X", va.getValues().get(0));

                }
                if (pi.getName().equals("zz_depends")) {

                    assertEquals(1, va.getValues().size());
                    assertEquals("Depends1", va.getValues().get(0));

                }

            }

        } catch (Exception e) {
            this.logFatalError(e.getMessage());

        } finally {
            if (objIdentity != null)
                m_objectservice
                        .delete(new ObjectIdentitySet(objIdentity),
                            null);
        }
}
```

Example 2-116: C#: Getting a snapshot of value constraints for objects with changed property settings

```
public void TestGetValueAssistSnapshotOfObject ()
        {
            String fileName = "test";
            ObjectIdentity objIdentity = null;
            try
            {
                objIdentity = new ObjectIdentity
                            (TestUtil.FirstRepositoryName);
                DataObject dataObject = new DataObject
                            (objIdentity, assistTypeName);
```

```
                PropertySet properties = dataObject.Properties;
                properties.Set("object_name", fileName);
                properties.Set("str_attr", "TEST_ASSIST");
                properties.Set("zz_depends", "Depends");
                properties.Set(new StringArrayProperty("zz_repeat_atozmap",
                                                        new String[] { "c" }));
                //this.logDebug("set the object attribute str_attr=TEST_ASSIT");
                //this.logDebug("set the object attribute zz_depends=Depends");
                //this.logDebug("set the object attribute zz_repeat_atozmap=c");
                DataObject result =
                objectService.Create(new DataPackage(dataObject),
                            null).DataObjects[0];
                objIdentity = result.Identity;
                Assert.NotNull(objIdentity);
                GetSchemaProfile().IncludeValues = true;
                OperationOptions option = new OperationOptions();

                ValueAssistRequest var =
                new ValueAssistRequest(objIdentity);
                var.Type = ValueAssistRequestType.OBJECT_IDENTITY;

                PropertySet propertySet = new PropertySet();
                option.SchemaProfile.PropertyFilter = PropertyInfoFilter.NON_INHERITED;
                ValueAssistSnapshot vas =

schemaService.GetValueAssistSnapshot(var,
                                                          propertySet, option);
                Assert.That(vas.TypeName, Is.EqualTo(assistTypeName));
                //            this.logDebug("Here are " + vas.getPropertyInfos().size()
                                            + " properties");
                Assert.That(vas.PropertyInfos.Count, Is.EqualTo(5));
                foreach (PropertyInfo pi in vas.PropertyInfos)
                {

                    ValueAssist va = pi.ValueAssist;
                    //                this.logDebug("property name:" + pi.getName());
                    if (va != null)
                        //                    this.logDebug(va.getValues().toString());
                        if (pi.Name.Equals("int_attr"))
                        {
                            Assert.That(va.Values.Count, Is.EqualTo(5));
                            Assert.That(pi.DefaultValues[0], Is.EqualTo("0"));
                            Assert.That(va.Values[0], Is.EqualTo("2"));
                        }
                    if (pi.Name.Equals("zz_dynamap"))
                    {
                        Assert.That(va.Values.Count, Is.EqualTo(4));
                        Assert.That(va.Values[0], Is.EqualTo("A"));


                    }
                    if (pi.Name.Equals("zz_depends"))
                    {

                        Assert.That(va.Values.Count, Is.EqualTo(1));
                        Assert.That(va.Values[0], Is.EqualTo("Depends2"));

                    }

                }

                propertySet.Set(new StringArrayProperty("zz_repeat_atozmap",
                        new String[] { "a" }));
                //            this.logDebug("setthe zz_repeat_atozamap={a}
                                            as changedPropetySet");
                propertySet.Set(new StringArrayProperty("str_attr",
                            new String[] { "a" }));
                //this.logDebug("setthe str_attr={a} as changedPropetySet");
                vas = schemaService.GetValueAssistSnapshot(var, propertySet, option);
                Assert.That(vas.TypeName, Is.EqualTo(assistTypeName));
```

```
                //this.logDebug("Here are " + vas.getPropertyInfos().size()
                            + " properties");
                Assert.That(vas.PropertyInfos.Count, Is.EqualTo(5));
                foreach (PropertyInfo pi in vas.PropertyInfos)
                {
                    ValueAssist va = pi.ValueAssist;
                    //                this.logDebug("property name:" + pi.getName());
                    if (va != null)
                        //                    this.logDebug(va.getValues().toString());
                        if (pi.Name.Equals("int_attr"))
                        {
                            Assert.That(va.Values.Count, Is.EqualTo(10));
                            Assert.That(pi.DefaultValues[0], Is.EqualTo("0"));
                            Assert.That(va.Values[0], Is.EqualTo("1"));

                        }
                    if (pi.Name.Equals("zz_dynamap"))
                    {
                        Assert.That(va.Values.Count, Is.EqualTo(3));
                        Assert.That(va.Values[0], Is.EqualTo("X"));

                    }
                    if (pi.Name.Equals("zz_depends"))
                    {

                        Assert.That(va.Values.Count, Is.EqualTo(1));
                        Assert.That(va.Values[0], Is.EqualTo("Depends1"));

                    }

                }

            }
            catch (Exception e)
            {
                Assert.Fail(e.Message);

            }
            finally
            {
                if (objIdentity != null)
                if (objIdentity != null)
                objectService.Delete(new ObjectIdentitySet
                            (objIdentity), null);
            }
}
```

2.2.7 Query service

The Query service provides an operation for executing general purpose queries (using Documentum Query Language) against a repository. Additional query functionality is provided by the Search service.

2.2.7.1 Query model

The Query class has two subclasses: StructuredQuery and PassthroughQuery. For Version 6, the Query service only accepts objects of class PassthroughQuery. Execution of a StructuredQuery is not supported.

2.2.7.2 QueryExecution

The QueryExecution class defines an object that is passed as an argument to the Query service, and which encapsulates settings that specify Query service behaviors. The following table summarizes the QueryExecution fields:

Field	Data Type	Description
queryId	String	Id of the query. This should be set to null for a new query or should be set to the queryId returned by the operation in the QueryResult for sequential processing of cached query results.
startingIndex	long	Specifies the position in the query results beginning at which to return data to the service client. Default value is 0. Normally used only in sequential processing of cached query results.
maxResultCount	int	Determines the maximum number of query results returned by the Query service. By default, the limit is set to 100. If this value is set to -1, the defined maximum limit is used, which is set in the `dfs-runtime.properties` file in the entry dfs.query_cache_policy.query_max_result. In Foundation SOAP API as shipped, this value is set to 500.

Field	Data Type	Description
maxResultPerSource	int	For Search service: the number of maximum number of results that can be returned to the client by any one of the managed or external repositories that are in the search scope. Not used by the Query service: should be set to -1. If set to the default (-1) there is no defined limit.
cacheStrategyType	CacheStrategyType	Specifies a service behavior for caching query results that can be sequentially processed in multiple service interactions. See "CacheStrategyType values" on page 142. Supported by Query service. Not supported by Search service in Foundation SOAP API version 6.

2.2.7.2.1 CacheStrategyType values

The following table describes the CacheStrategyType values:

CacheStrategyType value	Description
DEFAULT_CACHE_STRATEGY	The system default for caching query results, which is equal to NO_CACHE_STRATEGY.
BASIC_FILE_CACHE_STRATEGY	Cache query results on the remote file system. If the cached result does not exist the query is re-run.
BASIC_MEMORY_CACHE_STRATEGY	Cache query results in memory on the remote system. If the cached result does not exist the query is re-run.
NO_CACHE_STRATEGY	Do not cache query results, and void any previous query cache stored for this user and query.

2.2.7.3 PassthroughQuery

The PassthroughQuery type extends Query, and contains a queryString field that holds a DQL statement.

2.2.7.3.1 Example

Example 2-117: Java: PassthroughQuery

```
PassthroughQuery query = new PassthroughQuery();
query.setQueryString("select r_object_id, "
                     + "object_name from dm_cabinet";
query.setRepository(defaultRepositoryName);
```

Example 2-118: C#: PassthroughQuery

```
PassthroughQuery query = new PassthroughQuery();
query.QueryString = "select r_object_id, "
                    + "object_name from dm_cabinet";
query.AddRepository(DefaultRepository);
```

2.2.7.4 execute operation

2.2.7.4.1 Description

The execute operation runs a query against data in a repository and returns the results to the client as a QueryResult containing a DataPackage.

2.2.7.4.2 Java syntax

```
QueryResult execute(PassthroughQuery query,
                    queryExecution QueryExecution
                    OperationOptions operationOptions)
        throws ServiceException,
        QueryValidationException,
        CacheException
```

2.2.7.4.3 C# syntax

```
QueryResult execute(PassthroughQuery query,
                    queryExecution QueryExecution
                    OperationOptions operationOptions)
        throws ServiceException,
        QueryValidationException,
        CacheException
```

2.2.7.4.4 Parameters

Parameter	Data type	Description
query	PassthroughQuery	Contains a DQL statement that expresses the query.
queryExecution	QueryExecution	Object describing execution parameters.
operationOptions	OperationOptions	Contains profiles and properties that specify operation behaviors. In the case of the execute operation, the profiles primarily provide filters that modify the contents of DataPackage returned in the QueryResult.

2.2.7.4.5 Response

The execute operation returns a QueryResult, which contains:

- A queryId string matching the id in the query passed to the service. This aids the client in matching the query result to the query in batch operations.
- A DataPackage containing a DataObject for each repository object selected by the query. By default, each DataObject is contains a PropertySet and ObjectIdentity populated with the query results. This result can be modified by filter settings in profiles passed in OperationOptions.

The QueryResult object contains substantial additional information within a QueryStatus object, some of which is more relevant to the use of QueryResult in the Search service. For more information see "QueryResult" on page 306.

2.2.7.4.6 Basic PassthroughQuery

The following examples shows basic use of a PassthroughQuery. In this example the query result is not cached, and the entire result is returned to the client.

Example 2-119: Java: Executing a PassthroughQuery

```
public void basicPassthroughQuery()
    throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    IQueryService querySvc
        = serviceFactory.getRemoteService(IQueryService.class,
                                          serviceContext);
    PassthroughQuery query = new PassthroughQuery();
    query.setQueryString("select r_object_id, "
                         + "object_name from dm_cabinet");
    query.addRepository(defaultRepositoryName);
    QueryExecution queryEx = new QueryExecution();
    queryEx.setCacheStrategyType
        (CacheStrategyType.DEFAULT_CACHE_STRATEGY);
    OperationOptions operationOptions = null;
    QueryResult queryResult = querySvc.execute
```

```
        (query, queryEx, operationOptions);
    System.out.println("QueryId == " + query.
        getQueryString());
    System.out.println("CacheStrategyType == "
        + queryEx.getCacheStrategyType());
    DataPackage resultDp = queryResult.getDataPackage();
    List<DataObject> dataObjects = resultDp.getDataObjects();
    int numberOfObjects = dataObjects.size();
    System.out.println("Total objects returned is: "
        + numberOfObjects);
    for (DataObject dObj : dataObjects)
    {
        PropertySet docProperties = dObj.getProperties();
        String objectId = dObj.getIdentity().getValueAsString();
        String docName = docProperties.get("object_name").
                getValueAsString();
        System.out.println("Document " + objectId
                + " name is " + docName);
    }
}
```

Example 2-120: C#: Executing a PassthroughQuery

```
public void BasicPassthroughQuery()
{
    PassthroughQuery query = new PassthroughQuery();
    query.QueryString = "select r_object_id, "
                        + "object_name from dm_cabinet";
    query.AddRepository(DefaultRepository);
    QueryExecution queryEx = new QueryExecution();
    queryEx.CacheStrategyType = CacheStrategyType.
        DEFAULT_CACHE_STRATEGY;
    OperationOptions operationOptions = null;
    QueryResult queryResult = queryService.Execute
        (query, queryEx, operationOptions);
    Console.WriteLine("QueryId == " + query.QueryString);
    Console.WriteLine("CacheStrategyType == " +
        queryEx.CacheStrategyType);
    DataPackage resultDp = queryResult.DataPackage;
    List<DataObject> dataObjects = resultDp.DataObjects;
    int numberOfObjects = dataObjects.Count;
    Console.WriteLine("Total objects returned is: " +
        numberOfObjects);
    foreach (DataObject dObj in dataObjects)
    {
        PropertySet docProperties = dObj.Properties;
        String objectId = dObj.Identity.GetValueAsString();
        String docName = docProperties.Get("object_name").
                GetValueAsString();
        Console.WriteLine("Document " + objectId +
                " name is " + docName);
    }
}
```

2.2.7.4.7 Cached query processing

To process large result sets, the client can specify that they be cached on the remote system and process the query result sequentially in a loop. Each pass can examine a range of the query results determined by startingIndex position and maxQueryResultCount. When the startingIndex position is out of range, the execute operation will return a QueryResult containing zero DataObject instances.

Example 2-121: Java: Cached query

```
public void cachedPassthroughQuery()
    throws ServiceException
{
    ServiceFactory serviceFactory = ServiceFactory.getInstance();
    IQueryService querySvc =
        serviceFactory.getRemoteService(IQueryService.class,
                                        serviceContext);
    PassthroughQuery query = new PassthroughQuery();
    query.setQueryString("select r_object_id, "
                         + "object_name from dm_cabinet");
    query.addRepository(defaultRepositoryName);
    QueryExecution queryEx = new QueryExecution();
    OperationOptions operationOptions = null;
    queryEx.setCacheStrategyType(CacheStrategyType.
        BASIC_FILE_CACHE_STRATEGY);
    queryEx.setMaxResultCount(10);

    while (true)
    {
        QueryResult queryResult = querySvc.execute(query,
                                   queryEx,
                                   operationOptions);
        DataPackage resultDp = queryResult.getDataPackage();
        List<DataObject> dataObjects =
                resultDp.getDataObjects();
        int numberOfObjects = dataObjects.size();
        if (numberOfObjects == 0)
        {
            break;
        }
        System.out.println("Total objects returned is:
                " + numberOfObjects);
        for (DataObject dObj : dataObjects)
        {
            PropertySet docProperties = dObj.getProperties();
            String objectId = dObj.getIdentity().
                        getValueAsString();
            String cabinetName = docProperties.get
                    ("object_name")
                        .getValueAsString();
            System.out.println("Cabinet " + objectId
                        + " name is " + cabinetName);
        }
        queryEx.setStartingIndex(queryEx.
                getStartingIndex() + 10);
    }
}
```

Example 2-122: C#: Cached query

```
public void CachedPassthroughQuery()
{
    PassthroughQuery query = new PassthroughQuery();
```

```
    query.QueryString = "select r_object_id, "
                        + "object_name from dm_cabinet";
    query.AddRepository(DefaultRepository);
    QueryExecution queryEx = new QueryExecution();
    OperationOptions operationOptions = null;
    queryEx.CacheStrategyType = CacheStrategyType.
        BASIC_FILE_CACHE_STRATEGY;
    queryEx.MaxResultCount = 10;

    while (true)
    {
        QueryResult queryResult = queryService.Execute(query,
        queryEx,
        operationOptions);
        DataPackage resultDp = queryResult.DataPackage;
        List<DataObject> dataObjects = resultDp.DataObjects;
        int numberOfObjects = dataObjects.Count;
        if (numberOfObjects == 0)
        {
            break;
        }
        Console.WriteLine("Total objects returned is: "
                + numberOfObjects);
        foreach (DataObject dObj in dataObjects)
        {
            PropertySet docProperties = dObj.Properties;
            String objectId = dObj.Identity.GetValueAsString();
            String cabinetName =
            docProperties.Get("object_name").
                        GetValueAsString();
            Console.WriteLine("Cabinet " + objectId + " name is "
            + cabinetName);
        }
        queryEx.StartingIndex += 10;
    }
}
```

2.2.8 QueryStore service

The QueryStore service provides operations to handle queries such as:

- storing queries, with or without the corresponding results;
- listing saved queries;
- viewing a saved query, with the corresponding results, if any.

2.2.8.1 Saving queries

The QueryStore service allows you to save queries and to load saved queries. Saving a query means saving the query definition, but the result set can also be saved with the query, together with the query status. You cannot save the entire result set, you have to specify which results you want to save. A saved query can either be owned by one user or accessible to all users. Queries that are accessible to all users can be launched and edited by any user; however, it cannot be overwritten. If another user saves a saved query, a new SavedQuery object is created.

Note: Saved queries are private by default, to make them public, use the update operation of the Object service and give the “read” permission to the “dm_world” group.

2.2.8.2 Objects related to this service

This section briefly describes objects used by this service. For field-level information, refer to the Javadoc or Windows help.

2.2.8.2.1 SavedQuery

The SavedQuery class is used by the Query service as a container for a query that was executed and saved.

The definition of the saved query can either be a structured or a passthrough query. The query results and the query status, either global or per source, can be saved with the query definition.

The following table summarizes the SavedQuery fields:

Field	Data Type	Description
name	String	Specifies the name of the query.
description	String	Specifies the description as defined when the query was saved.
queryType	QueryType	Specifies the type of the query definition, possible values are: PASSTHROUGH, STRUCTURED or UNKNOWN.
savedWithResults	boolean	Specifies whether the query was saved with its results.
resultCount	int	Specifies the number of results.
richQuery	RichQuery	Specifies the query definition and its client properties.

2.2.8.2.2 RichQuery

The RichQuery class defines a query definition, either a structured or a passthrough query. To this query definition, it is possible to associate client custom properties and attributes that should be displayed for this query results.

2.2.8.2.3 SavedQueryFilter

SavedQueryFilter defines a filter that is used to restrict which saved queries to return according to their accessibility value. The two accessibility values are OWNED, to return only saved queries owned by the current user, or ALL, to return all saved queries (including the current user's personal saved queries).

2.2.8.3 saveQuery operation

This operation saves the specified query definition to the specified repository. It is not possible to specify where the saved query will be stored in the repository.

The result set can be saved with the query definition. In this case, the saved results correspond to the result identities and their metadata. Since results are cached, the system may relaunch the query to resolve result identities.

The saveQuery operation can also be used to update a saved query; that is you can decide to modify the query definition and save the query with this new definition, or you may want to save some of the results with the query. When updating a saved query, the query is re-executed.

2.2.8.3.1 Java syntax

```
ObjectIdentity saveQuery(DataObject object,
                         RichQuery query,
                         QueryExecution exec,
                         ObjectIdentitySet resultsIds,
                         OperationOptions options)
          throws ServiceException;
```

2.2.8.3.2 C# syntax

```
ObjectIdentity SaveQuery(DataObject object,
                         RichQuery query,
                         QueryExecution exec,
                         ObjectIdentitySet resultsIds,
                         OperationOptions options)
```

2.2.8.3.3 Parameters

This section describes the object parameter for the saveQuery operation. The Javadoc or Windows Help available under *<dfs-sdk>*\docs\ provide more information about the fields and methods.

Parameter	Data type	Description
object	DataObject	Specifies the repository to store the saved queries, the name (object_name attribute) and description (title attribute) of the saved query. If the saved query is updated, it is populated with the repository object identity of this saved query.
query	RichQuery	Specifies the query definition and its properties to store.
exec	QueryExecution	Specifies the query ID. This parameter is optional but it is required when you want to save the results (that is when resultsIds is not null) in order to resolve results identities.
resultsIds	ObjectIdentitySet	Specifies the identities of the query results, if any. If results are not saved, it is null.
options	OperationOptions	Not used in current Foundation SOAP API version, reserved for future use.

2.2.8.3.4 Response

Returns the identity of the saved query so that it can be used later to load the saved query.

2.2.8.3.5 Exceptions

The CoreServiceException exception is thrown if an error occurs.

2.2.8.3.6 Example

The following example demonstrates the operations of the QueryStore Service: saveQuery, loadSavedQuery and listSavedQueries.

In this example, a query is saved first with no results. It is then updated: the query is saved with the ten first results. Then the saved query is loaded and finally the ten first owned saved queries are listed.

Example 2-123: Java: Handling saved queries

```
public void saveNewQuery() throws Exception
    {
        try {
            String expectedName = "My Saved Query Name";
            String desc = "This Sample Saved Query searches
                    for Technical Specifications documents.";

            DataObject object = new DataObject
                    (new ObjectIdentity(MY_DOCBASE));
            PropertySet set = new PropertySet();
            set.set("object_name", expectedName);
            set.set("title", desc);
            object.setProperties(set);

            StructuredQuery query = buildQuery();
            RichQuery rQuery = buildRichQuery(query);
            QueryExecution queryExec = new QueryExecution(0,100,100);
            OperationOptionsAdapter options = new
                        OperationOptionsAdapter();
            QueryResult results = launchQuery(query, queryExec, options);
            queryExec.setQueryId(results.getQueryId());

            // Save the query with no results
            queryExec.setQueryId(results.getQueryId());
            ObjectIdentity objs = saveQuery(object, rQuery,
                    queryExec, null,
                    options);
            logMessage("Created SavedQuery #" +
                        GetIdUtil.getId(objs).getId());

            // Update the saved query
            object.setIdentity(objs);  // update the right SavedQuery
            object.getProperties().set("object_name",
                        "Updated SavedQuery");

            // Get the 10 first results ids
            ObjectIdentitySet resIdentities = getResultsIds(results.
            getDataPackage(),0,10);

            // Save the query with the 10 first results
            objs = saveQuery(object, rQuery, queryExec,
                        resIdentities, options);
            logMessage("Updated SavedQuery #" +
                        GetIdUtil.getId(objs).getId());

            // Load the saved query
            SavedQuery savedQuery = loadSavedQuery(objs,
                        new PagingInfo(0,10), null);
            logMessage("Loaded SavedQuery: " + savedQuery.getName() +
            " with "+ savedQuery.getResultCount() +" results.");

            // List the 10 first owned saved queries
            QueryExecution exec = new QueryExecution();
            exec.setStartingIndex(0);
            exec.setMaxResultCount(10);
            SavedQueryFilter filter = new SavedQueryFilter();
```

```
            filter.setAccessibility(SavedQueryAccessibility.OWNED);

            DataPackage savedQueries = listSavedQueries
                        (MY_DOCBASE, exec, filter);
                    logMessage("Listed "+ savedQueries.getDataObjects().size() +
             " Saved Queries.");
            for (DataObject r : savedQueries.getDataObjects())
            {
                System.out.println("SavedQuery [" + r.getIdentity()+ "]: " +
                r.getProperties().get(OBJECT_NAME_ATTRIBUTE));
            }
        }
        catch (Exception e)
        {
            e.printStackTrace();
            throw new RuntimeException(e);
        }
    }

    // ----------------------------------------
    //     Calls to the service
    // ----------------------------------------
    private DataPackage listSavedQueries (String repo,
    QueryExecution exec,
    SavedQueryFilter filter)
            throws ServiceException
    {
        return m_queryStoreService.listSavedQueries
                (repo, exec, filter, null);
    }


    private SavedQuery loadSavedQuery (ObjectIdentity objs,
    PagingInfo paging,
    OperationOptions options)
            throws ServiceException
    {
        return m_queryStoreService.loadSavedQuery(objs,
                paging, options);
    }


    private ObjectIdentity saveQuery (DataObject object,
    RichQuery rQuery,
    QueryExecution queryExec,
    ObjectIdentitySet resultsId,
    OperationOptionsAdapter options)
            throws ServiceException
    {
        return m_queryStoreService.saveQuery(object,
                rQuery,
                queryExec,
                resultsId,
                options);
    }


    public QueryResult launchQuery (Query query,
    QueryExecution queryExecution,
    OperationOptionsAdapter options)
        throws ServiceException
    {
        return m_searchService.execute(query,
             queryExecution, options);
    }



    // Utils
     private void logMessage (String message)
```

```
    {
        // Add your custom logging here.
        System.out.println(message);
    }

    private StructuredQuery buildQuery ()
    {
        StructuredQuery query = new StructuredQuery();
        query.setObjectType("dm_document");
        query.addRepository(MY_DOCBASE);
        ExpressionSet exprSet = new ExpressionSet();
        PropertyExpression propexpr = new
              PropertyExpression(OBJECT_NAME_ATTRIBUTE,
                Condition.CONTAINS,
                "Technical Specification");
        exprSet.addExpression(propexpr);
        query.setRootExpressionSet(exprSet);

        return query;
    }
    private RichQuery buildRichQuery (Query query)
    {
        RichQuery richQuery = new RichQuery();
        richQuery.setQuery(query);
        List<String> attrs = new ArrayList<String>();
        attrs.add(OBJECT_NAME_ATTRIBUTE);
        attrs.add(R_OBJECT_ID_ATTRIBUTE);
        attrs.add(R_MODIFY_DATE_ATTRIBUTE);
        attrs.add(R_CREATION_DATE_ATTRIBUTE);
        attrs.add(OWNER_NAME_ATTRIBUTE);
        richQuery.setDisplayedAttributes(attrs);
        return richQuery;
    }

    private ObjectIdentitySet getResultsIds (DataPackage dataPackage,
                                             int fromIndex,
                                             int toIndex)
    {
        final ObjectIdentitySet resIds = new ObjectIdentitySet();
        final List<DataObject> objects = dataPackage.getDataObjects();
        if (toIndex > objects.size()) {
            toIndex = objects.size();
        }
        if (fromIndex < objects.size()) {
            final List<DataObject> list = objects.subList
                    (fromIndex, toIndex);
            for (DataObject object:list) {
            resIds.addIdentity(object.getIdentity());
            }
        }
        return resIds;
    }

    private void initServices()
    {
        ServiceFactory serviceFactory =
              ServiceFactory.getInstance();
        try
        {
            if(remoteMode){
                m_searchService = serviceFactory.
                         getRemoteService(ISearchService.class,
                                          serviceContext);
                m_queryStoreService = serviceFactory.
                         getRemoteService(IQueryStoreService.class,
                                          serviceContext);
            }
            else{
                m_searchService = serviceFactory.
                         getLocalService(ISearchService.class,
```

```
                                                serviceContext);
                m_queryStoreService = serviceFactory.
                        getLocalService(IQueryStoreService.class,
                                                serviceContext);
            }
        }
        catch (ServiceInvocationException e)
        {
            e.printStackTrace();
            throw new RuntimeException(e);
        }
    }

    // Private Fields
    private IQueryStoreService m_queryStoreService ;
    private ISearchService m_searchService;


    // Constants
    private static final String MY_DOCBASE = "MyDocbase";

    private static final String OBJECT_NAME_ATTRIBUTE
        = "object_name";
    private static final String R_OBJECT_ID_ATTRIBUTE
        = "r_object_id";
    private static final String R_MODIFY_DATE_ATTRIBUTE
        = "r_modify_date";
    private static final String R_CREATION_DATE_ATTRIBUTE
        = "r_creation_date";
    private static final String OWNER_NAME_ATTRIBUTE
        = "owner_name";
```

2.2.8.4 listSavedQueries operation

The listSavedQueries operation provides a list of saved queries (SavedQuery objects), with the corresponding metadata, stored in the specified repository.

To get the result set for a query, use the loadSavedQuery operation.

2.2.8.4.1 Java syntax

```
DataPackage listSavedQueries(String repository,
                             QueryExecution execution,
                             SavedQueryFilter filter,
                             OperationOptions options)
                             throws ServiceException;
```

2.2.8.4.2 C# syntax

```
DataPackage ListSavedQueries(String repository,
                             QueryExecution execution,
                             SavedQueryFilter filter,
                             OperationOptions options)
```

2.2.8.4.3 Parameters

This section describes the object parameter for the listSavedQueries operation. The Javadoc or Windows Help available under *<dfs-sdk>*\docs\ provide more information about the fields and methods.

Parameter	Data type	Description
repository	String	The name of the managed repository where queries are stored.
execution	QueryExecution	Specifies the fields startingIndex and maxResultsCount to limit the number of SavedQuery objects to return.
filter	SavedQueryFilter	Specifies the filter to restrict the SavedQuery objects to return according to their accessibility value. "SavedQueryFilter" on page 149, provides more information about SavedQueryFilter objects.
options	OperationOptions	Reserved for future use.

2.2.8.4.4 Response

Returns a DataPackage that contains SavedQuery objects. Only the metadata associated to the SavedQuery object are returned; the loadSavedQuery operation should be used to get the entire set of results for this query.

2.2.8.4.5 Exceptions

The CoreServiceException exception is thrown when an error occurs while the specified repository is unreachable.

2.2.8.4.6 Example

The example "Example" on page 151, demonstrates the listSavedQueries operation.

2.2.8.5 loadSavedQuery operation

The loadSavedQuery operation loads a saved query with its metadata and the corresponding result set, if available.

This operation might be resource consuming depending on the number of saved results.

2.2.8.5.1 Java syntax

```
SavedQuery loadSavedQuery(ObjectIdentity savedQueryId,
                          PagingInfo pagingInfo,
                          OperationOptions options)
                     throws ServiceException;
```

2.2.8.5.2 C# syntax

```
SavedQuery LoadSavedQuery(ObjectIdentity savedQueryId,
                          PagingInfo pagingInfo,
                          OperationOptions options)
```

2.2.8.5.3 Parameters

This section describes the object parameter for the loadSavedQuery operation. The Javadoc or Windows Help available under <*dfs-sdk*>\docs\ provide more information about the fields and methods.

Parameter	Data type	Description
savedQueryId	ObjectIdentity	Specifies the unique identity of the SavedQuery object to load. It is the identity returned by the saveQuery operation.
pagingInfo	PagingInfo	Specifies the fields startIndex and pageSize to limit the number of results to return in a single call.
options	OperationOptions	Not used in current Foundation SOAP API version, reserved for future use.

2.2.8.5.4 Response

Returns a SavedQuery object with its status and the corresponding result set, if available.

2.2.8.5.5 Exceptions

The CoreServiceException exception is thrown if an error occurs, such as:

- The query definition cannot be read.
- The query status cannot be read.
- The query results cannot be read.

2.2.8.5.6 Example

The example "Example" on page 151, demonstrates the loadSavedQuery operation.

2.2.9 Virtual Document service

The VirtualDocumentService provides operations for managing virtual documents, such as modifying virtual documents by adding, removing, or reordering nodes, retrieving virtual documents from the repository, creating snapshots, and removing snapshots.

See also "Notes on checking in virtual documents" on page 82.

2.2.9.1 Understanding virtual documents

The following sections provide some basic conceptual information about virtual documents. For more information, see *(EDCCS250400-ORD)* and *(EDCPKCL250400-DGD)*.

2.2.9.1.1 What is a virtual document?

A virtual document is a hierarchically organized structure composed of component documents. The components of a virtual document are of type dm_sysobject, or a subtype of dm_sysobject (but excluding cabinets and folders). Most commonly, the components are of type dm_document or a subtype. The child components of a virtual document can be simple documents (that is, non-virtual documents), or they can themselves be virtual documents. Documentum CM Server does not impose any restrictions on the depth of nesting of virtual documents.

The root of a virtual document is version-specific and identified by an object identity (on Documentum CM Server an r_object_id). The child components of a virtual document are not version-specific, and are identified by an i_chronicle_id. The relationship between a parent component and its children are defined in containment objects (dmr_containment), each of which connects a parent object to a single child object. The order of the children of the parent object is determined by the order_no property of the containment object.

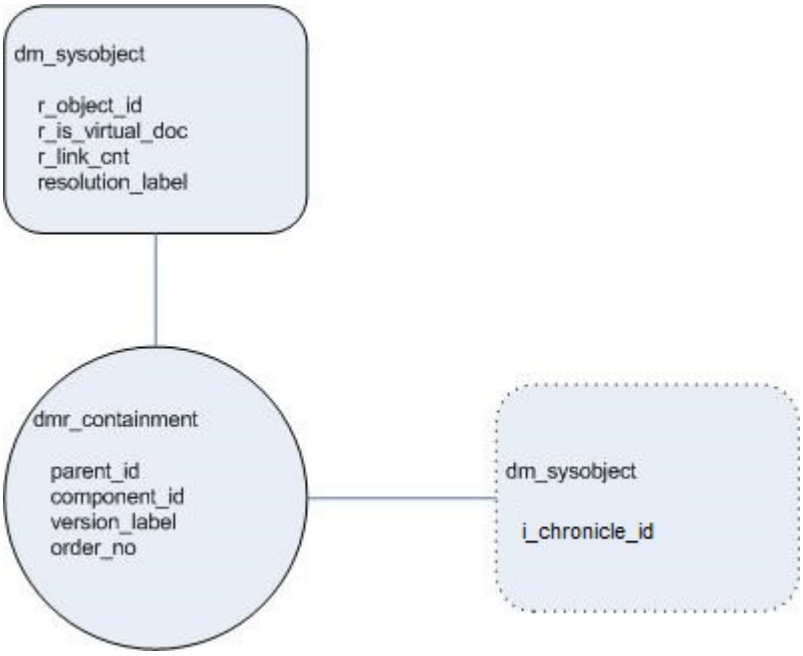

Figure 2-5: Containment object defines virtual document relationship

The version of the child component is determined at the time the virtual document is *assembled.* A virtual document is assembled when it is retrieved by a client, and when a snapshot of the virtual document is created. The assembly is determined at runtime by a binding algorithm governed by metadata set on the dmr_containment objects.

2.2.9.1.2 Use of virtual documents

Virtual documents provide a way to combine multiple documents in multiple formats into a single document. Each component exists as an independent object in the repository. Virtual documents allow users to:

- Share document components in multiple virtual documents to manage content redundancy. When a changed component is checked in, the change is reflected in all virtual documents that include the component.
- Combine different types of related content into the same document (as an organizational tool).
- Increase flexibility of user access (multiple users can simultaneously check out and maintain different parts of the virtual document).
- Save snapshots of the virtual document that reflect the state of all components at the time the snapshot is created.

For some content types, such as Microsoft Word files and XML files used in XML applications, virtual documents are patched as they are retrieved to a client, and

flattened into a single document. In other cases, the individual components of the virtual documents are retrieved as separate files.

2.2.9.1.3 Virtual document assembly and binding

A virtual document is assembled when it is retrieved by a client, and when a snapshot of the virtual document is created and stored in the repository.

2.2.9.1.4 Early and late binding

Each virtual document node can be early or late bound.

- In early binding, the binding label is set on the containment object when the node is created and stored persistently. (The binding level is stored in the version_label property of the dmr_containment object.)
- In late binding, the version of the node is determined at the time the virtual document is assembled, using a “preferred version” or late binding label passed at runtime. (If the version_label property of the dmr_containment object is empty or null, then the node is late bound.)

2.2.9.1.5 Binding rules and assembly logic

The logic that controls the assembly of the virtual document at the time it is retrieved is determined by settings on the containment objects.

API term	Documentum CM Server property	Description
binding	version_label	The early binding label of the virtual document node. If empty, then the node is late bound.
overrideLateBinding	use_node_vers_label	Override the late binding value for all descendants of this node, using the early bound label of this node.
includeBrokenBindings	none (provided by client API at runtime)	A broken binding occurs when there is no version label on the node corresponding to the lateBindingValue; if broken nodes are include, uses the CURRENT version of the node.

The following diagram shows the decision process when assembling a virtual document node:

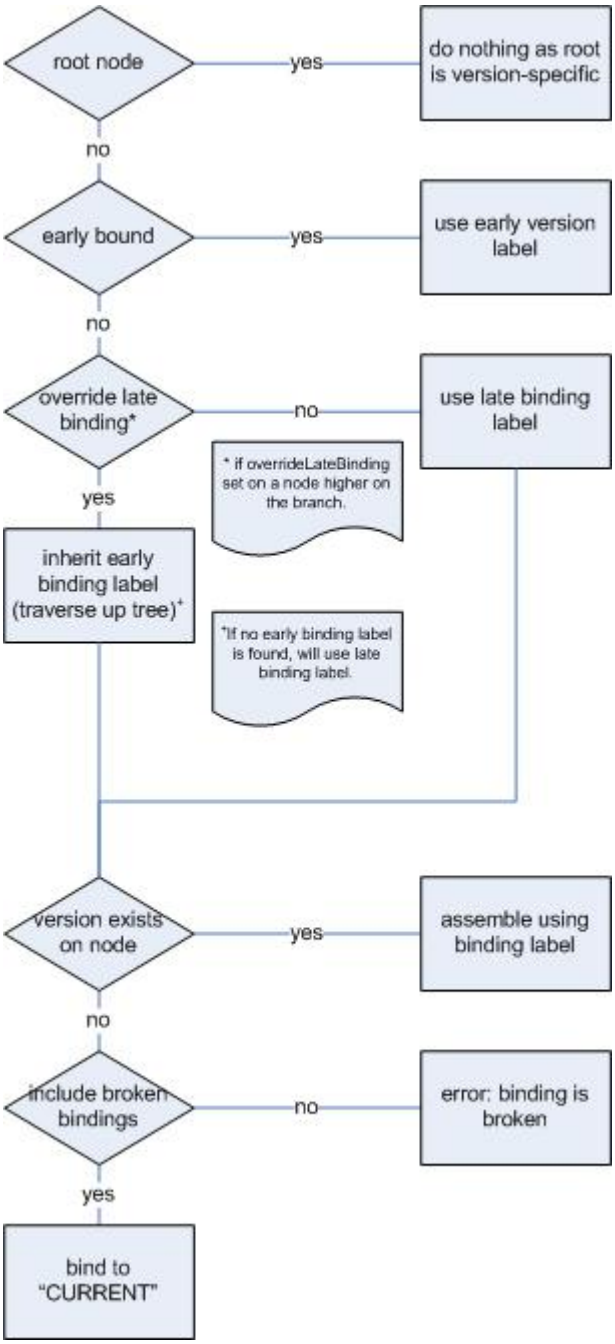

Figure 2-6: Assembly decision tree

2.2.9.1.6 Snapshots

Snapshots provide a way of persistently storing the results of virtual document assembly. The snapshot records the exact components of the virtual document at the time the snapshot was created, using version-specific object identities to represent each node.

Snapshots are stored in the repository as a set of assembly objects (dm_assembly) associated with a dm_sysobject. Each assembly object in a snapshot represents one node of the virtual document, and connects a parent document with a specific version of a child document.

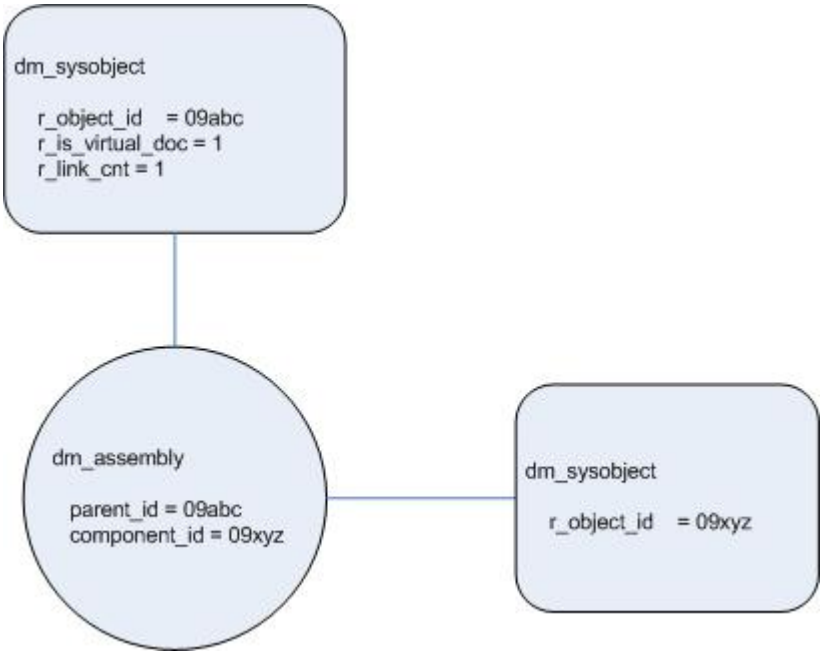

Figure 2-7: Assembly object defines assembly relationship

2.2.9.1.7 Inline and non-inline snapshots

There are two flavors of snapshots: inline and non-inline. A dm_assembly object (called an assembly) points to the virtual document from which the snapshot was derived using the book_id property.

In an inline snapshot, the root of the virtual document from which the snapshot is derived and the root of the snapshot are identical. A virtual document can have only one inline snapshot.

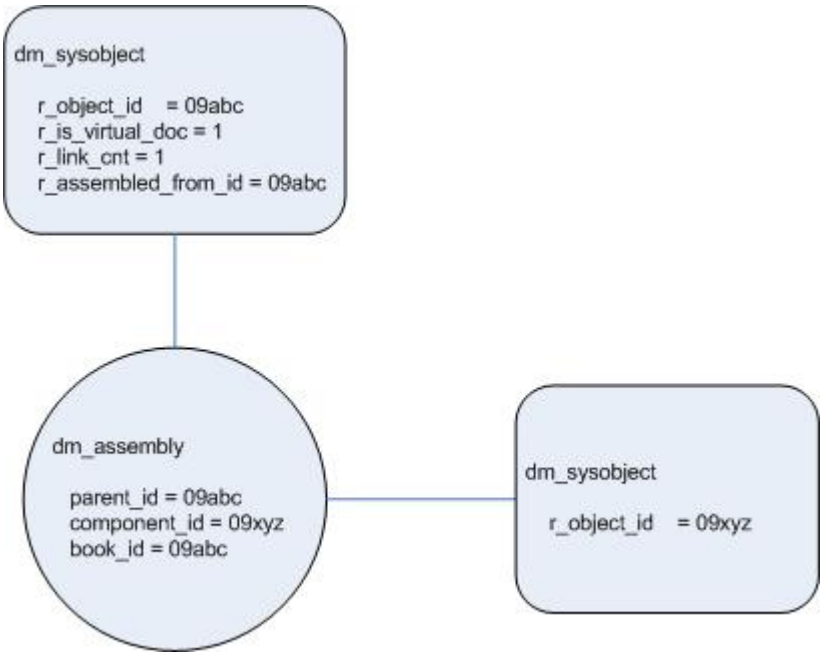

Figure 2-8: Inline snapshot

In a non-inline snapshot, the root of the virtual document from which the snapshot is derived and the root of the snapshot are not identical. You can make multiple non-inline snapshots of the same virtual document.

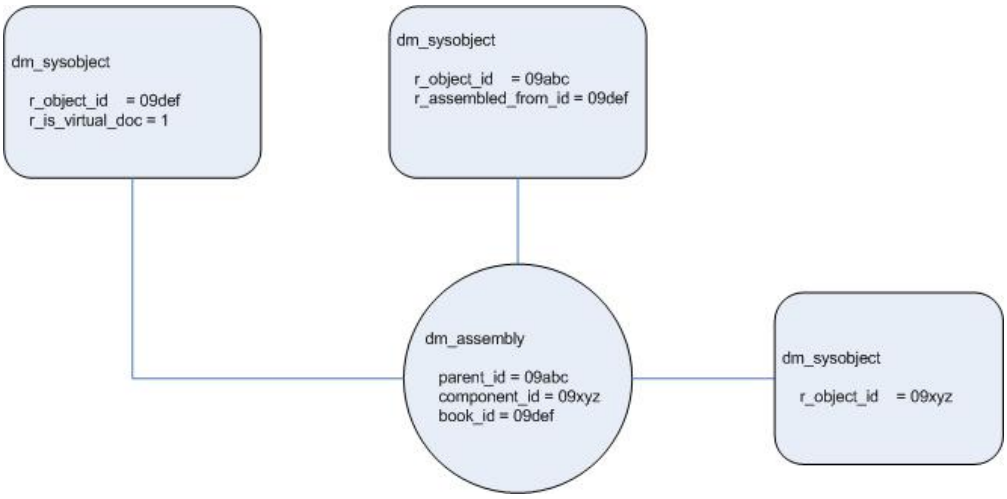

Figure 2-9: Non-inline snapshot

2.2.9.1.8 Following an assembly when assembling a Virtual Document node

In the case of an inline snapshot a dm_sysobject instance is simultaneously a virtual document (with associated containment objects) and a snapshot (with associated dm_assembly objects). When retrieving the virtual document, the API (either OpenText™ Documentum™ Content Management Foundation Java API or Foundation SOAP API) can specify whether to process the node using the relationships specified in the containment objects (following binding rules), or to process the node using the relationships specified in the assembly objects.

When retrieving a virtual document, the API enables you to specify a followAssembly (or in Foundation SOAP API, shouldFollowAssembly) setting for the entire virtual document. When this property is true, the inline assembly associated with the root node of the virtual document is used to retrieve the document, rather than the containment objects associated with the virtual document root.

2.2.9.1.9 Determining virtual document relationships when examining a dm_sysobject

In client applications it will often be necessary to obtain information about dm_sysobject instances in the repository and determine whether they are simple documents, virtual documents, snapshots, or inline snapshots (which are dm_sysobject instances that function as root of a virtual document and an assembly of that virtual document). This information can be determined by examining metadata on the dm_sysobject.

- If r_is_virtual_document = 1 OR r_link_cnt > 0, then the dm_sysobject is the root of a virtual document.
- If r_assembled_from_id has a value, then the dm_sysobject is the root of a snapshot assembled from the ID specified in r_assembled_from_id.

If both of the conditions are true, then the dm_sysobject is the root of an inline snapshot (that is, it is both a virtual document and a snapshot).

If none of the preceding conditions is true, then the dm_sysobject is a simple document.

2.2.9.2 Classes used by the Virtual Document Service

Class name	Description
VirtualDocumentNode	Uniquely identifies and provides information about a virtual document node.
VirtualDocumentInfo	Provides settings that determine how operations process this node, including version binding (during retrieve operations), and behavior when making copies of the virtual document.

Class name	Description
VdmChildrenActionInfo	Provides settings that specify a modification to the children of a virtual document node.

2.2.9.2.1 VirtualDocumentNode

Field name	Data type	Description
identity	ObjectIdentity	The identity of the node's parent object (dm_sysobject).
policy	VirtualDocumentInfo	Provides settings determining operation behavior when processing the virtual document node. See "VirtualDocumentInfo" on page 164.

2.2.9.2.2 VirtualDocumentInfo

Field name	Data type	Description
binding	String	The version label to use for early binding of a node. If this value is null, the node is late-bound. This can be set to VirtualDocumentInfo.BINDING_CURRENT to early bind to the current version of the object, or to BINDING_LATE, which specifies that the node will use the late binding version label, or to a version label string.
copyBehavior	CopyBehaviorMode	Determines whether a node is copied when the parent of the node is copied. COPY specifies that a copy (clone) of the object will be created. REFERENCE specifies that a reference to the object will be created. UNSPECIFIED allows the determination to be made by the application at runtime.

Field name	Data type	Description
overrideLateBinding	boolean	If true, use the early binding version label specified for this node for all descendant late-bound nodes. If false, use the early bound label for this node, but do not override descendant late-bound nodes.

2.2.9.2.3 VdmChildrenActionInfo

Field name	Data type	Description
action	VdmChildrenAction	Specifies the action to perform on the children of a virtual document. APPEND adds an existing child document to the end of the children list. INSERT inserts a new child into the list at position index. DELETE removes the child at position index. SET updates or replaces the child at position index.
index	int	The position in the list of children where the action will be applied. Ignored if action is APPEND. If the action is DELETE, and a documentNode is provided, then the operation compares the node at index to the provided node. If they do not match, an exception is thrown.
documentNode	VirtualDocumentNode	A child document node. This can be a new node (to be inserted or appended) or an existing node (to be deleted or set).

2.2.9.3 update operation

The update operation modifies (or creates) a virtual document. The operation is passed a DataObject representing the root document of the virtual document. If the object does not exist, it is created (provided that its ObjectIdentity has only the repositoryName set). An existing object will be updated with data provided in the DataObject passed to the operation.

To convert a simple document to a virtual document, you can either (1) pass a DataObject (which must be a SysObject or subtype) with its r_is_virtual_doc property set to 1, or (2) attach child nodes. If you attach child nodes, you do not have to set the r_is_virtual_doc property.

Note: The reason for this is that on Documentum CM Server an object is considered a virtual document if

```
r_is_virtual_doc == 1 || r_link_cnt > 0
```

Adding a child node to the parent increments the parent's r_link_cnt property; so either setting r_is_virtual_doc to 1 or adding a child is sufficient to ensure that the parent will be a virtual document.

The child nodes of the virtual document are updated, deleted, or set using data provided in a List<VdmChildrenActionInfo> (see "VdmChildrenActionInfo" on page 165.) The nodes are processed sequentially in the order in which they are contained in the List<VdmChildrenActionInfo>. You may need to take this into account when specifying indexes for processing INSERT or DELETE actions. Suppose for example that you have a parent with three child nodes, and you want to delete the first and third nodes. To accomplish this in left-to-right order you would specify indexes 0, then 1, because the child at index 2 will shift to index 1 when the child at index 0 is deleted. (To process from right-to-left you would specify 2, then 0.)

The update operation does not require that an existing object be checked out prior to the operation. If the object is not checked out, it will be checked out and checked in by the operation. If the object has been checked out by the user performing the update operation prior to the update operation, it will be checked in and the lock will not be preserved. This behavior can be changed using the retainLock setting in VdmUpdateProfile.

2.2.9.3.1 Java syntax

```
DataObject update(DataObject parent,
                  List<VdmChildrenActionInfo> children,
                  OperationOptions options)
        throws ServiceException,ServiceException
```

2.2.9.3.2 C# syntax

```
DataObject Update(DataObject parent,
                  List<VdmChildrenActionInfo> children,
                  OperationOptions options)
```

2.2.9.3.3 Parameters

Parameter	Data type	Description
parent	DataObject	Represents the root document (a dm_sysobject or subtype) of the virtual document.
children	List<VdmChildrenActionInfo>	A collection of VdmChildrenActionInfo instances, each of which contains information used to make a specific modification to the child list of the virtual document. See "VdmChildrenActionInfo" on page 165.
options	OperationOptions	Optional operation behaviors can be specified in a VdmUpdateProfile (see "VdmUpdateProfile" on page 168). A CheckinProfile and CheckoutProfile can be provided to specify options for the checkin and checkout of the updated object. The composition of the returned DataObject is governed by the standard ObjectService get operation profiles (PropertyProfile, RelationshipProfile, ContentProfile, PermissionProfile, and ContentTransferProfile).

2.2.9.3.4 VdmUpdateProfile

The VdmUpdateProfile class provides settings that govern the behavior of the update method.

Field name	Data type	Description
versionStrategy	VersionStrategy	Specifies an option for incrementing the version number of the virtual document root when it is checked in after the update.
retainLock	boolean	Determines whether the virtual document root will remain checked out after update. If set to true, by default only the root of the virtual document will remain checked out. You can use CheckinProfile and CheckoutProfile to specify alternative behavior (that is, leaving the children checked out).
labels	List<String>	A list of symbolic labels to apply to the virtual document root. If you provide this label and want the version to remain the CURRENT version, you must specifically add CURRENT to the label list.
updateMethod	ListUpdateMethod	Determines whether the child list will be replaced or supplemented by the list provided in the update operation. Values are MERGE and REPLACE.
convertToSimple	boolean	Determines whether the virtual document will be converted to a simple document if after the update it contains no children.

2.2.9.3.5 Returns

Returns a DataObject representing the virtual document after the update. The DataObject contains a list of ReferenceRelationship instances in which the virtual document child is the target object. Binding information for each virtual document node is included in the relationshipProperties list of the target object. The composition of the returned DataObject is governed by the standard ObjectService get operation profiles (PropertyProfile, RelationshipProfile, ContentProfile, PermissionProfile, and ContentTransferProfile).

2.2.9.3.6 Example

The following example adds a set of child documents to a virtual document node. If the parent node is a simple document, it will be converted to a virtual document. If the parent node does not exist in the repository, it will be created as a contentless object.

Example 2-124: Java: Populating a virtual document

```
public DataObject addChildNodes(DataObject parentObject,
ObjectIdentitySet childIdentities)
          throws ServiceException
{
    List<ObjectIdentity> idList = childIdentities.getIdentities();
    List<VdmChildrenActionInfo> caInfoList =
    new ArrayList<VdmChildrenActionInfo>();
    for (ObjectIdentity objIdentity : idList)
    {
        VirtualDocumentNode vdmNode = new VirtualDocumentNode();
        vdmNode.setIdentity(objIdentity);
        VirtualDocumentInfo vdmInfo = new VirtualDocumentInfo();
        vdmInfo.setBinding(VirtualDocumentInfo.BINDING_LATE);
        vdmInfo.setCopyBehavior(CopyBehaviorMode.UNSPECIFIED);
        vdmInfo.setFollowAssembly(false);
        vdmInfo.setOverrideLateBinding(false);
        vdmNode.setPolicy(vdmInfo);
        VdmChildrenActionInfo caInfo = new VdmChildrenActionInfo();
        caInfo.setAction(VdmChildrenAction.APPEND);
        caInfo.setDocumentNode(vdmNode);
        caInfoList.add(caInfo);
    }

  VdmUpdateProfile vdmUpdateProfile = new VdmUpdateProfile();
    List<String> versionLabels = new ArrayList<String>();
    versionLabels.add("testVersionLabel");

    // make sure to add the CURRENT label if you
    // want the virtual document to be CURRENT
    versionLabels.add("CURRENT");
    vdmUpdateProfile.setLabels(versionLabels);

    OperationOptions options = new OperationOptions();
    options.setVdmUpdateProfile(vdmUpdateProfile);
    return virtualDocumentService.update(parentObject,
    caInfoList, options);
}
```

Example 2-125: C#: Populating a virtual document

```
public DataObject AddChildNodes(DataObject parentObject,
ObjectIdentitySet childIdentities)
{
    List<ObjectIdentity> idList = childIdentities.Identities;
    List<VdmChildrenActionInfo> caInfoList =
        new List<VdmChildrenActionInfo>();
    foreach (ObjectIdentity objIdentity in idList)
    {
        VirtualDocumentNode vdmNode = new VirtualDocumentNode();
        vdmNode.Identity = objIdentity;
        VirtualDocumentInfo vdmInfo = new VirtualDocumentInfo();
        vdmInfo.Binding = VirtualDocumentInfo.BINDING_LATE;
        vdmInfo.CopyBehavior = CopyBehaviorMode.COPY;
        vdmInfo.OverrideLateBinding = false;
        vdmNode.Policy = vdmInfo;
        VdmChildrenActionInfo caInfo = new VdmChildrenActionInfo();
        caInfo.Action = VdmChildrenAction.APPEND;
        caInfo.Node = vdmNode;
        caInfoList.Add(caInfo);
    }

    VdmUpdateProfile vdmUpdateProfile = new VdmUpdateProfile();
    List<String> versionLabels = new List<String>();
    versionLabels.Add("testVersionLabel");

    // make sure to add the CURRENT label if you
    // want the virtual document to be CURRENT
    versionLabels.Add("CURRENT");
    vdmUpdateProfile.Labels = versionLabels;

    OperationOptions options = new OperationOptions();
    options.VdmUpdateProfile = vdmUpdateProfile;
    return virtualDocumentService.Update(parentObject,
        caInfoList, options);
}
```

2.2.9.4 retrieve operation

The retrieve gets a DataObject containing information about a virtual document, including its child nodes and binding rules. The retrieve operation can be used to get information using either a virtual document or a snapshot of a virtual document.

2.2.9.4.1 Java syntax

```
DataObject retrieve(ObjectIdentity parent,
                    OperationOptions options)
        throws CoreServiceException, ServiceException
```

2.2.9.4.2 C# syntax

```
DataObject Retrieve(ObjectIdentity parent,
                    OperationOptions options)
```

2.2.9.4.3 Parameters

Parameter	Data type	Description
parent	ObjectIdentity	The identity of the root document of the virtual document or snapshot to retrieve.
options	OperationOptions	May contain an instance of VdmRetrieveProfile. This operation will also process PropertyProfile, ContentProfile, PermissionProfile, RelationshipProfile to specify the data contained in the returned DataObject and ContentTransferProfile to specify content transfer options.

2.2.9.4.4 Returns

Returns a DataObject representing the virtual document. The DataObject contains a list of ReferenceRelationship instances in which the virtual document child is the target object. Binding information for each node is included in the relationshipProperties collection of the target object. Use profiles to specify other data returned as part of the DataObject.

2.2.9.4.5 VdmRetrieveProfile

The VdmRetrieveProfile can be used by the Virtual Document service retrieve operation, and can also be used by the Object service and VersionControl services when getting virtual documents from the repository. In the VirtualDocumentService it is used by the retrieve operation, and also in the createSnapshot operation, where it governs how the snapshot is assembled and the value returned by the operation.

Field name	Data type	Description
binding	String	Version label to use for late binding of virtual document nodes. Does not apply if shouldFollowAssembly is set to true, or the object being processed is a non-inline snapshot.

Field name	Data type	Description
shouldFollowAssembly	boolean	Sets operation behavior for retrieving nodes with associated assemblies (snapshots). If true, then retrieve the snapshot using the assemblies.

The shouldFollowAssembly setting is necessary because of the possibility of inline snapshots, in which the component in its capacity as a snapshot has associated assemblies (dm_assembly), and in its capacity as a virtual document has associated containment objects (dmr_containment). In this case the retrieve operation needs to be told whether to retrieve the virtual document using the associated containment objects, or from the snapshot, using the assemblies. In practice, this means that you must set shouldFollowAssembly to true whenever you pass a snapshot to the retrieve operation.

If the object being retrieved is a virtual document with no assemblies (that is, not a snapshot or inline snapshot), then shouldFollowAssembly is ignored, and the virtual document is retrieved using its associated containment objects.

2.2.9.4.6 Example

Example 2-126: Java: Virtual document information retrieval

```
public DataObject retrieveVdmInfo(ObjectIdentity objectIdentity,
boolean isSnapshot)
        throws ServiceException
{
    VdmRetrieveProfile retrieveProfile = new VdmRetrieveProfile();
    retrieveProfile.setShouldFollowAssembly(isSnapshot);
    retrieveProfile.setBinding("CURRENT");
    OperationOptions options = new OperationOptions();
    options.setVdmRetrieveProfile(retrieveProfile);

    DataObject resultDO = virtualDocumentService.retrieve
    (objectIdentity, options);
    List<Relationship> relationships = resultDO.getRelationships();
    System.out.println("Total relationships in virtual document = "
                    + relationships.size());

    int i = 0;
    for (Relationship r : relationships)
    {
        System.out.println();
        ReferenceRelationship refRel = (ReferenceRelationship)r;
        System.out.println("Child node "
                        + i++
                        + ": "
                        + refRel.getTarget().getValueAsString());
        PropertySet nodeProperties = refRel.getRelationshipProperties();
        Iterator propertyIterator = nodeProperties.iterator();
        while (propertyIterator.hasNext())
        {
            Property p = (Property)propertyIterator.next();
            System.out.print(p.getName() + ": ");
            System.out.println(p.getValueAsString());
        }
    }
```

```
    return resultDO;
}
```

Example 2-127: C#: Virtual document information retrieval

```
public DataObject RetrieveVdmInfo(ObjectIdentity objectIdentity,
Boolean isSnapshot)
{
    VdmRetrieveProfile retrieveProfile = new VdmRetrieveProfile();
    retrieveProfile.IsShouldFollowAssembly = isSnapshot;
    retrieveProfile.Binding = "CURRENT";
    OperationOptions options = new OperationOptions();
    options.VdmRetrieveProfile = retrieveProfile;

    DataObject resultDO = virtualDocumentService.Retrieve(objectIdentity, options);
    List<Relationship> relationships = resultDO.Relationships;
    Console.WriteLine("Total relationships in virtual document = "
        + relationships.Count);

    int i = 0;
    foreach (Relationship r in relationships)
    {
        Console.WriteLine();
        ReferenceRelationship refRel = (ReferenceRelationship)r;
        Console.WriteLine("Child node " + i++ + ": "
                + refRel.Target.GetValueAsString());
        PropertySet nodeProperties = refRel.RelationshipProperties;
        IEnumerator<Property> propertyEnumerator =
                nodeProperties.Properties.GetEnumerator();
        while (propertyEnumerator.MoveNext())
        {
            Property p = propertyEnumerator.Current;
            Console.WriteLine(p.Name + ": ");
            Console.WriteLine(p.GetValueAsString());
        }
    }
    return resultDO;
}
```

2.2.9.5 createSnapshot operation

The createSnapshot operations creates a snapshot of the virtual document and associates it with a document. If the document does not exist the associate document will be created and saved in the content repository before the resulting snapshot can be associated. If document exists in the repository, the createSnapshot operation will update the existing object using data passed in a DataObject before creating the snapshot.

The createSnapshot operation can create either an inline or a non-inline snapshot. In the first case, the identity of the associate document is the same as the identity of the parent document. In the latter case, the parent and associated documents are not identical.

The createSnapshot operation returns a DataObject containing the virtual document relationships (represented by ReferenceRelationship instances).

The assembly of the snapshot from the virtual document, and the object returned by createSnapshot, are governed by a VdmRetrieveProfile. If no VdmRetrieveProfile is provided in the OperationOptions object passed to createSnapshot, the operation creates one in which binding = “CURRENT” and shouldFollowAssembly = true. (Note that, as with the retrieve operation, the shouldFollowAssembly flag is ignored if the virtual document being processed has no assemblies.) Other profiles (such as PropertyProfile and PermissionProfile) can also be used to specify data returned in the DataObject.

2.2.9.5.1 Java syntax

```
DataObject createSnapshot(ObjectIdentity parent,
                          DataObject associate,
                          OperationOptions options)
        throws CoreServiceException, ServiceException
```

2.2.9.5.2 C# syntax

```
DataObject CreateSnapshot(ObjectIdentity parent,
                          DataObject associate,
                          OperationOptions options)
```

2.2.9.5.3 Parameters

Parameter	Data type	Description
parent	ObjectIdentity	The root document of the virtual document from which to derive the snapshot.
associate	DataObject	The object (a dm_sysobject or subtype) with which to associate the assemblies that comprise the snapshot. This can be the same object as parent, or a different object. If the object does not exist, it will be created.
options	OperationOptions	Contains profiles and properties that specify operation behaviors. Specifically, it can contain a VdmRetrieveProfile, as well as PropertyProfile, ContentProfile, PermissionProfile, and RelationshipProfile—which will determine how to populate the returned DataObject. It can contain ContentTransferProfile to specify content transfer options.

2.2.9.5.4 Returns

Returns a DataObject representing the virtual document from which the snapshot was created, guided by VdmRetrieveProfile. If VdmRetrieveProfile is not provided in OperationOptions, then a new VdmRetrieveProfile(true, “CURRENT”) will be constructed and used by the createSnapshot operation.

2.2.9.5.5 Examples

Example 2-128: Java: Creating a snapshot

```
public DataObject createSnapshotDemo(String vdmQualString,
                                     String snapshotName,
                                     String sourcePath)
        throws ServiceException
{
  // create ObjectIdentity of existing virtual document
    ObjectIdentity<Qualification> testVdmId =
    new ObjectIdentity<Qualification>();
    testVdmId.setRepositoryName(defaultRepositoryName);
    testVdmId.setValue(new Qualification<String>(vdmQualString));

    // create a new DataObject to use for the snapshot
    ObjectIdentity emptyIdentity = new
    ObjectIdentity(defaultRepositoryName);
    DataObject snapshotDO = new DataObject(emptyIdentity);
    snapshotDO.setType("dm_document");
    PropertySet parentProperties = new PropertySet();
    parentProperties.set("object_name", snapshotName);
    snapshotDO.setProperties(parentProperties);

    // link into a folder
    ObjectPath objectPath = new ObjectPath(sourcePath);
    ObjectIdentity<ObjectPath> sampleFolderIdentity =
    new ObjectIdentity<ObjectPath>(
                objectPath, defaultRepositoryName);
    ReferenceRelationship sampleFolderRelationship =
    new ReferenceRelationship();
    sampleFolderRelationship.setName(Relationship.RELATIONSHIP_FOLDER);
    sampleFolderRelationship.setTarget(sampleFolderIdentity);
    sampleFolderRelationship.setTargetRole(Relationship.ROLE_PARENT);
    snapshotDO.getRelationships().add(sampleFolderRelationship);

    // options are reserved for future use so just pass null
    OperationOptions options = null;
    return virtualDocumentService.createSnapshot(testVdmId,
    snapshotDO, options);
}
```

Example 2-129: C#: Creating a snapshot

```
public DataObject CreateSnapshotDemo(String vdmQualString,
                    String snapshotName, String sourcePath)
{
    // create ObjectIdentity of existing virtual document
    ObjectIdentity testVdmId = new ObjectIdentity();
    testVdmId.RepositoryName = DefaultRepository;
    testVdmId.Value = new Qualification(vdmQualString);

    // create a new DataObject to use for the snapshot
    ObjectIdentity emptyIdentity =
        new ObjectIdentity(DefaultRepository);
    DataObject snapshotDO = new DataObject(emptyIdentity);
```

```
    snapshotDO.Type = "dm_document";
    PropertySet parentProperties = new PropertySet();
    parentProperties.Set("object_name", snapshotName);
    snapshotDO.Properties = parentProperties;

    // link into a folder
    ObjectPath objectPath = new ObjectPath(sourcePath);
    ObjectIdentity sampleFolderIdentity =
        new ObjectIdentity(objectPath, DefaultRepository);
    ReferenceRelationship sampleFolderRelationship =
        new ReferenceRelationship();
    sampleFolderRelationship.Name = Relationship.RELATIONSHIP_FOLDER;
    sampleFolderRelationship.Target = sampleFolderIdentity;
    sampleFolderRelationship.TargetRole = Relationship.ROLE_PARENT;
    snapshotDO.Relationships.Add(sampleFolderRelationship);

    // options are reserved for future use so just pass null
    OperationOptions options = null;
    return virtualDocumentService.CreateSnapshot
        (testVdmId, snapshotDO, options);
}
```

Example 2-130: Java: Creating an inline snapshot

```
public DataObject createInlineSnapshotDemo
(String vdmQualString, String sourcePath)
        throws ServiceException
{
    // create ObjectIdentity of existing virtual document
      ObjectIdentity<Qualification> testVdmId =
        new ObjectIdentity<Qualification>();
      testVdmId.setRepositoryName(defaultRepositoryName);
      testVdmId.setValue(new Qualification<String>(vdmQualString));

      // create a new DataObject to use for the snapshot
      DataObject snapshotDO = new DataObject(testVdmId);

      // link into a folder
      ObjectPath objectPath = new ObjectPath(sourcePath);
      ObjectIdentity<ObjectPath> sampleFolderIdentity =
        new ObjectIdentity<ObjectPath>(
            objectPath, defaultRepositoryName);
      ReferenceRelationship sampleFolderRelationship =
        new ReferenceRelationship();
      sampleFolderRelationship.setName(Relationship.RELATIONSHIP_FOLDER);
      sampleFolderRelationship.setTarget(sampleFolderIdentity);
      sampleFolderRelationship.setTargetRole(Relationship.ROLE_PARENT);
      snapshotDO.getRelationships().add(sampleFolderRelationship);

      // options are reserved for future use so just pass null
      OperationOptions options = null;
      return virtualDocumentService.createSnapshot
        (testVdmId, snapshotDO, options);
}
```

Example 2-131: C#: Creating an inline snapshot

```
public DataObject CreateInlineSnapshotDemo(String vdmQualString,
String sourcePath)
{
    // create ObjectIdentity of existing virtual document
    ObjectIdentity testVdmId = new ObjectIdentity();
```

```
    testVdmId.RepositoryName = DefaultRepository;
    testVdmId.Value = new Qualification(vdmQualString);

    // create a new DataObject to use for the snapshot
    DataObject snapshotDO = new DataObject(testVdmId);

    // link into a folder
    ObjectPath objectPath = new ObjectPath(sourcePath);
    ObjectIdentity sampleFolderIdentity =
        new ObjectIdentity(objectPath,DefaultRepository);
    ReferenceRelationship sampleFolderRelationship = new ReferenceRelationship();
    sampleFolderRelationship.Name = Relationship.RELATIONSHIP_FOLDER;
    sampleFolderRelationship.Target = sampleFolderIdentity;
    sampleFolderRelationship.TargetRole = Relationship.ROLE_PARENT;
    snapshotDO.Relationships.Add(sampleFolderRelationship);

    // options are reserved for future use so just pass null
    OperationOptions options = null;
    return virtualDocumentService.CreateSnapshot(testVdmId,
        snapshotDO, options);
}
```

2.2.9.6 removeSnapshot operation

The remove operation removes the snapshot from the document with which they are associated (the document itself remains in the repository).

2.2.9.6.1 Java syntax

```
void removeSnapshot(ObjectIdentity associate,
                    OperationOptions options)
        throws CoreServiceException, ServiceException
```

2.2.9.6.2 C# syntax

```
void removeSnapshot(ObjectIdentity associate,
                    OperationOptions options)
```

2.2.9.6.3 Parameters

Parameter	Data type	Description
associate	ObjectIdentity	Uniquely identifies the object (dm_sysobject or subtype) with which the snapshot is associated.
options	OperationOptions	Reserved for future use.

2.2.9.6.4 Example

Example 2-132: Java: Removing a snapshot

```
public void removeSnapshot(String snapshotQualString)
throws ServiceException
{
  // create ObjectIdentity of existing snapshot
    ObjectIdentity<Qualification> testSnapshotId =
    new ObjectIdentity<Qualification>();
    testSnapshotId.setRepositoryName(defaultRepositoryName);
    testSnapshotId.setValue(new Qualification<String>(
            snapshotQualString));

    // remove snapshot
    virtualDocumentService.removeSnapshot(testSnapshotId, null);
}
```

Example 2-133: C#: Removing a snapshot

```
public void RemoveSnapshot(String snapshotQualString)
{
    // create ObjectIdentity of existing snapshot
    ObjectIdentity testSnapshotId = new ObjectIdentity();
    testSnapshotId.RepositoryName = DefaultRepository;
    testSnapshotId.Value = new Qualification(snapshotQualString);

    // remove snapshot
    virtualDocumentService.RemoveSnapshot(testSnapshotId, null);
}
```

2.2.10 RepositoryInquiry service

The RepositoryInquiry service gets data about the managed repositories currently available to the Foundation SOAP API server without requiring authentication. A managed repository is a OpenText Documentum CM Repository, as distinct from an external repository enabled with Documentum Federated Search Services.

2.2.10.1 getRepositoryList operation

The getRepositoryList operation returns a list of managed repositories.

2.2.10.1.1 Java syntax

```
List<Repository> getRepositoryList(OperationOptions options)
        throws CoreServiceException, ServiceException
```

2.2.10.1.2 C# syntax

```
List<Repository> GetRepositoryList(OperationOptions options)
```

2.2.10.1.3 Response

Returns a List of repository objects, which have the following properties:

Property	Data type	Description
name	String	The name of the repository.
type	RepositoryType	All repositories returned by this service will be of type RepositoryType.MANAGED.
properties	RepositoryProperty	An object that exposes detailed data about the repository capabilities.
servers	List<Server>	A list of servers that contain the following information: • name of the server • hostname of the server • version of the server • hostname of the docbroker

2.2.10.2 getRepositoryNameByObjectId operation

Takes a List of ObjectId instances and, for each item in the list, determines the repository in which the object resides. It then returns an ObjectIdentitySet that includes the ObjectId and repository name of each object.

2.2.10.2.1 Java syntax

```
ObjectIdentitySet getRepositoryNameByObjectId
(List objectIdList, OperationOptions options)
throws CoreServiceException
```

2.2.10.2.2 C# syntax

```
ObjectIdentitySet GetRepositoryNameByObjectId
(List objectIdList, OperationOptions options)
```

2.2.10.2.3 Parameters

Parameter	Data type	Description
objectIdList	List of ObjectId	A list of ObjectId instances for which repository names are requested.
options	OperationOptions	Reserved for future use.

2.2.10.2.4 Response

Returns an ObjectIdentitySet containing ObjectIdentity instances populated with the ObjectId passed by the client and the name of the repository where the object resides.

2.3 DocumentumUtil service

DocumentumUtilService provides the utility to change the system date format to IDfTime using the dateTimeToIDfTime operation.

2.3.1 dateTimeToIDfTime operation

The DocumentumUtilService has the dateTimeToIDfTime operation that provides the user to change the system date format to IDfTime. You need to create DateProperty and then add a date that you want in the IDfTime format. The operation changes the date format to IDfTime and returns in the string format.

2.3.1.1 Java syntax

```
public String dateTimeToIDFTime(DateProperty dteDateTime) throws CoreServiceException
```

2.3.1.2 C# syntax

```
string DateTimeToIDFTime(DateProperty dteDateTime);
```

2.3.1.3 Parameters

Parameter	Data type	Description
dteDateTime	DateProperty	Contains the key and value pair. Value is `java.util.Date`.

2.3.1.4 Response

Returns IDfTime in the string format. If the java.util.Date is DateTime.MinValue, then a nulldate is returned.

2.3.1.5 Examples

Example 2-134: Java: dateTimeToIDfTime

```
public String testDateTimeToIDFTime(Calendar dteDateTime)
{
 System.out.println("\nTesting date time to IDfTime...");
 String result="";
 try
 {
  DateProperty c = new DateProperty("dateTime",dteDateTime.getTime());
  result=documentumUtilServiceDateTimeToIDFTime.dateTimeToIDFTime(c);
 }
  catch(Exception e)
 {
  e.printStackTrace();
 }
  return result;
}
```

Example 2-135: C#: dateTimeToIDfTime

```
public void TestDateTimeToIDfTime()
{
 try
 {
  DateTime dateTime = new DateTime();
  Console.WriteLine("Date :" + dateTime);
  DateProperty dateProperty = new DateProperty("dateTime", new DateTime());

  Console.WriteLine("IDfTime is :" +
documentumUtilServiceDemo.DateTimeToIDfTime(dateProperty));
  Console.WriteLine("\nTest date time to IdfTime.");
 }
  catch (FaultException<SerializableException> ex)
 {
  Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
 }
  catch (Exception exx)
 {
  Console.WriteLine(exx.StackTrace);
 }
}
```

2.4 LoginTicket service

The LoginTicket service provides a set of operations for getting the login ticket for users to connect to Documentum CM Server. After getting the ticket, you can create different sessions for different users.

2.4.1 getDCTMLoginTicket operation

The getDCTMLoginTicket operation helps to generate the logic ticket for the OpenText Documentum CM session. To invoke this operation, you need the repository name and the user identity. Depending on the user identity, the login tickets are generated and used to create OpenText Documentum CM sessions.

2.4.1.1 Java syntax

```
public String getDCTMLoginTicket(String repositoryName, String loginUser) throws
CoreServiceException
```

2.4.1.2 C# syntax

```
string GetDCTMLoginTicket(string repositoryName, string loginUser);
```

2.4.1.3 Parameters

Parameter	Data type	Description
repositoryName	String	Name of the repository to which the user wants the OpenText Documentum CM session login ticket.
loginUser	String	Identity of the user.

2.4.1.4 Response

Returns either a OpenText Documentum CM login ticket for OpenText Documentum CM session or throws an exception.

2.4.1.5 Examples

Example 2-136: Java: getDCTMLoginTicket

```
public String testGetDCTMLoginTicket(String repositoryName, String userName)
{
 System.out.println("\nTesting get DCTM login ticket...");
 String result="";
 try
 {
  result=getDCTMLoginTicket.getDCTMLoginTicket(repositoryName,userName);
 }
  catch(Exception e)
 {
  e.printStackTrace();
```

```
 }
  return result;
}
```

Example 2-137: C#: getDCTMLoginTicket

```
public void TestGetDCTMLoginTicket()
{
 string repository=”testenv”;
 string username=”dmadmin”;
 try
 {
  Console.WriteLine("\nTest get DCTM login ticket for current user.");
  Console.WriteLine("\n DCTM Login ticket for current user is: " +
loginTicketServiceDemo.GetDCTMLoginTicket(repository,username));
 }
  catch (FaultException<SerializableException> ex)
 {
  Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
 }
  catch (Exception exx)
 {
  Console.WriteLine(exx.StackTrace);
 }
}
```

Chapter 3

Business process management services

3.1 Workflow service

The Workflow service provides the getProcessTemplates operation that retrieves all workflow process templates stored in the repository, the getProcessInfo operation that retrieves information about a specific workflow process template, and the startProcess operation that starts a workflow process instance.

Typically, the client calls the getProcessTemplates function to obtain a list of process templates available on a repository. Next, the client obtains default information about a process template by calling the getProcessInfo function. It then sets the attributes of the ProcessInfo object instance and passes the object to the startProcess operation to start the workflow.

Note that the user who executes the process must have the *Relate* and *Execute* permissions on the workflow process template.

Caution

When Foundation SOAP API SDK is installed after extracting the files packaged in ZIP file, all assemblies shipped with Foundation SOAP API-SDK installer are available in the *%DFS_SDK%* folder. Users can use the Task Management service *only* after adding the following assemblies (DLL files) to the .NET project available in the *..\<dfs-sdk>\lib\dotnet* folder:

- Emc.Documentum.FS.DataModel.Bpm.dll
- Emc.Documentum.FS.Services.Bpm.dll

3.1.1 Workflow SBO dependency

The Workflow service depends on the IStartWorkflow SBO that must be accessed from a global registry. This SBO is installed as part of the Workflow docapp that is installed during the installation of Documentum CM Server version 6.5. Therefore, the Workflow service is *not supported* on Documentum CM Server version 5.3x, and requires a global registry.

To access a global registry for local service invocation, the local `dfc.properties` in the Foundation SOAP API SDK must specify the global repository name, as well as the global registry user name and password. For remote service invocation, the `dfc.properties` file in `dfs.ear` deployed on the application server must have these settings.

The global registry settings would normally be set during Documentum CM Server and Foundation SOAP API installation; if they were set at install time there is no need to modify `dfc.properties` hosted by the application server. However, for local service invocation the user must modify the `dfc.properties` file in the SDK.

3.1.2 getProcessTemplates operation

3.1.2.1 Description

The getProcessTemplates operation is used to obtain a list of work process templates (dm_process objects) installed in the repository. If a folderPath String is passed to the getProcessTemplates operation, only the process templates within the folderPath are returned. In addition, all process templates in subfolders descending from the folderPath are returned.

3.1.2.2 Java syntax

```
DataPackage getProcessTemplates (String repositoryName,
String folderPath,
String additionalAttrs)
throws BpmServiceException;
```

3.1.2.3 Parameters

Parameter	Data type	Description
repositoryName	String	The name of the repository in which the process templates are stored.
folderPath	String	The path to the folder in which the process templates are linked. If the value is NULL, the operation will return all the process templates stored in the repository. For example / mycabinet/myfolder.

Parameter	Data type	Description
additionalAttrs	String	A comma-separated list of attribute names. By default, the getProcessTemplates operation returns only ObjectIdentity instances that represent dm_process repository objects, and does not return dm_process properties. When specified in the getProcessTemplates operation, the additionalAttrs parameter allows the client to pass a list of dm_process property names that are returned in each DataObject.

3.1.2.4 Returns

Returns a DataPackage containing DataObject instances that represent the dm_process repository objects. Properties (attributes) of the dm_process object are returned if specified in the additionAttrs argument.

3.1.2.5 Example

Example 3-1: Java: Getting process templates

```
public DataPackage processTemplates()
{
    try
    {
        ServiceFactory serviceFactory = ServiceFactory.getInstance();
        IWorkflowService workflowService
            = serviceFactory.getService
                        (IWorkflowService.class, serviceContext);
        DataPackage processTemplates
            = workflowService.getProcessTemplates(defaultRepositoryName,
              null,
              "object_name");
        for (DataObject dObj : processTemplates.getDataObjects())
        {
            System.out.println(dObj.getIdentity().getValueAsString());
            System.out.println(dObj.getProperties().get("object_name"));
        }

        return processTemplates;
    }
    catch (Exception e)
    {
        e.printStackTrace();
        throw new RuntimeException(e);
    }
}
```

Example 3-2: C#: Getting process templates

```
public DataPackage processTemplates()
{
    try
    {
        DataPackage processTemplates
            = workflowService.GetProcessTemplates(DefaultRepository,
              null,
              "object_name");
        foreach (DataObject dObj in processTemplates.DataObjects)
        {
            Console.WriteLine(dObj.Identity.GetValueAsString());
            Console.WriteLine(dObj.Properties.Get("object_name"));
        }
        return processTemplates;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        Console.WriteLine(e.StackTrace);
        throw new Exception(e.Message);
    }
}
```

3.1.3 getProcessInfo operation

3.1.3.1 Description

The getProcessInfo operation is used to obtain information about a specific process template. The user calls this operation after identifying a workflow process to start, by performing the getProcessTemplates operation. The getProcessInfo operation returns a data structure that the client uses to set values in the ProcessInfo object. These values are required to start a workflow. Subsequently, the client modifies these values and then passes the ProcessInfo object to the startProcess operation.

3.1.3.2 Java syntax

```
ProcessInfo getProcessInfo (ObjectIdentity process)
throws BpmServiceException;
```

3.1.3.3 Parameters

Parameter	Data type	Description
process	ObjectIdentity	The ObjectIdentity uniquely identifies a process template (dm_process object) installed in a repository. The getProcessTemplates operation returns ObjectIdentity instances. Only ObjectIdentity instances of ObjectId subtype, are supported.

3.1.3.4 Returns

Returns a ProcessInfo instance containing detailed information about a process template (dm_process repository object). The client takes the initial values (obtained by the getProcessInfo operation) from the process template, and sets these values in the ProcessInfo object. The client then modifies these values and passes the ProcessInfo object to the startProcess operation to start a workflow.

3.1.3.5 Example

Example 3-3: Java: Getting process information

```
public ProcessInfo processInfo(ObjectIdentity processId)
{
        try
        {
                ServiceFactory serviceFactory = ServiceFactory.getInstance();
                IWorkflowService workflowService
                = serviceFactory.getService(IWorkflowService.class, serviceContext);
                ProcessInfo processInfo = workflowService.getProcessInfo(processId);

                System.out.println("Process template "
                + processId.getValueAsString());
                System.out.println("Name is " + processInfo.getProcessInstanceName());
                System.out.println("isAliasAssignmentRequired == "
                + processInfo.isAliasAssignmentRequired());
                System.out.println("isPerformerAssignmentRequired == "
                + processInfo.isPerformerAssignmentRequired());
                return processInfo;
        }
        catch (Exception e)
        {
                e.printStackTrace();
                throw new RuntimeException(e);
        }
}
```

Example 3-4: C#: Getting process information

```
public ProcessInfo processInfo(ObjectIdentity processId)
{
        try
        {
                ProcessInfo processInfo = workflowService.GetProcessInfo(processId);
                Console.WriteLine("Process template "
```

```
                + processId.GetValueAsString());
                Console.WriteLine("Name is " + processInfo.ProcessInstanceName);
                Console.WriteLine("isAliasAssignmentRequired == "
                + processInfo.IsAliasAssignmentRequired);
                Console.WriteLine("isPerformerAssignmentRequired == "
                + processInfo.IsPerformerAssignmentRequired);
                return processInfo;
        }
        catch (Exception e)
        {
                Console.WriteLine(e.Message);
                Console.WriteLine(e.StackTrace);
                throw new Exception(e.Message);
        }
}
```

3.1.4 startProcess operation

3.1.4.1 Description

The client passes the ProcessInfo object to the startProcess operation to start the workflow. The startProcess operation executes a business process (workflow) based on the values set in the ProcessInfo object.

3.1.4.2 Java syntax

```
ObjectIdentity startProcess (ProcessInfo info)
throws BpmServiceException;
```

3.1.4.3 Parameters

Parameter	Data type	Description
info	ProcessInfo	A data structure containing information about the workflow process template. The client uses the getProcessInfo operation to get this structure for a specific process template, and sets these as initial values in the ProcessInfo object. The client then modifies these values and passes the ProcessInfo object to the startProcess operation to start a workflow.

3.1.4.4 Returns

Returns an ObjectIdentity uniquely identifying the instance of the process that was started. For further information on the ProcessInfo object, refer to the Javadocs.

3.1.4.5 Example

Example 3-5: Java: Starting a process

```
public void startProcess(String processId,
                                            String processName,
                                            String supervisor,
                                            ObjectId wfAttachment,
                                            List<ObjectId> docIds,
                                            String noteText,
                                            String userName,
                                            String groupName,
                                            String queueName) throws Exception
{
        ServiceFactory serviceFactory = ServiceFactory.getInstance();
        IWorkflowService workflowService
                = serviceFactory.getService(IWorkflowService.class,
                    serviceContext);
        // get the template ProcessInfo
        ObjectId objId = new ObjectId(processId);
        ProcessInfo info = workflowService
                .getProcessInfo(new ObjectIdentity<ObjectId>(objId,
                    defaultRepositoryName));

        // set specific info for this workflow
        info.setSupervisor(supervisor);
        info.setProcessInstanceName(processName + new Date());

        // workflow attachment
        info.addWorkflowAttachment("dm_sysobject", wfAttachment);

        // packages
        List<ProcessPackageInfo> pkgList = info.getPackages();
        for (ProcessPackageInfo pkg : pkgList)
        {
            pkg.addDocuments(docIds);
            pkg.addNote("note for " + pkg.getPackageName() + " " + noteText, true);
        }
        // alias
        if (info.isAliasAssignmentRequired())
        {
            List<ProcessAliasAssignmentInfo> aliasList
                    = info.getAliasAssignments();
            for (ProcessAliasAssignmentInfo aliasInfo : aliasList)
            {
                    String aliasName = aliasInfo.getAliasName();
                    String aliasDescription = aliasInfo.getAliasDescription();
                    int category = aliasInfo.getAliasCategory();
                    if (category == 1) // User
                    {
                        aliasInfo.setAliasValue(userName);
                    }
                    else if (category == 2 || category == 3)
                    // group, user or group
                    {
                        aliasInfo.setAliasValue(groupName);
                    }
            System.out.println("Set alias: "
                                                + aliasName
                                                  + ", description: "
                                                  + aliasDescription
                                                  + ", category: "
```

```
                                                + category
                                                + " to "
                                                + aliasInfo.getAliasValue());
            }
        }

        // Performer.
        if (info.isPerformerAssignmentRequired())
        {
            List<ProcessPerformerAssignmentInfo> perfList
                     = info.getPerformerAssignments();
            for (ProcessPerformerAssignmentInfo perfInfo : perfList)
            {
                    int category = perfInfo.getCategory();
                    int perfType = perfInfo.getPerformerType();
                    String name = "";
                    List<String> nameList = new ArrayList<String>();
                    if (category == 0) // User
                    {
                        name = userName;
                    }
                    else if (category == 1 || category == 2) // Group, user or group
                    {
                        name = groupName;
                    }
                    else if (category == 4) // work queue
                    {
                        name = queueName;
                    }
                        nameList.add(name);
                        perfInfo.setPerformers(nameList);
                        System.out.println("Set performer perfType: " + perfType +
                        ", category: " + category + " to " + name);
             }
         }
         ObjectIdentity wf = workflowService.startProcess(info);
         System.out.println("started workflow: " + wf.getValueAsString());
}
```

Example 3-6: C#: Starting a process

```
public void startProcess(String processId,
                                              String processName,
                                              String supervisor,
                                              ObjectId wfAttachment,
                                              List<ObjectId> docIds,
                                              String noteText,
                                              String userName,
                                              String groupName,
                                              String queueName)
{
            // get the template ProcessInfo
            ObjectId objId = new ObjectId(processId);
            ProcessInfo info = workflowService
                    .GetProcessInfo(new ObjectIdentity(objId, DefaultRepository));
            // set specific info for this workflow
            info.Supervisor = supervisor;
            info.ProcessInstanceName = processName + new DateTime();
            // workflow attachment
            info.AddWorkflowAttachment("dm_sysobject", wfAttachment);
            // packages
            List<ProcessPackageInfo> pkgList = info.Packages;
            foreach (ProcessPackageInfo pkg in pkgList)
            {
                    pkg.AddDocuments(docIds);
                    pkg.AddNote("note for " + pkg.PackageName + " "
```

```
                    + noteText, true);
            }
            // alias
            if (info.IsAliasAssignmentRequired())
            {
                    List<ProcessAliasAssignmentInfo> aliasList
                             = info.AliasAssignments;
                    foreach (ProcessAliasAssignmentInfo aliasInfo in aliasList)
                    {
                            String aliasName = aliasInfo.AliasName;
                            String aliasDescription = aliasInfo.AliasDescription;
                            int category = aliasInfo.AliasCategory;
                            if (category == 1) // User
                            {
                                    aliasInfo.AliasValue = userName;
                            }
                            else if (category == 2 || category == 3)
                            // group, user or group
                            {
                                    aliasInfo.AliasValue = groupName;
                            }
                            Console.WriteLine("Set alias: "
                                                                    + aliasName
                                                                    + ", description: "
                                                                    + aliasDescription
                                                                    + ", category: "
                                                                    + category
                                                                    + " to "
                                                                    + aliasInfo.AliasValue);
                    }
                }

          // Performer.
            if (info.IsPerformerAssignmentRequired())
            {
                    List<ProcessPerformerAssignmentInfo> perfList
                             = info.PerformerAssignments;
                    foreach (ProcessPerformerAssignmentInfo perfInfo in perfList)
                    {
                            int category = perfInfo.Category;
                            int perfType = perfInfo.PerformerType;
                            String name = "";
                            List<String> nameList = new List<String>();
                            if (category == 0) // User
                            {
                                    name = userName;
                            }
                            else if (category == 1 || category == 2)
                            // Group, user or group
                            {
                                    name = groupName;
                            }
                            else if (category == 4) // work queue
                            {
                                    name = queueName;
                            }
                            nameList.Add(name);
                            perfInfo.Performers = nameList;
                            Console.WriteLine("Set performer perfType: "
                            + perfType +
                                                                    ", category: " + category +
" to "
                                                                    + name);
                    }
                }
                ObjectIdentity wf = workflowService.StartProcess(info);
                Console.WriteLine("started workflow: "
                + wf.GetValueAsString());
}
```

3.1.5 terminateWorkflow operation

3.1.5.1 Description

The Terminate Workflow service terminates a workflow. You must be the workflow supervisor or a user with Sysadmin or Superuser privileges to terminate the workflow. There can be no active automatic activities. This operation halts, aborts, and then terminates the workflow. In addition, this operation sets the r_runtime_state attribute of workflow to `terminated`.

3.1.5.2 Java syntax

```
public Boolean terminateWorkflow(ObjectIdentity objectIdentity) throws
BpmServiceException
```

3.1.5.3 C# syntax

```
bool TerminateWorkflow(ObjectIdentity objectIdentity);
```

3.1.5.4 Parameters

Parameter	Data type	Description
objectIdentity	ObjectIdentity	Contains the workflow object ID and repository name.

3.1.5.5 Returns

Returns either true if workflow is terminated or throws an exception.

3.1.5.6 Example

Example 3-7: Java: terminateWorkflow

```
public void testTerminateWorkflow()
{
 System.out.println("\nTesting terminate workflows templates.");
 try
 {
 //first need to create workflow and then start it
  ObjectIdentity objectIdentity = workflowServiceDemo.createWorkflow();
//it won't terminate workflow if workflow is not executed
  workflowServiceDemo.terminateWorkflow(objectIdentity);
 }
  catch(Exception e)
 {
  System.out.println(e);
 }
  System.out.println("Completed terminateWorkflows");
}// complete code is available in dfs-sdk DfsSamples
```

Example 3-8: C#: terminateWorkflow

```
public void TestTerminateWorkflow()
{
 Console.WriteLine("\nTest terminate workflow");
 try
 {
 //first need to create workflow and then start it
 ObjectIdentity objectIdentity = workflowServiceDemo.createWorkflow();
 //it won't terminate workflow if workflow is not executed
 workflowServiceDemo.TerminatedWorkflow(objectIdentity);
 }
 catch (FaultException<SerializableException> ex)
 {
  Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
 }
  catch (Exception exx)
 {
  Console.WriteLine(exx.StackTrace);
 }
}
```

3.1.6 completeWorkitem operation

3.1.6.1 Description

This operation completes a workitem by passing the workitem ID. You must be the workflow supervisor or a user with Sysadmin or Superuser privileges to complete the workitem associated with workflow. When you start any workflow, Documentum CM Server creates workitem and queueitem. In addition, this operation sets the r_runtime_state attribute to `2` which means completed.

3.1.6.2 Java syntax

```
public Boolean completeWorkitem(ObjectIdentity objectIdentity) throws BpmServiceException
```

3.1.6.3 C# syntax

```
bool CompleteWorkitem(ObjectIdentity objectIdentity);
```

3.1.6.4 Parameters

Parameter	Data type	Description
objectIdentity	ObjectIdentity	Contains the workitem object ID and repository name.

3.1.6.5 Returns

Returns either true if workitem is completed or throws an exception.

3.1.6.6 Example

Example 3-9: Java: completeWorkitem

```
public void testCompleteWorkItem()
{
 /**
 * For testing this completeWorkitem operation, we have created a process template in
createWorkflow().
 */
 System.out.println("\nTesting work item complete.");
 try
 {
 ObjectIdentity workflowIdentity=workflowServiceDemo.createWorkflow();
 // After starting workflow we will have workitem. We need to fetch workitem and
complete it.

 ObjectIdentity
workitemIdentity=workflowServiceDemo.getWorkItemByWorkflowId(workflowIdentity);

 Boolean result=workflowServiceDemo.completeWorkitem(workitemIdentity);
 if(result){
 System.out.println("Workitem has id :"+workitemIdentity.getValueAsString()+"
completed");
 }
 else
 {
 System.out.println("Workitem has id :"+workitemIdentity.getValueAsString()+" not
completed");
 }
 }catch(Exception e){
 System.out.println(e);
 }
 System.out.println("Test completeWorkitem done");
}// complete code is available in dfs-sdk DfsSamples
```

Example 3-10: C#: completeWorkitem

```
public void TestCompleteWorkItem()
{
 /**
 * For testing this completeWorkitem operation, we have created a process template in
createWorkflow().
 */
 Console.WriteLine("\nTesting work item complete.");
 try
 {
 ObjectIdentity workflowIdentity = workflowServiceDemo.createWorkflow();
 // After starting workflow we will have workitem. We need to fetch workitem and
complete it.

 ObjectIdentity workitemIdentity =
workflowServiceDemo.getWorkItemByWorkflowId(workflowIdentity);

 Boolean result = workflowServiceDemo.CompleteWorkitem(workitemIdentity);
 if (result)
 {
 Console.WriteLine("Workitem has id :" + workitemIdentity.GetValueAsString() + "
completed");
 }
```

```
 else
 {
 Console.WriteLine("Workitem has id :" + workitemIdentity.GetValueAsString() + " not
completed");
 }
 }
 catch (FaultException<SerializableException> ex)
 {
 Console.WriteLine(String.Format("Got FaultException[{0}] with message: {1}\n",
ex.Detail, ex.Message));
 }
 catch (Exception exx)
 {
 Console.WriteLine(exx.StackTrace);
 }
 Console.WriteLine("Test completeWorkitem done");
}// complete code is available in dfs-sdk DotNetClientDocSamples
```

3.2 Task Management service

The Task Management service contains most of the functionality defined in IBM-WebService-HumanTask Specification v1.0 that includes task list and task management functions. For additional information, refer to Human task management.

The Task Management service provides functionality to retrieve custom task lists for a user, work queue, or work queue processor. It also provides functionality for handling and managing manual tasks and notifications.

A task list includes all information crucial for understanding the importance of and details about a given task or notification (for example, task priority, subject, and description). When a user selects a task, the user can also choose to open the task in the corresponding user interface and view all its details. This list of notifications or tasks can be sorted based on specific criteria. Typically, users access a task list and sort it or search for a specific task assigned to the user, and then begin processing the required task(s).

Caution

When Foundation SOAP API SDK is installed after extracting the files packaged in the ZIP file, all assemblies shipped with Foundation SOAP API-SDK installer are available in the *%DFS_SDK%* folder. Users can use the Task Management service *only* after adding the following assemblies (DLL files) to the .NET project available in the *..\<dfs-sdk>\lib\dotnet* folder:

- Emc.Documentum.FS.DataModel.Bpm.dll
- Emc.Documentum.FS.Services.Bpm.dll

3.2.1 Human task management

The Task Management service interface adheres to IBM-WebService-HumanTask Specification v1.0 and provides most of the functionality for handling tasks and notifications. The methods in this interface come from the standard WSDL. Support for this standard provides the ability to develop clients using web services-enabled client technology for human tasks managed by the Documentum Process Suite.

Human tasks, or manual tasks, enable the integration of human beings in the Business Process Management system. Human tasks are services "implemented" by people who are users who perform tasks based on the human roles assigned to them in the organization. A human task comprises two interfaces. The first interface exposes the service offered by the task, such as a translation service or an approval service. The second interface allows users to work with tasks, for example, to query for human tasks assigned to them, and to work on these tasks. A human task has users assigned to it. These assignments define who must be allowed to play a certain role on that task. Human tasks may also specify how task metadata must be rendered on different devices or applications making them portable and interoperable with different types of software. Human tasks can be defined to react on time-outs, by triggering an appropriate escalation action.

This also holds true for notifications. Notifications are a special type of human task that allow the sending of information about important business events to people. A Notification may have multiple recipients and optionally one or many business administrators. Notifications are always one-way. They are delivered in a fire-and-forget manner, where the sender pushes out notifications to people without waiting for the recipients to acknowledge the receipt of the notification.

3.2.2 Generic human roles

Generic human roles according to the IBM-WebService-HumanTask Specification v1.0, define what a user or users in a work queue can do with tasks and notifications, based on the roles assigned to them in the organization. The following generic human roles can be used in the various Task Management service operations:

- actualOwner – Refers to the user who owns the task. Use "actualOwner" while making a method call to list the tasks the user owns, or tasks that the user is responsible for in a specified work queue.
- businessAdministrators – Refers to work queue managers. Business Administrators play the same role as Task Stakeholders but at the task type level. Therefore, Business Administrators can perform the same operations as Task Stakeholders. Business Administrators can also observe the progress of notifications. Use "businessAdministrators" if the user is a work queue manager trying to view the work queue task list. Human task-compliant implementations must ensure that at runtime at least one person is associated with this role.
- taskStakeholders – Refers to work queue managers. Task Stakeholders are users who are ultimately responsible for the oversight and outcome of the task

instance. A Task Stakeholder can influence the progress of a task, for example, by adding attachments, forwarding the task, or simply observing the state changes of the task. This user can also perform administrative actions on the task instance and associated notification(s). Use “taskStakeholders” if the user is a work queue manager trying to view the work queue task list. Human task-compliant implementations must ensure that at least one person is associated with this role at runtime.

- potentialOwners – Refers to task performers or work queue processors who can claim tasks and complete them. Use “potentialOwners” if the user is a work queue processor trying to select a specific work queue task. Ensure that the work queue name is passed to the “workQueue” parameter. A Potential Owner becomes the Actual Owner of a task by explicitly claiming the task.

3.2.3 TaskListFactory SBO dependency

The Task Management service depends on the ITaskListFactory SBO that is accessed through the OpenText Documentum Content Management (CM) Foundation Java API BOF2 mechanism. This SBO is installed as part of the BPM docapp DAR that is installed by the Process Engine installer. To use this SBO, the Foundation SOAP API application must set the global repository in `dfc.properties` to the repository in which the BPM docapp is installed. Then the SBO is downloaded from the repository for use. Therefore, the Task Management service is *not supported* on Documentum CM Server version 5.3x, and requires a global registry.

To access a global registry for local service invocation, the local `dfc.properties` in the Foundation SOAP API SDK must specify the global repository name, as well as the global registry user name and password. For remote service invocation, the `dfc.properties` file in `dfs.ear` deployed on the application server must have these settings.

The global registry settings would normally be set during Documentum CM Server and Foundation SOAP API installation; if they were set at install time there is no need to modify `dfc.properties` hosted by the application server. However, for local service invocation the user must modify the `dfc.properties` file in the SDK.

3.2.4 claim operation

3.2.4.1 Description

The claim operation enables Potential Owners and Business Administrators to claim a task and complete it. When a task requires a specific user to complete the task, the user who must work on this task can acquire the task by performing the claim operation. The claim operation moves the task from the work queue to the user's Inbox, and other users in the work queue can no longer work on the claimed task.

A Potential Owner becomes the Actual Owner of a task by explicitly claiming it. Potential Owners can influence the progress of a task before it is claimed, for example, by changing the priority of the task, adding attachments or comments. The system transitions the claimed task to the “Reserved” state.

The claim operation is similar to the Acquire or Get Task operation in Documentum TaskSpace.

3.2.4.2 Java syntax

```
public void claim(String identifier)
throws BpmServiceException
```

3.2.4.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task that must be claimed.

3.2.4.4 Example

Example 3-11: Java: Claiming a task

The following Java samples are available for the claim operation:

- *Claiming a task retrieved by performing the getmytaskabstracts operation:*

```
import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
```

- *Claiming a task retrieved by performing the getmytasks operation:*

```
import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
```

- *Claiming a task retrieved by performing the query operation:*

```
import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
```

Example 3-12: C#: Claiming a task

The following C# samples are available for the claim operation:

- *Claiming a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
```

- *Claiming a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);
```

```
String taskId = task[0].id;

my_service.claim(taskId);
```

- *Claiming a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
("dmi_workitem.r_object_id,dmi_queue_item.
r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
```

3.2.5 start operation

3.2.5.1 Description

This operation is not supported. If the client calls this API, the UnsupportedOperationException fault is returned.

3.2.6 stop operation

3.2.6.1 Description

This operation is not supported. If the client calls this API, the UnsupportedOperationException fault is returned.

3.2.7 release operation

3.2.7.1 Description

The release operation is applicable to *work queue tasks only*. This operation enables the Actual Owner to release who claimed a work queue task that is in the *Reserved* or *InProgress* state. The task is released from the user's Inbox and moved to the work queue and made available to all Potential Owners in the work queue. A released task is transitioned to the *Ready* state.

The release operation is similar to the Unassign operation in Documentum TaskSpace.

3.2.7.2 Java syntax

```
public void release(String identifier)
throws BpmServiceException
```

3.2.7.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task that must be released.

3.2.7.4 Example

Example 3-13: Java: Releasing a task

The following Java samples are available for the release operation:

- *Releasing a task retrieved by performing the getmytaskabstracts operation:*

```
import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
(ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS",
"PotentialOwners",
null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.suspendUntil(taskId, untilTime);
my_service.resume(taskId);
```

```
my_service.forward(taskId);
my_service.release(taskId);
```

- *Releasing a task retrieved by performing the getmytasks operation:*

```
import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.release(taskId);
```

- *Releasing a task retrieved by performing the query operation:*

```
import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.release(taskId);
```

Example 3-14: C#: Releasing a task

The following C# samples are available for the release operation:

- *Releasing a task retrieved by performing the getmytaskabstracts operation:*

```
using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
```

```
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.release(taskId);
```

- *Releasing a task retrieved by performing the getmytasks operation:*

```
using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.release(taskId);
```

- *Releasing a task retrieved by performing the query operation:*

```
using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.release(taskId);
```

3.2.8 suspend operation

3.2.8.1 Description

The suspend operation enables Potential Owners, Actual Owner, or Business Administrators to pause a task that is in an active state such as “Ready”, “Reserved”, or “InProgress”, and transitions it to the “Suspended” state. The task performer may suspend a task to obtain some information for processing the task, or if the task performer runs into some problems that must be resolved before continuing to process the task. When suspended tasks are released, the state of the tasks changes to “Ready”.

The suspend operation is similar to the Suspend operation in Documentum TaskSpace.

3.2.8.2 Java syntax

```
public void suspend(String identifier)
throws BpmServiceException
```

3.2.8.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task that must be suspended.

3.2.8.4 Example

Example 3-15: Java: Suspending a task

The following Java samples are available for the suspend operation:

- *Suspending a task retrieved by performing the getmytaskabstracts operation:*

```
import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null, Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
my_service.suspend(taskId);
```

- *Suspending a task retrieved by performing the getmytasks operation:*

```
import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.suspend(taskId);
```

- *Suspending a task retrieved by performing the query operation:*

```
import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.suspend(taskId);
```

Example 3-16: C#: Suspending a task

The following C# samples are available for the suspend operation:

- *Suspending a task retrieved by performing the getmytaskabstracts operation:*

```
using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);
```

```
String taskId = task[0].id;

my_service.claim(taskId);
my_service.suspend(taskId);
```

- *Suspending a task retrieved by performing the getmytasks operation:*

```
using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.suspend(taskId);
```

- *Suspending a task retrieved by performing the query operation:*

```
using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.suspend(taskId);
```

3.2.9 suspendUntil operation

3.2.9.1 Description

The suspendUntil operation enables Potential Owners, Actual Owner, or Business Administrators to suspend a task that is in an active state such as “Ready”, “Reserved”, or “InProgress”, for a specific duration or until a specified time. The client must specify a duration or a fixed time. Such a task is transitioned to the “Suspended” state. Alternatively, the application will automatically resume the task when the specified time has lapsed, and transitions the task to the “Ready” state.

The suspendUntil operation is similar to the Suspend operation in Documentum TaskSpace.

3.2.9.2 Java syntax

```
public void suspendUntil(String identifier,
TTime time)
throws BpmServiceException
```

3.2.9.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task that must be suspended for a specific duration or until a specified time.
time	TTime	The duration for which a task must be suspended, or the time until when a task must be suspended.

3.2.9.4 Example

Example 3-17: Java: Suspending task for a specific time/duration

The following Java samples are available for the suspendUntil operation:

- *Suspending a task (for a specific time) retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
```

```
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
my_service.suspendUntil(taskId, untilTime);
```

- *Suspending a task (for a specific time) retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.suspendUntil(taskId, untilTime);
```

- *Suspending a task (for a specific time) retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.suspendUntil(taskId, untilTime);
```

Example 3-18: C#: Suspending task for a specific time/duration

The following C# samples are available for the suspendUntil operation:

- *Suspending a task (for a specific time) retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
```

```
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.suspendUntil(taskId, untilTime);
```

- *Suspending a task (for a specific time) retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.suspendUntil(taskId, untilTime);
```

- *Suspending a task (for a specific time) retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.suspendUntil(taskId, untilTime);
```

3.2.10 resume operation

3.2.10.1 Description

The resume operation enables Potential Owners, Actual Owner, and Business Administrators to resume the processing of tasks that have been paused or suspended. The state of the resumed task transitions to *InProgress*.

The resume operation is similar to the Unsuspend operation in the Documentum TaskSpace.

3.2.10.2 Java syntax

```
public void resume(String identifier)
throws BpmServiceException
```

3.2.10.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task that must be resumed.

3.2.10.4 Example

Example 3-19: Java: Resuming a task

The following Java samples are available for the resume operation:

- *Resuming a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
```

- *Resuming a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
```

- *Resuming a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
```

Example 3-20: C#: Resuming a task

The following C# samples are available for the resume operation:

- *Resuming a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
```

```
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
```

- *Resuming a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
```

- *Resuming a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
```

3.2.11 complete operation

3.2.11.1 Description

The complete operation enables the Actual Owner of a task to complete the processing of a task. The system sets the state of such a task to “Complete”. Completing a task sends it to the next user or activity in the workflow. Any changes made to attached files travel with the task if the version of the attached files and the checked in files are the same version.

This operation can be performed only on a task that does not require the signoff, set output paths, and select dynamic performer parameters to be set.

The complete operation is similar to the Finish operation in Documentum TaskSpace.

3.2.11.2 Java syntax

```
public void complete(String identifier,
Object TCompleteTaskData taskData)
throws BpmServiceException
```

3.2.11.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task that must be completed.
taskData	TCompleteTaskData	The TCompleteTaskData object holds information about requirements to complete a task.

3.2.11.4 Returns

If no output data is set, the complete operation will return the illegalArgumentFault exception.

3.2.11.5 Example

Example 3-21: Java: Completing a task

The following Java samples are available for the complete operation:

- *Completing a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.complete(taskId,null);
```

- *Completing a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.complete(taskId,null);
```

- *Completing a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();
```

```
String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.complete(taskId,null);
```

Example 3-22: C#: Completing a task

The following C# samples are available for the complete operation:

- *Completing a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.complete(taskId,null);
```

- *Completing a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.complete(taskId,null);
```

- *Completing a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.complete(taskId,null);
```

3.2.12 remove operation

3.2.12.1 Description

The remove operation is applicable to *notifications only*. Notification recipients perform this operation to permanently remove the notification from their task list or Inbox. Such a notification cannot be retrieved again.

The remove operation is similar to the Delete operation for notifications in Documentum TaskSpace.

3.2.12.2 Java syntax

```
public void remove(String identifier)
throws BpmServiceException
```

3.2.12.3 Parameters

Parameter	Data type	Description
identifier	String	The identifier that uniquely identifies the object ID of the notification that has been removed.

3.2.12.4 Example

Example 3-23: Java: Removing a notification

The following Java samples are available for the remove operation:

- *Removing a notification retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null, Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.remove(taskId);
```

- *Removing a notification retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName", Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.remove(taskId);
```

- *Removing a notification retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();
```

```
my_service.claim(taskId);
my_service.remove(taskId);
```

Example 3-24: C#: Removing a notification

The following C# samples are available for the remove operation:

- *Removing a notification retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.remove(taskId);
```

- *Removing a notification retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.remove(taskId);
```

- *Removing a notification retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
```

```
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.remove(taskId);
```

3.2.13 fail operation

3.2.13.1 Description

This operation enables the Actual Owner of a task to complete the execution of the task by raising a fault.

3.2.13.2 Java syntax

```
public void fail(String identifier,
String faultName,
Object faultData)
throws BpmServiceException
```

3.2.13.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task on which a fault is raised.
faultName	String	Name of the fault or exception.
faultData	Object	Content of the exception.

3.2.13.4 Returns

If the TaskManagementService interface does not define a fault, the operation returns the illegalOperationFault exception. If fault name or fault data is not defined, the operation returns the illegalArgumentFault exception. In OpenText Documentum CM 6.5, this function will always return the illegalOperationFault exception.

3.2.13.5 Example

Example 3-25: Java: Failing a task

The following Java samples are available for the fail operation:

- *Raising a fault for a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null, Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
```

- *Raising a fault for a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
```

- *Raising a fault for a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
```

```
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
```

Example 3-26: C#: Failing a task

The following C# samples are available for the fail operation:

- *Raising a fault for a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
```

- *Raising a fault for a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);
```

```
List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
```

- *Raising a fault for a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
```

3.2.14 setPriority operation

3.2.14.1 Description

The setPriority operation enables the Actual Owner and Business Administrators to change the priority of a task. The client must specify the integer value of the new priority. A user can change the priority of a task when the task becomes critical and must be escalated. The user can change the priority after the task is de-escalated.

3.2.14.2 Java syntax

```
public void setPriority(String identifier,
BigInteger priority)
throws BpmServiceException
```

3.2.14.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task to which a priority must be set.
priority	BigInteger	The value representing the new priority.

3.2.14.4 Example

Example 3-27: Java: Setting priority for a task

The following Java samples are available for the setPriority operation:

- *Setting priority for a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
```

- *Setting priority for a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners", "workqueueName",
    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
```

- *Setting priority for a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
```

Example 3-28: C#: Setting priority for a task

The following C# samples are available for the setPriority operation:

- *Setting priority for a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
```

- *Setting priority for a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
```

```
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
```

- *Setting priority for a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
```

3.2.15 addAttachment operation

3.2.15.1 Description

The addAttachment operation enables the Actual Owner and Business Administrators to add an attachment to a selected task.

The addAttachment operation is similar to the Add Attachments operation in Documentum TaskSpace.

3.2.15.2 Java syntax

```
public void addAttachment(String identifier,
String attachmentName,
String accessType,
ObjectIdentity attachment)
throws BpmServiceException
```

3.2.15.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task to which an attachment must be added.
attachmentName	String	The name of the attachment associated with the task.
accessType	String	The type of access allowed for the attachment associated with the task. The only valid value for the accessType parameter is “Objectid”. The UnsupportedOperationException fault is returned for all other values.
attachment	ObjectIdentity	The attachment or file included with a task.

3.2.15.4 Example

Example 3-29: Java: Adding attachment to a task

The following Java samples are available for the addAttachment operation:

- *Adding an attachment to a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null, Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
```

- *Adding an attachment to a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;
```

```
taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
```

- *Adding an attachment to a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
```

Example 3-30: C#: Adding attachment to a task

The following C# samples are available for the addAttachment operation:

- *Adding an attachment to a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
```

- *Adding an attachment to a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
```

- *Adding an attachment to a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
```

3.2.16 getAttachmentInfos operation

3.2.16.1 Description

The getAttachmentInfos operation retrieves the attributes of the attachments associated with a selected task.

3.2.16.2 Java syntax

```
public List<TAttachmentInfo> getAttachmentInfos(String identifier)
throws BpmServiceException
```

3.2.16.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task for which the attributes of all its attachments must be retrieved.

3.2.16.4 Example

Example 3-31: Java: Getting attachment information for a task

The following Java samples are available for the getAttachmentInfos operation:

- *Getting attachment information for a task retrieved by performing the getmytaskabstracts operation*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null, Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachmentInfos(taskId);
```

- *Getting attachment information for a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;
```

```
taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachmentInfos(taskId);
```

- *Getting attachment information for a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachmentInfos(taskId);
```

Example 3-32: C#: Getting attachment information for a task

The following C# samples are available for the getAttachmentInfos operation:

- *Getting attachment information for a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
```

```
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachmentInfos(taskId);
```

- *Getting attachment information for a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachmentInfos(taskId);
```

- *Getting attachment information for a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachmentInfos(taskId);
```

3.2.17 getAttachments operation

3.2.17.1 Description

The getAttachments operation retrieves the attributes and contents of all attachments associated with a selected task.

3.2.17.2 Java syntax

```
public List<TAttachment> getAttachments(String identifier,
String attachmentName)
throws BpmServiceException
```

3.2.17.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task with which attachments are associated.
attachmentName	String	The object name of the attachment. If NULL, the attributes and contents of all attachments associated with the task are returned.

3.2.17.4 Returns

A list TAttachmentInfo containing the attributes and contents of all attachments associated with a task, is returned.

3.2.17.5 Example

Example 3-33: Java: Getting attachments from a task

The following Java samples are available for the getAttachments operation:

- *Getting attachments of a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null, Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
```

```
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachments(taskId, null);
```

- *Getting attachments of a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachments(taskId, null);
```

- *Getting attachments of a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachments(taskId, null);
```

Example 3-34: C#: Getting attachments from a task

The following C# samples are available for the getAttachments operation:

- *Getting attachments of a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
```

```
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachments(taskId, null);
```

- *Getting attachments of a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachments(taskId, null);
```

- *Getting attachments of a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachments(taskId, null);
```

3.2.18 deleteAttachments operation

3.2.18.1 Description

The deleteAttachments operation deletes all attachments with a specified name from a selected task. If the task is associated with multiple attachments of the same name, all attachments are deleted. Documents in packages are not affected by this operation. Only workflow-related attachments can be deleted or removed from the task. This method call must not be applied to documents sent in packages. Attachments provided by the enclosing context are not affected by this operation.

3.2.18.2 Java syntax

```
public void deleteAttachments(String identifier,
String attachmentName)
throws BpmServiceException
```

3.2.18.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task from which attachments must be deleted.
attachmentName	String	The object name of the attachment(s) associated with the task. If this value is NULL, no attachment is deleted.

3.2.18.4 Example

Example 3-35: Java: Deleting attachments from a task

The following Java samples are available for the deleteAttachments operation:

- *Deleting attachments in a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
```

```
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.deleteAttachments(taskId, "attachment name");
```

- *Deleting attachments in a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.deleteAttachments(taskId, "attachment name");
```

- *Deleting attachments in a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.deleteAttachments(taskId, "attachment name");
```

Example 3-36: C#: Deleting attachments from a task

The following C# samples are available for the deleteAttachments operation:

- *Deleting attachments in a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
```

```
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.deleteAttachments(taskId, "attachment name");
```

- *Deleting attachments in a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.deleteAttachments(taskId, "attachment name");
```

- *Deleting attachments in a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.deleteAttachments(taskId, "attachment name");
```

3.2.19 addComment operation

3.2.19.1 Description

The addComment operation enables Potential Owners, Actual Owner, and Business Administrators to add a comment to the first package of a task. While in BPM client you can add separate comments to separate packages, this behavior is not supported by the addComment operation. This comment is then forwarded to all subsequent activities in the process.

The addComment operation is similar to the Add a comment operation in Documentum TaskSpace.

3.2.19.2 Java syntax

```
public void addComment(String identifier,
String text)
throws BpmServiceException
```

3.2.19.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task to which comments must be added.
text	String	The text content of the comment.

3.2.19.4 Example

Example 3-37: Java: Adding a comment to a task

The following Java samples are available for the addComment operation:

- *Adding a comment to a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null, Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addComment(taskId, "Text of the note");
```

- *Adding a comment to a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addComment(taskId, "Text of the note");
```

- *Adding a comment to a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addComment(taskId, "Text of the note");
```

Example 3-38: C#: Adding a comment to a task

The following C# samples are available for the addComment operation:

- *Adding a comment to a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);
```

```
List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addComment(taskId, "Text of the note");
```

- *Adding a comment to a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addComment(taskId, "Text of the note");
```

- *Adding a comment to a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addComment(taskId, "Text of the note");
```

3.2.20 getComments operation

3.2.20.1 Description

The getComments operation retrieves all comments associated with a task selected.

3.2.20.2 Java syntax

```
public List<TComment> getComments(String identifier)
throws BpmServiceException
```

3.2.20.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task from which all comments must be retrieved.

3.2.20.4 Returns

A list TComment containing information about all comments associated with the task, is returned.

3.2.20.5 Example

Example 3-39: Java: Getting comments associated with a task

The following Java samples are available for the getComments operation:

- *Getting comments associated with a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addComment(taskId, "Text of the note");
my_service.getComments(taskId);
```

- *Getting comments associated with a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addComment(taskId, "Text of the note");
my_service.getComments(taskId);
```

- *Getting comments associated with a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.addComment(taskId, "Text of the note");
my_service.getComments(taskId);
```

Example 3-40: C#: Getting comments associated with a task

The following C# samples are available for the getComments operation:

- *Getting comments associated with a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
```

```
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addComment(taskId, "Text of the note");
my_service.getComments(taskId);
```

- *Getting comments associated with a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addComment(taskId, "Text of the note");
my_service.getComments(taskId);
```

- *Getting comments associated with a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.addComment(taskId, "Text of the note");
my_service.getComments(taskId);
```

3.2.21 skip operation

3.2.21.1 Description

This operation is not supported. If the client calls this API, the UnsupportedOperationException fault is returned.

3.2.22 forward operation

3.2.22.1 Description

The forward operation enables Potential Owners, Actual Owner, or Business Administrators to forward a task in the *Ready*, *Reserved*, or *InProgress* state, to another organizational entity (user or work queue).

The forward operation is similar to the Forward operation in Documentum TaskSpace.

3.2.22.2 Java syntax

```
public void forward(String identifier,
TOrganizationalEntity organizationalEntity)
throws BpmServiceException
```

3.2.22.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task that must be forwarded.
organizationalEntity	TOrganizationalEntity	The name of the organizational entity to which the task is forwarded. The organizationalEntity can contain only one user or work queue name.

3.2.22.4 Example

Example 3-41: Java: Forwarding a task

The following Java samples are available for the forward operation:

- *Forwarding a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);
```

```
List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.forward(taskId);
```

- *Forwarding a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.forward(taskId);
```

- *Forwarding a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.forward(taskId);
```

Example 3-42: C#: Forwarding a task

The following C# samples are available for the forward operation:

- *Forwarding a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
```

```
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.forward(taskId);
```

- *Forwarding a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.forward(taskId);
```

- *Forwarding a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.forward(taskId);
```

3.2.23 delegate operation

3.2.23.1 Description

The delegate operation enables Potential Owners, Actual Owner, or Business Administrators of a task in the *Ready*, *Reserved*, or *InProgress* state, to assign the task to another user or work queue, making that user or work queue the Actual Owner of the task. A delegated task is transitioned to the *Ready* state.

The delegate operation is similar to the Delegate operation in Documentum TaskSpace.

3.2.23.2 Java syntax

```
public void delegate(String identifier,
TOrganizationalEntity organizationalEntity)
throws BpmServiceException
```

3.2.23.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task that must be delegated.
organizationalEntity	TOrganizationalEntity	The name of the organizational entity to which the task is delegated. The organizationalEntity can contain only one user or work queue name.

3.2.23.4 Example

Example 3-43: Java: Delegating a task

The following Java samples are available for the delegate operation:

- *Delegating a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null, Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();
```

```
my_service.claim(taskId);
my_service.delegate(taskId);
```

- *Delegating a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.delegate(taskId);
```

- *Delegating a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.delegate(taskId);
```

Example 3-44: C#: Delegating a task

The following C# samples are available for the delegate operation:

- *Delegating a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
```

```
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.delegate(taskId);
```

- *Delegating a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.delegate(taskId);
```

- *Delegating a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.delegate(taskId);
```

3.2.24 getRendering operation

3.2.24.1 Description

This operation is not supported. If the client calls this API, the UnsupportedOperationException fault is returned.

3.2.25 getRenderingTypes operation

3.2.25.1 Description

This operation is not supported. If the client calls this API, the UnsupportedOperationException fault is returned.

3.2.26 getTaskInfo operation

3.2.26.1 Description

The getTaskInfo operation retrieves information about tasks and notifications.

3.2.26.2 Java syntax

```
public TTask getTaskInfo(String identifier)
throws BpmServiceException
```

3.2.26.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task for which information must be retrieved.

3.2.26.4 Returns

Returns a data object of type TTask.

3.2.26.5 Example

Example 3-45: Java: Getting information about a task

The following Java samples are available for the getTaskInfo operation:

- *Getting information about a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
```

```
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getTaskInfo(taskId);
```

- *Getting information about a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName", Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getTaskInfo(taskId);
```

- *Getting information about a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getTaskInfo(taskId);
```

Example 3-46: C#: Getting information about a task

The following C# samples are available for the getTaskInfo operation:

- *Getting information about a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.getTaskInfo(taskId);
```

- *Getting information about a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.getTaskInfo(taskId);
```

- *Getting information about a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;
```

```
my_service.claim(taskId);
my_service.getTaskInfo(taskId);
```

3.2.27 getTaskDescription operation

3.2.27.1 Description

The getTaskDescription operation retrieves the presentation description of tasks and notifications. Presentation data is derived from the task definition or notification definition such as name, subject, or description.

3.2.27.2 Java syntax

```
public String getTaskDescription(String identifier,
String contentType)
throws BpmServiceException
```

3.2.27.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task whose description must be retrieved.
contentType	String	Optional. The presentation description of the task or notification in the specified mime type. By default, the content type is text/plain. Only “text/plain” content type is supported.

3.2.27.4 Returns

Returns presentation description of tasks and notifications of String type.

3.2.27.5 Example

Example 3-47: Java: Getting the description of a task

The following Java samples are available for the getTaskDescription operation:

- *Getting the description of a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
```

```
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);
String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getTaskDescription(taskId, "text/plain");
```

- *Getting the description of a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getTaskDescription(taskId, "text/plain");
```

- *Getting the description of a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getTaskDescription(taskId, "text/plain");
```

Example 3-48: C#: Getting the description of a task

The following C# samples are available for the getTaskDescription operation:

- *Getting the description of a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;
```

```
private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.getTaskDescription(taskId, "text/plain");
```

- *Getting the description of a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.getTaskDescription(taskId, "text/plain");
```

- *Getting the description of a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.getTaskDescription(taskId, "text/plain");
```

3.2.28 setFault operation

3.2.28.1 Description

This operation is not supported. If the client calls this API, the UnsupportedOperationException fault is returned.

3.2.29 deleteFault operation

3.2.29.1 Description

This operation is not supported. If the client calls this API, the UnsupportedOperationException fault is returned.

3.2.30 getInput operation

3.2.30.1 Description

The getInput operation enables Potential Owners, Actual Owner, or Business Administrators to get process data as input for the task being processed. Process data is based on the value of partName that is passed in to the function.

3.2.30.2 Java syntax

```
public Object getInput(String identifier,
String partName)
throws BpmServiceException
```

3.2.30.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task whose input message must be retrieved.
partName	String	The name of a package or process variable. For example, if the CustomerName process variable is defined the value “John Tesh”, and the getInput function is called with partName=CustomerName, this function will return “John Tesh”.

3.2.30.4 Returns

Returns process data based on the partName requested. If partName is a primitive type process variable, the corresponding process variable value is returned in its type. If it is a package, a String containing the XML is returned.

If a package name is passed in the partName parameter, the package information is returned. If a process variable name is passed in the partName parameter, the process variable value is returned.

3.2.30.5 Example

Example 3-49: Java: Getging input for a task

The following Java samples are available for the getInput operation:

- *Getting input for a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
```

- *Getting input for a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
```

- *Getting input for a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
```

```
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
```

Example 3-50: C#: Getting input for a task

The following C# samples are available for the getInput operation:

- *Getting input for a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
```

- *Getting input for a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;
```

```
my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
```

- *Getting input for a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
```

3.2.31 getOutput operation

3.2.31.1 Description

The getOutput operation enables the Actual Owner and Business Administrators to get process data as output for the task being processed. Process data is based on the value of partName that is passed in to the function.

3.2.31.2 Java syntax

```
public Object getOutput(String identifier,
String partName)
throws BpmServiceException
```

3.2.31.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task whose output message must be retrieved.
partName	String	Name of the package or process variable.

3.2.31.4 Returns

Returns any output type or NULL if the partName is not specified.

3.2.31.5 Example

Example 3-51: Java: Getting output for a task

The following Java samples are available for the getOutput operation:

- *Getting output for a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
my_service.getOutput(taskId, "Package0");
my_service.getOutput(taskId, "ProcessVariableName");
my_service.getOutput(taskId, "ProcessParameterName");
```

- *Getting output for a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
my_service.getOutput(taskId, "Package0");
my_service.getOutput(taskId, "ProcessVariableName");
my_service.getOutput(taskId, "ProcessParameterName");
```

- *Getting output for a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
```

```
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
my_service.getOutput(taskId, "Package0");
my_service.getOutput(taskId, "ProcessVariableName");
my_service.getOutput(taskId, "ProcessParameterName");
```

Example 3-52: C#: Getting output for a task

The following C# samples are available for the getOutput operation:

- *Getting output for a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
my_service.getOutput(taskId, "Package0");
my_service.getOutput(taskId, "ProcessVariableName");
my_service.getOutput(taskId, "ProcessParameterName");
```

- *Getting output for a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
```

```
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
my_service.getOutput(taskId, "Package0");
my_service.getOutput(taskId, "ProcessVariableName");
my_service.getOutput(taskId, "ProcessParameterName");
```

- *Getting output for a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
my_service.getOutput(taskId, "Package0");
my_service.getOutput(taskId, "ProcessVariableName");
my_service.getOutput(taskId, "ProcessParameterName");
```

3.2.32 setOutput operation

3.2.32.1 Description

The setOutput operation enables the Actual Owner of a task to set process data as output for the task being processed. Process data is based on the partName that is passed in to the function.

3.2.32.2 Java syntax

```
public void setOutput(String identifier,
String partName,
Object taskData)
```

3.2.32.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task whose output message must be specified.
partName	String	The name of the package or process variable.
taskData	Object	The output data of task. The value corresponding to `partName`.

3.2.32.4 Example

Example 3-53: Java: Setting output information for a task

The following Java samples are available for the setOutput operation:

- *Setting output for a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
my_service.getInput(taskId, "ProcessVariableName");
my_service.getInput(taskId, "ProcessParameterName");
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
```

- *Setting output for a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
my_service.getInput(taskId, "ProcessVariableName");
my_service.getInput(taskId, "ProcessParameterName");
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
```

- *Setting output for a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
my_service.getInput(taskId, "ProcessVariableName");
my_service.getInput(taskId, "ProcessParameterName");
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
```

Example 3-54: C#: Setting output information for a task

The following C# samples are available for the setOutput operation:

- *Setting output for a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
```

```
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
my_service.getInput(taskId, "ProcessVariableName");
my_service.getInput(taskId, "ProcessParameterName");
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
```

- *Setting output for a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
my_service.getInput(taskId, "ProcessVariableName");
my_service.getInput(taskId, "ProcessParameterName");
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
```

- *Setting output for a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);
```

```
String taskId = task[0].id;

my_service.claim(taskId);
my_service.getInput(taskId, "Package0");
my_service.getInput(taskId, "ProcessVariableName");
my_service.getInput(taskId, "ProcessParameterName");
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
```

3.2.33 deleteOutput operation

3.2.33.1 Description

This operation is not supported. If the client calls this API, the UnsupportedOperationException fault is returned.

3.2.34 getFault operation

3.2.34.1 Description

The getFault operation enables the Actual Owner and Business Administrators to retrieve the name and message of a fault (exception) that occurs when a task is not successfully executed.

3.2.34.2 Java syntax

```
public void getFault(String identifier,
Holder<String> faultName,
Holder<Object> faultData)
throws BpmServiceException
```

3.2.34.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task whose fault is retrieved.
faultName	Holder<String>	The name of the fault.
faultData	Holder<Object>	The fault message.

3.2.34.4 Example

Example 3-55: Java: Getting the fault for a task

The following Java samples are available for the getFault operation:

- *Getting fault for a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null,    Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
my_service.getFault(taskId, "name of the fault", "data about the fault");
```

- *Getting fault for a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
my_service.getFault(taskId, "name of the fault", "data about the fault");
```

- *Getting fault for a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
```

```
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
my_service.getFault(taskId, "name of the fault", "data about the fault");
```

Example 3-56: C#: Getting the fault for a task

The following C# samples are available for the getFault operation:

- *Getting fault for a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
my_service.getFault(taskId, "name of the fault", "data about the fault");
```

- *Getting fault for a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
```

```
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
my_service.getFault(taskId, "name of the fault", "data about the fault");
```

- *Getting fault for a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
my_service.getFault(taskId, "name of the fault", "data about the fault");
```

3.2.35 activate operation

3.2.35.1 Description

This operation is not supported. If the client calls this API, the UnsupportedOperationException fault is returned.

3.2.36 nominate operation

3.2.36.1 Description

The nominate operation enables Business Administrators to nominate an organizational entity (user or work queue) to process a task in the *Ready*, *Reserved*, or *InProgress* state. If the task is nominated to an individual, then the task state is set to *Ready*.

The nominate operation is similar to the Assign operation in Documentum TaskSpace.

3.2.36.2 Java syntax

```
public void nominate(String identifier,
TOrganizationalEntity organizationalEntity)
throws BpmServiceException
```

3.2.36.3 Parameters

Parameter	Data type	Description
identifier	String	The queue item object ID that identifies the task that must be nominated to an organizational entity.
organizationalEntity	TOrganizationalEntity	The organizational entity (user or work queue) to which the task is nominated.

3.2.36.4 Example

Example 3-57: Java: Nominating a task

The following Java samples are available for the nominate operation:

- *Nominating a task retrieved by performing the getmytaskabstracts operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null, Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.nominate(taskId, orgEntity);
```

- *Nominating a task retrieved by performing the getmytasks operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);
```

```
String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.nominate(taskId, orgEntity);
```

- *Nominating a task retrieved by performing the query operation:*

```
              import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.nominate(taskId, orgEntity);
```

Example 3-58: C#: Nominating a task

The following C# samples are available for the nominate operation:

- *Nominating a task retrieved by performing the getmytaskabstracts operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.nominate(taskId, orgEntity);
```

- *Nominating a task retrieved by performing the getmytasks operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
```

```
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.nominate(taskId, orgEntity);
```

- *Nominating a task retrieved by performing the query operation:*

```
              using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.nominate(taskId, orgEntity);
```

3.2.37 setGenericHumanRole operation

3.2.37.1 Description

This operation is not supported. If the client calls this API, the UnsupportedOperationException fault is returned.

3.2.38 getMyTaskAbstracts operation

3.2.38.1 Description

The getMyTaskAbstracts operation retrieves the abstracts or summary data of tasks or notifications. This operation is used to obtain the following data required to display: type of task, human interaction required for the task (role(s)), work queue to which the task belongs, state of the task, and maximum number of tasks to display in the task list. If a work queue is not specified, only tasks in the logged in user's personal Inbox are displayed.

When this operation is performed, the following results are displayed based on the role of the logged in user:

- If the user is a Business Administrator or a Task Stakeholder, all tasks in the specified work queue are listed.
- If the user is a Potential Owner and a work queue name is specified, all tasks in the specified work queue are listed. The user can then claim tasks from the list of tasks. However, if the user is a Potential Owner and a work queue name is not specified, all tasks in the user's Inbox are listed.
- If the user is the Actual Owner, all tasks in the user's Inbox are listed.

The generic human role must be passed in so that the logged in user can view different tasks based on the user's role. For example, if the logged in user who is a queue manager performs this operation into which the “businessAdministrators” human role is passed , all tasks in the specified work queue are listed.

3.2.38.2 Java syntax

```
public List<TTaskAbstract> getMyTaskAbstracts(String taskType,
String genericHumanRole,
String workQueue,
List<TStatus> status,
String whereClause,
String createdOnClause,
Integer maxTasks)
throws BpmServiceException;
```

3.2.38.3 Parameters

Parameter	Data type	Description
taskType	String	The type of tasks being queried. The task types are: • “TASKS” – All tasks • “NOTIFICATIONS” – All notifications • “ALL” – All tasks and notifications **Note:** When work queue tasks are queried, only tasks are returned and not notifications.

Parameter	Data type	Description
genericHumanRole	String	Generic human role defines what a person or a group of people performing a specific role in an organization, can do with tasks and notifications. For more information, see "Generic human roles" on page 198. The possible string values of this parameter are: • "actualOwner" • "businessAdministrators" • "taskStakeholders" • "potentialOwners"
workQueue	String	This is the name of the work queue that holds tasks to be performed. Optional if "potentialOwners" is specified in the genericHumanRole parameter. However, work queue name must be specified if the "businessAdministrators" or "taskStakeholders" generic role is specified in the genericHumanRole parameter.

status	List<TStatus>	Human tasks can have one of the following states: "Ready", "Reserved", "In Progress", and "Suspended".
		
		The mappings for the various states are as follows:
		`TStatus.READY -> IDfWorkitem.DF_WI_STATE_DORMANT`
		`TStatus.IN_PROGRESS -> DfWorkitem.DF_WI_STATE_ACQUIRED`
		`TStatus.SUSPENDED -> IDfWorkitem.DF_WI_STATE_PAUSED, IDfWorkitem.DF_WI_STATE_DHALTED, IDfWorkitem.DF_WI_STATE_PHALTED`
		`TStatus.FAILED -> IDfWorkitem.DF_WI_STATE_AHALTED, IDfWorkitem.DF_WI_STATE_FAULTED`
		`TStatus.COMPLETED -> IDfWorkitem.DF_WI_STATE_FINISHED`
		
		Any other task state is ignored.

whereClause	String	A DQL WHERE clause that limits the data loaded from source columns. This clause is used to run the getMyTaskAbstracts operation based on a single column using any of the following operators: equals ("="), not equals ("<>"), less than ("<"), greater than (">"), less than or equals ("<="), and greater than or equals (">="). For example, the whereClause can be defined as "dmi_workitem.r_priority>= 30" to query for task abstracts whose priority value is greater than or equal to "30".

Parameter	Data type	Description
createdOnClause	String	A DQL WHERE clause that indicates the time when the task was created. This clause is always used along with the WHERE clause. This clause is used to execute the getMyTaskAbstracts operation based on a single column using any of the following operators: equals (“=”), not equals (“<>”), less than (“<”), greater than (“>”), less than or equals (“<=”), and greater than or equals (“>=”). For example, the createdOnClause can be defined as “dmi_queue_item.date_sent = Date('1/21/2008')” to query for task abstracts whose sent date is “1/21/2008”.
maxTasks	Int	The number of tasks for which abstracts must be retrieved. The default value is 100, if no value is specified. If a value is specified for maxTasks, the number of task abstracts returned by the query will not exceed this limit.

3.2.38.4 Returns

Returns a list of tasks or notifications with summary data of each task or notification.

3.2.38.5 Example

Example 3-59: Java: Getting my task abstracts

```
  import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskAbstract;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTaskAbstract> taskList;

taskList = my_service.getMyTaskAbstracts("TASKS", "PotentialOwners",
    null, Status:IDfWorkitem.DF_WI_STATE_DORMANT,null, null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachments(taskId, null);
my_service.getTaskDescription(taskId, "text/plain");
my_service.deleteAttachments(taskId, "attachment name");
my_service.getTaskInfo(taskId);
my_service.addComment(taskId, "Text of the note");
my_service.getComments(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.suspendUntil(taskId, untilTime);
my_service.resume(taskId);
my_service.getInput(taskId, "Package0");
my_service.getInput(taskId, "ProcessVariableName");
my_service.getInput(taskId, "ProcessParameterName");
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
```

```
my_service.getOutput(taskId, "Package0");
my_service.getOutput(taskId, "ProcessVariableName");
my_service.getOutput(taskId, "ProcessParameterName");
my_service.forward(taskId);
my_service.claim(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
my_service.getFault(taskId, "name of the fault", "data about the fault");
my_service.nominate(taskId, orgEntity);
my_service.claim(taskId);
my_service.delegate(taskId);
my_service.claim(taskId);
my_service.remove(taskId);
my_service.claim(taskId);
my_service.release(taskId);
```

Example 3-60: C#: Getting my task abstracts

```
  using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();
my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskAbstract> taskAbstractList = my_service.GetMyTaskAbstracts
    ("ALL", "PotentialOwners", null,statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachments(taskId, null);
my_service.getTaskDescription(taskId, "text/plain");
my_service.deleteAttachments(taskId, "attachment name");
my_service.getTaskInfo(taskId);
my_service.addComment(taskId, "Text of the note");
my_service.getComments(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.suspendUntil(taskId, untilTime);
my_service.resume(taskId);
my_service.getInput(taskId, "Package0");
my_service.getInput(taskId, "ProcessVariableName");
my_service.getInput(taskId, "ProcessParameterName");
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
my_service.getOutput(taskId, "Package0");
my_service.getOutput(taskId, "ProcessVariableName");
my_service.getOutput(taskId, "ProcessParameterName");
my_service.forward(taskId);
my_service.claim(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
my_service.getFault(taskId, "name of the fault", "data about the fault");
my_service.nominate(taskId, orgEntity);
my_service.claim(taskId);
```

```
my_service.delegate(taskId);
my_service.claim(taskId);
my_service.remove(taskId);
my_service.claim(taskId);
my_service.release(taskId);
```

3.2.39 getMyTasks operation

3.2.39.1 Description

The getMyTasks operation retrieves details of tasks or notifications. This operation is used to obtain the following data required to display a task list, along with the following details of each task or notification: type of task, human interaction required for the task, work queue to which the task belongs, and state of the task.

When this operation is performed, the following results are displayed based on the role of the logged in user:

- If the user is a Business Administrator or a Task Stakeholder, all tasks in the specified work queue are listed.
- If the user is a Potential Owner and a work queue name is specified, all tasks in the specified work queue are listed. The user can then claim tasks from the list of tasks. However, if the user is a Potential Owner and a work queue name is not specified, all tasks in the user's Inbox are listed.
- If the user is the Actual Owner, all tasks in the user's Inbox are listed.

3.2.39.2 Java syntax

```
public List<TTask> getMyTasks(String taskType,
String genericHumanRole,
String workQueue,
List<TStatus> status,
String whereClause,
String createdOnClause,
Integer maxTasks)
throws BpmServiceException
```

3.2.39.3 Parameters

Parameter	Data type	Description
taskType	String	The type of tasks being queried. The task types are: • "TASKS" – All tasks • "NOTIFICATIONS" – All notifications • "ALL" – All tasks and notifications **Note:** When work queue tasks are queried, only tasks are returned and not notifications.

Parameter	Data type	Description
genericHumanRole	String	Generic human role defines what a person or a group of people performing a specific role in an organization, can do with tasks and notifications. For more information, see "Generic human roles" on page 198. The possible string values of this parameter are: • "actualOwner" • "businessAdministrators" • "taskStakeholders" • "potentialOwners"
workQueue	String	This is the name of the work queue that holds tasks to be performed. Optional if "potentialOwners" is specified in the genericHumanRole parameter. However, work queue name must be specified if the "businessAdministrators" or "taskStakeholders" generic role is specified in the genericHumanRole parameter.

status	List<TStatus>	Human tasks can have one of the following states: "Ready", "Reserved", "In Progress", and "Suspended".
		
		The mappings for the various states are as follows:
		`TStatus.READY -> IDfWorkitem.DF_WI_STATE_DORMANT`
		`TStatus.IN_PROGRESS -> DfWorkitem.DF_WI_STATE_ACQUIRED`
		`TStatus.SUSPENDED -> IDfWorkitem.DF_WI_STATE_PAUSED, IDfWorkitem.DF_WI_STATE_DHALTED, IDfWorkitem.DF_WI_STATE_PHALTED`
		`TStatus.FAILED -> IDfWorkitem.DF_WI_STATE_AHALTED, IDfWorkitem.DF_WI_STATE_FAULTED`
		`TStatus.COMPLETED -> IDfWorkitem.DF_WI_STATE_FINISHED`
		
		Any other task state is ignored.

whereClause	String	A DQL WHERE clause that limits the data loaded from source columns. This clause is used to run the getMyTasks operation based on a single column using any of the following operators: equals ("="), not equals ("<>"), less than ("<"), greater than (">"), less than or equals ("<="), and greater than or equals (">="). For example, the whereClause can be defined as "ddmi_workitem.r_priority>= 30" to query for tasks whose priority value is greater than or equal to "30".

Parameter	Data type	Description
createdOnClause	String	A DQL WHERE clause that indicates the time when the task was created. This clause is always used along with the WHERE clause. This clause is used to run the getMyTasks operation based on a single column using any of the following operators: equals (“=”), not equals (“<>”), less than (“<”), greater than (“>”), less than or equals (“<=”), and greater than or equals (“>=”). For example, the createdOnClause can be defined as “dmi_queue_item.date_sent = Date('1/21/2008')” to query for tasks whose sent date is “1/21/2008”.
maxTasks	Int	The number of task details the query must retrieve for a task in the task list. The default value is 100, if no value is specified. If a value is specified for maxTasks, the number of task details returned by the query will not exceed this limit.

3.2.39.4 Returns

Returns a list of tasks or notifications.

3.2.39.5 Example

Example 3-61: Java: Getting my tasks

```
import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTask;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

List<TTask> taskList;

taskList = my_service.getMyTasks("TASKS", "PotentialOwners",
    "workqueueName",    Status:IDfWorkitem.DF_WI_STATE_DORMANT,
    "dmi_workitem.r_runtime_state=0", null, 1);

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachments(taskId, null);
my_service.getTaskDescription(taskId, "text/plain");
my_service.deleteAttachments(taskId, "attachment name");
my_service.getTaskInfo(taskId);
my_service.addComment(taskId, "Text of the note");
my_service.getComments(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.suspendUntil(taskId, untilTime);
my_service.resume(taskId);
my_service.getInput(taskId, "Package0");
my_service.getInput(taskId, "ProcessVariableName");
my_service.getInput(taskId, "ProcessParameterName");
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
```

```
my_service.setOutput(taskId, "ProcessParameterName", 1);
my_service.getOutput(taskId, "Package0");
my_service.getOutput(taskId, "ProcessVariableName");
my_service.getOutput(taskId, "ProcessParameterName");
my_service.forward(taskId);
my_service.claim(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
my_service.getFault(taskId, "name of the fault", "data about the fault");
my_service.nominate(taskId, orgEntity);
my_service.claim(taskId);
my_service.delegate(taskId);
my_service.claim(taskId);
my_service.remove(taskId);
my_service.claim(taskId);
my_service.release(taskId);
```

Example 3-62: C#: Getting my tasks

```
using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);
ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTask> taskList = my_service.GetMyTasks("ALL", "PotentialOwners",
    null, statusList, "", "", 1);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachments(taskId, null);
my_service.getTaskDescription(taskId, "text/plain");
my_service.deleteAttachments(taskId, "attachment name");
my_service.getTaskInfo(taskId);
my_service.addComment(taskId, "Text of the note");
my_service.getComments(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.suspendUntil(taskId, untilTime);
my_service.resume(taskId);
my_service.getInput(taskId, "Package0");
my_service.getInput(taskId, "ProcessVariableName");
my_service.getInput(taskId, "ProcessParameterName");
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
my_service.getOutput(taskId, "Package0");
my_service.getOutput(taskId, "ProcessVariableName");
my_service.getOutput(taskId, "ProcessParameterName");
my_service.forward(taskId);
my_service.claim(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
my_service.getFault(taskId, "name of the fault", "data about the fault");
my_service.nominate(taskId, orgEntity);
```

```
my_service.claim(taskId);
my_service.delegate(taskId);
my_service.claim(taskId);
my_service.remove(taskId);
my_service.claim(taskId);
my_service.release(taskId);
```

3.2.40 query operation

3.2.40.1 Description

The query operation retrieves a specific list of tasks that conform to the search criteria specified in the query.

3.2.40.2 Java syntax

```
public TTaskQueryResultSet query(String selectClause,
String whereClause,
String orderByClause,
Integer maxTasks,
Integer taskIndexOffset)
throws BpmServiceException;
```

3.2.40.3 Parameters

Parameter	Data type	Description
selectClause	String	Mandatory. Represents the list of columns on which the query must be run. For example, the selectClause can be defined as “dmi_workitem.r_priority,Customer:id” to retrieve the priority and customer ID details of tasks in the task list.

Parameter	Data type	Description
whereClause	String	Mandatory. A DQL WHERE clause that limits the tasks to retrieve from source columns. This clause is used to run the query operation using any of the following operators: equals (“=”), not equals (“<>”), less than (“<”), greater than (“>”), less than or equals (“<=”), and greater than or equals (“>=”). For example, the whereClause can be defined as “((Customer:customerid ='001' and dmi_workitem.r_performer_name ='tuser1') or / boolean.bool_var=false)” to retrieve a task whose customer ID is 001, and performer name is 'tuser1' or a task where the value of the boolean.bool_var column is 'false'.
orderByClause	String	Optional. The “ORDER BY” clause sorts the results in the ascending or descending order based on one or more properties. This clause is used to run the query operation using any of the following operators: equals (“=”), not equals (“<>”), less than (“<”), greater than (“>”), less than or equals (“<=”), and greater than or equals (“>=”). For example, the orderByClause can be defined as “dmi_workitem.r_priority ASC, dmi_queue_item.date_sent DESC” to retrieve tasks sorted in the ascending order of priority, and descending order of the task sent date columns.

Parameter	Data type	Description
maxTasks	Int	The number of tasks the query must retrieve in the task list. The default value is 100, if no value is specified. If a value is specified for maxTasks, the number of task details returned by the query will not exceed this limit.
taskIndexOffset	Int	Used to perform multiple identical queries, and run queries on result sets where the maxTasks size exceeds the query limit. The default value is 0, if no value is specified.

3.2.40.4 Returns

Returns a TTaskQueryResultSet containing all tasks that conform to the search criteria in the query.

3.2.40.5 Example

Example 3-63: Java: Running a query to list tasks

```
import org.example.ws_ht.api.wsdl.client.ITaskManagementService;
import org.example.ws_ht.api.TTaskQueryResultSet;

ITaskManagementService my_service;
IServiceContext context = getServiceContext();
my_service = ServiceFactory.getInstance().getLocalService
    (ITaskManagementService.class, context);

TTaskQueryResultSet taskSet;
List<TTaskQueryResultRow> taskList;

taskSet = my_service.query("dmi_workitem.r_object_id,dmi_queue_item.
    r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0","dmi_queue_item.name",5,1);
taskList = taskSet.getRow();

String taskId = taskList.get(0).getId();

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachments(taskId, null);
my_service.getTaskDescription(taskId, "text/plain");
my_service.deleteAttachments(taskId, "attachment name");
my_service.getTaskInfo(taskId);
my_service.addComment(taskId, "Text of the note");
my_service.getComments(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.suspendUntil(taskId, untilTime);
my_service.resume(taskId);
my_service.getInput(taskId, "Package0");
my_service.getInput(taskId, "ProcessVariableName");
```

```
my_service.getInput(taskId, "ProcessParameterName");
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
my_service.getOutput(taskId, "Package0");
my_service.getOutput(taskId, "ProcessVariableName");
my_service.getOutput(taskId, "ProcessParameterName");
my_service.forward(taskId);
my_service.claim(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
my_service.getFault(taskId, "name of the fault", "data about the fault");
my_service.nominate(taskId, orgEntity);
my_service.claim(taskId);
my_service.delegate(taskId);
my_service.claim(taskId);
my_service.remove(taskId);
my_service.claim(taskId);
my_service.release(taskId);
```

Example 3-64: C#: Running a query to list tasks

```
using Example.Ws_ht.Api.Wsdl;
using Example.Ws_ht.Api;

private const String ModuleName = "bpm";
private const String ContextRoot = "http://127.0.0.1:8080/services";
private ITaskManagementService my_service;
private List<tStatus> statusList = new List<tStatus>(1);
statusList.Add(tStatus.READY);

ContextFactory factory = ContextFactory.Instance;
IServiceContext context = factory.NewContext();

my_service = serviceFactory.GetRemoteService<ITaskManagementService>
    (context, ModuleName, ContextRoot);

List<tTaskQueryResultRow> task = my_service.Query
    ("dmi_workitem.r_object_id,dmi_queue_item.r_object_id,dmi_queue_item.name",
    "dmi_workitem.r_runtime_state=0", null, 2, 0);

String taskId = task[0].id;

my_service.claim(taskId);
my_service.setpriority(taskId);
my_service.addAttachment(taskId, null, "objectid", objectIdentity);
my_service.getAttachments(taskId, null);
my_service.getTaskDescription(taskId, "text/plain");
my_service.deleteAttachments(taskId, "attachment name");
my_service.getTaskInfo(taskId);
my_service.addComment(taskId, "Text of the note");
my_service.getComments(taskId);
my_service.suspend(taskId);
my_service.resume(taskId);
my_service.suspendUntil(taskId, untilTime);
my_service.resume(taskId);
my_service.getInput(taskId, "Package0");
my_service.getInput(taskId, "ProcessVariableName");
my_service.getInput(taskId, "ProcessParameterName");
my_service.setOutput(taskId, "Package0", 1);
my_service.setOutput(taskId, "ProcessVariableName", 1);
my_service.setOutput(taskId, "ProcessParameterName", 1);
my_service.getOutput(taskId, "Package0");
my_service.getOutput(taskId, "ProcessVariableName");
my_service.getOutput(taskId, "ProcessParameterName");
my_service.forward(taskId);
```

```
my_service.claim(taskId);
my_service.complete(taskId,null);
my_service.fail(taskId, null, null);
my_service.getFault(taskId, "name of the fault", "data about the fault");
my_service.nominate(taskId, orgEntity);
my_service.claim(taskId);
my_service.delegate(taskId);
my_service.claim(taskId);
my_service.remove(taskId);
my_service.claim(taskId);
my_service.release(taskId);
```

Chapter 4

Collaboration services

4.1 Comment service

The Comment service provides a set of operations to associate a comment-thread with any repository object.

4.1.1 Dependencies

The Comment service has dependencies on the Collaboration Services module. Therefore, the Collaboration Services Archive (DAR) files are required to be deployed to the repository prior to using this service.

Note: The Collaboration services follows the Foundation SOAP API version in synchronization with other products.

4.1.2 createComment operation

Creates and associates a comment with the object represented by parentIdentity.

The creator of the first comment needs to have write permissions on the object the comment thread is associated with.

The creator permission is set to *delete* and *read* to all members that have read/relate/write/delete on the controlling sysobject.

4.1.2.1 Java syntax

```
createComment(ObjectIdentity parentIdentity,

Comment commentData)
```

4.1.2.2 Parameters

Parameter	Data type	Description
parentIdentity	ObjectIdentity	The parent object with which to associate a comment.
commentData	Comment	The comment to create and associate with the parent object.

4.1.2.3 Example

```
// 1. Create a new comment
Comment commentData1 = commentService.createComment(controlSysObjectIdentity, comment1);
```

```
// 2. Reply to a comment
Comment comment2 = new Comment();
comment2.setTitle("Comment 2");

RichText richText2 = new RichText();
richText2setBodyAsString(
     "<p>Test with anchor and image <a href='http://www.yahoo.com'>www.yahoo.com</a></p>
+
        "<h3>Images</h3><img src='cid:0' alt='test'><img src='cid:1' alt='test'>" +
        "<br>Text with this & that<div> DRLs:<a href='objectid:0b0193a880000df7'>DRL 1</
a>" +
        "<br><a href='http://www.yahoo.com'>Yahoo!</a></div>");
List<Content> imageList = new ArrayList<Content>();
image1 = new FileContent(getEnv().getInDataPath() + "\\test.jpg", "jpg");
image2 = new FileContent(getEnv().getInDataPath() + "\\test3.jpg", "jpg");
imageList.add(0, image1);
imageList.add(1, image2);
richText2.setContents(imageList);
comment2.setBody(richText2);

comment2.setReplyTo(commentData1.getIdentity());

Comment commentData2 = commentService.createComment(controlSysObjectIdentity, comment2);
```

4.1.3 enumComments operation

Returns a List of Comments associated with the object represented by parentIdentity.

4.1.3.1 Java syntax

```
public java.util.Lista,<comment>enumComments(ObjectIdentity parentIdentity)
throws ServiceException
```

4.1.3.2 Parameters

Parameter	Data type	Description
parentIdentity	ObjectIdentity	The parent object with which the comments are associated.

4.1.3.3 Example

```
// Return the list of comments associated to a controlling dm_sysobject
        List<Commment> comments = commentService.enumComments(controlSysObjectIdentity);
```

4.1.4 getComment operation

Retrieves a comment identified by the commentIdentity.

4.1.4.1 Java syntax

```
public Comment getComment(ObjectIdentity commentIdentity)
throws ServiceException
```

4.1.4.2 Parameters

Parameter	Data type	Description
commentIdentity	ObjectIdentity	Identity of the comment to retrieve.

4.1.4.3 Example

```
// Retrieve a comment
        Comment commentRetrieve = commentService.getComment(commentData1.getIdentity());
```

4.1.5 markRead operation

Marks all the comments associated with the object represented by parentIdentity as read.

4.1.5.1 Java syntax

```
public void markRead(ObjectIdentity parentIdentity)
throws ServiceException
```

4.1.5.2 Parameters

Parameter	Data type	Description
parentIdentity	ObjectIdentity	The parent object with which the comments are associated.

4.1.6 markUnread operation

Marks all the comments associated with the object represented by parentIdentity as Unread.

4.1.6.1 Java syntax

```
public void markUnread(ObjectIdentity parentIdentity)
throws ServiceException
```

4.1.6.2 Parameters

Parameter	Data type	Description
parentIdentity	ObjectIdentity	The parent object with which the comments are associated.

4.1.7 updateComment operation

Updates the comment's title and body attributes.

4.1.7.1 Java syntax

```
public void updateComment(Comment commentData)
throws ServiceException
```

4.1.7.2 Parameters

Parameter	Data type	Description
commentData	Comment	An object representing the comment to update.

Chapter 5

Analytics services

Content Intelligence Services relies on the Content Intelligence Services (CIS) server and its taxonomies to provide classification capabilities. Content Intelligence Services offers the "Analytics service" on page 293 service. The Analytics service allows to analyze a file in the repository and return classification information.

5.1 Analytics service

The Analytics service provides classification capabilities on documents available in an OpenText Documentum CM repository.

Successful use of the Analytics service requires the installation and configuration of the Content Intelligence Services (CIS) server. CIS server analyzes the content and metadata of documents and compares them against the definitions of categories. CIS configuration and category definition can be made in Documentum Administrator. For more information, see *(EDCSY250400-IGD)* and *OpenText Documentum Content Management - Administrator User Guide (EDCAC-UGD)*.

5.1.1 Classifying documents

The Analytics service enables you to obtain classification information based on their content and metadata. CIS server must be up and running to be called by the Analytics service. To be analyzed, the documents must be available in the repository and accessible by the CIS server (that is, CIS server must be configured for the repository). The taxonomies must be in the same repository and synchronized with the CIS server. All taxonomies synchronized in production mode are used. It is not possible to select only one taxonomy to run a test.

A list of document identities is passed to the CIS server, specifying the type of result expected. The results of the analysis are category associations for each document.

5.1.2 Objects related to this service

This section describes the objects related to this service.

5.1.2.1 Category

The main attributes of the Category object are its name and two threshold values. The thresholds allow to determine whether a document matches the category.

The following table summarizes the Category fields:

Field	Data Type	Description
identity	String	Specifies the Category ID.
name	String	Specifies the Category name.
path	List<String>	Reserved for future use.
candidateThreshold	int	Specifies the Category candidate threshold. It indicates the minimum score that a document must exceed or meet to be assigned to a category as a candidate requiring approval. This threshold value must be between 0 and 100. If not set, the candidate threshold specified for the parent taxonomy is used.
onTargetThreshold	int	Specifies the Category on-target threshold. It indicates the minimum score that a document must exceed or meet to be automatically assigned to a category. This threshold value must be between 0 and 100. If not set, the on-target threshold specified for the parent taxonomy is used.

5.1.2.2 CategoryAssign

A CategoryAssign is an object representing the category association for a given document. This category association is defined by the category itself and a numeric value that represents the score computed by CIS server to decide whether there is a match between the document and the category.

The following table summarizes the CategoryAssign fields:

Field	Data Type	Description
category	Category	The Category object corresponding to the Category on which the document matched.
score	int	The score of matching between the Category and the Document.

5.1.2.3 AnalyticsResult

An AnalyticsResult is an object that contains analytics information computed by the CIS server for a given document. The analytics information is typically a category association for the document.

The following table summarizes the AnalyticsResult fields:

Field	Data Type	Description
categoryAssignList	List<CategoryAssign>	The list of CategoryAssign for a document.
objectIdentity	ObjectIdentity	Object corresponding to the document.

5.1.3 analyze operation

The analyze operation allows to analyze documents' content and metadata, and compute classification information.

5.1.3.1 Java syntax

```
List<AnalyticsResult> analyze (ObjectIdentitySet sourceObjects,
OperationOptions options)
throws CIServiceException;
```

5.1.3.2 C# syntax

```
List<AnalyticsResult> Analyze (ObjectIdentitySet sourceObjects,
OperationOptions options)
```

5.1.3.3 Parameters

This section describes the object parameter for the analyze operation. The Javadoc or Windows Help available under <*dfs-sdk*>\docs\ provide more information about the fields and methods.

Parameter	Data type	Description
sourceObjects	ObjectIdentitySet	Contains a list of ObjectIdentity instances specifying the objects to classify.
options	OperationOptions	Contains profiles and properties that specify operation behaviors. Only the Category profile is applicable. Properties passed in OperationsOtions are the properties on which you want classification information such as category associations. Currently only category associations are available.

5.1.3.4 Profiles

Only the Category profile can be used: the only property you can request is category associations.

5.1.3.5 Response

A list of AnalyticsResult instances is returned.

5.1.3.6 Exceptions

The CIServiceException is thrown in various cases such as: when CIS server is not running or when the repository cannot be found.

5.1.3.7 Examples

This section provides Java and C# examples of how to use the analyze operation.

5.1.3.7.1 Getting a list of category assignments

The following examples describe how to call the Analytics service for a list of documents identified by their ID and how to look through the results:

Example 5-1: Java: Getting a list of category assignments

```
public List<AnalyticsResult> getCategoryList() throws Exception
  {
        // The repository that you want to run the query on
        final String MY_DOCBASE = "MSSQL60CIS4";

        // The username to login to the repository
        final String MY_LOGIN = "Administrator";

        // The password for the username
        final String MY_PASSWORD = "password";

        // The address where the Documentum Foundation Services services are located
        final String address = "http://127.0.0.1:7001/services";

        // The module name for the Analytics Services
        final String moduleName = "ci";

        // The document 1 ID
        String document_ID_1 = "091116ee8000229b";

        // The document 2 ID
        String document_ID_2 = "091116ee8000229c";

        // Create service context
        ContextFactory contextFactory = ContextFactory.getInstance();
        IServiceContext context = contextFactory.newContext();
        RepositoryIdentity repoId = new RepositoryIdentity();
        repoId.setRepositoryName(MY_REPOSITORY);
        repoId.setUserName(MY_LOGIN);
        repoId.setPassword(MY_PASSWORD);
        context.addIdentity(repoId);
        ContentTransferProfile profile = new ContentTransferProfile();
        profile.setTransferMode(ContentTransferMode.BASE64);
        context.setProfile(profile);
        context = contextFactory.register(context, moduleName, address);

        // Instantiate service proxy
        ServiceFactory serviceFactory = ServiceFactory.getInstance();

        /* Add the documents in an ObjectIdentitySet, in this example
         * you need the document's object ID.
         */
        ObjectIdentitySet documentsSet = new ObjectIdentitySet();
        documentsSet.addIdentity(new ObjectIdentity
                       <Object>(new ObjectId(document_ID_1), MY_REPOSITORY));
        documentsSet.addIdentity(new ObjectIdentity
                       <Object>(new ObjectId(document_ID_2), MY_REPOSITORY));


        /* Classification information and properties you want.
         * Currently the service returns only Categories.
         */
        OperationOptions operationOptions = new OperationOptions();
        PropertyProfile propProfile = new PropertyProfile();
        List<String> properties = new ArrayList<String>();
        properties.add("CATEGORIES");
        propProfile.setIncludeProperties(properties);
        operationOptions.setPropertyProfile(propProfile);


        IAnalyticsService classificationService;
```

```
        List<AnalyticsResult> classificationResult;

        classificationService = serviceFactory.
           getRemoteService(IAnalyticsService.class, context, moduleName, address);
        classificationResult = classificationService.
                                        analyze(documentsSet, operationOptions);

        // Look through Classification results.
        for (AnalyticsResult classResult : classificationResult)
        {
            // Get Category Assignments on documents and print the categories.
            List<CategoryAssign> catAssigns = classResult.getCategoryAssignList();
            System.out.println("The document with the ID: " +
                                classResult.getObjectIdentity().
                                                        getValueAsString() +
                                " matched with the following Categories:");
            for (CategoryAssign catAssign : catAssigns)
            {
                System.out.println("\t -" + catAssign.getCategory().getName() +
                                   " - Category ID " +
                                   catAssign.getCategory().getIdentity().
                                                        getValue());

            }
        }
        return classificationResult;
    }
```

Example 5-2: C#: Getting a list of category assignments

```
public List<AnalyticsResult> getCategoryList()
{
  // The repository that you want to run the query on
  String MY_REPOSITORY = "MSSQL60CIS4";

  //The username to login to the repository
  String MY_LOGIN = "Administrator";

  // The password for the username
  String MY_PASSWORD = "password";

  // The address where the Documentum Foundation Services services are located
  String address = "http://127.0.0.1:7001/services";

  // The module name for the Analytics  Services
  String moduleName = "ci";

  // The document 1 ID
  String document_ID_1 = "091116ee8000229b";

  // The document 2 ID
  String document_ID_2 = "091116ee8000229c";

  IServiceContext serviceContext;

  // Create service context
  ContextFactory contextFactory = ContextFactory.Instance;
  serviceContext = contextFactory.NewContext();
  RepositoryIdentity repositoryIdentity =
  new RepositoryIdentity(MY_REPOSITORY, MY_LOGIN, MY_PASSWORD, "");
  serviceContext.AddIdentity(repositoryIdentity);

  // Instantiate service proxy
  ServiceFactory serviceFactory = ServiceFactory.Instance;
  IAnalyticsService AnalyticsService = serviceFactory.
  GetRemoteService<IAnalyticsService>(serviceContext, moduleName, address);
```

```
    /* Add the documents in an ObjectIdentitySet, in this example
     * you need the document's object ID.
     */
    ObjectIdentitySet documentset = new ObjectIdentitySet();
    ObjectIdentity objectIdentity1 =
                      new ObjectIdentity(new ObjectId
                                         (document_ID_1, MY_REPOSITORY));
    ObjectIdentity objectIdentity2 =
                      new ObjectIdentity(new ObjectId
                                         (document_ID_2, MY_REPOSITORY));
    documentset.AddIdentity(objectIdentity1);
    documentset.AddIdentity(objectIdentity2);

    /* Classification information and properties you want.
     * Currently the service returns only Categories.
     */
    OperationOptions operationOptions = new OperationOptions();
    operationOptions.PropertyProfile.FilterMode =
             PropertyFilterMode.SPECIFIED_BY_INCLUDE;
    operationOptions.PropertyProfile.IncludeProperties.Add("CATEGORIES");

    List<AnalyticsResult> classificationResult;

    try
    {
      classificationResult = AnalyticsService.Analyze
            (documentset, operationOptions);
      if (classificationResult != null)
      {
        // Look through Classification results.
        for (int i = 0; i < classificationResult.Count; i++)
        {
          // Get Category Assignments on documents and print the categories.
          List<CategoryAssign> catAssigns =
                   classificationResult[i].CatAssignList;
          Console.WriteLine("The document with the ID " +
                            classificationResult[i].Identity.Value +
                            " matched with the following Categories:");
          for (int j = 0; j < catAssigns.Count; j++)
          {
            Console.WriteLine("\t" + catAssigns[j].Category.Name +
                              " - Category ID: " +
                              catAssigns[j].Category.Identity.RepositoryName);
          }
        }
        return classificationResult;
      }
      else
      {
        Console.Out.WriteLine("Unable to call the Analytics Service.");
        return null;
      }
    }
    catch (ServiceException e)
    {
        Console.WriteLine("Web Service call failed!");
        Console.WriteLine(e.Message);
        Console.WriteLine(e.InnerException.Message);
        Console.WriteLine(e.StackTrace);
        return null;
    }
}
```

Chapter 6

Search services.

Search services provides search capabilities against repositories, as well as against external sources, using Documentum Federated Search Services (FS2) server. The Search service allows to run full-text and database searches. It also allows to compute clusters on the search results.

6.1 Search service

The Search service provides full-text and structured search capabilities against multiple OpenText Documentum CM repositories (termed *managed repositories* in Foundation SOAP API), as well as against external sources (termed *external repositories*).

Successful use of the Search service is dependent on deployment and configuration of full-text indexing on OpenText Documentum CM repositories, installation of FS2 adapters on external repositories (registered with an FS2 server), and deployment of the Clustering SBO. For more information, see *(EDCSY250400-IGD)*, *OpenText Documentum xPlore - Installation Guide (EDCSRC220100-IGD)*, and *OpenText Documentum xPlore - Administration and Development Guide (EDCSRC220100-AGD)*.

To use the Search service it is also helpful to understand FTDQL queries, `dfc.properties` settings, and DQL hint file settings. For information about these topics, see *(EDCSRCH250400-AGD)*. For more information about FTDQL syntax, see *(EDCCS250400-DRD)*.

Note: The Object service get operation can return contents from both managed and external repositories based on the search results. For more information see "Getting content from external sources" on page 25.

6.1.1 Full-text and database searches

Search service queries can be run as full-text queries against a full-text index, as database queries against object attributes on a managed or external repository, or as standard queries against both a full-text index and a database.

Whether the search query is run as a full-text or database search depends on a number of different factors:

- The availability to the service of full-text indexed repositories.
- Settings in the DQL hints file, if present.
- The presence or absence of full-text expressions (a SEARCH DOCUMENT CONTAINS clause) in a DQL query.
- Explicit setting of setDatabaseSearch in a StructuredQuery.

Searches against a full-text index are case-insensitive. Database searches are by default case-sensitive.

If a query includes a full-text expression, either as a SEARCH DOCUMENT CONTAINS clause in PassthroughQuery, or as a FullTextExpression object in a StructuredQuery (see "FullTextExpression" on page 305), but is executed as a database query, the full-text expression is evaluated against the title, subject, and object_name properties of repository objects of type dm_sysobject. However if the repository does not support full-text queries, the query is not processed.

6.1.2 External source repositories

To run searches against external repositories, you must meet the following requirements:

- Install the Federated Search Services server. For information, see *(EDCSY250400-IGD)*.
- Install and configure FS2 adapters. For information, see *(EDCSY250400-IGD)*.
- Set the following properties in the `dfc.properties` file:
  - dfc.search.external_sources.enable=true
  - dfc.search.external_sources.host=<*fs2_host*>
  - dfc.search.external_sources.port=<*fs2_port*> (default is 3005)

6.1.3 Non-blocking (asynchronous) searches

Searches can either be blocking or non-blocking, depending on the Search Profile setting. By default, searches are blocking. Non-blocking searches allow to display results dynamically: the client application does not have to wait for all results to be returned before displaying the first results. The Search service supports non-blocking searches because:

- Foundation SOAP API relies on Foundation Java API, which supports asynchronous search execution;
- Query calls are non blocking: multiple successive calls can be made to get new results and the query status. The query status contains the status for each source repository, indicating if it is successful, if more results are expected, or if it failed with errors.

6.1.4 Caching mechanism

The Search service relies on a caching mechanism. The cache contains the search results populated in background for every search. The cache key is built with the queryId, the query definition and the number of results requested, which we call the search context. To leverage the cache, subsequent calls have to use the same search context. If one of the search context elements is different, the search is re-executed.

The cache is used to make successive calls. This way, the first results can be displayed while subsequent calls retrieve more results. If one source fails or takes too long to return results, the search is not blocked and the first available results are returned.

When a query is not found in the cache (cache miss), the operation, which contains the query execution parameters, re-executes the query.

The cache clean-up mechanism is both time-based and size-based. You can modify the cache clean-up properties by editing the `dfs-runtime.properties` file.

To modify the cache period, set the dfs.search_query_cache_house_keeper.period parameter. The default value is set to 10 (minutes) which lets enough time to compute clustering operations for the result set. Depending on the number of search operations run by users, you may need to reduce the cache period to avoid excessive memory usage.

To modify the cache size, set the dfs.search_query_cache_house_keeper.max_queries parameter. The default value is set to 100 (queries). As a guideline, one cache entry for a simple query on dm_document with 350 results uses around 1Mb of memory. For such queries, with the default cache size value of 100, the cache will not use more than 100 Mb of memory.

6.1.5 Computing clusters

The search results can be displayed in clusters. Clusters group results dynamically into categories based on the values of the results' attributes. The clustering information is returned as soon as enough results are gathered to compute clusters. Clusters can then be used to navigate into the search results. For each level of clusters, a strategy is used to defined which attributes will be used to compute the clusters. For example, you can define a first strategy to compute the first level of clusters on the values for Author, Source and Owner; then, a second strategy can be used to display clusters on a subset of the results using the values for Author, Format and Modified Date.

Clusters can be computed on search results, but they can also be computed on a subset of the results.

Note that query results are not cached; therefore, if they are no longer available in the search context, you need to execute again the query. The search context is the context in which the query was executed.

6.1.6 Clustering SBO dependency

The clustering operations of the Search service (getClusters and getSubclusters) depend on the Clustering SBO, which should be installed on a global registry. This SBO is installed as part of the Webtop Extended Search option with Documentum CM Server. For more information on how to install the Webtop Extended Search option, see *OpenText Documentum Web Development Kit - Development Guide (EDCPKCLWT-PGD)*.

6.1.7 Objects related to this service

This section briefly describes objects used by this service. For field-level information, refer to the Javadoc or Windows help.

6.1.7.1 PassthroughQuery

The PassthroughQuery object is a container for a DQL or FTDQL query string. It can be executed as either a full-text or database query, depending on factors specified in "Full-text and database searches" on page 301.

A PassthroughQuery can search multiple managed repositories, but does not run against external repositories. To search an external repository a client must use a StructuredQuery.

6.1.7.2 StructuredQuery

A structured query defines a query using an object-oriented model. The query is constrained by a set of criteria contained in an ExpressionSet object, and the scope of the query (the sources against which it is run), is defined by an ordered list of RepositoryScope objects.

6.1.7.3 ExpressionSet

An ExpressionSet is a collection of Expression objects, each of which defines either a full-text expression, or a search constraint on a single property. The Expression instances comprising the ExpressionSet are related to one another by a single logical operator (either AND or OR). The ExpressionSet as a whole defines the complete set of search criteria that will be applied during a search.

An ExpressionSet contains Expression instances, and it also extends the Expression class. This enables an ExpressionSet to nest ExpressionSet instances, permitting construction of arbitrarily complex expression trees. The top-level Expression passed contained in a StructuredQuery is referred to as the *root expression* of the expression tree.

6.1.7.4 RepositoryScope

RepositoryScope enables a search to be constrained to a specific folder of a repository. It can also exclude folders.

6.1.7.5 Expression

The Expression class is extended by three concrete classes: FullTextExpression, PropertyExpression, and ExpressionSet.

Because ExpressionSet extends Expression and contains a set of Expression instances, an ExpressionSet can nest ExpressionSet instances. This allows construction of arbitrarily complex expression trees. The top-level Expression passed contained in a StructuredQuery is referred to as the *root expression* of the expression tree.

6.1.7.5.1 FullTextExpression

FullTextExpression encapsulates a search string accessed using the getValue and setValue methods. This string supports the operators AND, OR, and NOT, as well as parentheses.

6.1.7.5.2 PropertyExpression

PropertyExpression provides a search constraint based on a single property.

6.1.7.5.3 ExpressionValue

The following table describes the concrete subtypes of the ExpressionValue class:

Table 6-1: ExpressionValue subtypes

Subtype	Description
SimpleValue	Contains a single String value.
RangeValue	Contains two String values representing the start and end of a range. The values can represent dates (using the DateFormat specified in the StructuredQuery) or integers.
ValueList	Contains an ordered List of String values.
RelativeDateValue	Contains a TimeUnit setting and an integer value representing the number of time units. TimeUnit values are MILLISECOND, SECOND, MINUTE, HOUR, DAY, ERA, WEEK, MONTH, YEAR. The integer value can be negative or positive to represent a past or future time.

6.1.7.5.4 Condition

Condition is an enumerated type that expresses the logical condition to use when comparing a repository value to a value in an Expression. A specific Condition is included in a PropertyExpression to determine precisely how to constrain the search on the property value.

6.1.7.6 QueryResult

The QueryResult class is used by both the Search and Query services as a container for the set of results returned by the execute operation. It also contains the queryId generated for this query and used to uniquely identified the query. The queryId is used as a key in the cache to identify the query, for a given user, for subsequent calls.

6.1.7.6.1 QueryStatus

QueryStatus contains status information returned by a search operation. The status information can be examined for each search source repository.

6.1.7.6.2 RepositoryStatusInfo

RepositoryStatusInfo contains data related to a query or search result regarding the status of the search in a specific repository. RepositoryStatusInfo instances are returned in a List<RepositoryStatusInfo> within a QueryResult, which is returned by a search or query operation.

6.1.7.6.3 RepositoryStatus

RepositoryStatus generally provides detail information about the status of a query that was executed, as pertains to a specific repository.

6.1.7.7 QueryCluster

The QueryCluster object is a container for ClusterTree objects for a given query. Another parameter is the queryId that is used to uniquely identified the query. It can be used to access any part of the result set such as the following set of results (for example, the ten following results after the ten first results) or to retrieve clusters on all or some of the results.

6.1.7.7.1 ClusterTree

A ClusterTree is a container for Cluster objects that are calculated according to a ClusteringStrategy. The field isRefreshable indicates if all clusters have been computed and the search is complete or if more results may be returned by the source.

6.1.7.7.2 Cluster

The Cluster class represents a cluster which is a group of objects having something in common. These objects are grouped into categories comparing the values of their attributes. The following table summarizes the Cluster fields:

Field	Data Type	Description
clusterValues	List<String>	Specifies the list of values that are used to generate the cluster name.
clusterSize	int	Specifies the number of objects in the cluster.
clusterObjectsIdentities	ObjectIdentitySet	Specifies a list of ObjectIdentity instances for the objects belonging to this cluster.

ClusteringStrategy

The ClusteringStrategy class is used by a ClusterTree object to set the strategy applied to calculate clusters that will belong to this ClusterTree. The clustering strategy can use tokenizers to group the clusters (for example, dates can be grouped into quarters). In this case, you define which tokenizer to apply for a given attribute.

The ClusteringStrategy class also controls the amount of data returned by the operation.

The following table summarizes the ClusteringStrategy fields:

Field	Data Type	Description
strategyName	String	Specifies the strategy name.
attributes	List<String>	Specifies the list of attributes used in this strategy.
clusteringRange	ClusteringRange	Specifies the number of clusters computed by the clustering service. Possible values are : LOW, MEDIUM, HIGH.
clusteringThreshold	int	Specifies the minimum number of results required to create a new cluster.

Field	Data Type	Description
returnIdentitySet	boolean	Specifies whether the object identities should be returned.
tokenizers	PropertySet	Specifies the tokenizer to apply. The ProperySet is a set of StringProperty where the name is the attribute name and the value is the tokenizer name to apply to this attribute. Available tokenizers are listed in "List of Tokenizers available for the clustering" on page 308.

Table 6-2: List of Tokenizers available for the clustering

Tokenizer name	Description
dm_object_name	Tokenizes an object name attribute. Strings are cleaned before being used: underscore characters are replaced by spaces and the extensions are removed.
dm_percentage	Tokenizes a score attribute or a numeric value between 0 and 1. The suffix "%" is added to the percentage.
dm_date_by_quarter	Tokenizes a date attribute to create cluster by Quarter (2006 Q1, 2006 Q2, 2006 Q3 ...)
dm_dynamic_size	Tokenizes a string size attribute and groups dynamically the input sizes.
dm_size_by_range	Tokenizes a string size attribute and creates predefined ranges. The ranges are 0 KB-100 KB, 100 KB-1 MB, 1 MB-10 MB, 10 MB-100 MB, >100 MB.
dm_date_by_day	Tokenizes a string date attribute according to the "dd/MM/yyyy" pattern.
dm_exact_match	Tokenizes any string and groups the ones that are exactly the same.
dm_text	Parses, lemmatizes and dynamically groups any string attribute.
dm_number	Tokenizes strings to obtain numbers and groups dynamically the input numbers.
dm_author	Tokenizes strings to obtain lists of authors. Groups dynamically the authors. By default, the author names are expected to start with the first name.

Tokenizer name	Description
dm_collection	Tokenizes strings of the form “category1:category2:category3” and groups dynamically according to the most significant categories or sub-categories.
dm_source	Tokenizes a r_source attribute, it generates a suitable source name for the external source.

Note: The values returned by the tokenizers are not localized.

6.1.8 getRepositoryList operation

The getRepositoryList operation provides list of managed and external repositories that are available to the service for searching.

6.1.8.1 Java syntax

```
List<Repository> getRepositoryList(OperationOptions options)
                            throws SearchServiceException
```

6.1.8.2 C# syntax

```
List<Repository> GetRepositoryList(OperationOptions options)
```

6.1.8.3 Parameters

Parameter	Data type	Description
options	OperationOptions	Contains profiles and properties that specify operation behaviors. This parameter is not used by the operation in current Foundation SOAP API version.

6.1.8.4 Response

Returns a List of Repository instances.

6.1.8.5 Example

The following example demonstrates the getRepositoryList operation:

Example 6-1: Java: Getting a repository list

```
public List<Repository> repositoryList()
{
    try
    {
        ServiceFactory serviceFactory = ServiceFactory.getInstance();
        ISearchService searchService
            = serviceFactory.getService(ISearchService.class, serviceContext);
        List<Repository> repositoryList = searchService.getRepositoryList
            (new OperationOptions());
        for (Repository r : repositoryList)
        {
            System.out.println(r.getName());
        }
        return repositoryList;
    }
    catch (Exception e)
    {
        e.printStackTrace();
        throw new RuntimeException(e);
    }
}
```

Example 6-2: C#: Getting a repository list

```
public List<Repository> RepositoryList()
{
    try
    {
        List<Repository> repositoryList = searchService.GetRepositoryList
            (new OperationOptions());
        foreach (Repository r in repositoryList)
        {
            Console.WriteLine(r.Name);
        }
        return repositoryList;
    }
    catch (Exception e)
    {
        Console.WriteLine(e.StackTrace);
        throw new Exception(e.Message);
    }
}
```

6.1.9 execute operation

The execute operation searches a repository or set of repositories and returns search results.

6.1.9.1 Java syntax

```
QueryResult execute(Query query,
                    QueryExecution execution,
                    OperationOptions options)
        throws SearchServiceException
```

6.1.9.2 C# syntax

```
QueryResult Execute(Query query,
                    QueryExecution execution,
                    OperationOptions options)
```

6.1.9.3 Parameters

Parameter	Data type	Description
query	Query	Either a PassthroughQuery (see "PassthroughQuery" on page 304) or a StructuredQuery (see "StructuredQuery" on page 304).
execution	QueryExecution	Object describing execution parameters. Query execution parameters are described in "QueryExecution" on page 141.
options	OperationOptions	Contains profiles and properties that specify operation behaviors. In the case of the execute operation, the profiles primarily provide filters that modify the contents of the DataPackage returned in QueryResult. An applicable profile is the SearchProfile. Note that in a PropertyProfile only the property filter mode SPECIFIED_BY_INCLUDE is supported for this operation. Other property filter modes are not supported.

6.1.9.4 Search profile

The SearchProfile allows to set the parameters for the search execution. Set the isAsyncCall parameter to indicate whether the search should be blocking or not.

6.1.9.5 Response

Returns a QueryResult instance. For information on QueryResult see "QueryResult" on page 306.

6.1.9.6 Examples

6.1.9.6.1 Simple passthrough query

Example 6-3: Java: Simple PassthroughQuery

```
public QueryResult simplePassthroughQuery()
{
    QueryResult queryResult;
    try
    {
        ServiceFactory serviceFactory = ServiceFactory.getInstance();
        ISearchService searchService
            = serviceFactory.getService(ISearchService.class, serviceContext);

        String queryString
            = "select distinct r_object_id from dm_document order by r_object_id ";
        int startingIndex = 0;
        int maxResults = 20;
        int maxResultsPerSource = 60;

        PassthroughQuery q = new PassthroughQuery();
        q.setQueryString(queryString);
        q.addRepository(defaultRepositoryName);

        QueryExecution queryExec = new QueryExecution(startingIndex,
                                                      maxResults,
                                                      maxResultsPerSource);
        queryExec.setCacheStrategyType(CacheStrategyType.NO_CACHE_STRATEGY);

        queryResult = searchService.execute(q, queryExec, null);

        QueryStatus queryStatus = queryResult.getQueryStatus();
        RepositoryStatusInfo repStatusInfo = queryStatus.
                                    getRepositoryStatusInfos().get(0);
        if (repStatusInfo.getStatus() == Status.FAILURE)
        {
            System.out.println(repStatusInfo.getErrorTrace());
            throw new RuntimeException("Query failed to return result.");
        }
        System.out.println("Query returned result successfully.");
        DataPackage dp = queryResult.getDataPackage();
        System.out.println("DataPackage contains " + dp.getDataObjects().size()
                                                                    + " objects.");
        for (DataObject dataObject : dp.getDataObjects())
        {
            System.out.println(dataObject.getIdentity());
        }
    }
    catch (Exception e)
    {
        e.printStackTrace();
        throw new RuntimeException(e);
    }
```

```
    return queryResult;
}
```

Example 6-4: C#: Simple PassthroughQuery

```
public QueryResult SimplePassthroughQuery()
{
    QueryResult queryResult;
    try
    {
        string queryString = "select distinct r_object_id from dm_document

order by r_object_id ";
        int startingIndex = 0;
        int maxResults = 20;
        int maxResultsPerSource = 60;

        PassthroughQuery q = new PassthroughQuery();
        q.QueryString = queryString;
        q.AddRepository(DefaultRepository);

        QueryExecution queryExec = new QueryExecution(startingIndex,
                                   maxResults,
                                   maxResultsPerSource);
        queryExec.CacheStrategyType = CacheStrategyType.NO_CACHE_STRATEGY;

        queryResult = searchService.Execute(q, queryExec, null);

        QueryStatus queryStatus = queryResult.QueryStatus;
        RepositoryStatusInfo repStatusInfo =
                queryStatus.RepositoryStatusInfos[0];
        if (repStatusInfo.Status == Status.FAILURE)
        {
            Console.WriteLine(repStatusInfo.ErrorTrace);
            throw new Exception("Query failed to return result.");
        }
        Console.WriteLine("Query returned result successfully.");
        DataPackage dp = queryResult.DataPackage;
        Console.WriteLine("DataPackage contains " + dp.DataObjects.Count

    + " objects.");
        foreach (DataObject dataObject in dp.DataObjects)
        {
            Console.WriteLine(dataObject.Identity);
        }
    }
    catch (Exception e)
    {
        Console.WriteLine(e.StackTrace);
        throw new Exception(e.Message);
    }
    return queryResult;
}
```

6.1.9.6.2 Structured query

Example 6-5: Java: Structured query

```
public void simpleStructuredQuery()
{
    try
    {
        ServiceFactory serviceFactory = ServiceFactory.getInstance();
        ISearchService searchService
            = serviceFactory.getService(ISearchService.class, serviceContext);

        String repoName = defaultRepositoryName;

        // Create query
        StructuredQuery q = new StructuredQuery();
        q.addRepository(repoName);
        q.setObjectType("dm_document");
        q.setIncludeHidden(true);
        q.setDatabaseSearch(true);
        ExpressionSet expressionSet = new ExpressionSet();
        expressionSet.addExpression(new PropertyExpression("owner_name",
                                                           Condition.CONTAINS,
                                                           "admin"));
        q.setRootExpressionSet(expressionSet);

        // Execute Query
        int startingIndex = 0;
        int maxResults = 20;
        int maxResultsPerSource = 60;
        QueryExecution queryExec = new QueryExecution(startingIndex,
                                                      maxResults,
                                                      maxResultsPerSource);
        QueryResult queryResult = searchService.execute(q, queryExec, null);

        QueryStatus queryStatus = queryResult.getQueryStatus();
        RepositoryStatusInfo repStatusInfo = queryStatus.
                                                 getRepositoryStatusInfos().get(0);
        if (repStatusInfo.getStatus() == Status.FAILURE)
        {
            System.out.println(repStatusInfo.getErrorTrace());
            throw new RuntimeException("Query failed to return result.");
        }

        // print results
        for (DataObject dataObject : queryResult.getDataObjects())
        {
            System.out.println(dataObject.getIdentity());
        }

    }
    catch (Exception e)
    {
        e.printStackTrace();
        throw new RuntimeException(e);
    }

    System.out.println("test completed - OK");
}
```

Example 6-6: C#: Structured query

```
public void SimpleStructuredQuery()
{
    try
    {
```

```
        String repoName = DefaultRepository;

        // Create query
        StructuredQuery q = new StructuredQuery();
        q.AddRepository(repoName);
        q.ObjectType = "dm_document";
        q.IsIncludeHidden = true;
        q.IsDatabaseSearch = true;
        ExpressionSet expressionSet = new ExpressionSet();
        expressionSet.AddExpression(new PropertyExpression("owner_name",
                                                           Condition.CONTAINS,
                                                           "admin"));
        q.RootExpressionSet = expressionSet;

        // Execute Query
        int startingIndex = 0;
        int maxResults = 20;
        int maxResultsPerSource = 60;
        QueryExecution queryExec = new QueryExecution(startingIndex,
                                                      maxResults,
                                                      maxResultsPerSource);
        QueryResult queryResult = searchService.Execute(q, queryExec, null);

        QueryStatus queryStatus = queryResult.QueryStatus;
        RepositoryStatusInfo repStatusInfo = queryStatus.RepositoryStatusInfos[0];
        if (repStatusInfo.Status == Status.FAILURE)
        {
            Console.WriteLine(repStatusInfo.ErrorTrace);
            throw new Exception("Query failed to return result.");
        }

        // print results
        foreach (DataObject dataObject in queryResult.DataObjects)
        {
            Console.WriteLine(dataObject.Identity);
        }

    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
        Console.WriteLine(e.StackTrace);
        throw new Exception(e.Message);
    }
}
```

6.1.10 stopSearch operation

The stopSearch operation allows to stop the execution of the query passed in as parameter. The execute operation must be called first to launch the query. After the query is stopped, results retrieved so far are available. It is then possible to call the operations getClusters, getSubclusters and getResultProperties passing in the Query and QueryExecution parameters of the stopped query. You can also restart the stopped search by calling the execute operation with the same query object and the same query execution object but without the queryId.

6.1.10.1 Java syntax

```
QueryStatus stopSearch(Query query,
                       QueryExecution execution)
            throws SearchServiceException
```

6.1.10.2 C# syntax

```
QueryStatus StopSearch(Query query,
                       QueryExecution execution)
```

6.1.10.3 Parameters

Parameter	Data type	Description
query	Query	Either a PassthroughQuery (see "PassthroughQuery" on page 304) or a StructuredQuery (see "StructuredQuery" on page 304).
execution	QueryExecution	Object describing execution parameters. Query execution parameters are described in "QueryExecution" on page 141.

6.1.10.4 Response

Returns a QueryStatus instance of the stopped query. For information on QueryStatus, see "QueryStatus" on page 306.

6.1.10.5 Example

The following example demonstrates the stopSearch operation:

6.1.10.5.1 Stopping a search

Example 6-7: Java: stopping a search

```
public QueryStatus stopSearch () throws ServiceException
    {
        // Specify query: can be either a PassthroughQuery or a StructuredQuery
        PassthroughQuery query = new PassthroughQuery();
        query.setQueryString("select * from dm_document");
        query.addRepository(getEnv().getDefaultDocbaseName());

        // Specify query execution
        QueryExecution queryExecution = new QueryExecution();
        queryExecution.setMaxResultCount(100);
        queryExecution.setMaxResultPerSource(350);

        // Set operations options
        OperationOptions operationOptions = new OperationOptions();
```

```
        SearchProfile searchProfile = new SearchProfile();
        searchProfile.setAsyncCall(true);
        operationOptions.setSearchProfile(searchProfile);

        PropertyProfile propertyProfile = new PropertyProfile();
        propertyProfile.setFilterMode(PropertyFilterMode.SPECIFIED_BY_INCLUDE);
        operationOptions.setPropertyProfile(propertyProfile);

        // Start the search
        QueryResult results =
            m_searchService.execute(query, queryExecution, operationOptions);

        // Set query id
        queryExecution.setQueryId(results.getQueryId());

        // Optional: check the status is RUNNING before stopping the search

        // Stop the search
        QueryStatus status = m_searchService.stopSearch(query, queryExecution);

        // Optional: check the status is STOPPED

        return status;
    }
```

6.1.11 getClusters operation

The getClusters operation enables to compute clusters on query results. The execute operation should be called first in order to run the query and get results. In this case, the getClusters operation uses the same Query and QueryExecution parameters.

These parameters are necessary to reexecute the query if the query has not run or if results are no longer available in the search context.

By setting, in the Search profile, whether the operation is blocking or non-blocking, clusters can be computed on the first available results or only when all results are returned. By default, the execution is synchronous and clusters are computed when all results are returned.

Note: You must install the Clustering SBO to compute clusters using the getClusters operation.

6.1.11.1 Java syntax

```
QueryCluster getClusters (Query query,
                          QueryExecution execution,
                          OperationOptions options)
        throws SearchServiceException;
```

6.1.11.2 C# syntax

```
QueryCluster GetClusters (Query query,
                          QueryExecution execution,
                          OperationOptions options)
```

6.1.11.3 Parameters

Parameter	Data type	Description
query	Query	Contains the query definition and the repositories against which the query is run.
execution	QueryExecution	Object describing execution parameters. Query execution parameters are described in "QueryExecution" on page 141.
options	OperationOptions	Contains profiles and properties that specify operation behaviors. Only the ClusteringProfile and the SearchProfile are applicable. If this object is null or if there is no ClusteringStrategy, no clusters are returned.

6.1.11.4 Clustering profile

The ClusteringProfile contains a list of ClusteringStrategy instances. The ClusteringStrategy is used to compute the ClusterTrees and controls the amount of data returned by the operation.

6.1.11.5 Response

Returns a QueryCluster object containing a list of ClusterTree objects and the id of the query. QueryCluster objects are described in the section "QueryCluster" on page 306.

6.1.11.6 Exceptions

The SearchServiceException exception is thrown in particular when the Clustering SBO is not installed.

6.1.11.7 Example

The following example demonstrates the getClusters operation:

Example 6-8: Java: Computing clusters on query results

```
public QueryCluster getClusters () throws ServiceException
    {
        OperationOptions options = new OperationOptions();

        // Can be either a PassthroughQuery or StructuredQuery
        PassthroughQuery query = new PassthroughQuery();
        query.setQueryString("select * from dm_document");
        query.addRepository(YOUR_REPOSITORY);

        // Get 50 results
        QueryExecution queryExec = new QueryExecution(0, 50, 50);
        QueryResult results = searchService.execute(query, queryExec, options);

        // Get generated queryId and set it for subsequent calls
        String queryId = results.getQueryId();
        queryExec.setQueryId(queryId);

        // Get query clusters
        // Set ClusteringStrategy
        ClusteringStrategy strategy = new ClusteringStrategy();
        strategy.setStrategyName("Name");
        List<String> attrs = new ArrayList<String>(2);
        attrs.add("object_name");
        strategy.setAttributes(attrs);
        strategy.setReturnIdentitySet(true);
        strategy.setClusteringRange(ClusteringRange.HIGH);

        // Set ClusteringProfile
        ClusteringProfile profile = new ClusteringProfile(strategy);
        options.setClusteringProfile(profile);

        QueryCluster queryCluster = searchService.getClusters(query,
                                                              queryExec,
                                                              options);
        return queryCluster;

}
```

6.1.12 getSubclusters operation

The getSubclusters operation enables to compute clusters on a subset of the result set. The subset is specified in the ObjectIdentitySet.

The execute operation should be called first in order to run the query and get results. In this case, the getSubclusters operation uses the same Query and QueryExecution parameters.

If the query has not run or if results are no longer available in the search context, the query is executed according to the Query, QueryExecution and OperationOptions parameters.

By setting, in the Search profile, whether the operation is blocking or non-blocking, clusters can be computed on the first available results or only when all results are

returned. By default, the execution is synchronous and clusters are computed when all results are returned.

Note: You must install the Clustering SBO to compute clusters using the getSubclusters operation.

6.1.12.1 Java syntax

```
QueryCluster getSubclusters (ObjectIdentitySet objectsToClusterize,
                             Query query,
                             QueryExecution execution,
                             OperationOptions options)
        throws SearchServiceException;
```

6.1.12.2 C# syntax

```
QueryCluster GetSubclusters (ObjectIdentitySet objectsToClusterize,
                             Query query,
                             QueryExecution execution,
                             OperationOptions options)
```

6.1.12.3 Parameters

Parameter	Data type	Description
objectsToClusterize	ObjectIdentitySet	Contains a list of ObjectIdentity instances specifying the objects on which the clusters are computed.
query	Query	Contains the query definition and the repositories against which the query is run.
execution	QueryExecution	Object describing execution parameters. Query execution parameters are described in "QueryExecution" on page 141.
options	OperationOptions	Contains profiles and properties that specify operation behaviors. Only the ClusteringProfile and the SearchProfile are applicable. If this object is null or if there is no ClusteringStrategy, no clusters are returned.

6.1.12.4 Clustering profile

The ClusteringProfile contains a list of ClusteringStrategy instances. The ClusteringStrategy is used to compute the ClusterTrees and controls the amount of data returned by the operation.

6.1.12.5 Response

Returns a QueryCluster object containing a list of ClusterTree objects and the id of the query. QueryCluster objects are described in the section "QueryCluster" on page 306.

6.1.12.6 Exceptions

The SearchServiceException exception is thrown in particular when the Clustering SBO is not installed.

6.1.12.7 Example

The following example demonstrates the getSubclusters operation:

Example 6-9: Java: Computing clusters on a subset of query results

```
public DataPackage getClusterObjects () throws ServiceException
    {
        OperationOptions options = new OperationOptions();

        // Can be either a PassthroughQuery or StructuredQuery
        PassthroughQuery query = new PassthroughQuery();
        query.setQueryString("select * from dm_document");
        query.addRepository(YOUR_REPOSITORY);

        // Get 50 results
        QueryExecution queryExec = new QueryExecution(0, 50, 50);
        QueryResult results = searchService.execute(query, queryExec, options);

        // Get generated queryId and set it for subsequent calls
        String queryId = results.getQueryId();
        queryExec.setQueryId(queryId);

        // Get query clusters
        // Set ClusteringStrategy
        ClusteringStrategy strategy = new ClusteringStrategy();
        strategy.setStrategyName("Name");
        List<String> attrs = new ArrayList<String>(2);
        attrs.add("object_name");
        strategy.setAttributes(attrs);
        strategy.setReturnIdentitySet(true);
        strategy.setClusteringRange(ClusteringRange.HIGH);

        // Set ClusteringProfile
        ClusteringProfile profile = new ClusteringProfile(strategy);
        options.setClusteringProfile(profile);

        QueryCluster queryCluster = searchService.getClusters(query,
                                                              queryExec,
                                                              options);

        // Get objects belonging to the first cluster
        DataPackage clusterObjects = new DataPackage();
        if (null != queryCluster.getClusterTrees() && !queryCluster.
```

```
                                                        getClusterTrees().isEmpty())
        {
           ClusterTree finalTree = queryCluster.getClusterTrees().get(0);
           if (null != finalTree.getClusters() && !finalTree.
                                                        getClusters().isEmpty())
           {
               Cluster cluster = finalTree.getClusters().get(0);
               clusterObjects = searchService.
                        getResultsProperties(cluster.getClusterObjectsIdentities(),
                                             query, queryExec, options);
           }
        }
        return clusterObjects;
}
```

6.1.13 getResultsProperties operation

The getResultsProperties operation enables to retrieve the results of a search in order to display them for example. This operation must be called after a call to the getClusters or getSubclusters operations. It can also be called after a search.

If the search context is no longer available, the query is executed according to the Query, QueryExecution and OperationOptions parameters. The search context is necessary to retrieve the results for the selected cluster.

6.1.13.1 Java syntax

```
DataPackage getResultsProperties (ObjectIdentitySet forClustersObjects,
                                  Query query,
                                  QueryExecution execution,
                                  OperationOptions options)
        throws SearchServiceException;
```

6.1.13.2 C# syntax

```
DataPackage GetResultsProperties (ObjectIdentitySet forClustersObjects,
                                  Query query,
                                  QueryExecution execution,
                                  OperationOptions options)
```

6.1.13.3 Parameters

Parameter	Data type	Description
forClustersObjects	ObjectIdentitySet	Contains a list of ObjectIdentity instances specifying the results to retrieve.
query	Query	Contains the query definition and the repositories against which the query is run.

Parameter	Data type	Description
execution	QueryExecution	Object describing execution parameters. Query execution parameters are described in "QueryExecution" on page 141.
options	OperationOptions	Contains profiles and properties that specify operation behaviors. If this object is null, default operation behaviors apply.

6.1.13.4 Response

Returns a DataPackage containing the query results, that is, the objects specified in the ObjectIdentitySet.

6.1.13.5 Exceptions

The SearchServiceException exception is thrown in particular when the Clustering docapp is not installed.

6.1.13.6 Example

The following example demonstrates the getResultsProperties operation:

Example 6-10: Java: Retrieving results of a selected cluster

```
public QueryCluster getSubClusters () throws ServiceException
    {
        OperationOptions options = new OperationOptions();

        // Can be either a PassthroughQuery or StructuredQuery
        PassthroughQuery query = new PassthroughQuery();
        query.setQueryString("select * from dm_document");
        query.addRepository(YOUR_REPOSITORY);

        // Ask for 100 results
        QueryExecution queryExec = new QueryExecution(0, 100, 100);
        QueryResult results = searchService.execute(query, queryExec, options);

        // Get generated queryId and set it for subsequent calls
        String queryId = results.getQueryId();
        queryExec.setQueryId(queryId);

        // Now get query clusters
        // Set ClusteringStrategy
        ClusteringStrategy strategy = new ClusteringStrategy();
        strategy.setStrategyName("Name");
        List<String> attrs = new ArrayList<String>();
        attrs.add("object_name");
        strategy.setAttributes(attrs);
        strategy.setReturnIdentitySet(true);
        strategy.setClusteringRange(ClusteringRange.HIGH);

        // Set ClusteringProfile with strategy
        ClusteringProfile profile = new ClusteringProfile(strategy);
        options.setClusteringProfile(profile);
```

```
        // Get clusters on results retrieved so far
        QueryCluster queryCluster = searchService.getClusters(query,
                                                              queryExec,
                                                              options);

        // Get the objects belonging to the first cluster
        // and calculate new clusters on this subset
        List<ClusterTree> clusterTrees = queryCluster.getClusterTrees();
        QueryCluster subClusters = new QueryCluster();
        if (null != clusterTrees && !clusterTrees.isEmpty())
        {

            // Get first ClusterTree
            ClusterTree firstTree = clusterTrees.get(0);
            List<Cluster> clusters = firstTree.getClusters();
            if (null != clusters && !clusters.isEmpty())
            {

               // Get first cluster
               Cluster cluster = clusters.get(0);

               // Get identities of objects belonging to this cluster
               ObjectIdentitySet ids = cluster.getClusterObjectsIdentities();

               // Create a new strategy to get clusters based on format
               ClusteringStrategy authorStrategy = new ClusteringStrategy();
               authorStrategy.setStrategyName("Format");
               List<String> authorAttrs = new ArrayList<String>
               authorAttrs.add("a_content_type");
               authorStrategy.setAttributes(authorAttrs);
               authorStrategy.setReturnIdentitySet(true);
               authorStrategy.setClusteringRange(ClusteringRange.HIGH);

               // Create new profile to take into account the new strategy
               ClusteringProfile newProfile =
                            new ClusteringProfile(authorStrategy);
               options.setClusteringProfile(newProfile);

               // Get new clusters calculated on the given subset of results
               subClusters = searchService.getSubclusters(ids,
                                                          query,
                                                          queryExec,
                                                          options);
            }
        }
        return subClusters;
}
```

Chapter 7

Transformation services

7.1 Profile service

The Profile web service (IProfileService) provides an interface to OpenText Documentum Content Management (CM) Transformation Services transformation profiles. The Profile web service enables applications to obtain available transformation profiles using various filtering mechanisms, as well as to update them if suitable permissions are held by the application session.

IProfileService provides the functionality to get, add, update and remove profiles and command line files for content transformations. The namespace for the Content Delivery service is: com.emc.documentum.fs.services.transformation.impl.

IProfileService is deployed using the Content Transformation WebServices installer.

This section describes different objects and operations related to IProfileService.

7.1.1 Objects related to this service

The objects related to this service are:

- LocalTextFile
- ParameterContent
- ParameterContentAttribute
- ParameterContentAttriToken
- ParameterControlType
- ParameterDataType
- ParameterDependency
- ParameterDependencyAction
- ParameterListValue
- ParameterRange
- Profile
- ProfileCommandFilePath
- ProfileFilter
- ProfileFormatPair
- ProfileInnerProfileEntry
- ProfileInnerTokenMapping

- ProfileOperation
- ProfileOuterProfileEntry
- ProfileParameter
- ProfileParamFile
- ProfileQueryFilter
- ProfileRequest

7.1.2 addProfile operation

The addProfile operation adds a OpenText Documentum CM Transformation Services profile to the repository from the client where the profile is saved.

7.1.2.1 Java syntax

```
ObjectIdentity addProfile(String repository, Profile profile, String folder)
        throws TransformationServiceException
```

7.1.2.2 Parameters

Parameter	Data type	Description
repository	String	The name of the repository.
profile	Profile	Object representing the profile.
folder	String	The repository folder path where theProfile is saved.

7.1.2.3 Response

The addProfile code returns an object representing the new profile id.

7.1.3 addProfiles operation

The addProfiles operation adds multiple profiles including command line files, user profiles, and system profiles to the repository from the client.

7.1.3.1 Java syntax

```
List<ObjectIdentity> addProfiles(List<ProfileParamFile> theParamFiles,
String repository)
throws TransformationServiceException
```

7.1.3.2 Parameters

Parameter	Data type	Description
theParamFiles	List<ObjectIdentity>	myProfile information.
repository	String	The name of the repository in which to add the profiles.

7.1.3.3 Response

The operation returns an array of identities representing the new profiles and command line files.

7.1.4 getProfileById operation

The getProfileById operation is used to get a specific profile which is queried by its saved Profile Id.

7.1.4.1 Java syntax

```
Profile getProfileById (ObjectIdentity profileId)

        throws TransformationServiceException
```

7.1.4.2 Parameters

Parameter	Data type	Description
profileId	ObjectIdentity	The id of the profile to retrieve.

7.1.4.3 Response

The getProfileById code returns a profile with the specific profileId queried.

7.1.5 getProfileByName operation

The getProfileByName operation is used to specify which profile should be invoked to process a source object. The profile is queried by the saved profile name and repository location. For example, only the profile with the id “flip” will process the source object.

7.1.5.1 Java syntax

```
Profile getProfileByName (String repository, String profileName)
        throws TransformationServiceException
```

7.1.5.2 Parameters

Parameter	Data type	Description
repository	String	the repository name
profileName	String	the profile name

7.1.5.3 Response

This code returns an instance of a profile with the profile name. Transformation Services products use the value returned by this method when it attempts to delegate the execution of profiles to the appropriate plug-ins.

7.1.6 getProfiles operation

The getProfiles operation is used to get an array of profiles which satisfy the query that specifies conditions passed with prParam.

7.1.6.1 Java syntax

```
List<Profile> getProfiles (String repository, ProfileRequest request)
        throws TransformationServiceException
```

7.1.6.2 Parameters

Parameter	Data type	Description
repository	String	the repository name to query
request	ProfileRequest	the query conditions

7.1.6.3 Response

The getProfiles code returns a List of profiles which satisfies your query conditions.

7.1.6.4 Example

The profiles returned will all be valid for execution as specified by the parameters. The parameters are based on a list of categories that include the user session, source format of the repository content, and the object to which the content is attached. The value returned by the getVersion() method is useful for keeping track of plug-ins as they evolve over time.

7.1.6.4.1 Profile service test case

This test case shows how to make a direct request to the web service.

Example 7-1: Java: Example of how to make direct runtime requests

```
// public class CTSWSProfileServiceTestCase
{
    protected static final String TEST_CONFIG = ctswsclient.properties;

    protected static final String REPOSITORY = "repository";
    protected static final String USER_NAME = "user";
    protected static final String PASSWORD = "password";
    protected static final String DOMAIN = "domain";
    protected static final String REPO_FOLDER = "repoFolder";
    protected static final String LOCAL_PROFILE = "localProfile";
    protected static final String LOCAL_CLF = "localCommandLineFile";
    protected static final String LOCAL_CLF_FORMAT = "localCLFFormat";
    protected static final String SUB_PROFILE_PATH = "subProfilePath";

    protected static final java.text.DateFormat s_df = new java.text.
SimpleDateFormat("HH:mm:ss.SSS");

    protected static PrintStream s_printStream = null;

    protected ICTSProfileService m_profileService;
    protected String m_repository;
    protected String m_user;
    protected String m_password;
    protected String m_domain;
    protected String m_repoFolder;
    protected String m_localProfile;
    protected String m_localCLF;
    protected String m_localCLFFormat;
    protected String m_subProfilePath;

    public static void main (String[] args)
            throws Exception
    {
        try
        {
            if (args != null && args.length > 0 && args[0] != null
&& args[0].length() > 0)
            {
                s_printStream = new PrintStream(args[0]);
                System.out.println("Test results will be saved in "
+ args[0]);
            }

            junit.textui.TestRunner.run(suite());
        }
        finally
```

```
        {
            if (s_printStream != null)
                s_printStream.close();
        }
    }

    public static Test suite()
    {
        return new TestSuite(CTSWSProfileServiceTestCase.class);
    }

    protected void setUp() throws Exception
    {

        Properties props = new Properties();

        props.load(getClass().getResourceAsStream(TEST_CONFIG));

        ContextFactory cf = ContextFactory.getInstance();
        IServiceContext context = cf.newContext();

        ContentTransferProfile transferProfile = new
ContentTransferProfile();
        transferProfile.setTransferMode(ContentTransferMode.MTOM);
        context.setProfile(transferProfile);

        m_repository = (String)props.get(REPOSITORY);
        m_user = (String)props.get(USER_NAME);
        m_password = (String)props.get(PASSWORD);
        m_domain = (String)props.get(DOMAIN);
        m_repoFolder = (String)props.get(REPO_FOLDER);
        m_localProfile = (String)props.get(LOCAL_PROFILE);
        m_localCLF = (String)props.get(LOCAL_CLF);
        m_localCLFFormat = (String)props.get(LOCAL_CLF_FORMAT);
        m_subProfilePath = (String)props.get(SUB_PROFILE_PATH);

        context.addIdentity(new RepositoryIdentity(m_repository,
m_user, m_password, m_domain));
        context = cf.register(context);

        ServiceFactory sf = ServiceFactory.getInstance();
        m_profileService = sf.getLocalService(ICTSProfileService.
class, context);
        //m_profileService = sf.getRemoteService(ICTSProfileService.
class, context);

        log("Initialized the Profile serivce");
    }

    protected void tearDown() throws Exception
    {
    }

    public void testGetProfiles()
            throws Exception
    {

        CTSProfileRequest pr = new CTSProfileRequest();
        executeGetProfiles(pr);

        String profileId = getProfileId();

        pr.setProfileId(profileId);
m_profileService.getProfileById(profileId);
        printProfileInfo(profile);

        pr = new CTSProfileRequest();
        String profileName = "register";
        pr.setProfileName(profileName);
        log(1, "### GetProfiles with profileName=%s", profileName);
```

```
        executeGetProfiles(pr);

        pr = new CTSProfileRequest();
        String profileLabel = "Adjust Contrast";
        pr.setProfileLabel(profileLabel);
        log(1, "### GetProfiles with profileLabel=%s", profileLabel);
        executeGetProfiles(pr);

        pr = new CTSProfileRequest();
        String sourceFormat = "illustrator10";
        pr.setSourceFormat(sourceFormat);
        pr.setPublicOnly(true);

        executeGetProfiles(pr);

        pr = new CTSProfileRequest();
        sourceFormat = "illustrator10";
        pr.setSourceFormat(sourceFormat);
        pr.addFilter(CTSProfile.FILTER_CTS_PRODUCT, CTSProfile.
CATEGORY_MTS);
        pr.addFilter(CTSProfile.FILTER_CTS_PRODUCT, CTSProfile.
CATEGORY_PUBLIC);
                executeGetProfiles(pr);

        pr = new CTSProfileRequest();
        String targetFormat = "pdf";
        pr.setSourceFormat(sourceFormat);
        pr.setPublicOnly(true);
        pr.setTargetFormat(targetFormat);


        pr = new CTSProfileRequest();
        String objectType = "dmc_content_collection";
        pr.setObjectType(objectType);
        pr.setPublicOnly(true);
        pr.addFilter(CTSProfile.FILTER_VIRTUAL_DOCUMENT,
CTSProfile.CATEGORY_VIRTUAL_DOCUMENT);
                        objectType, CTSProfile.FILTER_
VIRTUAL_DOCUMENT, CTSProfile.CATEGORY_VIRTUAL_DOCUMENT);
        executeGetProfiles(pr);

        pr.addFilter(CTSProfile.FILTER_CTS_PRODUCT,
CTSProfile.CATEGORY_MTS);

                objectType, CTSProfile.FILTER_VIRTUAL_
DOCUMENT, CTSProfile.CATEGORY_VIRTUAL_DOCUMENT,
                CTSProfile.FILTER_CTS_PRODUCT, CTSProfile.
CATEGORY_MTS);
        executeGetProfiles(pr);

        pr = new CTSProfileRequest();
        pr.setSourceFormat(sourceFormat);
        pr.setTargetFormat(targetFormat);
        CTSProfileQueryFilter  qf1 = new
CTSProfileQueryFilter(CTSProfile.FILTER_APP_PRODUCT,
"=", CTSProfile.CATEGORY_MTS);
        CTSProfileQueryFilter  qf2 = new
CTSProfileQueryFilter(CTSProfile.FILTER_VISIBILITY,
"=", CTSProfile.CATEGORY_SYSTEM);
        CTSProfileQueryFilter  qf3 = new
CTSProfileQueryFilter(qf1, "AND", qf2);
        CTSProfileQueryFilter  qf4 = new
CTSProfileQueryFilter(CTSProfile.FILTER_COLLECTION,
"=", CTSProfile.CATEGORY_COLLECTION);
        CTSProfileQueryFilter  qf5 = new
CTSProfileQueryFilter(qf3, "OR", qf4);
        pr.addFilter(qf5);


        pr.setObjectType(objectType);
```

```
        executeGetProfiles(pr);
    }

    public void testManageProfiles() throws Exception
    {
        CTSProfile profile;



        String profileName = "register";
        log(1, "### GetProfileByName=%s", profileName);
        profile = m_profileService.getProfileByName
(m_repository, profileName);
        printProfileInfo(profile);

        String newProfileName = "myRegister";
        profile.setName(newProfileName);
                m_profileService.addProfile
(m_repository, profile, m_repoFolder);
        profile = m_profileService.getProfileByName
(m_repository, newProfileName);
        printProfileInfo(profile);

        if (profile != null)
        {
            profile.getName());
            m_profileService.removeProfile(profile.getProfileId(),
true, false);
            log("Querying the removed profile: %s",
profile.getName());
            profile = m_profileService.getProfileByName
(m_repository, profile.getName());
            printProfileInfo(profile);
        }
        String clfId = m_profileService.addCommandLineFile
(new CTSLocalTextFile(m_localCLF), m_repository, m_repoFolder,

null, m_localCLFFormat, null);
        log("Updating the command line file: %s", clfId);
        String newCLFId = m_profileService.updateCommandLineFile
(clfId, new CTSLocalTextFile(m_localCLF), IDfCheckinOperation.
NEXT_MINOR, null);
        log("Removing the current version of the command line
file: %s", newCLFId);
        m_profileService.removeCommandLineFile(newCLFId,
false, true);
        log("Removing all versions of the command line
file: %s", clfId);
        m_profileService.removeCommandLineFile(clfId,
true, true);

                String profileId = m_profileService.
addLocalProfile(m_repository, new CTSLocalTextFile(m_localProfile),
m_repoFolder);
        CTSProfile localProfile = m_profileService.getProfileById
(profileId);
        printProfileInfo(localProfile);

                localProfile.addProfileFilter(CTSProfile.
FILTER_APP_PRODUCT, "MyProduct");
        String newProfileId = m_profileService.updateProfile
(localProfile, CTSProfileService.SAME_VERSION);
        log("new profileId=%s", newProfileId);
        localProfile = m_profileService.getProfileByName
(m_repository, localProfile.getName());
        printProfileInfo(localProfile);


        m_profileService.removeProfile(localProfile.getProfileId(),
```

```
true, false);
        log("Querying the removed profile: %s", localProfile.getName());
        localProfile = m_profileService.getProfileByName(m_repository,
localProfile.getName());
        printProfileInfo(localProfile);

        CTSProfile newProfile = new CTSProfile();
        newProfile.setName("myRoot");
        newProfile.setLabel("myRoot Label");
        newProfile.setDescription("myRoot Description");
        newProfile.setTranscodeName("myTranscode");
        newProfile.setTranscodeLabel("myTranscode Label");
        newProfile.setOperation(CTSProfileOperation.TRANSFORM);
        newProfile.addFormatPair("mpeg", "mpeg");
        newProfile.addProfileFilter(CTSProfile.FILTER_VISIBILITY,
CTSProfile.CATEGORY_PUBLIC);
        newProfile.addProfileFilter(CTSProfile.FILTER_CTS_PRODUCT,
CTSProfile.CATEGORY_MTS);

        CTSProfileOuterProfileEntry ope = new CTSProfileOuterProfileEntry
(false);
        CTSProfileInnerProfileEntry ipe = new CTSProfileInnerProfileEntry
(m_repoFolder + "/my-sub", false, true);
        ipe.addInnerTokenMapping(new CTSProfileInnerTokenMapping
("false", "overwrite_rendition", true));
        ope.appendInnerProfileEntry(ipe);

        newProfile.setOuterProfileEntry(ope);

        ArrayList params = new ArrayList();
        params.add(new CTSProfileParamFile(newProfile, m_repoFolder));
        params.add(new CTSProfileParamFile(CTSProfileParamFile.
PT_PROFILE, m_subProfilePath, m_repoFolder, null, null, null));
        params.add(new CTSProfileParamFile(CTSProfileParamFile.
PT_COMMAND_LINE_FILE, m_localCLF, m_repoFolder, null, m_localCLFFormat,
null));

        List resultIds = m_profileService.addProfiles(params,
m_repository);
        for (int i = 0; i < params.size(); i++)
        {
            if (params.get(i).getProfileType() == CTSProfileParamFile.
PT_PROFILE)
                printProfileInfo(m_profileService.getProfileById
(resultIds.get(i)));
            else
                log("A command line file was saved with %s",
resultIds.get(i));
        }

        for (int i = 0; i < params.size(); i++)
        {
            if (params.get(i).getProfileType() == CTSProfileParamFile.
PT_PROFILE)
                m_profileService.removeProfile(resultIds.get(i), true,
false);
            else
                m_profileService.removeCommandLineFile(resultIds.get(i),
true, false);
        }
    }

    /*
    public void tstEcho()
    {
        try
        {
            String result = m_profileService.echo("Hello");

            log("echo result = " + result);
```

```
        }
        catch (Exception e)
        {
            log("echo error: " + e.toString());
            e.printStackTrace();
        }
    }
    */

    protected void executeGetProfiles(CTSProfileRequest request)
    {
        try
        {
            List  profiles = m_profileService.getProfiles(null, request);

            if (profiles != null)
            {
                for (CTSProfile profile : profiles)
                    printProfileInfo(profile);
            }
        }
        catch(Exception e)
        {
            log("getProfiles error: " + e.toString());
            e.printStackTrace();
        }
    }

    protected String getProfileId()
            throws Exception
    {
        IDfCollection coll = null;
        IDfQuery query = new DfQuery();
        String profileId = null;
        IDfSession session = null;
        IDfClient client = DfClient.getLocalClient();
        IDfSessionManager sm = client.newSessionManager();

        query.setDQL("SELECT r_object_id FROM dm_media_profile
WHERE ANY (filter_names='Public' AND filter_values='Public')");

        try
        {
            IDfLoginInfo li = new DfLoginInfo();

            li.setUser(m_user);
            li.setPassword(m_password);
            li.setDomain(m_domain);

            sm.setIdentity(m_repository, li);
            session = sm.getSession(m_repository);

            coll = query.execute(session, IDfQuery.DF_READ_QUERY);
            if (coll.next())
                profileId = coll.getString("r_object_id");
        }
        catch (Exception e)
        {
            log("Error in getProfileId: " + e.toString());
        }
        finally
        {
            if (coll != null)
                coll.close();
            if (session != null)
                sm.release(session);
        }

        return profileId;
    }
```

```
    protected static void printProfileInfo(CTSProfile profile)
    {
        if (profile == null)
        {
            log(1, "printProfileInfo: null profile !!!%n");
            return;
        }

        String  filterMsg = "";
        if (profile.getProfileFilters() != null)
        {
            filterMsg = String.format("%n\t[Filters] ");
            for (CTSProfileFilter filter: profile.getProfileFilters())
            {
                String filterValues = null;
                for (String filterValue : filter.getFilterValues())
                {
                    if (filterValues == null)
                        filterValues = filterValue;
                    else
                        filterValues += "," + filterValue;
                }
                filterMsg += "(" + filter.getFilterName() + ":
" + filterValues + ")";
            }
        }

        String formatMsg = "";
        if (profile.getFormatPairs() != null)
        {
            formatMsg = String.format("%n\t[Formats]");
            for (CTSProfileFormatPair format: profile.getFormatPairs())
            {
                formatMsg += String.format("%n\t%s:", format.
getSourceFormat());

                List targets = format.getTargetFormats();
                for (String target: targets)
                    formatMsg += " " + target;
            }
        }

        String paramMsg = "";
        if (profile.getParameters() != null)
        {
            paramMsg = String.format("%n\t[Parameters]");
            for (CTSProfileParameter param: profile.getParameters())
            {
                paramMsg += String.format("%n\t%s: %s, %s, %s, %s",
                        param.getName(), param.getLabel(), param.
getDescription(), param.getDataType(), param.getControlType());
                if (param.getList() != null)
                {
                    for (CTSParameterListValue plv: param.getList())
                        paramMsg += "(" + plv.getOptionName() + "," +
plv.getOptionValue() + ")";
                }
                if (param.getRange() != null)
                    paramMsg += "(" + param.getRange().getMaxValue()
+ "," + param.getRange().getMinValue() + ")";
                if (param.getRelatedProfileParameter() != null)
                    paramMsg += ", Related: " + param.
getRelatedProfileParameter().getName();
            }
        }

        String cmdMsg = "";
        if (profile.getCommandFilePaths() != null)
        {
```

```
            cmdMsg = String.format("%n\t[Command File Paths]");
            for (CTSProfileCommandFilePath cmdPath: profile.
getCommandFilePaths())
                cmdMsg += String.format("%n\t%s: %s", cmdPath.
getMPType(), cmdPath.getFilePath());
        }

        String outerMsg = "";
        if (profile.getOuterProfileEntry() != null)
        {
            CTSProfileOuterProfileEntry    ope = profile.
getOuterProfileEntry();
            outerMsg = String.format("%n\t[%s Profile]",
ope.getIsChainProfile() ? "Chain" : "Sequence");
            for (CTSProfileInnerProfileEntry ipe: ope.
getInnerProfileEntries())
            {
                outerMsg += String.format("%n\t%s, ut:%s,
woc:%s", ipe.getPath(), ipe.getUseTargetFormat(), ipe.getWaitOnCompletion());
                if (ipe.getInnerTokenMappings() != null)
                {
                    for (CTSProfileInnerTokenMapping itm:
ipe.getInnerTokenMappings())
                        outerMsg += String.format("%n\t\tinner-token:%s,
local-token:%s, literal:%s", itm.getInnerProfileToken(),
itm.getLocalProfileToken(), itm.getLiteral());
                }
            }
        }

        log("Profile Name: %s(%s), Version:%s,
folder:%s%n\tLabel: %s, Desc: %s%n\trelated=%s,
notify=%s, op=%s, trans=(%s,%s), taskImpl=%s%s%s%s%s%s%s",
                profile.getName(), profile.getProfileId(),
profile.getVersionLabels(), profile.getRepositoryFolder(),
                profile.getLabel(), profile.getDescription(),
                profile.getRelatedObjectsOnly(), profile.
getNotifyResult(), profile.getOperation(),
                profile.getTranscodeName(), profile.
getTranscodeLabel(), profile.getTaskImpl(),
                filterMsg, formatMsg, paramMsg, cmdMsg, outerMsg);
    }

    protected static void log(int newLines, String format,
Object ... args)
    {
        for (int i = 0; i < newLines; i++)
        {
            System.out.println();
            if (s_printStream != null)
                s_printStream.println();
        }

        log(format, args);
    }

    protected static void log(String format, Object ... args)
    {
        String msg = "[" + s_df.format(Calendar.getInstance().
getTime()) + "] " + String.format(format, args);

        System.out.println(msg);
        if (s_printStream != null)
            s_printStream.println(msg);
    }
}
```

7.1.7 removeProfile operation

This removeProfile operation removes a profile with a specific profileId. All versions of the profile will be removed, if specified.

7.1.7.1 Java syntax

```
void removeProfile(ObjectIdentity profileId,
                   boolean allVersions,
                   boolean safeRemove)
        throws TransformationServiceException
```

7.1.7.2 Parameters

Parameter	Data type	Description
profileId	ObjectIdentity	the profile id
allVersions	boolean	if all versions of profile should be deleted
safeRemove	boolean	if it requires to check the validity

7.1.7.3 Response

The removeProfile code removes a profile with a specific profile id.

7.1.8 updateProfile operation

The updateProfile operation updates a specified saved profile.

7.1.8.1 Java syntax

```
ObjectIdentity updateProfile(Profile theProfile, int version)
        throws TransformationServiceException
```

7.1.8.2 Parameters

Parameter	Data type	Description
theProfile	Profile	theProfile to be updated
version	int	version number to be assigned to the updated profile

7.1.8.3 Response

The updateProfile code returns the identity of the new version object.

7.1.8.4 Exceptions

If the updateProfile code fails to update theProfile, it throws the following exception:

```
TransformationServiceException
```

7.2 Transformation service

The Transformation web service (ITransformationService) enables applications to request transformations from the Transformation Services suite of products in both synchronous and asynchronous modes.

ITransformationService provides functionality to add, get, delete and transform jobs to the server in a synchronous and asynchronous mode.

Transformation web services is implemented as a pojo service. Transformation Services is deployed using the Content Transformation WebServices installer.

This section describes different objects and operations related to ITransformationService.

7.2.1 Objects related to this service

The Object service provides a set of basic operations on repository objects. The objects related to the Transformation web service include:

- ContentProperty

  Encapsulates the transformation property

- FileTargetInfo

  Used for realtime requests where File Output is expected

- JobFilter

  Used as a filter class

- JobTicket

  Encapsulates the transformation request

- RepositoryTargetInfo

  Used when Repo Output is expected

- SourceContent

7.2.2 addJob operation

The addJob operation creates an asynchronous transformation job in the repository and returns a Job Id for tracking purposes. It adds a transformation request(s) to the dm_queue. The source document is in the repository. Transformation Services polls the transformation at its own time. JobTicket encapsulates the profile being executed and the required parameters for the request.

7.2.2.1 Java syntax

```
ObjectIdentity addJob(JobTicket jobTicket)
        throws
TransformationServiceException
```

7.2.2.2 Parameters

The addJob interface contains one method that is important for identifying the file path of the source file being transformed. The local file path is configured in the jobTicket file.

Parameter	Data type	Description
jobTicket	JobTicket	Representation of the job to add.

7.2.2.3 Response

The method returns a unique ObjectIdentity representing a jobTicket which is used for finding the job status of the transformation.

7.2.2.4 Exceptions

If the operation fails to create the jobTicket, it throws the following exception:

```
TransformationServiceException
```

7.2.2.5 Example

The operation submits an asynchronous transformation request to the transformation service. This will create a transformation request in the repository and the job id will be returned. The job id can be later used to find the status of the job. The transformation request details are sent in as JobTicket.

7.2.2.5.1 Retrieving an object from the repository

If the source file is taken from the repository, the source id is mentioned in the job ticket. If the file is sent as an attachment, the local file path is configured in the jobticket file. The following example uses jobId to reference a repository object and retrieve it from the repository:

Example 7-2: Java: Sample of Basic Object Retrieval

```
/**
        * tests AddJob asynchronous webservice method...
              This method intakes the jobticket object.
              Make sure you pass in
        * all the required entries in the job ticket object
        * @throws Exception if it fails
        */
    ITransformationService myTransformationService;
    IProfileService myProfileService;
    IObjectService myDFSCoreService;
    private void setUp() throws Exception
       {
           myConfig = new ClientConfig();
           ContextFactory cf = ContextFactory.getInstance();
           IServiceContext context = cf.newContext();
           ContentTransferProfile transferProfile =
                   new ContentTransferProfile();
           transferProfile.setTransferMode( ContentTransferMode.BASE64);
           context.setProfile(transferProfile);

           myRepository = myConfig.getRepositoryName();
           myUsername = myConfig.getUsername();
           myPassword = myConfig.getPassword();
           myDomain = myConfig.getDomain();

           context.addIdentity(myConfig.getRepositoryIdentity());
           context = cf.register(context, "transformation",
                   myConfig.getDFSServiceURL());
           ServiceFactory sf = ServiceFactory.getInstance();
           String theServiceType =  myConfig.getServiceType();
           if( theServiceType.equals( "remote"))
           {
               myTransformationService = sf.getRemoteService
                       ( ITransformationService.class, context,

                       "transformation",
            myConfig.getDFSServiceURL());
            myProfileService = sf.getRemoteService
                       (IProfileService.class, context,
           "transformation",
            myConfig.getDFSServiceURL());
           myDFSCoreService =  sf.getRemoteService(IObjectService.
                   class, context,
             "core",
               myConfig.getDFSServiceURL()); }
           else
                   { myProfileService = sf.getLocalService( IProfileService.
                   class, context);
          myTransformationService = sf.getLocalService
                   ( ITransformationService.class, context);
          myDFSCoreService =  sf.getLocalService(IObjectService.class,
                                                      context );
           }log("Initialized the Transformation And Profile services");
           }public void testAddJob() throws Exception
         //create a jobticket object for this asynchronous request
           JobTicket theJobTicket = new JobTicket();
           String theSourceObjectId = "0907e97280017a53";
           String theSourceFormat = "jpeg";
```

```
            String theTargetFormat = "gif";
            String theProfileName = "transformTo";
            String theRenditionName = "my_name";
            String theRenditionDescription = "my_description";
            int thePriority = 9;
            //required paramters
            theJobTicket.setSourceObjectId( theSourceObjectId );
            theJobTicket.setSourceFormat( theSourceFormat );
            theJobTicket.setTargetFormat( theTargetFormat);
            Profile theProfile = myProfileService.getProfileByName
                    ( myRepository, theProfileName);
            theJobTicket.setProfile( theProfile );
            List theProfileParameters = new ArrayList();
            ProfileParameter theProfileParameter =
                    new ProfileParameter();
            theProfileParameter.setName( "doc_token_targetFormat");
            theProfileParameter.setValue( "gif");
            theProfileParameters.add( theProfileParameter );
            theJobTicket.setProfileParameter( theProfileParameters );
        //required.. make sure you pass in all the
            // required profile parameter info for this transformation
                        request

            // see the usage of some optional parameters here
            theJobTicket.setRenditionName( theRenditionName );
                    //optional
            theJobTicket.setRenditionDescription
                ( theRenditionDescription); //optional
            theJobTicket.setNotifyUser( true );
            theJobTicket.setPriority( thePriority );

          //make an asynchronous request here and get the unique job id
            ObjectIdentity  theJobId = myTransformationService.
                    addJob( theJobTicket );if( theJobId != null )
```

7.2.3 cleanUpJobs operation

The cleanUpJobs operation deletes all the transformation related objects by a specified date. This operation is used to set the conditions (based on time) of clearing a record of transformations. The operation clean ups all the jobs and their associated objects from the queue, which were created before the target date.

7.2.3.1 Java syntax

```
boolean cleanUpJobs(Date date)
        throws
TransformationServiceException
```

7.2.3.2 Parameters

The *cleanUpJobs* interface contains the dateFilter method identifies the objects to be cleaned-up by the date specified.

Parameter	Data type	Description
date	Date	Specifies date prior to which to clean up transformations.

7.2.3.3 Example(s)

7.2.3.3.1 Deleting job file entries

Use the CleanUpJob webservices method to clean up all the job file entries, and associated jobs, that were created prior to the clean up date.

Example 7-3: Java: Basic object delete

```
        // Let us delete all the objects created prior to today..
        JobFilter theDateFilter = new JobFilter();
        theDateFilter.setDate( new Date() );
        boolean result  = myTransformationService.cleanUpJobs
            ( theDateFilter );}
```

7.2.4 deleteJob operation

The deleteJob operation deletes only a specific transformation by JobId before it is processed. If the job has already started, it will not be deleted.

7.2.4.1 Java syntax

```
boolean deleteJob(ObjectIdentity jobId)
        throws
TransformationServiceException
```

7.2.4.2 Parameters

The deleteJob interface contains one method that identifies the transformation to be deleted by object identity.

Parameter	Data type	Description
jobId	ObjectIdentity	ObjectIdentity instance specifying the job to be deleted.

7.2.5 getJobInfo operation

The getJobInfo operation queries the job details of a requested transformation based on the Job Id. This operation provides the job status about the specified object, for example, getJobInfo will inform if the job is pending, in progress, failed, or complete.

7.2.5.1 Java syntax

```
JobInfo getJobInfo(ObjectIdentity objectIdentuty)
        throws
TransformationServiceException
```

7.2.5.2 Parameters

Parameter	Data type	Description
objectIdentity	ObjectIdentity	The identity of the job about which to retrieve info

7.2.5.3 Response

The method returns a JobInfo instance with data about the job.

7.2.5.4 Example

7.2.5.4.1 Testing GetJob asynchronous webservice

This example gets the JobStatus method to test the GetJob asynchronous webservice method.

Example 7-4: Java: Sample job information retrieval

```
/**
     * Gets the Job Status..method to test GetJob asynchronous webservice
method.
     * @throws Exception if it fails*/
    ITransformationService myTransformationService;
     IProfileService myProfileService;
     IObjectService myDFSCoreService;
     private void setUp() throws Exception
        {
            log(1, "TransformServiceTestCase started");
            myConfig = new ClientConfig();
            ContextFactory cf = ContextFactory.getInstance();
            IServiceContext context = cf.newContext();
            ContentTransferProfile transferProfile = new
ContentTransferProfile();
            transferProfile.setTransferMode( ContentTransferMode.BASE64);
            context.setProfile(transferProfile);

            myRepository = myConfig.getRepositoryName();
            myUsername = myConfig.getUsername();
            myPassword = myConfig.getPassword();
            myDomain = myConfig.getDomain();

            context.addIdentity(myConfig.getRepositoryIdentity());
```

```
            context = cf.register(context, "transformation",
myConfig.getDFSServiceURL());
            ServiceFactory sf = ServiceFactory.getInstance();
            String theServiceType =  myConfig.getServiceType();
            if( theServiceType.equals( "remote"))
            {
                myTransformationService = sf.getRemoteService
( ITransformationService.class, context,




                                  "transformation",

 myConfig.getDFSServiceURL());
                myProfileService = sf.getRemoteService
(IProfileService.class, context,
                      "transformation",
                        myConfig.getDFSServiceURL());
                myDFSCoreService =  sf.getRemoteService
(IObjectService.class, context,
        "core",
  myConfig.getDFSServiceURL());
            }
            else
            {
                myProfileService = sf.getLocalService
( IProfileService.class, context);
                myTransformationService = sf.getLocalService
( ITransformationService.class, context);
                myDFSCoreService =  sf.getLocalService
(IObjectService.class, context );
            }
            log("Initialized the Transformation And Profile services");
        }
    public void testGetJob() throws Exception
    {
        if( myJobIds != null && myJobIds.size() > 0)
        {
            ObjectIdentity theJobId = myJobIds.get( 0 );
            JobInfo theJobInfo  = myTransformationService.getJobInfo
( theJobId );
            printRepoJobInfo( theJobInfo );
        }
}
```

7.2.6 importAndAddJob operation

The importAndAddJob operation submits a transformation request on a source file to the Transformation Services server asynchronously. The user submits the source file which is imported to the repository from the client's machine prior to adding it to the queue for transformation. The operation gives back JobId for status tracking.

7.2.6.1 Java syntax

```
ObjectIdentity importAndAddJob(DataPackage sourceObjects, JobTicket jobTicket)
        throws
TransformationServiceException
```

7.2.6.2 Parameters

The importAndAddJob interface contains two methods that are required to identify the source file and the transformation job ticket (to follow-up on the status of the transformation).

Parameter	Data type	Description
sourceObjects	DataPackage	DataPackage the source file Content
JobTicket	JobTicket	the xml job ticket

7.2.6.3 Response

The method returns an ObjectIdentity value that identifies the created job id.

7.2.6.4 Example

You add a transformation job to the Transformation Services server queue for a source file from the client machine and you get the transformation requested.

7.2.6.4.1 Importing a non-repository file

This is a sample of a transformation request with a file being imported.

Example 7-5: Java: Sample send transformation with a non-repository file

```
/**
      * tests ImportAndAddJob asynchronous webservice method.
            The source can be sent along with the request, and,
      *  source is not required to be in the repository
      *
      *   additional optional parameters for this method
      *   theJobTicket.setStoreResultInRepo( true );
      *
      * @throws Exception if it fails
      */
    ITransformationService myTransformationService;
     IProfileService myProfileService;
     IObjectService myDFSCoreService;
     private void setUp() throws Exception
        {
            log(1, "TransformServiceTestCase started");
            myConfig = new ClientConfig();
            ContextFactory cf = ContextFactory.getInstance();
            IServiceContext context = cf.newContext();
            ContentTransferProfile transferProfile = new
ContentTransferProfile();
            transferProfile.setTransferMode( ContentTransferMode.BASE64);
            context.setProfile(transferProfile);
```

```
            myRepository = myConfig.getRepositoryName();
            myUsername = myConfig.getUsername();
            myPassword = myConfig.getPassword();
            myDomain = myConfig.getDomain();

            context.addIdentity(myConfig.getRepositoryIdentity());
            context = cf.register(context, "transformation",
myConfig.getDFSServiceURL());
            ServiceFactory sf = ServiceFactory.getInstance();
            String theServiceType =  myConfig.getServiceType();
            if( theServiceType.equals( "remote"))
            {
             myTransformationService = sf.getRemoteService
( ITransformationService.class, context,
         "transformation",
           myConfig.getDFSServiceURL());
           myProfileService = sf.getRemoteService(IProfileService.
class, context,
           "transformation",
         myConfig.getDFSServiceURL());
          myDFSCoreService =  sf.getRemoteService(IObjectService.
class, context,
           "core",
          myConfig.getDFSServiceURL());
            }
            else
            {
           myProfileService = sf.getLocalService( IProfileService.
class, context);
            myTransformationService = sf.getLocalService
( ITransformationService.class, context);
            myDFSCoreService =  sf.getLocalService(IObjectService.
class, context );
            }
            public void testImportAndAddJob()
             throws Exception
     {
         //create a jobticket object for this asynchronous request
         JobTicket theJobTicket = new JobTicket();
         //get the source format here
         String theSourceFormat = myConfig.getTestFileFormat();
         String theTargetFormat = "gif";
         String theProfileName = "flip";
         String theRenditionName = "my_name";
         String theRenditionDescription = "my_description";
         String theParamTokenName = "doc_token_direction";
         String theParamTokenValue = "vertical";
         int thePriority = 9;
         //required paramters
         theJobTicket.setSourceFormat( theSourceFormat );
         theJobTicket.setTargetFormat( theTargetFormat);
        // theJobTicket.setRepoFolderName( "/india/Sheeba_test2");

         Profile theProfile = myProfileService.getProfileByName
( myRepository, theProfileName);
         theJobTicket.setProfile( theProfile );
         List theProfileParameters = new ArrayList();
         ProfileParameter theProfileParameter =  new ProfileParameter();
         theProfileParameter.setName( "doc_token_direction");
         theProfileParameter.setValue( "vertical");
         theProfileParameters.add( theProfileParameter );
         theJobTicket.setProfileParameter( theProfileParameters );
 //required.. make sure you pass in all the
         // required profile parameter info for this transformation
request

         // see the usage of some optional parameters here
         theJobTicket.setRenditionName( theRenditionName );
 //optional
```

```
         theJobTicket.setRenditionDescription( theRenditionDescription);
//optional
         theJobTicket.setNotifyUser( true );
         theJobTicket.setPriority( thePriority );

         //additional optional parameters for this method
         theJobTicket.setStoreResultInRepo( true );

         //and here comes the source file to be imported
         DataPackage theDataPackage = createDataPackage
( myConfig.getTestFileName(), myConfig.getTestFileFormat() );
         ObjectIdentity theJobId = myTransformationService.
importAndAddJob( theDataPackage, theJobTicket );
         if( theJobId != null )
         {
             JobInfo theJobInfo = myTransformationService.
getJobInfo( theJobId );
             printRepoJobInfo( theJobInfo );
         }
}
```

7.2.7 transformJob operation

The transformJob operation submits a transformation request directly to the Transformation Services server synchronously. The call is made directly to the Transformation Services server and you get the result back. This operation is ideal when a user wants to see a sample of the transformation before committing the source document or its transformation to the repository. There are four ways to submit your request:

- File Input File Output

  The source file is retrieved from the client's file system; the transformation is stored in the client machine and is not stored in the repository.

- File Input Repo Output

  The source file is retrieved from the client's file system; the transformation is stored in the repository. (The user can provide a target repository location for output files.)

- Repo Input Repo Output

  The source file is retrieved from the repository; the transformation is added to the source file in the repository.

- Repo Input File Output

  The source file is retrieved from the repository; the transformation goes to the client's file system (not the repository).

7.2.7.1 Java syntax

```
JobInfo transformJob(JobTicket jobTicket)
        throws TransformationServiceException
```

7.2.7.2 Parameters

Parameter	Data type	Description
jobTicket	JobTicket	Represents the transformation job.

7.2.7.3 Response

The transformJob operation returns a JobInfo object representing the executed job.

7.2.7.4 Example

Key points to remember when implementing transformJob webservice:

- *File Output* – the results of this method go through theJobInfo.getFileTargetInfos
- *Repo Output* – the results of this method go through theJobInfo.getRepositoryTargetInfos

This section includes code examples for:

- File Input File Output
- File Input Repo Output
- Repo Input Repo Output
- Repo Inout File Output

7.2.7.4.1 File Input File Output

The sample shows how to make a real-time transformation request of a file originating from, and going back to, the client machine.

Example 7-6: Java: File Input Repo Output

```
/**
     * ---------------------------------------------------------
        * Let us test the four main permutations of synchronus real
        time requests.
     * 1. File Input File Output -
                    theJobTicket.setSourceContent( theDatas );
     *  theJobTicket.setStoreResultInRepo( false );
     *  2. File Input Repo Output - theJobTicket.
                    setSourceContent( theDatas );
     *   theJobTicket.setStoreResultInRepo( true );
     * 3. Repo Input Repo Output  - theJobTicket.
                setSourceObjectId( theSourceObjectId );
     * theJobTicket.setStoreResultInRepo( true );
     * 4. Repo Inout File Output - theJobTicket.
                setSourceObjectId( theSourceObjectId );
```

```
     * theJobTicket.setStoreResultInRepo( false );
     * ------------------------------------------------------------
* Common Key points:
     * File Outputs : get the result through theJobInfo.
            getFileTargetInfos()
     *
     * Repo Outputs : get the result through theJobInfo.
            getRepositoryTargetInfos()
     */
    ITransformationService myTransformationService;
     IProfileService myProfileService;
     IObjectService myDFSCoreService;
     private void setUp() throws Exception
        {
            log(1, "TransformServiceTestCase started");
            myConfig = new ClientConfig();
            ContextFactory cf = ContextFactory.getInstance();
            IServiceContext context = cf.newContext();
            ContentTransferProfile transferProfile =
newContentTransferProfile();
            transferProfile.setTransferMode( ContentTransferMode.BASE64);
            context.setProfile(transferProfile);

            myRepository = myConfig.getRepositoryName();
            myUsername = myConfig.getUsername();
            myPassword = myConfig.getPassword();
            myDomain = myConfig.getDomain();

            context.addIdentity(myConfig.getRepositoryIdentity());
            context = cf.register(context, "transformation",
myConfiggetDFSServiceURL());
            ServiceFactory sf = ServiceFactory.getInstance();
            String theServiceType =  myConfig.getServiceType();
            if( theServiceType.equals( "remote"))
            {
                myTransformationService = sf.getRemoteServic
( ITransformationService.class, context,
"transformation",
   myConfig.getDFSServiceURL());
                myProfileService = sf.getRemoteService
(IProfileService.class, context,
 "transformation",

   myConfig.getDFSServiceURL());
                myDFSCoreService =  sf.getRemoteService
(IObjectService.class, context,
  "core",
myConfig.getDFSServiceURL());
            }
            else
            {
                myProfileService = sf.getLocalServic
( IProfileService.class, context);
                myTransformationService = sf.getLocalService
( ITransformationService.class, context);
                myDFSCoreService =  sf.getLocalService
(IObjectService.class, context );
            }
            log("Initialized the Transformation And Profile services");
        }
    /**
     *  1. File Input Repo Output
     *
     * * ******  Key Points *******
     * theJobTicket.setStoreResultInRepo( true );
     *
    * and you can get the output through the theJobInfo.getTargetInfos()
     *
     * @throws Exception if it fails
     */
```

```
    public void testFileInRepoOut() throws Exception
    {

        //this is my source file
        String theFilename = "R:\\MediaServer\\Main\\WebServices
\\data\\test.jpeg";
        String theSourceFormat = "jpeg";
        JobTicket theJobTicket = new JobTicket();
        //this is important
        theJobTicket.setStoreResultInRepo( true);

        //sets the source content here
        List theSourceDatas = new ArrayList();
        SourceContent theSource = new SourceContent();
        //or this.. if the source file is available on the
local file system
        DataHandlerContent imageContent = new DataHandlerContent
(new DataHandler(new FileDataSource(theFilename)), theSourceFormat);
        theSource.setSourceFileContent( imageContent );
        theSource.setSourceFileName( theFilename );
        theSourceDatas.add( theSource );
        theJobTicket.setSourceContents( theSourceDatas );
        theJobTicket.setRepoFolderName("/india/Sheeba_test1");
        //sets all other
        String theTargetFormat = "gif";
        String theProfileName = "flip";
        theJobTicket.setSourceFormat( theSourceFormat );
        theJobTicket.setTargetFormat( theTargetFormat);
        Profile theProfile = myProfileService.getProfileByName
( myRepository, theProfileName);
        theJobTicket.setProfile( theProfile );

        //add the profile parameter info
        List theProfileParameters = new ArrayList();
        ProfileParameter theProfileParameter =  new
ProfileParameter();
        theProfileParameter.setName( "doc_token_direction");
        theProfileParameter.setValue( "vertical");
        theProfileParameters.add( theProfileParameter );
        theProfileParameter =  new ProfileParameter();
        theProfileParameter.setName( "doc_token_targetFormat");
        theProfileParameter.setValue( theTargetFormat);
//this has to be the dos extension
        theProfileParameters.add( theProfileParameter );
        theJobTicket.setProfileParameter( theProfileParameters );

        // see the usage of some optional parameters here
        JobInfo  theJobInfo = myTransformationService.transformJob
            ( theJobTicket );
        if( theJobInfo != null )
        {
           if( theJobInfo.getJobStatus().equals( "Completed")){
            printRepoJobInfo(theJobInfo);}
            else{
           }
        }
    }


    protected  void printRepoJobInfo( JobInfo theJobInfo)
throws Exception
        {

            String theServerName = theJobInfo.getCTSServerName();
              String theJobId = theJobInfo.getJobId();
               in thePriority = theJobInfo.getJobPriority();
               String theStatus = theJobInfo.getJobStatus();
                String theParameters = theJobInfo.getParameters();
               String theProfileName = theJobInfo.getProfileName();
               String theQueued = theJobInfo.getQueueID();
```

```
               Date theSignOffTime = theJobInfo.getQueueItemSignedOffTime();
                long theSourceFileSize = theJobInfo.getSourceFileSize();
                String theSourceFormat = theJobInfo.getSourceFormat();
               String theSourceId = theJobInfo.getSourceObjectId();
               String theUserName = theJobInfo.getUserName();
               if( theJobInfo.getJobStatus().equals( "Failed" ))
            {
              String theJobInfoXML = theJobInfo.getXMLContent();
                                 }
            else if( theJobInfo.getJobStatus().equals( "Completed" ))
            {
                if( theJobInfo.getRepositoryTargetInfos() != null &&
theJobInfo.getRepositoryTargetInfos().size() > 0)
                {
           for( int i = 0; i < theJobInfo.getRepositoryTargetInfos().
size(); i++ )
                        {
          RepositoryTargetInfo theRepositoryTargetInfo = theJobInfo.
getRepositoryTargetInfos().get( i );
           String theObjectType = theRepositoryTargetInfo.getDfType();

          List theProperties = theRepositoryTargetInfo.
getContentProperties();
                            if( theProperties != null && theProperties.
size() > 0 )
                            {
                                for( ContentProperty theProperty :
theProperties)
                                {
                                                                    }
                        }
                }
                String theJobInfoXML = theJobInfo.getXMLContent();
                log(1, "### JobInfoXML " + theJobInfoXML );
            }
        }


    private URL[] exportObject(TargetInfo theTargetInfo, String
SourceObjectId, String theLocalFolder) throws Exception
    {
        File dsfFile = null;
        String theTargetFormat = theTargetInfo.getTargetFormat();
        String theTargetPageModifier = theTargetInfo.
getTargetPageModifier();
        int thePage = theTargetInfo.getPage();
        String theDfType = theTargetInfo.getDfType();
        String theObjectId;
        ServiceFactory serviceFactory = ServiceFactory.getInstance();
        ContextFactory theContextFactory = ContextFactory.getInstance();
        IServiceContext theContext = theContextFactory.newContext();
        ContentProfile theProfile = new ContentProfile
( FormatFilter.SPECIFIED, theTargetFormat,
          PageFilter.SPECIFIED, thePage,
         PageModifierFilter.SPECIFIED, theTargetPageModifier);
        theContext.setProfile(theProfile);
        theContext.addIdentity(myConfig.getRepositoryIdentity());
        theContext = theContextFactory.register(theContext,
"core", myConfig.getDFSServiceURL());
        IObjectService theObjSvc;
        try
        {
            ObjectIdentitySet theIdentities = null;
            ObjectIdentity objectIdentity = null;
            if( theDfType.equals("DM_CONTENT"))
            {
                theObjectId = SourceObjectId;
                objectIdentity = new ObjectIdentity(new
ObjectId(theObjectId),
                        myConfig.getRepositoryName());
```

```
            }
            else if( theTargetInfo != null)
            {

                objectIdentity = new ObjectIdentity(new
ObjectId(theTargetInfo.getTargetId()),
                           myConfig.getRepositoryName());
            }
            theIdentities = new ObjectIdentitySet(objectIdentity);
            if( myConfig.getServiceType().equalsIgnoreCase( "remote" ))
            {
                // Remote service invocation
                theObjSvc = serviceFactory.getRemoteService
(IObjectService.class, theContext,
                      "core", myConfig.getDFSServiceURL());
                System.out.println("Remote\n--------------------");
            }
            else
            {
                // Local service invocation
                theObjSvc = serviceFactory.getLocalService
(IObjectService.class, theContext);
                System.out.println("Local\n----------------------");
            }
            OperationOptions oo = new OperationOptions();
            DataPackage dataPackage = theObjSvc.get(theIdentities, oo);
            dsfFile = dataPackage.getDataObjects().get(0).getContents
().get(0).getAsFile();
        }
        catch (Exception e)
        {
            e.printStackTrace();
        }
        catch (Throwable t)
        {
            t.printStackTrace();
        }
        if( dsfFile != null && dsfFile.exists() )
        {
            try
            {
                //move the file if needed
                if( theLocalFolder != null )
                {
                    try
                    {
                        String theNewFile = moveFileToLocalFolder
(theLocalFolder,
                                  dsfFile.getAbsolutePath(),
                                   theTargetInfo.getTargetFormat());
                        if( theNewFile != null  )
                        {
                            return new URL[]{new File(theNewFile).toURL()};
                        }
                    }
                    finally
                    {
                        //we can delete this duplicate copy now
                        dsfFile.delete();
                    }
                }
                else
                {
                    System.out.println("exported file : " +
dsfFile.getAbsolutePath());
                    return new URL[]{dsfFile.toURL()};
                }
            }
            catch (Exception e)
            {
```

```
                //todo
            }
            System.out.println("Image exported from content
repository via UCF successfully by Documentum Foundation Services.");
        }
        return null;
}
```

7.2.7.4.2 File Input Repo Output

The sample shows how to make a real-time request retrieving an object from the client machine and placing the rendition in the repository location mentioned in the Jobticket.

Example 7-7: Java: File Input Repo Output

```
 //this is my source file
             String theSourceObjectId = "0907e97280017a53";
             String theSourceFormat = "jpeg";
             JobTicket theJobTicket = new JobTicket();

             //this is important
             theJobTicket.setStoreResultInRepo( false);
             theJobTicket.setSourceObjectId( theSourceObjectId );

             //sets all other
             String theTargetFormat = "tiff";
             String theProfileName = "flip";
             theJobTicket.setSourceFormat( theSourceFormat );
             theJobTicket.setTargetFormat( theTargetFormat);
             Profile theProfile = myProfileService.getProfileByName
( myRepository, theProfileName);
             theJobTicket.setProfile( theProfile );

             //add the profile parameter info
             List theProfileParameters = new ArrayList();
             ProfileParameter theProfileParameter =  new ProfileParameter();
             theProfileParameter.setName( "doc_token_angle");
             theProfileParameter.setValue( "130");
             theProfileParameters.add( theProfileParameter );
             theProfileParameter =  new ProfileParameter();
             theProfileParameter.setName( "doc_token_targetFormat");
             theProfileParameter.setValue( theTargetFormat);
             theProfileParameters.add( theProfileParameter );
             theJobTicket.setProfileParameter( theProfileParameters );
             theProfileParameter =  new ProfileParameter();
             theProfileParameter.setName( "doc_token_direction");
             theProfileParameter.setValue( "vertical");
             theProfileParameters.add( theProfileParameter );
             // see the usage of some optional parameters here
             JobInfo  theJobInfo = myTransformationService.transformJob
( theJobTicket );
             if( theJobInfo != null )
             {
                                 if( theJobInfo.getJobStatus().equals
( "Failed"))
                 {
                                     }
                 else
                 {
                     printFileJobInfo(theJobInfo);
                 }
             }
     }
```

```
        protected  void printFileJobInfo( JobInfo theJobInfo)
throws Exception
        {
                        if( theJobInfo != null )
            {
               List theTargetInfos = theJobInfo.getFileTargetInfos();
               for(FileTargetInfo theTargetInfo : theTargetInfos)
               {
                   Content theTargetContent = theTargetInfo.
getTargetContent();
                   if (theTargetContent instanceof DataHandlerContent)
                   {
                       DataHandlerContent dataHandlerContent =
(DataHandlerContent)theTargetContent;
                       String theFormat = dataHandlerContent.getFormat();
                       if( dataHandlerContent.canGetAsFile() )
                       {
                           File theTempFile = dataHandlerContent.
getAsFile();
                           String theNewName =
getAbsoluteNameWithoutExtension(theTempFile.getAbsolutePath())+".
"+theFormat;
                           theTempFile.renameTo( new File( theNewName) );
                           System.out.println("SUCCESS The result is
available at : " + theNewName);
                       }
                   }
                    List theProperties = theTargetInfo.
getContentProperties();
                    if( theProperties != null && theProperties.
size() > 0 )
                    {
                        for( ContentProperty theProperty : theProperties)
                        {
                       }
                    }
               }
            }
        }
```

7.2.7.4.3 Repo Input Repo Output

The sample shows how to make a real-time transformation request for a repository object as a source for transformation with the output being stored in the repository.

Example 7-8: Java: Repo Input Repo Output

```
 //this is my source file
        String theSourceObjectId = "0907e97280017a53";
        String theSourceFormat = "jpeg";
        JobTicket theJobTicket = new JobTicket();
        //this is important
        theJobTicket.setStoreResultInRepo( true);
        theJobTicket.setSourceObjectId( theSourceObjectId );

        //sets all other
        String theTargetFormat = "png";
        String theProfileName = "rotate";
        theJobTicket.setSourceFormat( theSourceFormat );
        theJobTicket.setTargetFormat( theTargetFormat);
        Profile theProfile = myProfileService.getProfileByName
                ( myRepository, theProfileName);
```

```
        theJobTicket.setProfile( theProfile );

        //add the profile parameter info
        List theProfileParameters = new ArrayList();
        ProfileParameter theProfileParameter =  new ProfileParameter();
        theProfileParameter.setName( "doc_token_angle");
        theProfileParameter.setValue( "60");
        theProfileParameters.add( theProfileParameter );
        theProfileParameter =  new ProfileParameter();
        theProfileParameter.setName( "doc_token_targetFormat");
        theProfileParameter.setValue( theTargetFormat);
        theProfileParameters.add( theProfileParameter );
        theJobTicket.setProfileParameter( theProfileParameters );

        // see the usage of some optional parameters here
         JobInfo  theJobInfo = myTransformationService.transformJob
( theJobTicket );
        if( theJobInfo != null )
        {
          if( theJobInfo.getJobStatus().equals( "Failed"))
            {
                            }
            else
            {
                printRepoJobInfo(theJobInfo);
            }
        }

    }



    protected  void printRepoJobInfo( JobInfo theJobInfo)  throws
Exception
        {

            String theServerName = theJobInfo.getCTSServerName();
            String theJobId = theJobInfo.getJobId();
             int thePriority = theJobInfo.getJobPriority();
             String theStatus = theJobInfo.getJobStatus();
             String theParameters = theJobInfo.getParameters();
              String theProfileName = theJobInfo.getProfileName();
            String theQueued = theJobInfo.getQueueID();
            Date theSignOffTime = theJobInfo.getQueueItemSignedOffTime();
             long theSourceFileSize = theJobInfo.getSourceFileSize();
             String theSourceFormat = theJobInfo.getSourceFormat();
              String theSourceId = theJobInfo.getSourceObjectId();
              String theUserName = theJobInfo.getUserName();
              if( theJobInfo.getJobStatus().equals( "Failed" ))
            {

       String theJobInfoXML = theJobInfo.getXMLContent();
                        }
            else if( theJobInfo.getJobStatus().equals( "Completed" ))
            {
                if( theJobInfo.getRepositoryTargetInfos() != null &&
theJobInfo.getRepositoryTargetInfos().size() > 0)
                {
                    for( int i = 0; i < theJobInfo.
getRepositoryTargetInfos().size(); i++ )
                    {
                        RepositoryTargetInfo theRepositoryTargetInfo =
                                theJobInfo.getRepositoryTargetInfos().get( i );
                        String theObjectType =
theRepositoryTargetInfo.getDfType();



           int thePageNo = theRepositoryTargetInfo.getPage();
```

```
                        long theTargetSize = theRepositoryTargetInfo.
getTargetFileSize();


                  String theTargetFormat = theRepositoryTargetInfo.
getTargetFormat();

                                String theTargetId =
theRepositoryTargetInfo.getTargetId();

                               String theTargetPageModifier =
theRepositoryTargetInfo.getTargetPageModifier();
                                                 String
theTargetProfileName = theRepositoryTargetInfo.getProfileName();

                      URL[] theUrls = exportObject(
theRepositoryTargetInfo, theSourceId, myConfig.getClientCache() );

                List theProperties = theRepositoryTargetInfo.
getContentProperties();
         if( theProperties != null && theProperties.size() > 0 )
                        {
           for( ContentProperty theProperty : theProperties)
                           {
                                                                 }
                        }
                     }
                }
                String theJobInfoXML =
theJobInfo.getXMLContent();
```

7.2.7.4.4 Repo Input File Output

The sample shows how to retrieve an object from the repository with the transformation being stored in the client's file system.

Example 7-9: Java: Repo Input File Output

```
 *
     * ******  Key points *******
     * theJobTicket.setStoreResultInRepo( false );
     *
     * You can get the output through theJobInfo.getTargetContent();
     *
     * @throws Exception if it fails.
     */
     public void testRepoInFileOut() throws Exception
     {
                 //this is my source file
             String theSourceObjectId = "0907e97280017a53";
             String theSourceFormat = "jpeg";
             JobTicket theJobTicket = new JobTicket();

             //this is important
             theJobTicket.setStoreResultInRepo( false);
             theJobTicket.setSourceObjectId( theSourceObjectId );

             //sets all other
             String theTargetFormat = "tiff";
             String theProfileName = "flip";
             theJobTicket.setSourceFormat( theSourceFormat );
             theJobTicket.setTargetFormat( theTargetFormat);
             Profile theProfile = myProfileService.getProfileByName
```

```
( myRepository, theProfileName);
             theJobTicket.setProfile( theProfile );

             //add the profile parameter info
             List theProfileParameters = new ArrayList();
             ProfileParameter theProfileParameter =  new
ProfileParameter();
             theProfileParameter.setName( "doc_token_angle");
             theProfileParameter.setValue( "130");
             theProfileParameters.add( theProfileParameter );
             theProfileParameter =  new ProfileParameter();
             theProfileParameter.setName( "doc_token_targetFormat");
             theProfileParameter.setValue( theTargetFormat);
             theProfileParameters.add( theProfileParameter );
             theJobTicket.setProfileParameter( theProfileParameters );
             theProfileParameter =  new ProfileParameter();
             theProfileParameter.setName( "doc_token_direction");
             theProfileParameter.setValue( "vertical");
             theProfileParameters.add( theProfileParameter );
             // see the usage of some optional parameters here
             log(1, "### Source Format=%s", theSourceFormat );
             log(1, "### Target Format=%s", theTargetFormat );
             log(1, "### Profile Name=%s", theProfileName );
             log(1, "### Source ObjectId=%s", theSourceObjectId );
             JobInfo  theJobInfo = myTransformationService.
transformJob( theJobTicket );
             if( theJobInfo != null )
             {
                 log(1, "### JobId " + theJobInfo.getJobStatus() );
                 if( theJobInfo.getJobStatus().equals( "Failed"))
                 {
                     log(1, "### JobId " + theJobInfo.
getJobErrorDetails());
                 }
                 else
                 {
                     printFileJobInfo(theJobInfo);
                 }
             }
     }


        protected  void printFileJobInfo( JobInfo theJobInfo)
throws Exception
        {
                         {
               List theTargetInfos = theJobInfo.getFileTargetInfos();
               for(FileTargetInfo theTargetInfo : theTargetInfos)
               {
                   Content theTargetContent = theTargetInfo.
getTargetContent();
                   if (theTargetContent instanceof DataHandlerContent)
                   {
                       DataHandlerContent dataHandlerContent =
(DataHandlerContent)theTargetContent;
                       String theFormat = dataHandlerContent.
getFormat();
                       if( dataHandlerContent.canGetAsFile() )
                       {
                           File theTempFile = dataHandlerContent.
getAsFile();
                           String theNewName =
getAbsoluteNameWithoutExtension(theTempFile.getAbsolutePath())+".
"+theFormat;
                           theTempFile.renameTo( new File
( theNewName) );
                           System.out.println("SUCCESS The
result is available at : " + theNewName);
                       }
                   }
```

```
                    List theProperties = theTargetInfo.
getContentProperties();
                    if( theProperties != null && theProperties.
size() > 0 )
                    {
                        for( ContentProperty theProperty :
theProperties)
                        {
                            log(1, "### Property Name :
" + theProperty.getName() );
                            log(1, "### Property Type :
" + theProperty.getType() );
                            log(1, "### Property Value :
" + theProperty.getValue() );
                        }
                    }
                }
            }
        }
```

Chapter 8

Enterprise Integration services

8.1 ERP Integration Service

The ERP Integration Service is a SOAP-based API used for Documentum Content Services for SAP (CS SAP) Agent functions. The service functions are used to execute Documentum Content Services for SAP actions and SAP queries using a predefined interface. The service is a multi-layered service library, built on the Foundation Java API, JAX-WS, and EI Core Library and the Documentum Content Services for SAP Java Agent.

The ERP Integration class is a JAX-WS and Foundation Java API-based web service and operations provide SAP integration functions, such as:

- Executing preconfigured actions used to establish links and replicate data between SAP and OpenText Documentum CM.
- Executing external (SAP) queries and returning results.

8.1.1 Prerequisites

ERP Integration Service is compatible with D6.x Stack products (Documentum CM Server, Repository, and Foundation Java API) and SAP R/3, SAP 4.6C, or later, versions. Archive Services for SAP (AS SAP) needs to be installed for Archivelink scenarios.

8.1.2 Installation

This section describes installation procedure.

8.1.2.1 Preparing the repository

Ensure the Enterprise_Integration_Services and HVPS dar are installed before using the Documentum Content Services for SAP ERP Integration Service.

8.1.2.2 Server-side deployment

ERP integration is packaged with Process Services for SAP as:

- An EAR file (erp-integration.ear) deployable to the J2EE container.
- A WAR file (erp-integration.war) deployable to Tomcat server.

Application archive contains all dependency libraries, including Foundation Java API; there is no need to install Foundation Java API on the application server.

The deployment procedure details depend on the server and are described in the corresponding vendor's documentation.

For example, one way of deploying applications on the JBOSS server is to copy an EAR file to the autodeploy folder {JBOSS_HOME}\server\default\deploy of the default deployment folder. Others involve using JBOSS scripting tools. The vendor's documentation provides more details.

8.1.2.2.1 Deployment on Tomcat

Follow these steps to deploy Tomcat:

1. Ensure Apache Tomcat is configured to deploy the `erp-integration.war` file in unpacked mode. This is the Apache Tomcat default setting.
2. Copy erp-integration.war to <TOMCAT-HOME>\webapps.
3. Change the context path of the application by adding the following configuration under the Host tag of the `server.xml`:

```
    <Context path="/erp" docBase="erp-integration" debug="0" privileged="true">
</Context>
```

4. Restart Tomcat.

After deployment, the ERP Services WSDL is available at the following URL:

```
http://[host]:[port]/erp/erpintegration?wsdl
```

where:

- *host* is the address associated with the service host.
- *port* is the port number associated with the service at installation.

In addition to the EAR/WAR file, an SAP JCO library needs to be installed on the application server. To install SAP JCO:

- Download distribution from SAP: service.sap.com/swdc. The OpenText Documentum CM ERP Services is compatible with SAP JCO version 2.1.3. If you want to use a different version, the EAR file needs to be re-packaged by replacing `sapjco.jar` file in the APP_INF directory.
- Follow SAP JCO installation instructions found in the archive.

8.1.2.3 Building remote clients for CS SAP services

Documentum Content Services for SAP Services distribution does not contain any client. Anyone who uses services must create the client side proxy and client application. To create client side proxies for using wsdl, one of the following can be used:

- JAX-WS tools, as described in your JAX_WS documentation.
- ABAP development workbench proxy creation, as described in ABAP documentation.
- NWDI tools, Adaptive web service model as mentioned in SAP Webdynpro application documentation.

8.1.2.4 Configuration

Configuration parameters are maintained in multiple configuration files, packaged inside service EAR/WAR archives. To change configuration, unzip the archive in a separate folder (preserving folder hierarchy), edit configuration file, package WAR/EAR and deploy it.

8.1.2.4.1 dfc.properties

Edit this file to specify a host/port of the docbroker and other Foundation Java API settings.

8.1.2.4.2 log4j2.properties

Used to configure the location and logging levels. To diagnose ERP services issues, set the following loggers at DEBUG levels:

- com.documentum.ei
- com.emc.documentum.erpservice

8.1.2.4.3 security-handler-chain

Specifies the authentication method. See "Authentication" on page 362 section for details.

8.1.2.4.4 Usernametoken-security.xml

Configuration of the WSS UsernameToken authentication profile. The corresponding WSS documentation provides details.

8.1.2.4.5 Authentication

Authentication method is specified in the security-handler chain configuration file. Only UsernameToken is currently supported.

8.1.2.4.6 WSS_UsernameToken

A SOAP message-based authentication, when user credentials are sent in the special SOAP header. The WSS specification provides details. If this method is selected, profile-specific parameters must be specified in the usernametoken-security.xml.

8.1.3 Description of services

The related Javadoc provides a detailed description of service methods and its parameters.

8.1.3.1 executeAction

This function executes a preconfigured action. The action can belong to one of five Documentum Content Services for SAP action types:

- Link Documentum
- Link SAP
- Replicate Documentum
- Replicate SAP
- Check DIR Link

Action configuration is stored in a repository object. Such objects can be created using Documentum Content Services for SAP WebAdmin and used by other products from Documentum Content Services for SAP suite.

8.1.3.1.1 Java syntax

```
public List<String> executeAction(String repositoryId
                                                          String serverConfig,
                                                        String userConfig,
                                                        String actionName,
                                                        int commitFrequency)
        throws Exception
```

8.1.3.1.2 Parameters

The following table provides the description of each of the parameters:

Note: In the current release, commitFrequency value is ignored by the server. The user can pass any value but only 0 is considered internally.

Table 8-1: executeAction parameters

Parameter	Data type	Description
repositoryId	String	A string that contains a name of the repository.
serverConfig	String	A String that contains a name of the configuration object specifying SAP server parameters.
userConfig	String	A String that contains a name of the configuration object specifying user login info.
actionName	String	A String that contains a name of the action configuration object.
commitFrequency	int	A number that controls the granularity of a transaction. Possible values: • 0 – "One big commit" after all records are processed • N (>0) – Commit after each N records

8.1.3.1.3 Response

Returns an array of strings; each string contains a result code for a document from the list.

8.1.3.2 executeERPQuery

This function executes a preconfigured external query, for example, SAP, type identified by queryTypeName. SAP Query type configuration is stored in a repository object which can be created using Documentum Content Services for SAP WebAdmin. Configuration usually includes the name of the SAP function (BAPI) and a description of its parameters. Values of query parameters should be supplied as PropertySet object.

8.1.3.2.1 Java syntax

```
public ResultTable executeERPQuery(String repositoryId
                                                                    serverConfig,
                                                                    userConfig,
                                                                   queryTypeName,
                                                                    List<String>
parameterMapping)
        throws Exception
```

8.1.3.2.2 Parameters

The following table provides the description of each of the parameters:

Table 8-2: executeERPQuery parameters

Parameter	Data type	Description
repositoryId	String	A String that contains a name of the repository.
serverConfig	String	A String that contains a name of the configuration object specifying SAP server parameters.
userConfig	String	A String that contains a name of the configuration object specifying user login info.
queryTypeName	String	A String that contains a name of the config object (one of the types: sap_query_type_plm, sap_query_plm_type_table).
parameterMapping	List	Substitute values for query parameters.

8.1.3.2.3 Response

Returns a Resulttable of ResultTableRow objects, each of which contains a List of String values from the corresponding query results.

8.1.4 Application-level service examples

The ERP Integration service includes high-level business integration functions such as Create Link, Execute Query, and Replicate Data to and from OpenText Documentum CM. Each function generally performs an atomic or transactional operation on a pair of objects, or sequence of such operations.

These service functions are built on top of the Foundation Java API, are accessible through a SOAP protocol, and are published as WSDL descriptors. To provide functionality consistent with the Documentum Content Services for SAP Agent, including all customizations, the ERP Integration service calls the Documentum Content Services for SAP Agent framework when performing basic functions such as link, replicate, and validate.

8.1.4.1 Link document to SAP (Link Documentum)

This function executes a preconfigured Link Documentum action. For more information about Link Documentum, see *(EDCCOSAPCS250400-AGD)*.

8.1.4.1.1 Usage examples

Usage examples include Messaging Service Link and ArchiveLink.

8.1.4.1.2 Messaging Service link

Used to create a Messaging Service link between a folder containing supporting documentation in the OpenText Documentum CM repository and SAP objects such as a master record of material or fixed assets stored in R/3 system, enabling the opening of the supporting documents in the SAP GUI.

8.1.4.1.3 ArchiveLink

Used to create an ArchiveLink relation between a report, such as an archived financial document in the OpenText Documentum CM repository, and the SAP R/3 record, enabling search and retrieval of the report in SAP GUI.

8.1.4.1.4 Input parameters

The input parameters are:

- SAP Server connection parameters (sap_server_config or sap_user_config).
- Name of the action configuration object.
- Identity of the document being linked. If not specified (null), documents are selected as results of the DQL query, as defined by the action configuration.

8.1.4.1.5 Results

Returns a list of strings identifying the objects created. For example:

- Messaging Service link – List of DIR keys
- ArchiveLink – List of ARC_DOC_IDs
- Incoming document – List of workflow IDs

8.1.4.2 Link SAP object to Documentum Query (Link SAP)

This function executes a preconfigured Link SAP action. For more information about Link SAP, see *(EDCCOSAPCS250400-AGD)*.

8.1.4.2.1 Usage example

Used to create a Messaging Service relation between the SAP object and the parameterized DQL query to establish a dynamic link between objects in SAP R/3 and documents in a OpenText Documentum CM repository. SAP GUI is used to browse object master records and select the linked documents, populating the query parameters using attributes of the SAP object. A OpenText Documentum CM repository search is performed returning a list of relevant documents.

8.1.4.2.2 Input parameters

The input parameters are:

- SAP Server connection parameters (sap_server_config or sap_user_config).
- Name of the action configuration object.
- Name of the preconfigured SAP Query object which returns a list of SAP object identities linked to the Documentum query. If not specified, SAP query is used as configured by the action.

8.1.4.2.3 Results

Returns a list of DIR object keys.

8.1.4.3 Download SAP data to OpenText Documentum CM attributes (Replicate SAP)

This function executes a preconfigured Replicate SAP action. For more information about Replicate SAP, see *(EDCCOSAPCS250400-AGD)*.

8.1.4.3.1 Usage example

Used to schedule a periodic Replicate OpenText Documentum CM job required to keep SAP R/3 master record folders up-to-date with SAP R/3 master record values. These folders store documentation for SAP materials, one per material, in a OpenText Documentum CM repository.

Note: Set the object_name to the name of the SAP object.

8.1.4.3.2 Input parameters

The input parameters are:

- SAP Server connection parameters (sap_server_config or sap_user_config).
- Name of the action configuration object.
- Identity of the document (replication target). If not specified (null), documents are selected as results of the DQL query, as defined by the action configuration.
- Name of the preconfigured SAP Query object which returns a list of SAP object identities linked to the OpenText Documentum CM query. If not specified, SAP query is used as configured by the action.

8.1.4.3.3 Results

Returns a list of DIR object keys.

8.1.4.4 Update SAP DIR status according to document attributes (Replicate OpenText Documentum CM)

This function executes a preconfigured Replicate OpenText Documentum CM action. For more information about Replicate OpenText Documentum CM, see *(EDCCOSAPCS250400-AGD)*.

8.1.4.4.1 Usage example

Used to create a Messaging Service link between a OpenText Documentum CM repository documentation object, such as documentation for a piece of equipment in R/3, making the document accessible through SAP GUI. When the document is being revised, the DIR status is set to not accessible. When revisions are complete and new version checked in, the DIR status is set to available.

8.1.4.4.2 Input parameters

The input parameters are:

- SAP Server connection parameters (sap_server_config or sap_user_config).
- Name of the action configuration object.
- Identity of the document (replication target). If not specified (null), documents are selected as a result of the DQL query, as defined by the action configuration.

8.1.4.4.3 Results

Returns a result code. For example:

- OK
- FAIL

8.1.4.5 Verify document/SAP Object Link (Verify Links)

This function executes a preconfigured Verify Links action. For more information about Verify Links, see *(EDCCOSAPCS250400-AGD)*.

8.1.4.5.1 Usage example

Used to create a Verify Link job that corrects Messaging Service link problems, by either creating missing records or removing orphaned ones, when pieces of the Messaging Service link (sap_link_relation and DIR) are damaged and inconsistent.

8.1.4.5.2 Input parameters

The input parameters are:

- SAP Server connection parameters (sap_server_config or sap_user_config).
- Name of the action configuration object.
- Identity of the document (replication target). If not specified (null), documents are selected as results of the DQL query, as defined by the action configuration.

8.1.4.5.3 Results

Returns a list of result codes, one for each link verified. For example:

- OK
- REPAIRED
- CREATED

8.1.4.6 Execute SAP query

This function executes:

- A preconfigured SAP query by specifying the name of the sap_query object
- A preconfigured SAP query type by specifying the name of the sap_query_type_plm or sap_query_plm_type_table object and mapping rules for parameters

8.1.4.6.1 Usage examples

Usage includes PLM Query and PLM Table Query.

8.1.4.6.2 PLM query

A client application needs to retrieve a set of attributes from one or more SAP R/3 objects.

8.1.4.6.3 PLM table query

A client application retrieves a set of rows from a SAP R/3 table.

8.1.4.6.4 Input parameters

Input parameters are included for SAP query and SAP query type.

8.1.4.6.5 SAP query

Input parameters are:

- SAP Server connection parameters (sap_server_config or sap_user_config)
- Name of the SAP query configuration object
- Maximum number of rows to be returned

8.1.4.6.6 SAP query type

Input parameters are:

- SAP Server connection parameters (sap_server_config or sap_user_config)
- Name of the SAP query type configuration object
- A set of mapping rules for input parameters
- Maximum number of rows to be returned

8.1.4.6.7 Results

Returns a query resultset with attributes configured by the SAP query type object.

8.1.4.7 Get list of related objects

This function returns a list of identities of SAP objects that are related to a given document using a Messaging Service or ArchiveLink relation.

8.1.4.7.1 Usage examples

A client application verifies whether the document is already linked to objects stored in a SAP R/3 system.

8.1.4.7.2 Input parameters

The input parameters are:

- SAP Server connection parameters (sap_server_config or sap_user_config)
- Identity of the document

8.1.4.7.3 Results

Returns a resultset of identities of SAP objects related to the document.

8.1.5 Sample applications

8.1.5.1 Java Webdynpro sample application

A sample ERP Integration Service Java Webdynpro application is available by extracting the `ERPServices_JAVA_EP_SAMPLE.zip` file.

1. Extract `ERPServices_JAVA_EP_SAMPLE.zip`.
2. Create a new workspace in NetWeaver Developer Studio(NWDS) “sample”.
3. On the system, navigate to `C:\sample.jdi\` and replace the LocalDevelopment folder from the `ERPServices_JAVA_EP_SAMPLE.zip` file.

   Note: While copying the LocalDevelopment folder, make sure that your workspace is not in use.

4. Open SAP NetWeaver developer studio.
5. Navigate to **File > Import > General** and select **Existing Projects into Workspace**.
6. In the resultant window select the root directory `C:\ sample.jdi\ LocalDevelopment`.
7. Click **Finish**.

Create a Service Destination for ERP Services as described:

1. Log in to NetWeaver administrator using http://nwserver_ip:port /nwa and navigate to *SOA Management>Destination Management* template.
2. Click **create destination**:
   - Destination Type: WSDL
   - Destination Name: ERP_Destination
   - URL: http://<host>:<port>/erp/erpintegration?wsdl (ERP Services WSDL)
   - System: Java
   - Authentication: Message Authentication
   - User Id/password (Basic)
3. In NWDS, navigate to *Web Dynpro>Applications*.
4. Right-click ERPDemoApplication and click **Deploy new archive and run**.
5. Default browser opens and shows the first screen of the application.
6. Proceed with execution of actions by providing inputs in the screen.

8.1.5.2 ABAP Webdynpro sample application

A sample ERP Integration Service ABAP Webdynpro program is available by extracting the `ERPServices_ABAP_EP_SAMPLE.zip` file.

Follow the instructions to create a ABAP application:

1. Open ABAP workbench using transaction se80. Create a package such as ZERP_PACKAGE.
2. Right-click ZERP_PACKAGE and click **Proxy Object**.
3. Select URL/HTTP destination on WSDL source popup.
4. Type package as *ZERP_PACKAGE* and prefix as ZERP for web service proxy to be generated.
5. After creating the web service proxy, click **save**.

   Now you should see ZERPCO_ERPINTEGRATION under Client Proxies. Select **ZERPCO_ERPINTEGRATION** and click **activate**. This generates all the required proxy classes for ERP service.
6. Create a Logical Port LP_PORT:
   - Transaction code to create logical port is LPCONFIG.
   - Select proxy class for ERP Services as “ZERPCO_ERPINTEGRATION” and type a logical port such as LP_PORT.
   - Click *yes* to select LP_PORT as default port.
   - In the next screen fill Description as ERP demo port.
   - On the Global setting tab: select *web service infrastructure* under runtime tab and under error tab select *No Log* and *No trace*.

- Click **save**.

7. On ABAP workbench, right-click *ZERP_PAKAGE* and select a program to create ABAP program.
8. Create program ZERPPROG and save.
9. Copy and paste the contents from ERPServices_ABAP_SAMPLE.txt and change additional header part in the program for usernameToken and then save the program:.

```
<wsse:Username>username*</wsse:Username>
<wsse:Password Type="http://docs.oasis-open.org/
wss/2004/01/oasis-200401-wss-username-token-profile-1.0#
PasswordText">password*</wsse:Password>
```

   where:

   username* is the username for usernameToken and

   password* is the password for usernameToken

10. Activate and execute the ABAP program.
11. Proceed with execution of actions by providing the inputs in the execution screen.

8.1.6 Known problems and limitations

This section lists the known problems and limitations.

8.1.6.1 No DfRegistryWin32 on java.library.path

As a workaround, add the following setting in `dfc.properties` file packaged in the application archive:

```
Dfc.registry.mode = File
```

Chapter 9

Compliance management services

9.1 Policy service

The Policy Service provides the operations for retrieving information about policies, applying a policy to an object and removing the policy from an object.

The Policy Service enables the management of objects using policies such as Retention Policy, Containment Policy, Security Policy and Naming Policy. These policies can be applied to any type of object (dm_cabinet, dm_folder or dm_document for example) including physical objects (dmc_prm_physical_document or dmc_prm_physical_folder for example) in the repository. After a policy is applied to an object, the object is then considered a record.

Records are determined by the business need of the customer. Customers decide which policies to associate to a document to make it a record. For more information, see *(EDCRM250400-UGD)*.

The policy types supported are:

- Retention
- Containment
- Security
- Security Level
- Restrictive Markings
- Shared Markings
- Attribute Markings
- Naming

9.1.1 Prerequisites and dependencies

The Policy Service has dependencies on the business logic in the OpenText Documentum Content Management (CM) Records Manager and Retention Policy Services BOF modules. Therefore, the OpenText Documentum CM Records Manager and Retention Policy Services docapps, or Documentum Archive (DAR) files, are required to be deployed to the repository prior to using this service. As well, the records or retention policies will have to be created first using the Records Manager Administrator application for this operation to execute successfully. A global repository is mandatory to support privileged Foundation Java API since Records Manager and OpenText Documentum Content Management (CM) Retention Policy Services access privileged code. The Foundation Java API client must be registered

using Documentum Administrator on all the Records Manager and Retention Policy Services repositories only (not the global repository). For more information, see *(EDCRM250400-IGD)*.

Note: If the customer requires just the Retention Policy, only the Retention Policy Services docapp or Documentum Archive (DAR) file needs to be deployed to the repository.

9.1.2 Objects related to this service

9.1.2.1 ObjectStatusFilter

A ObjectStatusFilter enum is used in PolicyFilter to specify the policy status to be included in a query operation.

- ENABLED, returns a list of enabled policies
- DISABLED, returns a list of disabled policies
- ALL, returns a list of both enabled and disabled policies

9.1.2.2 ObjectInheritanceFilter

A ObjectInheritanceFilter enum is used in AppliedPolicyFilter to filter the policies based on the policy application.

- DIRECT, returns a list of policies that are directly applied to the object.
- INHERITED, returns a list policies that are inherited by an object from its parent
- ALL, returns a list of policies that are both directly applied and inherited

9.1.2.3 PolicyFilter

PolicyFilter is used to specify what policies the getPolicies operation will return. It has three properties:

- Policy type filter: an enum to indicate whether the filter will be based on a specific policy type or any policy type. The default value is set to PolicyTypeFilter.ANY which indicates that all policy types will be included in the filter.
- Policy type: an enum representing the types of policy to filter. The default value is NULL which indicates that PolicyType is ignored. However, PolicyType must be specified if the PolicyTypeFilter is set to PolicyTypeFilter.SPECIFIC.
- Policy status: indicates the policy status to filter. The default value is ObjectStatusFilter.ENABLED.

  If ObjectStatusFilter is set to ENABLED, only enabled policies are included. If ObjectStatusFilter is set to DISABLED, only disabled policies are included. And, all policies are included regardless of its status if ObjectStatusFilter is set to ALL.

9.1.2.4 PolicyTypeFilter

PolicyTypeFilter enum is used in conjunction with PolicyType. The values are:

- SPECIFIED: Only the policy type that is specified in the PolicyType attribute is included.
- ANY: All policy types are included. The PolicyType attribute is ignored.

9.1.2.5 PolicyType

PolicyType enum is used to specify the policy type that can be applied to an object using the Policy Service. The policy types supported are:

- Retention
- Containment
- Security
- Security Level
- Restrictive Markings
- Shared Markings
- Attribute Markings
- Naming

9.1.2.6 AppliedPolicyFilter

AppliedPolicyFilter is used to specify policies the getAppliedPolicies operation will return. It has three properties:

- Policy type filter: an enum to indicate whether the filter will be based on a specific policy type or any policy type. The default value is set to PolicyTypeFilter.ANY which indicates that all policy types will be included in the filter.
- Policy type: an enum representing the types of policy to filter. The default value is NULL which indicates that PolicyType is ignored. However, PolicyType must be specified if the PolicyTypeFilter is set to PolicyTypeFilter.SPECIFIC.
- Policy strategy: indicates the policy strategy filter. The default value is ObjectInheritanceFilter.DIRECT.

  If ObjectInheritanceFilter is set to DIRECT, only policies that were directly applied to the object are included. If ObjectInheritanceFilter is set to INHERITED, only policies that are inherited by an object from its parent are included. And all policies are included (both direct and inherited) if ObjectInheritanceFilter is set to ALL.

9.1.2.7 PolicyProcessInfo

PolicyProcessInfo controls specific behaviors of the apply policy operation. This is applicable to all policy types.

PolicyProcessInfo has one property:

- Policy identity: The identity of the policy that will be applied to an object.

9.1.2.8 ApplyRetentionPolicyProcessInfo

ApplyRetentionPolicyProcessInfo controls specific behaviors of the apply policy operation in particular for retention policies. It has two properties:

- Policy identity: The identity of the policy that will be applied to an object.
- Chronological start date: A Date representing the start date that will be used for calculating the qualification date for the first phase of a retention policy that uses the chronological aging method. It overrides the default base-date mapping configured in the system.

Note: The value for this field is ignored if the first phase has a condition applied (event based aging).

9.1.3 getPolicies operation

The getPolicies operation is used to get a list of record and/or retention policies that are available (enabled or disabled or all policies) in the repository. By default, it returns all enabled policies in the repository. The specific return results of the getPolicies operation are controlled by the PolicyFilter.

A user who is a member of the Record Manager role (dmc_rm_recordsmanager, by default is also a member of Retention Manager role, dmc_rps_retentionmanager) or a member of Privilege User role (dmc_rm_privilegeduser, by default is also a member of the Power User role, dmc_rps_poweruser) can perform this operation.

However, for the Retention policy type, it is sufficient for a user to be a member of either Retention Manager role or Power User role.

The getPolicies operation returns an ObjectIdentitySet that you can use to apply a policy to an object.

9.1.3.1 Java syntax

```
public ObjectIdentitySet getPolicies (String repositoryName,
PolicyFilter policyFilter) throws PolicyServiceException
```

9.1.3.2 Parameters

Parameter	Data type	Description
repositoryName	RepositoryName	The name of the repository to query.
policyFilter	PolicyFilter	The object that defines which policies will be returned as results. If this object is NULL, all enabled policies in the repository will be returned.

9.1.3.3 Response

Returns an ObjectIdentitySet uniquely identifying the instances of policies that were returned from the query based on the PolicyFilter. Refer to the PolicyFilter for details.

9.1.3.4 Exceptions

PolicyServiceException is thrown in various situations such as when invalid policy filter is provided (unknown or missing policy type information for example) or if an error is encountered during query execution.

9.1.3.5 Example

Example 9-1: Java: Getting policy information

```
public List <ObjectIdentity> getPolicyObjects () throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPolicyService policyService = serviceFactory.getRemoteService(
      IPolicyService.class, serviceContext, "policy",
      "http://localhost:8888/services");

      // create policy filter
      PolicyFilter policyFilter =
      new PolicyFilter(PolicyTypeFilter.SPECIFIED, PolicyType.RETENTION_POLICY,
      ObjectStatusFilter.ENABLED);
      return policyService.getPolicies(MY_REPOSITORY, policyFilter).getIdentities();

    }
```

Example 9-2: C#: Getting policy information

```
public List<ObjectIdentity> GetPolicyObjects()
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.Instance;
      IPolicyService policyService = serviceFactory.GetRemoteService<IPolicyService>(
      serviceContext, "policy", "http://localhost:8888/services");
      PolicyFilter policyFilter = new PolicyFilter(PolicyTypeFilter.SPECIFIED,
      PolicyType.RETENTION_POLICY, ObjectStatusFilter.ENABLED);
      return policyService.GetPolicies(myRepository, policyFilter).Identities;

    }
```

9.1.4 getAppliedPolicies operation

The getAppliedPolicies operation is used to get a list of record and/or retention policies (enabled or disabled or all) that have been applied to a particular object in the repository.

A user who is a member of the Record Manager Role (by default is also a member of Retention Manager Role) or a member of Privilege User Role (by default is also a member of the Power User Role) can perform this operation.

However, for the Retention policy type, it is sufficient for a user to be a member of either Retention Manager Role or Power User Role or Compliance Officer Role or Vital Records Administrator Role.

The getAppliedPolicies operation returns an ObjectIdentitySet.

9.1.4.1 Java syntax

```
public ObjectIdentitySet getAppliedPolicies (ObjectIdentity objectIdentity,
AppliedPolicyFilter policyFilter) throws PolicyServiceException
```

9.1.4.2 Parameters

Parameter	Data type	Description
objectIdentity	ObjectIdentity	Identifies the object instance from which the policies have been applied.
policyFilter	AppliedPolicyFilter	The object that defines which policies will be returned as results. If this object is NULL, all enabled policies that were directly applied to the object will be returned.

9.1.4.3 Response

Returns an ObjectIdentitySet uniquely identifying the instances of applied policies that were returned from the query based on the AppliedPolicyFilter. Refer to the AppliedPolicyFilter for details.

9.1.4.4 Exceptions

PolicyServiceException is thrown in various situations such as when an invalid policy filter is provided (invalid or missing policy type information for example) or if error encountered during query execution or if the object instance to query does not exist.

9.1.4.5 Example

Example 9-3: Java: Getting applied policy information

```
public List<ObjectIdentity> getObjectAppliedPolicies (ObjectIdentity
objectIdentity) throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPolicyService policyService = serviceFactory.getRemoteService(
      IPolicyService.class, serviceContext, "policy",
      "http://localhost:8888/services");

      // create policy filter
      AppliedPolicyFilter appliedPolicyFilter = new AppliedPolicyFilter(
      PolicyTypeFilter.SPECIFIED, PolicyType.RETENTION_POLICY,
      ObjectInheritanceFilter.DIRECT);
      return policyService.getAppliedPolicies(objectIdentity,
      appliedPolicyFilter).getIdentities();
    }
```

Example 9-4: C#: Getting applied policy information

```
public List<ObjectIdentity> GetObjectAppliedPolicies(ObjectIdentity
objectIdentity)
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.Instance;
      IPolicyService policyService = serviceFactory.GetRemoteService
      <IPolicyService>(
      serviceContext, "policy", "http://localhost:8888/services");
      AppliedPolicyFilter appliedPolicyFilter = new AppliedPolicyFilter
      (PolicyTypeFilter.SPECIFIED, PolicyType.RETENTION_POLICY,
      ObjectInheritanceFilter.ENABLED);
      return policyService.GetAppliedPolicies(objectIdentity,
      appliedPolicyFilter).Identities;

    }
```

9.1.5 apply operation

The Policy Service apply operation is used to apply a retention or record policy to an object in the repository which can be instances of any types (dm_cabinet, dm_folder or dm_document for example) including physical object types (dmc_prm_physical_document or dmc_prm_physical_folder for example). If the object contains child objects then the policy will be propagated to them.

A user who is a member of the Record Manager Role (by default is also a member of Retention Manager Role) or a member of Privilege User Role (by default is also a member of the Power User Role) can perform this operation.

However, for the Retention policy type, it is sufficient for a user to be a member of either Retention Manager Role or Power User Role.

9.1.5.1 Java syntax

```
public void apply (ObjectIdentity targetObjectIdentity, PolicyProcessInfo
policyProcessInfo) throws PolicyServiceException
```

9.1.5.2 Parameters

Parameter	Data type	Description
objectIdentity	ObjectIdentity	The identity of the object instance in which the policy will be applied to.
policyProcessInfo	PolicyProcessInfo	The object containing the specific policy processing information that controls the behavior of the apply operation. Use PolicyProcessInfo for default apply operation (that is, when no policy specific parameter is required except for the policy identity in the apply operation) or use emc.documentum.fs. datamodel.records.policy. ApplyRetentionPolicyProcess Info for retention policy if required.

9.1.5.3 Exceptions

PolicyServiceException is thrown in various situations such as when an attempt to apply a disabled policy or if policy does not exist or if error is encountered during policy propagation to children.

9.1.5.4 Example

Example 9-5: Java: Applying policy to object

```
public void applyPolicy (ObjectIdentity targetObjectIdentity,
ObjectIdentity policyIdentity) throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPolicyService policyService = serviceFactory.getRemoteService(
      IPolicyService.class, serviceContext, "policy",
      "http://localhost:8888/services");

      // create policy process info
      PolicyProcessInfo policyProcessInfo = new PolicyProcessInfo();
      policyProcessInfo.setPolicyIdentity(policyIdentity);
      policyService.apply(targetObjectIdentity, policyProcessInfo);
    }
```

Example 9-6: C#: Applying policy to object

```
public void ApplyPolicy(ObjectIdentity targetObjectIdentity,
ObjectIdentity policyIdentity)
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.Instance;
      IPolicyService policyService = serviceFactory.GetRemoteService
      <IPolicyService>( serviceContext, "policy",
      "http://localhost:8888/services");
      PolicyProcessInfo theProcessInfo = new PolicyProcessInfo();
      theProcessInfo.PolicyIdentity = policyIdentities[0];
      policyService.Apply( targetObjectIdentity, theProcessInfo);
    }
```

9.1.6 remove operation

The removePolicy operation is used to remove a record or retention policy from an object which can be an instance of any type (dm_cabinet, dm_folder or dm_document for example) including physical object types (dmc_prm_physical_document or dmc_prm_physical_folder for example). If its child objects inherited this policy, it will be removed as well. Policies inherited by child objects can only be removed when the policy is removed from the parent object.

A user who is a member of the Records Manager Role (by default is also a member of Retention Manager Role) can perform this operation.

However, for the Retention policy type, it is sufficient for a user to be a member of Retention Manager Role.

9.1.6.1 Java syntax

```
public void remove (ObjectIdentity targetObjectIdentity, ObjectIdentity
policyIdentity) throws PolicyServiceException
```

9.1.6.2 Parameters

Parameter	Data type	Description
targetObjectIdentity	ObjectIdentity	The identity of the object instance in which the policy will be removed from.
policyIdentity	ObjectIdentity	Contains the identity of the policy instance to be removed from the target object.

9.1.6.3 Exceptions

PolicyServiceException is thrown in various situations such as when an attempt to remove policy from an object that does not exist or if policy does not exist or if error is encountered during propagation of policy changes to children.

9.1.6.4 Example

Example 9-7: Java: Removing policy from object

```
public void removePolicy (ObjectIdentity targetObjectIdentity,
ObjectIdentity policyIdentity) throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPolicyService policyService = serviceFactory.getRemoteService(
      IPolicyService.class, serviceContext, "policy",
      "http://localhost:8888/services");

      // remove policy from target object
      policyService.remove(targetObjectIdentity, policyIdentity);
    }
```

Example 9-8: C#: Removing policy from object

```
public void RemovePolicy(ObjectIdentity targetObjectIdentity,
ObjectIdentity policyIdentity)
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.Instance;
      IPolicyService policyService = serviceFactory.GetRemoteService
```

```
      <IPolicyService>( serviceContext, "policy",
      "http://localhost:8888/services");
      policyService.Remove( targetObjectIdentity, policyIdentity);
    }
```

9.2 Formal Record Service

The Formal Record Service provides functionality to create formal records by declaring one or more source objects as a formal record to a file plan location of choice. The user can declare a single formal record to group the objects when multiple objects are selected. A user can create a regular formal record or a Department of Defence (DoD) compliant record, that can be classified or non-classified according to the level of security ranking needed. All formal records declared are associated to a form template used to capture the metadata. The user declaring the formal record selects the appropriate form template, on which to enter the metadata, according to the object type being declared. Four form templates available out-of-the-box are included for declaring formal records, one for regular formal records and three for DoD compliant formal records:

- Regular Formal Record
  - Formal Record (dmc_rm_formal_record)
- DoD Compliant Formal Records
  - Record DoD 5015 Chapter 2 Record (rm_dod5015ch2record)
  - Email Record DoD 5015 Chapter 2 (rm_dod5015ch2email)
  - Record DoD 5015 Ch4 (rm_dod5015ch4record)

The *Record DoD 5015 Ch4* form template defines *No Markings* as the option used to declassify a document. The lowest security level selected to classify a document can be replaced with the *No Markings* option to declassify the document and make it nonclassified.

You must manually update the chapter 4 form template, for systems which do not use *No Markings*.

Currently, the Formal Record Service only supports source objects that have been archived/imported into an OpenText Documentum CM repository. Source objects from external repositories cannot be handled at this time.

A formal record is a snapshot taken at a particular point in time of one or more documents to capture the information, content and metadata. A new formal record or snapshot needs to be taken, if necessary, to capture any subsequent changes to the source document. The same source document(s) can be associated to as many formal records as needed to capture any ongoing updates. For example, the snapshot of formal record A points to version 1 of the source document, the snapshot of formal record B points to version 2 of the same source document, the snapshot of formal

record C points to version 3 of the same source document, and so on. The source document(s) can be unlinked (only if the user has Unlink privileges) to remove the source document(s) from its original location in a folder, after it has been declared a formal record.

To declare a formal record, a user must perform the following steps in the order indicated:

1. The formal record object instance is created based on the form template defined for a certain formal record object type.

   All formal records are associated to a particular form template type as listed previously.

2. Populate the mandatory and optional attributes of the formal record object using the Foundation SOAP API Object Service.

   Note: It is the responsibility of the client application to ensure that the validation rules are satisfied for the mandatory and optional attributes associated with the formal record object before proceeding to declare a formal record.

3. Declare a formal record. The Formal Record Service creates a snapshot for the record from one or more of the source document(s) and links it to a file plan (the location to which the record will be declared).

   Note: No validation will be performed in this operation.

   Validation of the formal record attributes should be done in step 2.

9.2.1 Prerequisites and dependencies

The Formal Record Service depends on the business logic in the Records Manager BOF modules. In addition, the four out-of-the-box form template types that can be used to declare a formal record must be deployed. Therefore, the Records Manager DAR (Documentum Archive), Records Manager Forms Adaptor DAR, Forms DAR, Records Manager Default DAR, and optional DoD Chapter 2 and Chapter 4 DAR files are required to be deployed to the repository prior to using this service. A policy managed file plan must also be available before this operation can return any results. This can be set up by applying a policy to a file plan folder using the Policy Service's apply operation or from the Records Manager Administrator (RMA) application. A global repository is mandatory to support privileged DFC since Records Manager needs to access privileged code. The Foundation SOAP API client will need to be registered using Documentum Administrator on all the Records Manager repositories only (not including the global repository). For more information, see *(EDCRM250400-UGD)*.

Note: Records Manager docapps or DAR files have dependencies on the Retention Policy Services docapp or DAR. This implies that Retention Policy Services docapp or DAR has to be applied prior to Records Manager docapps or DAR files.

9.2.2 Objects related to this service

- CreateFormalRecordProcessInfo

  Specifies the data required for creating a formal record.

- DeclareFormalRecordProcessInfo

  Specifies the data required for declaring a formal record.

9.2.3 getValidFormalRecordTypes operation

This operation is used to get a list of valid formal record object types that can be declared in the specified file plan location in the repository.

A user who is a member of Record Manager Role or a member of Privilege User Role or a member of Records Contributor Role.

9.2.3.1 Java syntax

```
public List<String> getValidFormalRecordTypes (ObjectIdentity folderIdentity)
throws FormalRecordServiceException
```

9.2.3.2 Parameters

Parameter	Data type	Description
folderIdentity	ObjectIdentity	Identity of the file plan folder instance.

9.2.3.3 Response

A list of valid formal record object types supported by the fileplan.

9.2.3.4 Exceptions

FormalRecordServiceException: This exception is thrown if an exception occurs while attempting to get list of valid formal record types.

9.2.4 getFormalRecordTemplates operation

The Formal Record Service getFormalRecordTemplates operation is used to get a list of ObjectIdentitiy instances of the formal record templates associated to a particular object type (which can be dmc_rm_formal_record or its subtypes).

A user who is a member of the Records Manager role, Privilege User role, or the Records Contributor role can perform this operation. The user must be in the Form User (form_user) role to create formal records.

9.2.4.1 Java syntax

```
public ObjectIdentitySet getFormalRecordTemplates (String objectType,
String repositoryName) throws FormalRecordServiceException
```

9.2.4.2 Parameters

Parameter	Data type	Description
repositoryName	String	The name of the repository to query.
objectType	String	The formal record object type. It must be a subtype of dmc_rm_formal_record.

9.2.4.3 Response

The ObjectIdentitySet returned represents a collection of formal record template identities.

9.2.4.4 Exceptions

FormalRecordServiceException is thrown in various situations such as when an error occurs while attempting to get list of valid formal record types or when a user is not in the required role membership.

9.2.4.5 Example

Example 9-9: Java: Getting formal record templates

```
public List<ObjectIdentity> getFormalRecordTemplatesInFilePlan()
throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IFormalRecordService formalRecordService = serviceFactory.
      getRemoteService(
      IFormalRecordService.class, serviceContext, "formalrecord",
      "http://localhost:8888/services");

      // get list of formal record templates specific to a fileplan
      ObjectPath folderObjectPath = new ObjectPath("/Temp/FormalFileplan");
      ObjectIdentity<ObjectPath> folderPathIdentity =
      new ObjectIdentity<ObjectPath>(
      folderObjectPath, MY_REPOSITORY);
      List<String> formalRecordTypes = formalRecordService.
      getValidFormalRecordTypes(folderPathIdentity);
      List<ObjectIdentity> templateIdentities =
      new ArrayList<ObjectIdentity>();
      if (formalRecordTypes != null && !formalRecordTypes.isEmpty())
```

```
        {
            for (String type : formalRecordTypes)
            {
                ObjectIdentitySet templateIdentitySet =
                formalRecordService.getFormalRecordTemplates(type, MY_REPOSITORY);
                templateIdentities.addAll(templateIdentitySet.getIdentities());
            }
        }
      return templateIdentities;
    }
```

Example 9-10: C#: Getting formal record templates

```
public List<ObjectIdentity> getFormalRecordTemplatesInFilePlan ()
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IFormalRecordService formalRecordService = serviceFactory.
      GetRemoteService<IFormalRecordService>(
      serviceContext, "formalrecord", "http://localhost:8888/services");

      ObjectIdentity fileplanIdentity = new ObjectIdentity();
      fileplanIdentity.RepositoryName = MY_REPOSITORY;
      fileplanIdentity.ValueType = ObjectIdentityType.OBJECT_PATH;
      fileplanIdentity.valueTypeSpecified = true;
      fileplanIdentity.Value = new ObjectPath("/Temp/FormalFileplan");
      List<string> formalRecordTypes =
      formalRecordService.GetValidFormalRecordTypes(fileplanIdentity);

      List<ObjectIdentity> templateIdentities = new List<ObjectIdentity>();
      if (formalRecordTypes != null && formalRecordTypes.Count >0)
         {
            foreach (String type in formalRecordTypes)
             {
                ObjectIdentitySet templateIdentitySet = formalRecordService.
                GetFormalRecordTemplates(type, MY_REPOSITORY);
                templateIdentities.AddRange(templateIdentitySet.Identities);
             }

         }

      return templateIdentities;

    }
```

9.2.5 createFormalRecord operation

The Formal Record Service createFormalRecord operation is used to create an instance of the formal record object type with attributes as defined in the form template. The formal record must be populated with the required attributes by a client application, after which it can be declared as a formal record (using declareFormalRecord operation). Also note that the file plan specified must have at least one policy applied (retention policy, containment policy, security policy or naming policy for example). Depending on the policies defined on the file plan (in particular containment or security policies), it may prevent declaration of a particular formal record type in the file plan location. If the system default (dmc_rm_docbaseconfig for example) is configured to indicate that retention is mandatory on a file plan folder to declare a formal record, then declaring a formal record to a managed folder without retention will be prevented. If the folder setting for mandatory retention is disabled then declaring a formal record to a managed folder with or without retention will be allowed. The client application *must* populate (using the Foundation SOAP API Object Service) and validate the values of the formal record attributes to ensure the mandatory and optional data is fulfilled before it can be declared as a formal record.

A user who is a member of the Records Manager role, Privilege User role, or the Records Contributor role can perform this operation. The user must be in the Form User (form_user) role to create formal records.

9.2.5.1 Java syntax

```
public DataObject createFormalRecord (CreateFormalRecordProcessInfo
formalRecordProcessInfo,
OperationOptions operationOptions)
throws FormalRecordServiceException
```

9.2.5.2 Parameters

Parameter	Data type	Description
formalRecordProcessInfo	CreateFormalRecordProcessInfo	The formal record info object specifies the data required for creating a formal record.
operationOptions	OperationOptions	The object that can contain the following profiles: PropertyProfile, Permission Profile, RelationshipProfile and ContentProfile to control the composition of the DataObject returned. By default, only non-system properties will be returned as part of the DataObject.

9.2.5.3 Response

The DataObject returned is an instance representing a newly created formal record with none of its attributes populated (other than the object name). The client can control the complexity of DataObject returned using Profile settings passed in OperationOptions.

9.2.5.4 Exceptions

FormalRecordServiceException is thrown in various situations such as when the form template does not exist or if the formal record object type cannot be declared in the file plan location due to restriction defined by policies or or if user is not in the required role membership.

9.2.5.5 Example

Example 9-11: Java: Creating formal record operation

```
public DataObject createFormalRecordObject() throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IFormalRecordService formalRecordService = serviceFactory.
      getRemoteService (IFormalRecordService.class, serviceContext,
      "formalrecord", "http://localhost:8888/services");

      // get list of formal record templates for Chapter 2 Formal Record Type
      String chapter2FormalRecordType = "rm_dod5015ch2record";
      ObjectIdentitySet templateIdentitySet = formalRecordService.
      getFormalRecordTemplates(chapter2FormalRecordType, MY_REPOSITORY);
      List<ObjectIdentity> templateIdentities = templateIdentitySet.
      getIdentities();

      CreateFormalRecordProcessInfo formalRecordInfo =
      new CreateFormalRecordProcessInfo ();
      formalRecordInfo.setFormalRecordName("FormalRecord-1");
      formalRecordInfo.setFilePlanPath("/Temp/FormalFileplan");
      formalRecordInfo.setTemplateIdentity(templateIdentities.get(0));
      return (formalRecordService.createFormalRecord(formalRecordInfo, null));

    }
```

Example 9-12: C#: Creating formal record operation

```
public DataObject CreateFormalRecordObject()
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
```

```
        serviceContext.AddIdentity(repository);

        ServiceFactory serviceFactory = ServiceFactory.getInstance();
        IFormalRecordService formalRecordService = serviceFactory.
        GetRemoteService<IFormalRecordService>(
        serviceContext, "formalrecord", "http://localhost:8888/services");

        // get list of formal record templates for Chapter 2 Formal Record Type
        String chapter2FormalRecordType = "rm_dod5015ch2record";
        ObjectIdentitySet templateIdentitySet =
        formalRecordService.GetFormalRecordTemplates(chapter2FormalRecordType,
        MY_REPOSITORY);
        List<ObjectIdentity> templateIdentities = templateIdentitySet.Identities;

        CreateFormalRecordProcessInfo formalRecordInfo =
        new CreateFormalRecordProcessInfo ();
        formalRecordInfo.formalRecordName = "FormalRecord-1";
        formalRecordInfo.filePlanPath = "/Temp/FormalFileplan";
        formalRecordInfo.TemplateIdentity = templateIdentities[0];

        return (formalRecordService.CreateFormalRecord(formalRecordInfo, null));

    }
```

9.2.6 declareFormalRecord operation

The Formal Record Service declareFormalRecord operation is used to declare one or more source objects as a formal record, with a certain object type (associated to a formal record template), in a certain file plan location. Source objects must exist in the OpenText Documentum CM repository. Only the *CURRENT* version of the source objects is supported. The source object has to be of the type, *dm_sysobject* or its subtype and it cannot be a snapshot (this means you are not permitted to select a formal record and then declare it as a formal record).

9.2.6.1 Java syntax

```
public void declareFormalRecord (ObjectIdentity formalRecordIdentity,
DeclareFormalRecordProcessInfo formalRecordProcessInfo)
throws FormalRecordServiceException
```

9.2.6.2 Parameters

Parameter	Data type	Description
formalRecordIdentity	ObjectIdentity	Identifies the newly created formal record object with its mandatory and optional attributes fulfilled.
formalRecordProcessInfo	DeclareFormalRecordProcess Info	The formal record info object specifies the data required for declaring a formal record.

9.2.6.3 Exceptions

FormalRecordServiceException is thrown in various situations such as when the source object is not a CURRENT version or if an error is encountered while propagating policies to its children (that is, the source documents) or if the source document is a snapshot (another formal record for example) or the formal record object type cannot be declared in the file plan location due to restriction defined by policies or if the folder setting for mandatory retention is enabled and user declares a formal record to a managed folder without retention or if user is not in the required role membership.

9.2.6.4 Example

Example 9-13: Java: Declaring formal record

```
public void declareFormalRecord (ObjectIdentity formalRecordIdentity,
ObjectIdentitySet sourceDocuments)throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IFormalRecordService formalRecordService = serviceFactory.
      getRemoteService(
      IFormalRecordService.class, serviceContext, "formalrecord",
      "http://localhost:8888/services");
      IObjectService objectService = serviceFactory.getRemoteService(
      IObjectService.class, serviceContext, "core",
      "http://localhost:8888/services");

      // Fetch formal record object - the formal record identity
      comes from data object returned by the createFormalRecord operation.
      PropertyProfile propertyProfile = new PropertyProfile();
      propertyProfile.setFilterMode(PropertyFilterMode.ALL);
      OperationOptions operationOptions = new OperationOptions();
      operationOptions.setPropertyProfile(propertyProfile);

      DataPackage dataPackage =
      objectService.get(new ObjectIdentitySet(formalRecordIdentity),
      operationOptions);
      DataObject formalRecordObject = dataPackage.getDataObjects().get(0);

      if (formalRecordObject != null)
        {
            // populate formal record object with the appropriate data
            PropertySet fRecordProperties = formalRecordObject.getProperties();
            fRecordProperties.set("subject", "My FormalRecord Test");
            fRecordProperties.set("originating_org", "My Org");
            fRecordProperties.set("media_type", "My Media Type");
            fRecordProperties.set("application_format", "My Format");
            fRecordProperties.set("publication_date", new Date());
            fRecordProperties.set("received_date", new Date());
            String[] authors = {"author1", "author2"};
            StringArrayProperty authorList = new StringArrayProperty("authors",
            authors);
            fRecordProperties.set(authorList);

            // update the formal record object in the repository
            DataPackage updatedDataPackage =
```

```
            objectService.update(new DataPackage(formalRecordObject),
            operationOptions);
            formalRecordObject = updatedDataPackage.getDataObjects().get(0);

            // setup parameters required by the declareFormalRecord operation
            DeclareFormalRecordProcessInfo declareFormalRecordInfo =
            new DeclareFormalRecordProcessInfo();

            // set source documents required for declare formal record operation
            declareFormalRecordInfo.setSourceObjectIdentities(sourceDocuments);
            declareFormalRecordInfo.setFilePlanPath("/Temp/FormalFileplan");
            declareFormalRecordInfo.setUnlinkSource(true);

            formalRecordService.declareFormalRecord(formalRecordObject.getIdentity(),
            declareFormalRecordInfo);

         }
    }
```

Example 9-14: C#: Declaring formal record

```
public void declareFormalRecord(ObjectIdentity formalRecordIdentity,
ObjectIdentitySet sourceDocuments)
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IFormalRecordService formalRecordService = serviceFactory.
      GetRemoteService<IFormalRecordService>(
      serviceContext, "formalrecord", "http://localhost:8888/services");
      IObjectService objectService = serviceFactory.
      GetRemoteService<IObjectService>(
      serviceContext, "core", "http://localhost:8888/services");

      // fetch formal record object - the formal record identity comes
      from data object returned by the createFormalRecord operation.
      PropertyProfile propertyProfile = new PropertyProfile();
      propertyProfile.FilterMode =PropertyFilterMode.ALL;
      propertyProfile.filterModeSpecified = true;
      OperationOptions operationOptions = new OperationOptions();
      theOptions.PropertyProfile = propertyProfile;

      ObjectIdentitySet theSet = new ObjectIdentitySet(formalRecordIdentity);
      DataPackage theData = objectService.Get( theSet, operationOptions );
      DataObject formalRecordObject = theData.DataObjects[0];

      if (formalRecordObject != null)
        {
            // populate formal record with required attributes
            PropertySet fRecordProperties = new PropertySet();
            fRecordProperties.Set<string>("subject", "My FormalRecord Test");
            fRecordProperties.Set<string>("originating_org", "My Org");
            fRecordProperties.Set<string>("media_type", "My Media Type");
            fRecordProperties.Set<DateTime>("publication_date", DateTime.Now);
            fRecordProperties.Set<DateTime>("received_date", DateTime.Now);
            fRecordProperties.Set<string>("authors", "author1");
            formalRecordObject.Properties = fRecordProperties;

            // update formal record object in the repository
            DataPackage updatedDataPackage = objectService.Update(new
```

```
            DataPackage(formalRecordObject), operationOptions);
            formalRecordObject = updatedDataPackage.DataObjects[0];

            // set up declare formal record parameters
            DeclareFormalRecordProcessInfo formalRecordInfo =
            new DeclareFormalRecordProcessInfo();
            formalRecordInfo.SourceObjectIdentities= sourceDocuments;
            formalRecordInfo.filePlanPath = "/Temp/FormalFileplan";
            formalRecordInfo.isUnlinkSource = true;

            formalRecordService.DeclareFormalRecord(formalRecordObject.
            Identity, formalRecordInfo);
        }

    }
```

9.3 Retention Markup Service

The Retention Markup Service provides the ability to apply one or more retention markups to objects that are retained in a repository. The object retained can be a document or a folder. Specific markups prevent specific actions when applied to a retained object:

- promotion of the object to the next phase in a retention lifecycle is prevented when the retention markup for **Freeze** is applied
- disposition (destruction or final transfer of objects) is prevented when the retention markup for **Hold** or **Permanent** is applied
- privileged delete is prevented when the retention markup for **Hold** or **Permanent** is applied
- removal of the retention policy is prevented when the retention markup for **Hold** or **Permanent** is applied

Retention markups can only be applied to objects that are under retention! There are four retention markups designated as follows:

- **Hold**, prevents disposition, removal of the policy, and privileged delete

  Prevents disposition such that the destruction or transfer of the object being retained in the final phase is stopped. A hold however does not affect aging of the object as the qualification and promotion processes continue normally until the object enters final phase where the hold then prevents disposition. A hold prevents removing the retainer from the object (document or folder) and also prevents a privileged delete. This retention markup can be used to hold the object according to a court order for investigative purposes such that it is not destroyed or transferred until the investigation is over and all holds placed against the object are removed.

- **Freeze**, prevents promotion

  Stops the promotion of an object from one phase to the next phase. It prevents the retainers applied to the object from qualifying for promotion to the next phase in the retention lifecycle. It does not however prevent disposition of an

object that is already in the final phase, removing the retainer, or performing a privileged delete.

- **Review**, triggers review notifications

  Causes a review notification to be sent out to reviewers after a certain date and time is reached. It does not prevent promotion or disposition. This retention markup does not prevent the object from reaching final phase for disposition where it can be destroyed. Use a hold or permanent markup in combination with this retention markup if you need to prevent the object from undergoing disposition.

- **Permanent**, prevents disposition

  Prevents disposition such that the destruction or transfer of the object being retained in the final phase is stopped. Applying this retention markup to an object under retention indicates that the object shall be retained indefinitely and will not undergo disposition until this retention markup is removed. It is very much similar to a hold and both behave the same. The permanent markup can only be removed by a Retention manager whereas a hold can be removed by a Retention manager and a Compliance officer.

None of the date calculations are affected when an object has a hold retention markup and the object ages as it would normally. A freeze does not affect previous aging but since it prevents promotion it stops successive aging - this is unique to the Freeze retention markup. Although multiple retention markups can be placed on an object, all of the retention markups, with designation of hold or permanent, must be removed for the disposition process to occur. You must be in the role of either a Retention Manager or a Compliance Officer to administer retention markups. Either role has the permissions to view, apply, and remove retention markups. However, only Retention Managers are allowed to apply and remove retention markups with the Permanent designation. Retention markups are applied at either the document or folder level. If a retention markup is applied at the folder level, all the documents within the folder have the retention markup applied to them. Only child documents in the folder are affected. Sub-folders are not affected in any way and do not inherit the retention markup when it is applied to their parent folder.

Note: In order for Centera to support retention holds, the administrator has to ensure that Centera is configured, according to Centera documentation, to accept retention holds. The Centera cluster has to be configured to grant retention hold privilege. Retention hold on Centera is usually configured by the Centera Administrator. A Centera Hold clip is created if you set the designation to either a Hold or Permanent markup.

9.3.1 Prerequisites and dependencies

The Retention Markup Service has dependencies on the business logic in the Retention Policy Services BOF modules. Therefore, the Retention Policy Services docapps are required to be deployed to the repository prior to using this service. As well, the retention markups must be created first using the Retention Policy Services Administrator GUI for this operation to execute successfully, that is to return any result. A global repository is mandatory to support privileged DFC since Retention Policy Services accesses privileged code. The Foundation Java API client must be registered using Documentum Administrator on all the Retention Policy Services repositories only (not including the global repository).

9.3.2 Objects related to this service

The following objects are related to this service:

- RetentionMarkup

  This is a representation of the Retention Markup object in the repository.

- ObjectStatusFilter

  The ObjectStatusFilter enum is used in RetentionMarkupFilter to specify the retention markup status to be included in the query operation.

- ObjectInheritanceFilter

  The ObjectInheritanceFilter enum is used in AppliedRetentionMarkupFilter to filter the markups based on the markup application.

- BaseMarkupFilter

  This is the base class for all Retention Markup filters such as RetentionMarkupFilter and AppliedRetentionMarkupFilter.

- RetentionMarkupFilter

  RetentionMarkupFilter is used to specify the retention markup information to be returned from the getRetentionMarkups operation.

- AppliedRetentionMarkupFilter

  AppliedRetentionMarkupFilter is used to specify the retention markup information returned from the getAppliedRetentionMarkups operation.

Refer to the Javadocs for complete details.

9.3.3 getRetentionMarkups operation

The getRetentionMarkups operation is used to get a list of retention markups (enabled or disabled or both) from a repository. This operation by default returns a list of all enabled retention markups in the repository. Use the *RetentionMarkupFilter* to obtain results other than the default list.

This operation has the following five internal properties:

- includeHold

  A boolean representing retention markup **Hold** designation to be returned.
- includeFreeze

  A boolean representing retention markup **Freeze** designation to be returned.
- includeReview

  A boolean representing retention markup **Review** designation to be returned.
- includePermanent

  A boolean representing retention markup **Permanent** designation to be returned.
- status

  Indicates the retention markup status filter. The default value is ObjectStatusFilter.ENABLED.

The markup designation flags will be joined by an OR operator. For example, to get a list of retention markup with designation of **Hold** OR **Review**, set includeHold and includeReview to *True* and set includeFreeze and includePermanent to *False*. If none of the markup designation flags are set, the default is set to *True*. This indicates that all retention markups in the repository will be returned by the getRetentionMarkups operation.

This operation can be performed by members of the following roles:

- Retention Manager
- Compliance Officer
- Vital Record Administrator (can only view markups with Review designation)

9.3.3.1 Java syntax

```
public List<RetentionMarkup> getRetentionMarkups (
String repositoryName, RetentionMarkupFilter markupFilter)
throws RetentionMarkupServiceException
```

9.3.3.2 Parameters

Parameter	Data type	Description
repositoryName	String	The name of the repository to query.
markupFilter	RetentionMarkupFilter	The RetentionMarkupFilter defines which data will be returned as results. If this parameter is set to NULL, all enabled retention markups (regardless of designations) will be returned.

9.3.3.3 Response

Returns a list of RetentionMarkup objects retrieved from the repository.

9.3.3.4 Exceptions

RetentionMarkupServiceException is thrown in various situations such as when an error occurs while querying for a list of retention markups or invalid input data or if user is not in the required role membership.

9.3.3.5 Example

Example 9-15: Java: Get retention markups information

```
public void getRetentionMarkups () throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      ServiceContext serviceContext = contextFactory.newContext();
      positoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);


      ServiceFactory serviceFactory
            = ServiceFactory.getInstance();
      IRetentionMarkupService markupService
            = serviceFactory.getRemoteService(
      IRetentionMarkupService.class, serviceContext, "retentionmarkup",
      "http://localhost:8888/services");

      RetentionMarkupFilter markupFilter = new RetentionMarkupFilter();
      List<RetentionMarkup> retentionMarkups
            = markupService.
      getRetentionMarkups(MY_REPOSITORY, markupFilter);
      if (retentionMarkups != null && !retentionMarkups.isEmpty())
```

```
        {
           for (RetentionMarkup markup : retentionMarkups)
             {
                System.out.println(
                markup.getMarkupIdentity().
                            getValueAsString() + "isEnabled: "
                + markup.isEnabled());
             }
        }
    }
```

Example 9-16: C#: Get retention markups information

```
public void GetRetentionMarkups()
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.Instance;
      IRetentionMarkupService markupService = serviceFactory.
      GetRemoteService<IRetentionMarkupService>(  serviceContext,
      "retentionmarkup", "http://localhost:8888/services");

      RetentionMarkupFilter filter = new RetentionMarkupFilter();
      List<RetentionMarkup> retentionMarkups = markupService.
      GetRetentionMarkups(MY_REPOSITORY, filter);
      if (retentionMarkups != null && retentionMarkups.Count > 0)
            {
               foreach (RetentionMarkup markupInfo in retentionMarkups)
                {
                  Console.WriteLine(markupInfo.GetMarkupIdentity().
                  GetValueAsString());
                }
            }
        }
```

9.3.4 getAppliedRetentionMarkups operation

The getAppliedRetentionMarkups operation is used to get a list of retention markups in the repository (enabled or disabled or both) that have been directly applied to a particular object. This operation by default returns a list of all enabled retention markups that have been directly applied to a particular object. Use the *AppliedRetentionMarkupFilter* to obtain results other than the default list.

This operation has the following five properties:

- includeHold

  A boolean representing retention markup **Hold** designation to be returned.

- includeFreeze

  A boolean representing retention markup **Freeze** designation to be returned.

- includeReview

  A boolean representing retention markup **Review** designation to be returned.

- includePermanent

  A boolean representing retention markup **Permanent** designation to be returned.

- markupstrategy

  Indicates the retention markup inheritance filter. The default value is ObjectInheritanceFilter.DIRECT.

  If the retention markup strategy filter is ObjectInheritanceFilter.DIRECT, only retention markups that are directly applied to the object are returned.

  If the retention markup strategy filter is ObjectInheritanceFilter.INHERITED, only retention markups that were inherited by an object from its parent are returned.

  If the retention markup strategy filter is ObjectInheritanceFilter.ALL, both direct and inherited retention markups are returned.

The markup designation flags will be joint by an OR operator. For example, to get a list of retention markup with designation of **Hold** and **Review**, set includeHold and includeReview to *True* and set includeFreeze and includePermanent to *False*. If none of the markup designation flags are set, the default is set to *True*. This indicates that all retention markups in the respository will be returned by the getAppliedRetentionMarkups operation.

This operation can be performed by members of the following roles:

- Retention Manager
- Compliance Officer
- Vital Record Administrator (can only view markups with Review designation)

9.3.4.1 Java syntax

```
public List<RetentionMarkup> getAppliedRetentionMarkups (
ObjectIdentity objectIdentity, AppliedRetentionMarkupFilter markupFilter)
throws RetentionMarkupServiceException
```

9.3.4.2 Parameters

Parameter	Data type	Description
objectIdentity	ObjectIdentity	Identifies the object instance to query. This object must have retention markup applied.

Parameter	Data type	Description
markupFilter	AppliedRetentionMarkupFilter	The AppliedRetentionMarkupsFilter defines which data will be returned as results. If this parameter is set to NULL, all enabled retention markups that were directly applied to the object will be returned.

9.3.4.3 Response

Returns a list of RetentionMarkup objects that have been applied to a particular object.

9.3.4.4 Exceptions

RetentionMarkupServiceException is thrown in various situations such as when an error occurs while querying for a list of retention markups or invalid input data or if user is not in the required role membership.

9.3.4.5 Example

Example 9-17: Java: Get applied retention markups information

```
public void getAppliedRetentionMarkups (ObjectIdentity objectIdentity)
throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IRetentionMarkupService markupService = serviceFactory.getRemoteService(
      IRetentionMarkupService.class, serviceContext, "retentionmarkup",
      "http://localhost:8888/services");

      AppliedRetentionMarkupFilter appliedMarkupFilter =
      new AppliedRetentionMarkupFilter(true, false, true, false,
      ObjectInheritanceFilter.DIRECT);
      List<RetentionMarkup> retentionMarkups =
      markupService.getAppliedRetentionMarkups(objectIdentity,
      appliedMarkupFilter);
      if (retentionMarkups != null && !retentionMarkups.isEmpty())
        {
            for (RetentionMarkup markup : retentionMarkups)
              {
                  System.out.println(
                  markup.getMarkupIdentity().getValueAsString() + "isEnabled: "
                  + markup.isEnabled());
              }
        }
    }
```

Example 9-18: C#: Get applied retention markups information

```
public void GetAppliedRetentionMarkups(ObjectIdentity
objectIdentity)
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.Instance;
      IRetentionMarkupService markupService = serviceFactory.
      GetRemoteService<IRetentionMarkupService>(  serviceContext,
      "retentionmarkup",
      "http://localhost:8888/services");

      // setup applied retention markup
      AppliedRetentionMarkupFilter filter = new AppliedRetentionMarkupFilter();
      filter.isIncludeHold = true;
      filter.isIncludeFreeze = false;
      filter.isIncludeReview = true;
      filter.isIncludePermanent = false;
      filter.MarkupStrategy = ObjectInheritanceFilter.DIRECT;
      filter.MarkupStrategySpecified = true;

      List<RetentionMarkup> retentionMarkups = markupService.
      GetAppliedRetentionMarkups(
      objectIdentity, filter);

      if (retentionMarkups != null && retentionMarkups.Count > 0)
            {
               foreach (RetentionMarkup markupInfo in retentionMarkups)
                {
                  Console.WriteLine(markupInfo.GetMarkupIdentity().
                  GetValueAsString());
                }
            }
        }
```

9.3.5 apply operation

Applies a retention markup to a retained object in the repository which can be an instance of any types (dm_cabinet, dm_folder or dm_document for example) including physical object types (dmc_prm_physical_document or dmc_prm_physical_folder for example). If the object contains child objects then the retention markup will be propagated to its children that are documents but not subfolders.

This operation can be performed by members of the following roles:

- Retention Manager

  Only Retention Managers are allowed to apply retention markups with the **Permanent** designation.
- Compliance Officer
- Vital Record Administrator

Vital Record Administrators can only apply a retention markup with the **Review** designation.

9.3.5.1 Java syntax

```
public void apply (ObjectIdentity targetObjectIdentity, ObjectIdentity
retentionMarkupIdentity) throws RetentionMarkupServiceException
```

9.3.5.2 Parameters

Parameter	Data type	Description
targetObjectIdentity	ObjectIdentity	Identifies the object in which the retention markup will be applied to.
retentionMarkupIdentity	ObjectIdentity	Contains the unique identifier of the retention markup for the apply operation.

9.3.5.3 Exceptions

RetentionMarkupServiceException is thrown in various situations such as when a user attempt to apply a retention markup that have been disabled, or a user attempt to apply a retention markup that does not exist, or an error is encountered during retention markup propagation to child documents or invalid input data is provided or if the user is not in the required role membership.

9.3.5.4 Example

Example 9-19: Java: Apply retention markup

```
public void applyRetentionMarkups (ObjectIdentity targetObjectIdentity)
throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IRetentionMarkupService markupService = serviceFactory.
      getRemoteService(
      IRetentionMarkupService.class, serviceContext,
      "retentionmarkup", "http://localhost:8888/services");

      RetentionMarkupFilter markupFilter = new RetentionMarkupFilter
      (true, false, false, false, ObjectStatusFilter.ENABLED );
      List<RetentionMarkup> markupList = markupService.
      getRetentionMarkups(MY_REPOSITORY, markupFilter);
      for (RetentionMarkup markup : markupList)
         {
            markupService.apply(targetObjectIdentity,
            markup.getMarkupIdentity());
```

```
        }
    }
```

Example 9-20: C#: Apply retention markup

```
public void ApplyRetentionMarkups(ObjectIdentity targetObjectIdentity)
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.Instance;
      IRetentionMarkupService markupService = serviceFactory.
      GetRemoteService<IRetentionMarkupService>(  serviceContext,
      "retentionmarkup", "http://localhost:8888/services");

      RetentionMarkupFilter filter = new RetentionMarkupFilter();
      filter.isIncludeHold = true;
      filter.isIncludeFreeze = false;
      filter.isIncludePermanent = false;
      filter.isIncludeReview = false;
      filter.MarkupStatus = ObjectStatusFilter.ENABLED;
      filter.MarkupStatusSpecified = true;
      List<RetentionMarkup> retentionMarkups = markupService.
      GetRetentionMarkups(MY_REPOSITORY, filter);

      if (retentionMarkups != null && retentionMarkups.Count > 0)
            {
              foreach (RetentionMarkup markupInfo in retentionMarkups)
                {
                  markupService.Apply(targetObjectIdentity.Identity,
                  markupInfo.MarkupIdentity);
                }
            }
        }
```

9.3.6 remove operation

Remove a retention markup from a retained object in the repository which can be an instance of any types (dm_cabinet, dm_folder or dm_document for example) including physical object types (dmc_prm_physical_document or dmc_prm_physical_folder for example). If the object contains child objects then the retention markup will be removed from its children that are documents but not subfolders.

This operation can be performed by members of the following roles:

- Retention Manager

  Only Retention Managers are allowed to remove retention markups with the **Permanent** designation.

- Compliance Officer

- Vital Record Administrator

  Vital Record Administrators can only remove a retention markup with the **Review** designation.

9.3.6.1 Java syntax

```
public void remove (ObjectIdentity targetObjectIdentity, ObjectIdentity
retentionMarkupIdentity) throws RetentionMarkupServiceException
```

9.3.6.2 Parameters

Parameter	Data type	Description
targetObjectIdentity	ObjectIdentity	Identifies the object in which the retention markup will be removed from.
retentionMarkupIdentity	ObjectIdentity	Contains the unique identifier of the retention markup for the remove operation.

9.3.6.3 Exceptions

RetentionMarkupServiceException is thrown in various situations such as when a user attempt to remove a retention markup that does not exist, or an error is encountered during retention markup removal from its child documents or invalid input data is provided or if the user is not in the required role membership.

9.3.6.4 Example

Example 9-21: Java: Removing retention markups from an object

```
public void removeRetentionMarkups (ObjectIdentity targetObjectIdentity)
throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IRetentionMarkupService markupService = serviceFactory.getRemoteService(
      IRetentionMarkupService.class, serviceContext, "retentionmarkup",
      "http://localhost:8888/services");

      // Remove all retention markups that were directly applied to
      object - use default filter
      AppliedRetentionMarkupFilter appliedMarkupFilter =
      new AppliedRetentionMarkupFilter();
      List<RetentionMarkup> markupList = markupService.
      getAppliedRetentionMarkups(
      targetObjectIdentity, appliedMarkupFilter);
```

```
      for (RetentionMarkup markup : markupList)
         {
            markupService.remove(targetObjectIdentity, markup.
            getMarkupIdentity());
         }
    }
```

Example 9-22: C#: Removing retention markups from an object

```
public void RemoveRetentionMarkups(ObjectIdentity targetObjectIdentity)
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.Instance;
      IRetentionMarkupService markupService = serviceFactory.GetRemoteService
      <IRetentionMarkupService>( serviceContext, "retentionmarkup",
      "http://localhost:8888/services");

      // Remove all retention markups that were directly applied to an
      object - use default filter
      AppliedRetentionMarkupFilter filter = new AppliedRetentionMarkupFilter();

      List<RetentionMarkup> retentionMarkups = markupService.
      GetAppliedRetentionMarkups(
      targetObjectIdentity, filter);

      if (retentionMarkups != null && retentionMarkups.Count > 0)
            {
              foreach (RetentionMarkup markupInfo in retentionMarkups)
                {
                  markupService.Remove(targetObjectIdentity, markupInfo.
                  MarkupIdentity);
                }
            }
     }
```

9.3.7 getRetentionMarkupProperties operation

This operation is used to get a list of properties used in the query. It returns the properties used in the getRetentionMarkupsQuery operation. A list of retention markup properties is returned.

9.3.7.1 Java syntax

```
public List<String> getRetentionMarkupProperties ()
```

9.3.7.2 Example

Note: This example gets a list of retention markups using Query Service with getRetentionMarkupProperties method and getRetentionMarkupsQuery method.

Example 9-23: Java: Get list of retention markups

```
public void getRetentionMarkupsUsingObjectService() throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IRetentionMarkupService markupService = serviceFactory.getRemoteService(
      IRetentionMarkupService.class, serviceContext, "retentionmarkup",
      "http://localhost:8888/services");
      IQueryService queryService = serviceFactory.getRemoteService (IQueryService.
      class, serviceContext, “core”, "http://localhost:8888/services");

      // use default value of markup filter
      RetentionMarkupFilter markupFilter = new RetentionMarkupFilter();
      PassthroughQuery query = new PassthroughQuery(
      Arrays.asList(MY_REPOSITORY), markupService.
      getRetentionMarkupsQuery(markupFilter));

      // setup query cache strategy
      QueryExecution queryExec = new QueryExecution();
      queryExec.setCacheStrategyType(CacheStrategyType.
      BASIC_MEMORY_CACHE_STRATEGY);
      queryExec.setMaxResultCount(MAX_QUERY_RESULT_COUNT);

      // setup property profile to include specific markup properties
      PropertyProfile propertyProfile = new PropertyProfile(PropertyFilterMode.
      SPECIFIED_BY_INCLUDE);
      propertyProfile.setIncludeProperties(markupService.
      getRetentionMarkupProperties());
      OperationOptions operationOptions = new OperationOptions();
      operationOptions.setPropertyProfile(propertyProfile);

      List<DataObject> markupList = new ArrayList<DataObject>>();
      while (true)
        {
          QueryResult result = queryService.execute(query, queryExec,
          operationOptions);
          List<DataObject> resultDataObjects = result.getDataObjects();

          if (resultDataObjects.isEmpty())
            {
              break;
            }

          markupList.addAll(resultDataObjects);

          queryExec.setStartingIndex(queryExec.getStartingIndex() +
```

```
          MAX_QUERY_RESULT_COUNT);
        }

      System.out.println("List retention markups in the repository: ");
      System.out.println("---------------------------------------- ");
      int count = 0;
      for (DataObject markup : markupList)
        {
           count++;
           System.out.println("Markup# " + count + " : " + markup.toString());
        }
    }
```

9.3.7.3 Example

Note: This example gets a list of retention markups using Query Service with getRetentionMarkupProperties method and getRetentionMarkupsQuery method.

Example 9-24: C#: Get list of retention markups

```
public void GetRetentionMarkupsUsingObjectService()
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.Instance;
      IRetentionMarkupService markupService = serviceFactory.
      GetRemoteService<IRetentionMarkupService>( serviceContext,
      "retentionmarkup", "http://localhost:8888/services");
      IQueryService queryService = serviceFactory.GetRemoteService
      <IQueryService>(serviceContext, "core",
      "http://localhost:8888/services");

      int pageSize = 25;
      RetentionMarkupFilter markupFilter = new RetentionMarkupFilter();
      string theQueryString = markupService.GetRetentionMarkupsQuery
      (markupFilter);
      PassthroughQuery thePassthroughQuery = new PassthroughQuery();
      thePassthroughQuery.AddRepository(MY_REPOSITORY);
      thePassthroughQuery.QueryString = theQueryString;

      OperationOptions theOptions = new OperationOptions();
      PropertyProfile thePropertyProfile = new PropertyProfile();
      thePropertyProfile.FilterMode = PropertyFilterMode.
      SPECIFIED_BY_INCLUDE;
      thePropertyProfile.IncludeProperties = markupService.
      GetRetentionMarkupProperties(myRepository);
      theOptions.PropertyProfile = thePropertyProfile;

      QueryExecution theQueryExecution = new QueryExecution();
      theQueryExecution.MaxResultCount = pageSize;
      theQueryExecution.StartingIndex = 0;
      theQueryExecution.CacheStrategyType = CacheStrategyType.
      BASIC_MEMORY_CACHE_STRATEGY;

      List<DataObject>theQueryResult = new List<DataObject>();
      while (true)
```

```
        {
            QueryResult theResult = queryService.Execute(thePassthroughQuery,
            theQueryExecution, theOptions);
            if (theResult != null)
             {
                List<DataObject> theObjects = theResult.DataObjects;
                if (theObjects != null && theObjects.Count > 0)
                  {
                    if (theQueryExecution.QueryId == null)
                    theQueryExecution.QueryId = theResult.QueryId;

                    theQueryResult.AddRange(theObjects);
                    if (theObjects.Count == pageSize)
                     {
                        theQueryExecution.StartingIndex = theQueryExecution.
                        StartingIndex
                        + theResult.DataObjects.Count;
                        continue;
                     }
                   }
               }
            break;
        }

      int count = 0;
      foreach( DataObject data in theQueryResult)
        {
           count++;
           Console.Write("Markup# {0}: ", count);
           Console.WriteLine(data.Properties.GetValueAsString());
        }

    }
```

9.3.8 getRetentionMarkupsQuery operation

This operation returns a query string to get a specific set of retention markups in the repository. The intent of providing this operation is to enable the client/end-user to setup and perform their query execution directly using Query Service.

The RetentionMarkupFilter is used to define which data will be returned as results. If this object is NULL, all enabled retention markups in the repository will be returned.

9.3.8.1 Java syntax

```
public String getRetentionMarkupsQuery (RetentionMarkupFilter markupFilter)
```

9.3.8.2 Parameters

Parameter	Data type	Description
markupFilter	RetentionMarkupFilter	The RetentionMarkupFilter that defines which data will be returned as results. If this object is NULL, all enabled retention markups in the repository will be returned.

9.3.8.3 Example

Note: This example gets a list of retention markups using Query Service with getRetentionMarkupProperties method and getRetentionMarkupsQuery method.

Example 9-25: Java: Get list of retention markups

```
public void getRetentionMarkupsUsingObjectService() throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IRetentionMarkupService markupService = serviceFactory.getRemoteService(
      IRetentionMarkupService.class, serviceContext, "retentionmarkup",
      "http://localhost:8888/services");
      IQueryService queryService = serviceFactory.getRemoteService (IQueryService.
      class, serviceContext, “core”, "http://localhost:8888/services");

      // use default value of markup filter
      RetentionMarkupFilter markupFilter = new RetentionMarkupFilter();
      PassthroughQuery query = new PassthroughQuery(
      Arrays.asList(MY_REPOSITORY), markupService.
      getRetentionMarkupsQuery(markupFilter));

      // setup query cache strategy
      QueryExecution queryExec = new QueryExecution();
      queryExec.setCacheStrategyType(CacheStrategyType.
      BASIC_MEMORY_CACHE_STRATEGY);
      queryExec.setMaxResultCount(MAX_QUERY_RESULT_COUNT);

      // setup property profile to include specific markup properties
      PropertyProfile propertyProfile = new PropertyProfile(PropertyFilterMode.
      SPECIFIED_BY_INCLUDE);
      propertyProfile.setIncludeProperties(markupService.
      getRetentionMarkupProperties());
      OperationOptions operationOptions = new OperationOptions();
      operationOptions.setPropertyProfile(propertyProfile);

      List<DataObject> markupList = new ArrayList<DataObject>>();
      while (true)
        {
          QueryResult result = queryService.execute(query, queryExec,
          operationOptions);
          List<DataObject> resultDataObjects = result.getDataObjects();
```

```
          if (resultDataObjects.isEmpty())
            {
              break;
            }

          markupList.addAll(resultDataObjects);

          queryExec.setStartingIndex(queryExec.getStartingIndex() +
          MAX_QUERY_RESULT_COUNT);
        }

      System.out.println("List retention markups in the repository: ");
      System.out.println("---------------------------------------- ");
      int count = 0;
      for (DataObject markup : markupList)
        {
           count++;
           System.out.println("Markup# " + count + " : " + markup.toString());
        }
    }
```

9.3.8.4 Example

Note: This example gets a list of retention markups using Query Service with getRetentionMarkupProperties method and getRetentionMarkupsQuery method.

Example 9-26: C#: Get list of retention markups

```
public void GetRetentionMarkupsUsingObjectService()
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.Instance;
      IRetentionMarkupService markupService = serviceFactory.
      GetRemoteService<IRetentionMarkupService>( serviceContext,
      "retentionmarkup", "http://localhost:8888/services");
      IQueryService queryService = serviceFactory.GetRemoteService
      <IQueryService>(serviceContext, "core",
      "http://localhost:8888/services");

      int pageSize = 25;
      RetentionMarkupFilter markupFilter = new RetentionMarkupFilter();
      string theQueryString = markupService.GetRetentionMarkupsQuery
      (markupFilter);
      PassthroughQuery thePassthroughQuery = new PassthroughQuery();
      thePassthroughQuery.AddRepository(MY_REPOSITORY);
      thePassthroughQuery.QueryString = theQueryString;

      OperationOptions theOptions = new OperationOptions();
      PropertyProfile thePropertyProfile = new PropertyProfile();
      thePropertyProfile.FilterMode = PropertyFilterMode.
      SPECIFIED_BY_INCLUDE;
      thePropertyProfile.IncludeProperties = markupService.
      GetRetentionMarkupProperties(myRepository);
      theOptions.PropertyProfile = thePropertyProfile;
```

```
    QueryExecution theQueryExecution = new QueryExecution();
    theQueryExecution.MaxResultCount = pageSize;
    theQueryExecution.StartingIndex = 0;
    theQueryExecution.CacheStrategyType = CacheStrategyType.
    BASIC_MEMORY_CACHE_STRATEGY;

    List<DataObject>theQueryResult = new List<DataObject>();
    while (true)
      {
          QueryResult theResult = queryService.Execute(thePassthroughQuery,
          theQueryExecution, theOptions);
          if (theResult != null)
           {
              List<DataObject> theObjects = theResult.DataObjects;
              if (theObjects != null && theObjects.Count > 0)
                {
                  if (theQueryExecution.QueryId == null)
                  theQueryExecution.QueryId = theResult.QueryId;

                  theQueryResult.AddRange(theObjects);
                  if (theObjects.Count == pageSize)
                   {
                      theQueryExecution.StartingIndex = theQueryExecution.
                      StartingIndex
                      + theResult.DataObjects.Count;
                      continue;
                   }
                 }
              }
          break;
      }

    int count = 0;
    foreach( DataObject data in theQueryResult)
      {
         count++;
         Console.Write("Markup# {0}: ", count);
         Console.WriteLine(data.Properties.GetValueAsString());
      }

  }
```

9.4 Physical Records Library Service

The Physical Records Library Service is used to generate user requests for physical objects and to look at listings of the physical objects requested. Typically, physical objects are real life objects that are represented and tracked in an electronic system. The following list includes examples of physical objects:

- warehouses
- bays
- bins
- shelves
- boxes
- folders
- non-electronic documents

The operations that are supported by this web service in 6.6 are:

- Make a library request to ask for a set of physical objects.
- Get a list of library requests associated to a specific user or to all users.
- Cancel a library request.

Physical objects can be requested and subsequently converted to a charge out by a Physical Records Administrator. Boxes and folders for example, and the documents inside a folder can be requested and listed using the Physical Records Library Service. In general, only physical objects that have been configured to be charged out can be requested. This can be configured when the physical object is initially created. Making a library request is similar to making a reservation. Although similar, a reservation implies the physical objects are available and reserved whereas, physical objects requested according to a library request may or may not be available. Making a library request is basically making a request for a resource (physical item) on a certain date. The Administrator can convert the library request to a charge-out when the physical object is available or the library request can be cancelled if the requested item is no longer required.

A user must be in the appropriate user role(s) in order to perform any of the Physical Records Library Service operations. Here is a summary of the user role and function:

- Members of the following roles can make library requests:
  - Physical Records Manager
  - Library Administrator
  - Library User
- Members of the following roles can get a list of library requests associated to a specific user (their own library request for example) or all users:
  - Physical Records Manager
  - Library Administrator
  - Library User (can only get list of own library requests)
- Members of the following roles can cancel library requests:
  - Physical Records Manager (can cancel any users request)
  - Library Administrator (can cancel any users requests)
  - Library User (can only cancel their own library request)

9.4.1 Prerequisites and dependencies

The Library Request Service depends on the business logic in the Physical Records Manager BOF modules. Therefore, the Physical Records Manager docapp in addition to the Retention Policy Services docapp are required to be deployed to the repository prior to using this service. As well, a global repository is mandatory to support privileged DFC since Physical Records Manager accesses privileged code. The Foundation Java API client must be registered from Documentum Administrator on all the Physical Records Manager repositories only (not the global repository). For more information, see *(EDCRM250400-IGD)*.

Also, the physical objects must be created in the system using the Records Manager application prior to using the Library Request Service. It is therefore assumed that the physical objects exist in the system for the library request operations to execute successfully.

9.4.2 Objects related to this service

The following objects are related to this service:

- LibraryRequestInfo

  This object is the base class that defines information required for processing a library request (make a library request for example). It is also a base class for the LibraryRequest class that represents the library request object in the repository.

- LibraryRequest

  This object represents the Library Request object in the repository.

- NotificationPreference

  This is an Enum used to specify the preferred method for sending/receiving notifications.

- ShippingPreference

  This is an Enum used to specify the preferred method for shipping items to a requestor.

- StateType

  This is an Enum used to indicate the process or lifecycle state of a library request.

- StateTypeFilter

  This is an Enum used in conjunction with StateType.

For complete details if needed, refer to the Javadocs.

9.4.3 createRequest operation

This operation is used to create/make a request for a physical object. A user can make library requests for themselves or for someone else depending on their user role.

The user must have at least READ permission on the requested object(s). As well, the requested object must be a physical object and it must be flagged as an item that can be charged-out. Users who are members of the Library User Role or the Physical Manager Role or the Library Administrator Role can make a library request.

9.4.3.1 Java syntax

```
public ObjectIdentity createRequest (LibraryRequestInfo libraryRequestInfo)
throws PhysicalRecordsLibraryServiceException
```

9.4.3.2 Parameters

Parameter	Data type	Description
libraryRequestInfo	LibraryRequestInfo	A data structure containing information about the library request process template. The client creates this structure and datafills the values in the LibraryRequestInfo object. The client then passes the LibraryRequestInfo object to the createRequest operation to make a library request.

9.4.3.3 Response

The object identity of the new library request.

9.4.3.4 Exceptions

PhysicalRecordsLibraryServiceException: This exception is thrown when a user attempts to make a library request with invalid library request information or if the user is not in the required role membership.

9.4.3.5 Example

Example 9-27: Java: Create request

```
public ObjectIdentity createLibraryRequest () throws ServiceException
    {
      LibraryRequestInfo requestInfo = new LibraryRequestInfo();
      requestInfo.setName("MyLibraryRequest");
      requestInfo.setNotificationPreference(NotificationPreference.PHONE);
      requestInfo.setPhoneNumber("(613)270-4141");
      requestInfo.setShippingPreference(ShippingPreference.PICKUP_AT_LOCATION);

      Date requestDate = new Date();
      requestInfo.setRequestedDate(requestDate);

      // set contact identity
      requestInfo.setRequestor(MY_CONTACT_IDENTITY);

      // set requested items
      ObjectIdentitySet item = new ObjectIdentitySet(MY_REQUESTED_ITEM_IDENTITY);
      requestInfo.setRequestedItems(item);

      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPhysicalRecordsLibraryService libraryService = serviceFactory.getRemoteService(
      IPhysicalRecordsLibraryService.class, serviceContext, "physicallibrary",
      "http://localhost:8888/services");

      return libraryService.createRequest(requestInfo);
    }
```

Example 9-28: C#: Create request

```
public ObjectIdentity CreateLibraryRequest()
    {
      LibraryRequestInfo requestInfo = new LibraryRequestInfo();
      requestInfo.name = ("MyLibraryRequest");
      requestInfo.NotificationPreference =NotificationPreference.
      PHONE;
      requestInfo.phoneNumber = "(613) 270-8888";
      requestInfo.NotificationPreferenceSpecified = true;
      requestInfo.ShippingPreference = ShippingPreference.
      PICKUP_AT_LOCATION;
      requestInfo.ShippingPreferenceSpecified = true;

      requestInfo.RequestedDate = new Date();
      requestInfo.RequestedDateSpecified = true;

      // set contact identity
      requestInfo.Requestor = MY_CONTACT_IDENTITY;

      // set requested item
      ObjectIdentitySet item = new ObjectIdentitySet
      (MY_REQUESTED_ITEM_IDENTITY);
      requestInfo.RequestedItems = item;

      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
```

```
    repository.RepositoryName = MY_REPOSITORY;
    repository.UserName = MY_LOGIN;
    repository.Password = MY_PASSWORD;
    IServiceContext serviceContext = contextFactory.NewContext();
    serviceContext.AddIdentity(repository);

    ServiceFactory serviceFactory = ServiceFactory.Instance;
    IPhysicalRecordsLibraryService libraryService = serviceFactory.
    GetRemoteService<IPhysicalRecordsLibraryService>( serviceContext,
    "physicallibrary", "http://localhost:8888/services");
    return libraryService.CreateRequest(requestInfo);
  }
```

9.4.4 getLibraryRequests operation

This operation is used to get a list of library requests for all users.

Users who are members of the Physical Manager Role or Library Administrator Role or the Inventory Manager Role can request a list of the library requests for all users.

9.4.4.1 Java syntax

```
public List<LibraryRequest> getLibraryRequests (
String repositoryName, StateTypeFilter stateTypeFilter, StateType stateType)
throws PhysicalRecordsLibraryServiceException
```

9.4.4.2 Parameters

Parameter	Data type	Description
repositoryName	String	The name of the repository to query.
stateTypeFilter	StateTypeFilter	To filter the library requests based on its process/lifecycle state. If StateTypeFilter.SPECIFIED is used, only the policy type that is specified in the PolicyType attribute is included. The StateTypeFilter.ANY will include all state types and the StateType attribute is ignored.
stateType	StateType	Defines the library request state.

9.4.4.3 Response

The list of all library requests in the repository.

9.4.4.4 Exceptions

PhysicalRecordsLibraryServiceException: This exception is thrown when the user is unable to query the list of library requests due to an error or invalid data or if the user is not a member in the required role membership.

9.4.4.5 Example

Example 9-29: Java: Getting library request information

```
public List<LibraryRequest> getLibraryRequests () throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPhysicalRecordsLibraryService libraryService = serviceFactory.
      getRemoteService(
      IPhysicalRecordsLibraryService.class, serviceContext, "physicallibrary",
      "http://localhost:8888/services");

      // get all library requests in the repository
      return libraryService.getLibraryRequests(MY_REPOSITORY, StateTypeFilter.
      ANY, null);

    }
```

Example 9-30: C#: Getting library request information

```
public List<LibraryRequest> getLibraryRequests()
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.Instance;
      IPhysicalRecordsLibraryService libraryService = serviceFactory.
      GetRemoteService<IPhysicalRecordsLibraryService>( serviceContext,
      "physicallibrary", "http://localhost:8888/services");

      return libraryService.GetLibraryRequests(MY_REPOSITORY,
      StateTypeFilter.ANY, StateType.SUBMITTED);
    }
```

9.4.5 cancelRequest operation

This operation is used to cancel a library request that was created earlier. You can cancel a library request only if the library request is in submitted state. You can not cancel a library request if the library request is in processing or completed state.

Users who are members of the Library User Role or the Physical Manager Role or the Library Administrator Role can cancel a library request.

9.4.5.1 Java syntax

```
public void cancelRequest (ObjectIdentity libraryRequestIdentity)
throws PhysicalRecordsLibraryServiceException
```

9.4.5.2 Parameters

Parameter	Data type	Description
libraryRequestIdentity	ObjectIdentity	The identity of the library request to be cancelled

9.4.5.3 Exceptions

PhysicalRecordsLibraryServiceException: This exception is thrown if the library request does not exist or if an error is encountered during the library request cancellation or if the user is not in the required role membership.

9.4.5.4 Example

Example 9-31: Java: Cancel request

```
public void cancelRequest (ObjectIdentity requestIdentity)
throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPhysicalRecordsLibraryService libraryService = serviceFactory.
      getRemoteService(
      IPhysicalRecordsLibraryService.class, serviceContext,
      "physicallibrary", "http://localhost:8888/services");

      libraryService.cancelRequest(requestIdentity);
    }
```

Example 9-32: C#: Cancel request

```
public void CancelRequests(ObjectIdentity requestIdentity)
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.Instance;
      IPhysicalRecordsLibraryService libraryService = serviceFactory.
      GetRemoteService<IPhysicalRecordsLibraryService>( serviceContext,
      "physicallibrary", "http://localhost:8888/services");

      libraryService.CancelRequest(requestIdentity);
    }
```

9.4.6 getUserLibraryRequests

This operation is used to get a list of library request associated to a user. There is a restriction where library users can only access their own library requests or those library requests made on behalf of someone else. A list of library requests associated to a particular user is returned.

Users who are a member of the Library User Role, the Physical Manager Role or Library Administrator Role can perform this operation. A Library User can only get their list of library requests or the library request made on behalf of someone else.

The Physical Records Library Service depends on the business logic in the Physical Records Manager BOF modules. Therefore, the Physical Records Manager docapp in addition to the Retention Policy Services docapp are required to be deployed to the repository prior to using this service.

9.4.6.1 Java syntax

```
public List<LibraryRequest> getUserLibraryRequests (
ObjectIdentity requestorIdentity, StateTypeFilter stateTypeFilter,
StateType stateType) throws PhysicalRecordsLibraryServiceException
```

9.4.6.2 Parameters

Parameter	Data type	Description
requestorIdentity	ObjectIdentity	The identity of the requestor of the library request.
stateTypeFilter	StateTypeFilter	To filter the library requests based on its process/lifecycle state.
stateType	StateType	Defines the library request state.

9.4.6.3 Response

A list of Library Request information for a particular requestor is returned.

9.4.6.4 Exceptions

PhysicalRecordsLibraryServiceException: This exception is thrown if the operation is unable to query the list of library requests due to error or invalid data or if the user is not in the required role membership.

9.4.7 getLibraryRequestProperties

This operation returns the properties used in the getLibraryRequestsQuery operation or getUserLibraryRequestsQuery operation.

Any user in the repository can perform this operation.

9.4.7.1 Java syntax

```
public List<String> getLibraryRequestProperties (String repositoryName)
throws PhysicalRecordsLibraryServiceException
```

9.4.7.2 Parameters

Parameter	Data type	Description
repositoryName	String	The name of the repository to query.

9.4.7.3 Response

A list of strings where each element is an attribute of the library request object.

9.4.7.4 Exceptions

PhysicalRecordsLibraryServiceException: This exception is thrown if an error is encountered while generating a list of Library Request properties.

9.4.7.5 Example

Note: This example gets a list of library requests using Query Service with getLibraryRequestProperties method and getLibraryRequestsQuery method.

Example 9-33: Java: Get list of library requests

```
public void getLibraryRequestsUsingQueryService() throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPhysicalRecordsLibraryService libraryService = serviceFactory.
      getRemoteService(
      IPhysicalRecordsLibraryService.class, serviceContext, "physicallibrary",
      "http://localhost:8888/services");
      IQueryService queryService = serviceFactory.getRemoteService
      (IQueryService.class, serviceContext, “core”,
      "http://localhost:8888/services");

      // setup query
      PassthroughQuery query = new PassthroughQuery(
      Arrays.asList(MY_REPOSITORY), libraryService.getLibraryRequestsQuery
      (MY_REPOSITORY, StateTypeFilter.ANY, StateType.SUBMITTED));

      // setup query cache strategy
      int pageSize = 25;
      QueryExecution queryExec = new QueryExecution();
      queryExec.setCacheStrategyType(CacheStrategyType.
      BASIC_MEMORY_CACHE_STRATEGY);
      queryExec.setMaxResultCount(pageSize);

      // setup property profile
      PropertyProfile propertyProfile = new PropertyProfile
      (PropertyFilterMode.SPECIFIED_BY_INCLUDE);
      propertyProfile.setIncludeProperties(libraryService.
      getLibraryRequestProperties(MY_REPOSITORY));
      OperationOptions operationOptions = new OperationOptions();
      operationOptions.setPropertyProfile(propertyProfile);

      List<DataObject> requestList = new ArrayList<DataObject>();
      while (true)
        {
            QueryResult result = queryService.execute(query, queryExec,
            operationOptions);
            List<DataObject> resultDataObjects = result.getDataObjects();

            if (resultDataObjects.isEmpty())
```

```
              {
                 break;
              }

            requestList.addAll(resultDataObjects);

            queryExec.setStartingIndex(queryExec.getStartingIndex()
            + pageSize);
        }

      System.out.println("List library requests in the repository: ");
      System.out.println("---------------------------------------- ");
      int count = 0;
      for (DataObject request : requestList)
        {
           count++;
           System.out.println("Request# " + count + " : " + request.toString());
        }
    }
```

9.4.7.6 Example

Note: This example gets a list of library requests using Query Service with getLibraryRequestProperties method and getLibraryRequestsQuery method.

Example 9-34: C#: Get list of library requests

```
public void GetLibraryRequestsUsingQueryService()
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPhysicalRecordsLibraryService libraryService = serviceFactory.
      GetRemoteService<IPhysicalRecordsLibraryService>(
      serviceContext, "physicallibrary", "http://localhost:8888/services");
      IQueryService queryService = serviceFactory.
      GetRemoteService<IQueryService>(serviceContext, "core",
      "http://localhost:8888/services");

      int pageSize = 25;
      string theQueryString = libraryService.GetLibraryRequestsQuery
      (myRepository, StateTypeFilter.ANY,
      StateType.SUBMITTED);
      PassthroughQuery thePassthroughQuery = new PassthroughQuery();
      thePassthroughQuery.AddRepository(myRepository);
      thePassthroughQuery.QueryString = theQueryString;

      OperationOptions theOptions = new OperationOptions();
      PropertyProfile thePropertyProfile = new PropertyProfile();
      thePropertyProfile.FilterMode = PropertyFilterMode.
      SPECIFIED_BY_INCLUDE;
      thePropertyProfile.IncludeProperties = libraryService.
      GetLibraryRequestProperties(myRepository);
      theOptions.PropertyProfile = thePropertyProfile;

      QueryExecution theQueryExecution = new QueryExecution();
      theQueryExecution.MaxResultCount = pageSize;
      theQueryExecution.StartingIndex = 0;
```

```
      theQueryExecution.CacheStrategyType = CacheStrategyType.
      BASIC_MEMORY_CACHE_STRATEGY;
      List<DataObject> theQueryResult = new List<DataObject>();
      while (true)
        {
            QueryResult theResult = queryService.Execute(thePassthroughQuery,
            theQueryExecution, theOptions);
            if (theResult != null)
              {
                  List<DataObject> theObjects = theResult.DataObjects;
                  if (theObjects != null && theObjects.Count > 0)
                 {
                  if (theQueryExecution.QueryId == null)
                  theQueryExecution.QueryId = theResult.QueryId;

                  theQueryResult.AddRange(theObjects);
                  if (theObjects.Count == pageSize)
                    {
                        theQueryExecution.StartingIndex = theQueryExecution.
                        StartingIndex + theResult.DataObjects.Count;
                        continue;
                    }
                 }
              }
            break;
        }

      int count = 0;
      foreach( DataObject data in theQueryResult)
        {
           count++;
           Console.Write("Request# {0}: ", count);
           Console.WriteLine(data.Properties.GetValueAsString());

           Console.WriteLine();
        }
    }
```

9.4.8 getLibraryRequestsQuery

This operation returns a query string to get a list of all the library requests in a repository.

9.4.8.1 Java syntax

```
public String getLibraryRequestsQuery (String repositoryName,
StateTypeFilter stateTypeFilter, StateType stateType)
throws PhysicalRecordsLibraryServiceException
```

9.4.8.2 Parameters

Parameter	Data type	Description
repositoryName	String	The name of the repository to query.
stateTypeFilter	StateTypeFilter	Filters library requests based on its process/lifecycle state.
stateType	StateType	Defines the library request state.

9.4.8.3 Response

Query string.

9.4.8.4 Exceptions

PhysicalRecordsLibraryServiceException: This exception is thrown if the operation is unable to generate a query string.

9.4.8.5 Example

Note: This example gets a list of library requests using Query Service with getLibraryRequestProperties method and getLibraryRequestsQuery method.

Example 9-35: Java: Get list of library requests

```
public void getLibraryRequestsUsingQueryService() throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPhysicalRecordsLibraryService libraryService = serviceFactory.
      getRemoteService(
      IPhysicalRecordsLibraryService.class, serviceContext, "physicallibrary",
      "http://localhost:8888/services");
      IQueryService queryService = serviceFactory.getRemoteService
      (IQueryService.class, serviceContext, “core”,
      "http://localhost:8888/services");

      // setup query
      PassthroughQuery query = new PassthroughQuery(
      Arrays.asList(MY_REPOSITORY), libraryService.getLibraryRequestsQuery
      (MY_REPOSITORY, StateTypeFilter.ANY, StateType.SUBMITTED));

      // setup query cache strategy
      int pageSize = 25;
      QueryExecution queryExec = new QueryExecution();
      queryExec.setCacheStrategyType(CacheStrategyType.
      BASIC_MEMORY_CACHE_STRATEGY);
      queryExec.setMaxResultCount(pageSize);
```

```
      // setup property profile
      PropertyProfile propertyProfile = new PropertyProfile
      (PropertyFilterMode.SPECIFIED_BY_INCLUDE);
      propertyProfile.setIncludeProperties(libraryService.
      getLibraryRequestProperties(MY_REPOSITORY));
      OperationOptions operationOptions = new OperationOptions();
      operationOptions.setPropertyProfile(propertyProfile);

      List<DataObject> requestList = new ArrayList<DataObject>();
      while (true)
        {
            QueryResult result = queryService.execute(query, queryExec,
            operationOptions);
            List<DataObject> resultDataObjects = result.getDataObjects();

            if (resultDataObjects.isEmpty())
              {
                 break;
              }

            requestList.addAll(resultDataObjects);

            queryExec.setStartingIndex(queryExec.getStartingIndex()
            + pageSize);
        }

      System.out.println("List library requests in the repository: ");
      System.out.println("---------------------------------------- ");
      int count = 0;
      for (DataObject request : requestList)
        {
           count++;
           System.out.println("Request# " + count + " : " + request.toString());
        }
    }
```

9.4.8.6 Example

Note: This example gets a list of library requests using Query Service with getLibraryRequestProperties method and getLibraryRequestsQuery method.

Example 9-36: C#: Get list of library requests

```
public void GetLibraryRequestsUsingQueryService()
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPhysicalRecordsLibraryService libraryService = serviceFactory.
      GetRemoteService<IPhysicalRecordsLibraryService>(
      serviceContext, "physicallibrary", "http://localhost:8888/services");
      IQueryService queryService = serviceFactory.
      GetRemoteService<IQueryService>(serviceContext, "core",
      "http://localhost:8888/services");

      int pageSize = 25;
      string theQueryString = libraryService.GetLibraryRequestsQuery
      (myRepository, StateTypeFilter.ANY,
```

```
      StateType.SUBMITTED);
      PassthroughQuery thePassthroughQuery = new PassthroughQuery();
      thePassthroughQuery.AddRepository(myRepository);
      thePassthroughQuery.QueryString = theQueryString;

      OperationOptions theOptions = new OperationOptions();
      PropertyProfile thePropertyProfile = new PropertyProfile();
      thePropertyProfile.FilterMode = PropertyFilterMode.
      SPECIFIED_BY_INCLUDE;
      thePropertyProfile.IncludeProperties = libraryService.
      GetLibraryRequestProperties(myRepository);
      theOptions.PropertyProfile = thePropertyProfile;

      QueryExecution theQueryExecution = new QueryExecution();
      theQueryExecution.MaxResultCount = pageSize;
      theQueryExecution.StartingIndex = 0;
      theQueryExecution.CacheStrategyType = CacheStrategyType.
      BASIC_MEMORY_CACHE_STRATEGY;
      List<DataObject> theQueryResult = new List<DataObject>();
      while (true)
        {
            QueryResult theResult = queryService.Execute(thePassthroughQuery,
            theQueryExecution, theOptions);
            if (theResult != null)
              {
                  List<DataObject> theObjects = theResult.DataObjects;
                  if (theObjects != null && theObjects.Count > 0)
                {
                  if (theQueryExecution.QueryId == null)
                  theQueryExecution.QueryId = theResult.QueryId;

                  theQueryResult.AddRange(theObjects);
                  if (theObjects.Count == pageSize)
                    {
                        theQueryExecution.StartingIndex = theQueryExecution.
                        StartingIndex + theResult.DataObjects.Count;
                        continue;
                    }
                }
              }
            break;
        }

      int count = 0;
      foreach( DataObject data in theQueryResult)
        {
           count++;
           Console.Write("Request# {0}: ", count);
           Console.WriteLine(data.Properties.GetValueAsString());

           Console.WriteLine();
        }
    }
```

9.4.9 getUserLibraryRequestQuery

This operation returns a query string to get a list of all the library requests in a repository.

9.4.9.1 Java syntax

```
public String getUserLibraryRequestsQuery (ObjectIdentity requestorIdentity,
StateTypeFilter stateTypeFilter, StateType stateType)
throws PhysicalRecordsLibraryServiceException
```

9.4.9.2 Parameters

Parameter	Data type	Description
requestorIdentity	ObjectIdentity	The object identity of the requestor.
stateTypeFilter	StateTypeFilter	Filters library requests based on its process/lifecycle state.
stateType	StateType	Defines the library request state.

9.4.9.3 Response

Query string.

9.4.9.4 Exceptions

PhysicalRecordsLibraryServiceException: This exception is thrown if an error is encountered during the conversion of the object identity to IDfId.

9.4.9.5 Example

Note: This example gets user specific library requests using Query Service with getLibraryRequestProperties method and getUserLibraryRequestsQuery method.

Example 9-37: Java: Get list of user specific library requests

```
public void getUserLibraryRequestsUsingQueryService(ObjectIdentity
contactIdentity) throws ServiceException
    {
      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPhysicalRecordsLibraryService libraryService = serviceFactory.
      getRemoteService(
```

```
    IPhysicalRecordsLibraryService.class, serviceContext,
    "physicallibrary", "http://localhost:8888/services");
    IQueryService queryService = serviceFactory.getRemoteService
    (IQueryService.class, serviceContext, “core”,
    "http://localhost:8888/services");

    // setup query
    PassthroughQuery query = new PassthroughQuery(
    Arrays.asList(MY_REPOSITORY), libraryService.
    getUserLibraryRequestsQuery(contactIdentity, StateTypeFilter.ANY, null));

    // setup query cache strategy
    int pageSize = 25;
    QueryExecution queryExec = new QueryExecution();
    queryExec.setCacheStrategyType(CacheStrategyType.
    BASIC_MEMORY_CACHE_STRATEGY);
    queryExec.setMaxResultCount(pageSize);

    // setup property profile
    PropertyProfile propertyProfile = new PropertyProfile
    (PropertyFilterMode.SPECIFIED_BY_INCLUDE);
    propertyProfile.setIncludeProperties(libraryService.
    getLibraryRequestProperties(MY_REPOSITORY));
    OperationOptions operationOptions = new OperationOptions();
    operationOptions.setPropertyProfile(propertyProfile);

    List<DataObject> requestList = new ArrayList<DataObject>();
    while (true)
      {
          QueryResult result = queryService.execute(query, queryExec,
          operationOptions);
          List<DataObject> resultDataObjects = result.getDataObjects();

          if (resultDataObjects.isEmpty())
            {
              break;
            }

          requestList.addAll(resultDataObjects);

          queryExec.setStartingIndex(queryExec.getStartingIndex()
          + pageSize);
      }

    System.out.println("List library requests in the repository: ");
    System.out.println("---------------------------------------- ");
    int count = 0;
    for (DataObject request : requestList)
      {
          count++;
          System.out.println("Request# " + count + " : " + request.toString());
      }
  }
```

9.4.9.6 Example

Note: This example gets user specific library requests using Query Service with getLibraryRequestProperties method and getUserLibraryRequestsQuery method.

Example 9-38: C#: Get list of user specific library requests

```
public void GetUserLibraryRequestsUsingQueryService(ObjectIdentity
contactIdentity)
    {
      ContextFactory contextFactory = ContextFactory.Instance;
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.RepositoryName = MY_REPOSITORY;
      repository.UserName = MY_LOGIN;
      repository.Password = MY_PASSWORD;
      IServiceContext serviceContext = contextFactory.NewContext();
      serviceContext.AddIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IPhysicalRecordsLibraryService libraryService = serviceFactory.
      IPhysicalRecordsLibraryService libraryService = serviceFactory.
      GetRemoteService<IPhysicalRecordsLibraryService>(
      serviceContext, "physicallibrary", "http://localhost:8888/services");
      "http://localhost:8888/services");
      IQueryService queryService = serviceFactory.
      GetRemoteService<IQueryService>(serviceContext, "core",
      "http://localhost:8888/services");

      int pageSize = 25;
      thePropertyProfile.IncludeProperties = libraryService.
      GetUserLibraryRequestsQuery(contactIdentity, StateTypeFilter.
      ANY, StateType.SUBMITTED));
      PassthroughQuery thePassthroughQuery = new PassthroughQuery();
      thePassthroughQuery.AddRepository(myRepository);
      thePassthroughQuery.QueryString = theQueryString;

      OperationOptions theOptions = new OperationOptions();
      PropertyProfile thePropertyProfile = new PropertyProfile();
      thePropertyProfile.FilterMode = PropertyFilterMode.
      SPECIFIED_BY_INCLUDE;
      thePropertyProfile.IncludeProperties = libraryService.
      GetLibraryRequestProperties(myRepository);
      theOptions.PropertyProfile = thePropertyProfile;

      QueryExecution theQueryExecution = new QueryExecution();
      theQueryExecution.MaxResultCount = pageSize;
      theQueryExecution.StartingIndex = 0;
      theQueryExecution.CacheStrategyType = CacheStrategyType.
      BASIC_MEMORY_CACHE_STRATEGY;
      List<DataObject> theQueryResult = new List<DataObject>();
      while (true)
        {
            QueryResult theResult = queryService.Execute(thePassthroughQuery,
            theQueryExecution, theOptions);
            if (theResult != null)
             {
                  List<DataObject> theObjects = theResult.DataObjects;
                  if (theObjects != null && theObjects.Count > 0)
                {
                  if (theQueryExecution.QueryId == null)
                  theQueryExecution.QueryId = theResult.QueryId;

                  theQueryResult.AddRange(theObjects);
                  if (theObjects.Count == pageSize)
                    {
                        theQueryExecution.StartingIndex = theQueryExecution.
```

```
                        StartingIndex + theResult.DataObjects.Count;
                        continue;
                    }
                }
            }
            break;
        }

        int count = 0;
        foreach( DataObject data in theQueryResult)
          {
             count++;
             Console.Write("Request# {0}: ", count);
             Console.WriteLine(data.Properties.GetValueAsString());

             Console.WriteLine();
          }
      }
```

9.5 Federated Proxy Service

The Federated Proxy Service provides the ability for enterprises to centrally manage the retention of simple content objects held in managed repositories external to the repository. This is achieved using proxy objects.

A proxy object is a persistent object in the repository that represents the external object. The client application is responsible for creating the proxy object(s) and associating the content and metadata that describes the external object(s). A client can define their own type for their proxy object. It can contain as many attributes from the remote system as required by the client.

In 6.6 however, only the object type of dm_sysobject and its subtypes are supported. This restriction is enforced by the business logic.

After the proxy object is created in the repository, the Federated Proxy Service can be invoked to declare a proxy object. A declared proxy object implies that the proxy object is instrumented with the necessary aspect. The proxy object is subsequently registered with an external object by the proxy registration job in order to support asynchronous registration process. At this point, after a successful registration, the external object becomes a managed object that can be administered by the Retention Policy Service.

Users of this service must be a member of the VRM Master Contributor Role.

9.5.1 Prerequisites and dependencies

The Federated Proxy Service depends on the business logic in the Federated Records Service BOF modules. Therefore, the FRS DAR files are required to be deployed to the repository prior to using this service.

The FRS configuration of system objects have to be created first using the FRS Administrator client before this operation can be executed successfully.

A global repository is also mandatory to support privileged DFC since FRS accesses privileged code. The Foundation Java API client must be registered using Documentum Administrator on all the FRS repositories, not including the global repository.

9.5.2 Objects related to this service

The following object is related to this service: DeclareProxyProcessInfo

This object defines the data required to declare a proxy object. This include the external object identity and external repository identity.

Refer to the FRS Javadocs for complete details.

9.5.3 declareProxy operation

This operation is used to declare a proxy object. It means that the proxy object will be instrumented with an aspect (dmc_vrm_external_content) and is qualified to be process by the proxy registration job at a later time. The proxy registration job registers proxy objects with the corresponding external objects.

9.5.3.1 Java syntax

```
public void declareProxy (ObjectIdentity proxyIdentity,
DeclareProxyProcessInfo declareProxyInfo) throws FederatedProxyServiceException
```

9.5.3.2 Parameters

Parameter	Data type	Description
proxyIdentity	ObjectIdentity	The object instance of a persistent object in the repository that represents the external object.
declareProxyInfo	DeclareProxyProcessInfo	Provides data required for declaring a proxy object.

9.5.3.3 Exceptions

FederatedProxyServiceException: This exception is thrown if an error is encountered while declaring a proxy object.

9.5.3.4 Example

Example 9-39: Java: Declaring a proxy object

```
public void declareProxyObject () throws ServiceException
    {
      // create proxy object
      ObjectIdentity objIdentity = new ObjectIdentity(MY_REPOSITORY);
      DataObject dataObject = new DataObject(objIdentity, "dm_document");
      PropertySet properties = dataObject.getProperties();
      properties.set("object_name", "MyProxyObject");

      DataPackage dataPackage = new DataPackage(dataObject);
      dataPackage = objectService.create(dataPackage, null);
      DataObject proxyObject = dataPackage.getDataObjects().get(0);

      // setup declare proxy process information
      DeclareProxyProcessInfo declareProxyInfo =
      new DeclareProxyProcessInfo();
      declareProxyInfo.setExternalObjectIdentifier
      (EXTERNAL_OBJECT_IDENTITY);
      declareProxyInfo.setExternalRepositoryIdentifier
      (REMOTE_REPOSITORY_IDENTITY);

      // Create new Service Context
      ContextFactory contextFactory = ContextFactory.getInstance();
      IServiceContext serviceContext = contextFactory.newContext();
      RepositoryIdentity repository = new RepositoryIdentity();
      repository.setRepositoryName(MY_REPOSITORY);
      repository.setUserName(MY_LOGIN);
      repository.setPassword(MY_PASSWORD);
      serviceContext.addIdentity(repository);

      ServiceFactory serviceFactory = ServiceFactory.getInstance();
      IFederatedProxyService proxyService = serviceFactory.getRemoteService(
      IFederatedProxyService.class, serviceContext, "federatedproxy",
      "http://localhost:8888/services");

      proxyService.declareProxy(proxyObject.getIdentity(), declareProxyInfo);
    }
```

Example 9-40: C#: Declaring a proxy object

```
public void DeclareProxyObject()
    {
      ObjectIdentity objIdentity = new ObjectIdentity(MY_REPOSITORY);
      DataObject dataObject = new DataObject(objIdentity, "dm_document");
      PropertySet properties = dataObject.Properties;
      properties.Set("object_name", "MyProxyObject");
      DataPackage dataPackage = new DataPackage(dataObject);
      dataPackage = objectService.Create(dataPackage, null);
      DataObject proxyObject = dataPackage.DataObjects[0];

      DeclareProxyProcessInfo declareProxyInfo =
      new DeclareProxyProcessInfo();
      declareProxyInfo.externalObjectIdentifier = EXTERNAL_OBJECT_IDENTITY;
      declareProxyInfo.externalRepositoryIdentifier =REMOTE_REPOSITORY_IDENTITY;

      ContextFactory contextFactory = ContextFactory.Instance;
```

```
    RepositoryIdentity repository = new RepositoryIdentity();
    repository.RepositoryName = MY_REPOSITORY;
    repository.UserName = MY_LOGIN;
    repository.Password = MY_PASSWORD;
    IServiceContext serviceContext = contextFactory.NewContext();
    serviceContext.AddIdentity(repository);

    ServiceFactory serviceFactory = ServiceFactory.Instance;
    IFederatedProxyService proxyService = serviceFactory.
    GetRemoteService<IFederatedProxyService>(  serviceContext,
    "federatedproxy", "http://localhost:8888/services");

    proxyService.DeclareProxy(proxyObject.Identity,declareProxyInfo);
  }
```

9.6 Electronic Signature Service

The Electronic Signature Service, alternately the eSign Service, provides functionality for dispersing sign off notifications and for obtaining online signatures, in the form of electronic signatures, whether to approve or reject a controlled document driven within an Documentum Compliance Manager (DCM) lifecycle. Customers can apply an electronic signature to a piece of content in a way that provides an indisputable record of a user approving (signing) a piece of content. Users are able to view all signatures applied to that particular version of the content, on a signature page attached to the PDF rendition of the content, either at the beginning or at the end of the rendition. Users can verify that the rendition to which the initial signature is applied is the actual rendition (for example, content that has not changed since the rendition) of the content in whichever format the content is in. They can verify if both the content and the associated signed PDF rendition are the same from the initial signature through to the final signature and thereafter until disposition, to make sure it has not been altered in any way.

DCM helps customers achieve compliance with external regulations and internal policies while maintaining high product and service quality standards. Users can create, review, revise, approve and distribute controlled content online within an audited environment. In place of elaborate manual or email-driven processes for review, approval and distribution, DCM creates a Web-driven knowledge chain that links disconnected processes for collecting, sharing, and applying content to meet stringent quality goals and compliance requirements. DCM is applicable across many market segments where customers face controlled content challenges.

9.6.1 Prerequisites and dependencies

Electronic Signature Service requires a Dar called pssESign to be deployed to the repository prior to using this service. A global repository is mandatory to support privileged DFC because the Electronic Signature Service must access privileged code. The Foundation Java API client (dfc.jar) must be registered using Documentum Administrator on all the repositories that contain objects to which you want to apply electronic signatures when using this service.

9.6.2 Objects related to this service

The SignatureFormat enum is used in ElectronicSignatureInfo to specify the format of the rendition to which the electronic signatures will be applied. Currently, only the PDF format is supported.

- *PDF*

  DEFAULT - is same as PDF in this case.

- A *HashAlgorithm* enum is used in ElectronicSignatureInfo to specify the hash algorithm.

  Currently only SHA1 is supported.

  SHA1

  DEFAULT - is same as SHA1 in this case.

- A *ElectronicSignatureMethod* enum is used in ElectronicSignatureInfo to specify the signing method name.

  Currently only PSS_ESIGN is supported.

  PSS_ESIGN

  DEFAULT - is same as PSS_ESIGN in this case.

- *ElectronicSignatureInfo*

  ElectronicSignatureInfo is used to pass the signature information to electronic signature service for adding a new signature. The information includes:

  – object Id: an ObjectIdentity represents the object to be signed.

  – userLoginName: a string that represents the signatory login name.

  – userLoginDomain: a string that represents the signatory login domain.

  – password: a string that represents the signatory login password.

  – signatureJustification: a string that represents the reason for the signature.

  – signatureFormat: a SignatureFormat represents the signing format.

  – hashAlgorithm: a HashAlgorithm represents the hash algorithm.

  – signatureMethod: an ElectronicSignatureMethod represents the signing method.

- applicationProperties: a PropertySet represents the custom application properties. This parameter is reserved for future enhancement.
- signForGroup: a string that represents a group name for which a signatory signs.
- signAsGroup: a string that represents a group name as which signatory signs.

9.6.3 add operation

Use the *add* operation to add a new electronic signature by providing signature information. The add operation returns the object ID of the audit trail that is generated after a signature is added. The *add* method/operation performs the following actions to generate an electronic signature for a given object:

1. Authenticates the user and verifies that the user has at least Relate permission on the document to be signed.
2. Verifies that the document is not checked out. A checked out document cannot be signed by the *add* method call.
3. Verifies that the pre_signature hash argument, if any, in the method, matches a hash of the content in the repository.
4. If the content has been previously signed, the method retrieves all the audit trail entries for the previous dm_addesignature events on this content.
5. Verifies that the most recent audit trail entry is signed (by the OpenText Documentum Content Management (CM) Server) and that the signature is valid.
6. Verifies that the entries have consecutive signature numbers.
7. Verifies that the hash in the audit trail entry matches the hash of the document content.
8. Copies the content to be signed to a temporary directory location and calls the signature creation method. The signature creation method generates the signature page using the signature page template and adds the page to the content.
9. Replaces the content in the temporary location with the signed content.
10. If the signature creation method returns successfully, the method replaces the original content in the repository with the signed copy. If the signature is the first signature applied to that particular version of the document, the method appends the original, unsigned content to the document as a rendition with the page modifier set to dm_sig_source.
11. Creates the audit trail entry recording the dm_addesignature event. The entry also includes a hash of the newly signed content.

9.6.3.1 Java syntax

```
ObjectIdentity add( ElectronicSignatureInfo info)throws
com.emc.documentum.fs.services.compliance.ElectronicSignatureServiceException;
```

9.6.3.2 C# syntax

```
ObjectIdentity Add(ElectronicSignatureInfo info)
```

9.6.3.3 Parameters

Parameter	Data type	Description
electronicSignatureInfo	ElectronicSignatureInfo	Electronic signature information.

9.6.3.4 Response

ObjectIdentity: audit trail ID for newly added signature.

9.6.3.5 Exceptions

ElectronicSignatureServiceException: is thrown when the add new signature operation fails. The failure could be caused by object changes since the last signature. It could also be thrown when another signatory is signing the same document.

9.6.3.6 Example

Example 9-41: Java: Getting formal record templates

```
ServiceFactory serviceFactory = ServiceFactory.getInstance();
IElectronicSignatureService mySvc;
mySvc = serviceFactory.getRemoteService(IElectronicSignatureService.class,
context, "compliance", "http://localhost:8080/services");

ElectronicSignatureInfo info = new ElectronicSignatureInfo();
ObjectId id = new ObjectId( doc.getObjectId().getId());
ObjectIdentity objId = new ObjectIdentity(id, docbaseName);
info.setDocId( objId);
info.setUserLoginName( userName);
info.setUserPassword( passWord);
info.setESignatureMethod( ESignatureMethod.PSS_ESIGN);
info.setHashAlgorithm( HashAlgorithm.SHA1 );
info.setSignatureFormat( SignatureFormat.PDF);
info.setSignatureJustification( "WS testing");

ObjectIdentity auditTrail = mySvc.add( info);
```

Example 9-42: C#: Getting formal record templates

```
ServiceFactory theServiceFactory = ServiceFactory.Instance;
IElectronicSignatureService eSignatureService =
theServiceFactory.GetRemoteService<IElectronicSignatureService>(
```

```
theContext, COMPLIANCE_MODULE, myServerUrl);
ElectronicSignatureInfo signatureInfo = new ElectronicSignatureInfo();
signatureInfo.userLoginName = myUser;
signatureInfo.userPassword = myPassword;
signatureInfo.userLoginDomain = myDomain;
signatureInfo.docId = myObjectId;
signatureInfo.signatureJustification = "Dot Net Test";
signatureInfo.ESignatureMethod = ESignatureMethod.PSS_ESIGN;
signatureInfo.ESignatureMethodSpecified = true;
signatureInfo.hashAlgorithm = HashAlgorithm.SHA1;
signatureInfo.hashAlgorithmSpecified = true;
signatureInfo.preSignatureHash = null;
signatureInfo.signatureFormat = SignatureFormat.PDF;
signatureInfo.signatureFormatSpecified = true;
signatureInfo.applicationProperties = null;
signatureInfo.signForGroup = "administrator";
signatureInfo.signAsGroup = null;
ObjectIdentity auditId = eSignatureService.Add(signatureInfo);
```

9.6.4 verify operation

The *verify* operation examines the audit trail entries to find all entries for the specified object(s) with the event name dm_addesignature. The method does not calculate the hash values of the source content and the signed content, but does compare the calculated hash values stored within all audit trail entries to verify that the corresponding hash values do match.

This operation also checks that the signatures on the object are numbered consecutively.

9.6.4.1 Java syntax

```
public boolean verify(ObjectIdentity objectId)throws
com.emc.documentum.fs.services.compliance.ElectronicSignatureServiceException;
```

9.6.4.2 C# syntax

```
bool Verify(ObjectIdentity objectId);
```

9.6.4.3 Parameters

Parameter	Data type	Description
objectIdentity	ObjectIdentity	The object ID represents an object passed for signature verification.

9.6.4.4 Response

Boolean. True if verification succeeds. Is otherwise False.

9.6.4.5 Exceptions

Exception if passed object can not be found.

9.6.4.6 Example

Example 9-43: Java: Getting applied policy information

```
IElectronicSignatureService mySvc;
mySvc = serviceFactory.getRemoteService(IElectronicSignatureService.class,
context, "compliance", "http://localhost:8080/services");
boolean verified = mySvc.verify(myObjectId);
```

Example 9-44: C#: Getting applied policy information

```
ServiceFactory theServiceFactory = ServiceFactory.Instance;
IElectronicSignatureService eSignatureService =  theServiceFactory.
GetRemoteService<IElectronicSignatureService>(
theContext, COMPLIANCE_MODULE, myServerUrl);
bool isTestSuccess = eSignatureService.Verify(myObjectId);
(theContext, COMPLIANCE_MODULE, myServerUrl);
bool isTestSuccess = eSignatureService.Verify(myObjectId);
```

Chapter 10

Content delivery services

Content delivery services in this namespace are delivered with Interactive Delivery Services (IDS). These services are bundled with the IDS source installer and are deployed on the source host only. For more information about the deployment, see *OpenText Documentum Interactive Delivery Services - Installation Guide (EDCPBWEB160705-IGD)*.

10.1 Content Delivery service

The Content Delivery service offers the ability to publish repository content to a website, or to ingest modified web content into the repository. Specific content, such as objects, folders and cabinets, or entire publishing folders can be published from source to target. Target-side content changes can be routed to the repository through a workflow so that only approved content is ingested.

Content Delivery service is implemented as a POJO service.

Note: This service is not packaged with client-side support for this release.

10.1.1 Dependencies and prerequisites

As a precondition for these services, Documentum Interactive Delivery Services must be installed and configured on both the source and target hosts. If a database is used, it must exist on the target host.

A site publishing configuration must be created with Documentum Administrator for any objects being published by this service. The ingest operation requires a pre-existing workflow.

10.1.2 Objects related to this service

This section describes the objects used by the Content Delivery service.

Refer to the Javadoc for field-level information.

10.1.2.1 IngestStatus

The IngestStatus class is a response of the ingest operation, and acts as a container for the value of the ingest status. It sets the ingest status as a boolean parameter, and it returns the workflow ID for the object.

This class includes the following methods:

- getMessage()
- getStatus()
- getWorkFlowId()
- setMessage(String)
- setStatus(IngestStatus.IngestionStatus)
- setWorkFlowId(ObjectIdentity)

10.1.2.2 PublishInfo

PublishInfo is an input parameter that is normally passed in the publish operation. It controls specific behaviors of the publish operation. It has two internal properties:

- methodTraceLevel

  Method tracing can be set to levels from 0 to 10. Default value is 0.

- forceRefresh

  Specifies whether the object(s) should be refreshed. Default value is false.

This class includes the following methods:

- getMethodTraceLevel()
- isForceRefresh()
- setForceRefresh(boolean)
- setMethodTraceLevel(String)

10.1.2.3 PublishSiteInfo

PublishSiteInfo is an input parameter that is normally passed in the publishSite operation. It controls specific behaviors of the publishSite operation. It has four internal properties:

- fullRefresh

  The default is FALSE. If set to TRUE, the entire site is deleted and the source data set is republished.

  If you perform a full refresh and you modify the attributes to be exported using IAPI, you should also set recreatePropertySchema to TRUE in order to destroy and recreate the database tables.

You must have Superuser privileges to specify a fullRefresh.

- methodTraceLevel

  The default is 0. When methodTraceLevel is set, debug tracing for the publish operation is enabled. Method tracing can be set to levels from 0 to 10.

- updatePropertySchema

  The default value is FALSE. When set to TRUE, the database schema is updated without a full refresh publishing operation.

- recreatePropertySchema

  The default is FALSE. If set to TRUE, then the database tables at the target host are destroyed and recreated during a refresh.

  Setting recreatePropertySchema to TRUE forces a fullRefresh publishing operation and causes the database schema to reflect that of the repository types at the time of the publish, but you must restart the IDS source software first. You should set recreatePropertySchema to TRUE any time you use IAPI to modify the attributes to be exported.

This class includes the following methods:

- getMethodTraceLevel()
- isFullRefresh()
- isRecreatePropertySchema()
- isUpdatePropertySchema()
- setFullRefresh(boolean)
- setMethodTraceLevel(String)
- setRecreatePropertySchema(boolean)
- setUpdatePropertySchema(boolean)

10.1.2.4 PublishStatus

PublishStatus is a response of the publishSite and publish operations, and acts as a container for the value of the publish status. It sets the publish status as a boolean parameter.

This class includes the following methods:

- getMessage()
- getStatus()
- setMessage(String)
- setStatus(PublishStatus.Status)

10.1.3 publishSite operation

This operation publishes the contents of a publishing folder for which a site publishing configuration has been created using Documentum Administrator. Various publishing options are available with this operation.

When this service is invoked, Interactive Delivery Services publishes content and/or metadata of a Web Cabinet from the repository to a target web server or application server. If the website has been published before, the service publishes only new or modified content. If metadata is published, the metadata is transferred in an ICE format XML file; on subsequent publish requests, only new or modified metadata is exported.

This operation has rollback capability. Metadata is rolled back to a previous state if there are any exceptions on subsequent publish requests.

10.1.3.1 Java syntax

```
public PublishStatus publishSite (ObjectIdentity configId,
                                          PublishSiteInfo publishSiteInfo)
                        throws ScsServiceException
```

10.1.3.2 C# syntax

```
PublishStatus PublishSite(ObjectIdentity configId,
                                          PublishSiteInfo publishSiteInfo)
```

10.1.3.3 Parameters

Parameter	Data type	Description
configId	ObjectIdentity	The IDS configuration ID to which publishing is invoked. This represents a unique site configuration, created in Documentum Administrator, for the target location.
publishSiteInfo	PublishSiteInfo	See "PublishSiteInfo" on page 442 for details.

10.1.3.4 Response

This service operation returns a PublishStatus object which contains the publish status (SUCCESS, WARNING, ERROR, or FATAL) and, in the case of a known error, a message.

10.1.3.5 Exceptions

This operation throws an ScsServiceException if publishing fails.

10.1.3.6 Example

This example publishes new or changed content files only for a preconfigured website. The level of tracing output is set to the highest level (10).

Example 10-1: Java: Publish a site

```
{
            PublishSiteInfo publishSiteInfo = new PublishSiteInfo();
            publishSiteInfo.setFullRefresh(false);
            publishSiteInfo.setMethodTraceLevel("10");
            publishSiteInfo.setUpdatePropertySchema(false);
            publishSiteInfo.setRecreatePropertySchema(false);
            ObjectIdentity objectIdentity = new ObjectIdentity();
            objectIdentity.setRepositoryName("my_repository_name");
            ObjectId configId = new ObjectId("080010a280000d37");
            objectIdentity.setValue( configId );
            PublishStatus publishStatus = mySvc.publishSite
            (objectIdentity, publishSiteInfo);
            System.out.println("Publish Status : "+ publishStatus.getStatus());
            System.out.println("Message : " + publishStatus.getMessage());
        }
        catch (ScsServiceException e)
        {
            String[] message = (String[])e.getMessageArgs();
            System.out.println(message[0]);
            e.printStackTrace();

        }
        catch (Throwable t)
        {
            t.printStackTrace();
        }
```

Example 10-2: C#: Publish a site

```
{
            PublishSiteInfo publishSiteInfo = new PublishSiteInfo();
            publishSiteInfo.fullRefresh = true;
            publishSiteInfo.methodTraceLevel="10";
            publishSiteInfo.updatePropertySchema=false;
            publishSiteInfo.recreatePropertySchema = false;
            ObjectId configId = new ObjectId("080010a280000d37");
            ObjectIdentity objectIdentity = new ObjectIdentity(configId,
                     "documentum");
            PublishStatus publishStatus = mySvc.PublishSite
                     (objectIdentity, publishSiteInfo);
            Console.WriteLine("Publish Status :
                     "+ publishStatus.status);
            Console.WriteLine("Message :
```

```
                    " + publishStatus.message);


        }
```

10.1.4 publish operation

This operation is similar to "publishSite operation" on page 444 except that it publishes a specific object, folder, or cabinet from the repository rather than an entire website. Object ID is used to specify the content to be published.

A site publishing configuration must exist for the content. Various publishing options are available with this operation.

Note: The publish operation can publish an *entire site* if the ID of the site (root folder) is chosen. However, this use is not recommended because it results in a full refresh without the option to update or recreate property schema. For this reason, entire sites should be published using the "publishSite operation" on page 444.

10.1.4.1 Java syntax

```
public PublishStatus publish (ObjectIdentity configId,
                                        DataPackage dataPackage,
                                        PublishInfo publishInfo)
                        throws ScsServiceException
```

10.1.4.2 C# syntax

```
PublishStatus Publish(ObjectIdentity configId,
                                        DataPackage dataPackage,
                                        PublishInfo publishInfo)
```

10.1.4.3 Parameters

Parameter	Data type	Description
configId	ObjectIdentity	The IDS configuration ID to which publishing is invoked. This represents a unique site configuration, created in Documentum Administrator, for the target location.
publishInfo	PublishInfo	See "PublishInfo" on page 442 for details.

Parameter	Data type	Description
dataPackage	DataPackage	DataPackage containing one or more DataObject. A DataObject can represent an existing object, folder, or cabinet in the repository. If the DataObject specifies the object ID of a folder, everything under the folder is also refreshed. The folder must be located beneath the root publishing folder in the webc config object. Multiple DataObject can be used to specify the multiple object IDs, allowing multiple single-item publishes, but this cannot be combined with the object ID of a folder.

10.1.4.4 Response

This service operation returns a PublishStatus object which contains the publish status (SUCCESS, WARNING, ERROR, or FATAL) and, in the case of a known error, a message.

10.1.4.5 Exceptions

This operation throws an ScsServiceException if publishing fails.

10.1.4.6 Example

In this example, two specific objects are published from a preconfigured website. The level of tracing output is set to the highest level (10).

Example 10-3: Java: Publish specified objects

```
{
            PublishInfo publishInfo = new PublishInfo();
            publishInfo.setMethodTraceLevel("10");
            publishInfo.setForceRefresh (false);
            ObjectIdentity objectIdentity = new ObjectIdentity();
            objectIdentity.setRepositoryName("my_repository_name");
            ObjectId configId = new ObjectId("080010a280000d37");
            objectIdentity.setValue( configId );
            DataPackage dataPackage = new DataPackage();

            ObjectIdentity documentIdentity1 = new ObjectIdentity("my_repository_name");
            ObjectId documentId1 = new ObjectId("090010a280004aff");
            documentIdentity1.setValue(documentId1);
            DataObject dataObject1 = new DataObject(documentIdentity1);
            dataPackage.addDataObject(dataObject1);

            ObjectIdentity documentIdentity2 = new ObjectIdentity("my_repository_name");
            ObjectId documentId2 = new ObjectId("090010a280004b00");
            documentIdentity2.setValue(documentId2);
```

```
            DataObject dataObject2 = new DataObject(documentIdentity2);
            dataPackage.addDataObject(dataObject2);

            PublishStatus publishStatus = mySvc.publish(objectIdentity,
             dataPackage, publishInfo);
            System.out.println("Publish Status : "+ publishStatus.getStatus());
            System.out.println("Message : " + publishStatus.getMessage());
        }
        catch (ScsServiceException e)
        {
            String[] message = (String[])e.getMessageArgs();
            System.out.println(message[0]);
            e.printStackTrace();

        }
        catch (Throwable t)
        {
            t.printStackTrace();
        }
```

Example 10-4: C#: Publish specified objects

```
{
            PublishInfo publishInfo = new PublishInfo();
            publishInfo.forceRefresh = true;
            publishInfo.methodTraceLevel = "10";

            DataPackage dataPackage = new DataPackage();
            ObjectId documentId1 = new ObjectId("090010a280004aff");
            ObjectIdentity documentIdentity1 =
                     new ObjectIdentity(documentId1,"documentum");
            DataObject dataObject1 = new DataObject(documentIdentity1);
            dataPackage.AddDataObject(dataObject1);


            ObjectId documentId2 = new ObjectId
                     ("090010a280004b00");
            ObjectIdentity documentIdentity2 =
                     new ObjectIdentity(documentId2,"documentum");
            DataObject dataObject2 = new DataObject(documentIdentity2);
            dataPackage.AddDataObject(dataObject2);

            ObjectId configId = new ObjectId("080010a280000d37");
            ObjectIdentity objectIdentity =
                     new ObjectIdentity(configId, "documentum");
            PublishStatus publishStatus =
                     mySvc.Publish(objectIdentity,dataPackage, publishInfo);
            Console.WriteLine("Publish Status : " + publishStatus.status);
            Console.WriteLine("Message : " + publishStatus.message);


        }
```

10.1.5 ingest operation

This operation ingests modified content and/or metadata to the repository by routing it through a workflow. This API will be invoked in order to bring modified data (such as comments or form results) into the repository. The content was originally published to the target using the Interactive Delivery Services publishing mechanism.

The object being ingested will pass through a predefined workflow before getting checked in to the repository; the workflow will provide the option to either accept or discard the changes. A set of attributes associated with the incoming object relates to the earlier object that was published. Modified metadata (user attributes and application attributes only) will be either appended (for repeating attributes) or updated (for single attributes). Any changes to system and internal attributes are discarded.

The workflow is packaged with the IDS DocApp as part of the IDS source installation.

To use a *custom* workflow with this operation, use the ingest_workflow argument. This argument is set in the Site Publishing/IDS Administration node of Documentum Administrator; it appears on the Extra Arguments tab for the repository configuration. This is an optional setting that is used only when a custom workflow is required.

Note: It is not possible to set the ingest_workflow argument in individual site configurations; this argument is set at the repository level only.

10.1.5.1 Java syntax

```
public IngestStatus ingest (DataPackage dataPackage,
                                    OperationOptions options)
                        throws ScsServiceException
```

10.1.5.2 C# syntax

```
IngestStatus Ingest(DataPackage dataPackage,
                                    OperationOptions options)
```

10.1.5.3 Parameters

Parameter	Data type	Description
dataPackage	DataPackage	DataPackage contains modified (either content is modified or metadata is modified or both) or new DataObject.
options	OperationOptions	An object containing profiles and properties that specify operation behaviors. If this object is null, default operation behaviors will take effect.

10.1.5.4 Response

This service operation returns an IngestStatus object which contains the ingest status (SUCCESS, WARNING, ERROR, or FATAL); a workflow ID; and, in the case of a known error, a message.

10.1.5.5 Exceptions

This operation throws an ScsServiceException if ingest fails.

10.1.5.6 Example

In this example, one modified object is being ingested to the repository.

Example 10-5: Java: Ingest a modified object

```
{

            DataPackage dataPackage = new DataPackage();

            ObjectIdentity documentIdentity1 = new ObjectIdentity
            ("my_repository_name");
            ObjectId documentId1 = new ObjectId("090010a280004aff");
            documentIdentity1.setValue(documentId1);
            DataObject dataObject1 = new DataObject(documentIdentity1);
            dataObject1.getContents().add(new FileContent
            ("c:\\My_Test.txt", "crtext"));
            dataPackage.addDataObject(dataObject1);

            PropertySet propertySet = new PropertySet();
            StringArrayProperty strArrayProperty = new StringArrayProperty
            ("authors", new String[]{"first_name","last_name" });
            propertySet.set(strArrayProperty);
            dataObject1.setProperties(propertySet);

            OperationOptions options = new OperationOptions();
            IngestStatus ingestStatus = mySvc.ingest(dataPackage, options);

            System.out.println("Ingest Status : "+ ingestStatus.getStatus());
            System.out.println("Workflow Id : "+ ((ObjectId)
                ingestStatus.getWorkFlowId().getValue()).getId());
            System.out.println("Message : " + ingestStatus.getMessage());
```

```
        }
        catch (ScsServiceException e)
        {
            String[] message = (String[])e.getMessageArgs();
            System.out.println(message[0]);
            e.printStackTrace();

        }
        catch (Throwable t)
        {
            t.printStackTrace();
        }
```

Example 10-6: C#: Ingest a modified object

```
{
        DataPackage dataPackage = new DataPackage();
        ObjectId documentId1 = new ObjectId("090010a280004afe");
        ObjectIdentity documentIdentity1 = new ObjectIdentity
        (documentId1, "documentum");
        DataObject dataObject1 = new DataObject(documentIdentity1);
        dataObject1.Contents.Add(new FileContent("c:\\Test.txt", "crtext"));

        dataPackage.AddDataObject(dataObject1);

        PropertySet propertySet = new PropertySet();
        StringArrayProperty strArrayProperty =
        new StringArrayProperty("authors", new String[]
        {"author first name","author last name" });
        propertySet.Set(strArrayProperty);
        dataObject1.Properties = propertySet;
        OperationOptions options = new OperationOptions();
        IngestStatus ingestStatus = mySvc.Ingest(dataPackage, options);

        Console.WriteLine("Ingest Status : "+ ingestStatus.status);
        Console.WriteLine("Workflow Id : " +
        ((ObjectId) ingestStatus.workFlowId.Value).Id);
        Console.WriteLine("Message : " + ingestStatus.message);
    }
```